Programmierung für Ingenieurinnen und Ingenieure

Prof. Dr. Maik Weber

2026-10-19

Inhaltsverzeichnis

1. Vorwort **7**

2. Einleitung **8**
2.1. Was ist Programmierung? . . . 8
Exemplarische Anwendungsbereiche der Programmierung . . . 8
2.2. Welche Bedeutung hat Programmierung für Ingenieurinnen und Ingenieure? . . 9
2.3. Grundbegriffe: Algorithmus und Programm . . . 12
2.4. Von Assembler bis Python: Ein kurzer Überblick über Programmiersprachen . 14
Historie der Programmiersprachen . . . 15
Interpretiert oder kompiliert - das ist hier die Frage! . . . 15
Welche Programmiersprache darf es denn sein? . . . 16

3. Vorbereitungen **18**
3.1. Anaconda installieren . . . 18
3.2. Die IDE - unsere Werkbank fürs Programmieren! . . . 19
Was ist eine IDE? . . . 20
Vorteile einer IDE . . . 20
IDEs für die Python-Entwicklung . . . 20

4. Einführung ins Programmieren **23**
4.1. Das erste Programm: Hello, World . . . 23
4.2. Python als Taschenrechner . . . 25
4.3. Zuweisungen . . . 29
Einschub: Benutzereingaben . . . 33
4.4. Vergleiche . . . 35
4.5. Zum Abschluss des Kapitels: ein wenig Fachsprache . . . 40

5. Datentypen und ihre Repräsentation **43**
5.1. Grundlegende Datentypen . . . 43
Statisch oder dynamisch typisiert? . . . 46
5.2. Casting (Typumwandlung) . . . 47
Implizites Casting . . . 47
Explizites Casting . . . 48
5.3. Listen . . . 51
Methoden der Liste . . . 53

5.4. Instanzen und Referenzen . . . 56
Ein vereinfachtes Speichermodell . . . 56
Instanzen . . . 57
Referenzen . . . 57
Identität einer Instanz . . . 58
Seiteneffekte – wenn mehrere Referenzen auf dieselbe Instanz zeigen . . . 59
Mutable und immutable Datentypen . . . 62
5.5. Binäre Darstellung von Daten . . . 64
Binäre Darstellung von Ganzzahlen . . . 65
Binäre Darstellung von Gleitkommazahlen . . . 66
Binäre Darstellung von Strings . . . 68

6. Kontrollstrukturen **71**
6.1. Verzweigungen . . . 71
Verzweigungen mit if und else . . . 73
Verzweigungen mit if, elif und else . . . 75
Zusammengesetzte Bedingungen mit *and*, *or* und *not* . . . 79
6.2. Schleifen . . . 80
While-Schleife . . . 81
For-Schleife . . . 82
Den Schleifenfluss kontrollieren mit *break* und *continue* . . . 87
Verschachtelte Schleifen . . . 91
6.3. Anwendungen in der Mathematik . . . 93
Das Newton-Verfahren . . . 93
Lineare Algebra mit Listen . . . 95

7. Funktionen **99**
7.1. Prozedurale Programmierung . . . 99
Was ist eine Funktion? . . . 100
7.2. Built-in Functions . . . 100
Funktionen für Ein- und Ausgabe . . . 101
Funktionen für Typumwandlungen (Casting) . . . 101
Allgemeine Informationsfunktionen . . . 101
Numerische Funktionen . . . 102
Nützliche Funktionen für Sequenzen . . . 102
Funktionen für verschiedene Zahlensysteme . . . 103
Funktionen für Zeichencodierung . . . 103
7.3. Eigene Funktionen programmieren . . . 103
Parameter und Argumente . . . 104
Mehrere Parameter . . . 105
Rückgabewerte . . . 106
Keyword-Argumente . . . 108

7.4. Dynamische Typisierung und mögliche Gefahren . . . 109
Ergänzung: Mit `isinstance` den Typ von Parametern prüfen . . . 111
7.5. Funktionen mit beliebig vielen Parametern . . . 111
7.6. Scope . . . 112
Das Keyword `global` in Python . . . 114
7.7. Funktionsaufrufe mit mutable Datentypen . . . 117
7.8. Docstrings und Typ-Annotationen: Meine Funktion für andere dokumentieren . 119

8. Datenstrukturen in Python 122
8.1. Ein Sensor speichert stündlich Temperaturdaten – Anwendung der Liste . . . 123
Speichern der Daten . . . 123
Sortieren der Messungen . . . 123
Werte extrahieren – Indexzugriff und List Slicing . . . 124
8.2. Analyse der verschiedenen Messwerte – Anwendung des Sets . . . 128
8.3. Hinzufügen von Zeitstempeln – Anwendung des Dictionaries . . . 130
8.4. Speichern einer zweiten Messgröße – Anwendung des Tupels (engl.: tuple) . . . 133
8.5. Daten mehrerer Sensoren – verschachtelte Dictionaries . . . 137
8.6. Mutable und immutable Datenstrukturen . . . 138
8.7. Sicheres Kopieren von Listen . . . 140
8.8. List Comprehensions . . . 142
Syntax . . . 142
Beispiele aus der Ingenieurpraxis . . . 143
Performance . . . 147
Fazit für die Praxis . . . 148
8.9. Eleganter iterieren: `enumerate()` und `zip()` . . . 149

9. Die Verwendung von Bibliotheken 152
9.1. Einführung . . . 152
Aliase . . . 153
Installation . . . 154
Kurze Begriffsklärung: Modul, Paket, Bibliothek . . . 155
9.2. NumPy . . . 155
Überblick . . . 155
Arrays . . . 156
Demo: Matrixoperationen mit Arrays . . . 161
Demo: Vektorisierung . . . 165
Demo: Broadcasting . . . 166
9.3. Matplotlib . . . 168
Eine Funktion plotten . . . 169
Messwerte in einem Streudiagramm darstellen . . . 171
Ein einfaches Balkendiagramm . . . 173
9.4. pandas . . . 176

10. Objektorientierte Programmierung (OOP) 182
10.1. Grundidee und Prinzipien . . . 182
Kurze Historie . . . 185
10.2. Bestandteile einer Klasse: Methoden und Attribute . . . 185
10.3. Erste eigene Klassen . . . 187
10.4. UML-Klassendiagramme . . . 191
Grundstruktur eines Klassendiagramms . . . 191
Beziehungen zwischen Klassen . . . 194
Ergänzungen . . . 194
10.5. Beispiel: Bibliotheksverwaltung . . . 195
Die Klasse `Library` . . . 195
Die Klasse `Medium` und ihre Kindklassen: `Book` und `Magazine` . . . 196
Mit `super()` auf die Basisklasse zugreifen . . . 196
Die Implementierung . . . 196
10.6. Public, Protected, Private . . . 201
public - der Standard . . . 201
protected - nur für den internen Gebrauch . . . 201
private - Gehört nur einer einzigen Klasse! . . . 202
Getter und Setter . . . 203
10.7. Magic Methods . . . 203
10.8. Abstrakte Klassen . . . 207

Anhang 211

A. Dateien lesen und schreiben 211
A.1. Dateien, Pfade und das Arbeitsverzeichnis . . . 211
A.2. Textdateien schreiben . . . 212
A.3. Textdateien lesen . . . 213
A.4. CSV-Dateien mit dem Modul `csv` . . . 215

B. Fehlerbehandlung 218
B.1. Arten von Fehlern . . . 218
B.2. Eine Fehlermeldung lesen . . . 218
B.3. Fehler abfangen mit `try` und `except` . . . 220
Mehrere Fehlertypen und die Fehlermeldung selbst . . . 220
`else` und `finally` . . . 221
B.4. Fehler selbst auslösen mit `raise` . . . 221
B.5. Typische Fehler bei der Fehlerbehandlung . . . 222

C. Interaktivität und grafische Oberflächen 225
C.1. Ereignisgesteuerte Programmierung . . . 225

C.2. Interaktive Notebooks mit ipywidgets . . . 226
Parameterstudien mit `interact` . . . 226
C.3. Desktop-Anwendungen mit tkinter . . . 229
Beispiel: ein Biegespannungsrechner . . . 230
C.4. Weitere Werkzeuge . . . 232

D. MicroPython: Python auf Mikrocontrollern **235**
D.1. Was ist gleich, was ist anders? . . . 235
D.2. Das „Hello, World“ der Mikrocontroller: eine blinkende LED . . . 236
D.3. Eine Spannung messen . . . 236
D.4. REPL und `main.py` . . . 237
D.5. Zum Weiterlesen . . . 238

1. Vorwort

Version: 4.0

Willkommen zu dieser Einführung in die Programmierung! Dieses Skript bildet die Basis für die Vorlesungen „Ingenieurinformatik 1“ und „Grundlagen der Programmierung“ an der Hochschule Trier. Keine Sorge, Vorkenntnisse in Programmierung oder Informatik sind nicht erforderlich!

Wir werden die grundlegenden Konzepte und deren praktische Umsetzung in der Programmiersprache Python erkunden. Der Kurs richtet sich an Studierende der Ingenieurwissenschaften in den ersten beiden Semestern, ist aber auch für Interessierte mit anderen Hintergründen geeignet.

Obwohl Python unsere exemplarische Programmiersprache ist, handelt es sich nicht um einen reinen Python-Kurs. Unser Ziel ist es, ein umfassendes Grundverständnis der Programmierung zu vermitteln: Wie funktioniert sie? Welche fundamentalen Konzepte gibt es in nahezu allen Programmiersprachen und wie können diese genutzt werden, um Probleme zu lösen? Um den Blick für das große Ganze zu bewahren, ziehen wir immer wieder Vergleiche mit anderen Programmiersprachen und vermeiden es, zu tief in die spezifischen Details von Python einzutauchen.

Nach erfolgreichem Abschluss des Kurses werden Sie in der Lage sein, die Themen, die für Ihren persönlichen Weg von besonderer Bedeutung sind, eigenständig zu vertiefen. Das Gelernte wird Ihnen auch ein sicheres Fundament geben, wenn Sie eine andere Sprache als Python lernen müssen (oder wollen). Abschließend sei noch auf den Folgekurs „Ingenieurinformatik 2“ hingewiesen, der auf diesem Kurs aufbaut und den Fokus stärker auf Anwendung und Problemlösung legt.

Die Codebeispiele wurden mit Python 3.13 erstellt und ausgeführt. Neben Python selbst werden die Pakete NumPy, pandas und Matplotlib benötigt, für Anhang C außerdem ipywidgets; in Anaconda und Google Colab sind sie bereits enthalten. Mit anderen Versionen können einzelne Ausgaben leicht abweichen.

No one is perfect: Trotz sorgfältiger Durchsicht werden sich Fehler eingeschlichen haben (wie übrigens auch in jedem größeren Programm). Hinweise bitte an den Autor: m.weber@hochschule-trier.de

Viel Erfolg bei diesem Kurs – oder wie man unter Programmierer*innen sagt: Happy Coding!

2. Einleitung

Lernziele

Nach diesem Kapitel können Sie …

- erklären, was Programmieren ist und wo es in den Ingenieurwissenschaften eingesetzt wird
- die Begriffe *Algorithmus* und *Programm* erklären und die Eigenschaften eines Algorithmus nennen
- Programmiersprachen nach ihrer Nähe zur Maschine unterscheiden (Maschinencode, Assembler, Hochsprache) und ihre Entwicklung bis heute in groben Zügen skizzieren
- den Unterschied zwischen interpretierten und kompilierten Sprachen erklären
- einschätzen, welche Programmiersprache sich für welchen Zweck eignet

2.1. Was ist Programmierung?

Programmierung ist die Kunst und Wissenschaft, Computer dazu zu bringen, bestimmte Aufgaben auszuführen. Dies geschieht, indem man ihnen genaue Anweisungen in Form von Code gibt. Dabei werden Logik, Problemlösungsfähigkeiten und technisches Know-how kombiniert, um Software-Anwendungen, Websites, Spiele und vieles mehr zu entwickeln. Programmieren ist nicht nur eine technische Fertigkeit, sondern auch ein kreativer Prozess, der neue Lösungen für reale Herausforderungen ermöglicht.

In der Praxis bedeutet Programmieren weit mehr als nur das Schreiben von Code. Es umfasst das Verstehen von Problemen, das Entwerfen effizienter Lösungen sowie das Testen und Optimieren von Programmen. Gerade in den angewandten Wissenschaften spielt Programmieren eine zentrale Rolle, um praxisnahe Lösungen zu entwickeln – sei es die Automatisierung eines Fertigungsprozesses, die Analyse komplexer Daten oder die Optimierung einer Maschine.

Exemplarische Anwendungsbereiche der Programmierung

Automatisierung: Vereinfachen von Routineaufgaben.

Robotik: Steuern von Robotern und automatisierten Maschinen.

Messtechnik und Prüfstände: Erfassen, Auswerten und Visualisieren von Messdaten.

Simulation und Berechnung: Nachbilden technischer Systeme, z.B. Festigkeits-, Strömungs- oder Schaltungssimulationen.

Embedded Systems und Steuerungstechnik: Programmieren von Mikrocontrollern, Sensoren und speicherprogrammierbaren Steuerungen (SPS). Wie Python auf einem Mikrocontroller aussieht, zeigt Anhang D.

CAD und Konstruktion: Automatisieren wiederkehrender Konstruktions- und Auslegungsaufgaben über Skript-Schnittstellen.

Datenanalyse: Analysieren großer Datenmengen.

Künstliche Intelligenz: Entwickeln von KI-Systemen, z.B. zur Klassifizierung oder Generierung von Bildern.

Cybersecurity: Schützen von Computersystemen und Netzwerken vor Angriffen.

Medizinische Informatik: Entwickeln von Softwarelösungen für das Gesundheitswesen.

E-Commerce: Erstellen von Online-Shops und Zahlungsplattformen.

Webentwicklung: Erstellen und Gestalten von Websites und Webanwendungen.

Spieleentwicklung: Entwickeln von Computerspielen.

Mobile Apps: Programmieren von Anwendungen für Smartphones und Tablets.

Programmieren zu lernen ist vergleichbar mit dem Erlernen einer neuen Sprache: Je mehr Sie üben, desto besser können Sie Ihre Ideen in funktionsfähige Programme umsetzen. Lassen Sie sich dabei nicht von der Vielzahl an Programmiersprachen abschrecken. Ähnlich wie beim Sprachenlernen erleichtert das Wissen über eine Sprache das Erlernen einer weiteren: Wer Spanisch beherrscht, wird sich leichter in Italienisch zurechtfinden. Und selbst bei einer völlig anderen Sprache wie Japanisch profitieren Sie von Ihrer allgemeinen Sprachlernerfahrung. Deshalb beginnt man mit einer ausgewählten Programmiersprache und baut auf diesem Wissen auf, wenn man weitere benötigt.

2.2. Welche Bedeutung hat Programmierung für Ingenieurinnen und Ingenieure?

Programmieren ist längst kein reines IT-Thema mehr – es ist ein unverzichtbares Werkzeug für moderne Ingenieurinnen und Ingenieure. Ob Maschinenbau, Elektrotechnik, Fahrzeugtechnik oder Wirtschaftsingenieurwesen – in nahezu jedem ingenieurwissenschaftlichen Bereich spielen Software und Algorithmen eine zentrale Rolle. Eine Auswahl an Argumenten, warum Grundfertigkeiten der Programmierung für Ingenieurinnen und Ingenieure unverzichtbar sind:

1. **Automatisierung und Effizienzsteigerung**: Ingenieurinnen und Ingenieure nutzen Programmierung, um wiederkehrende Aufgaben zu automatisieren und somit die Effizienz zu steigern. Dies kann die Steuerung von Maschinen und Prozessen umfassen, aber auch die Erstellung von Berichten und Analysen. Durch Automatisierung können Fehler reduziert und Zeit gespart werden, was zu einer höheren Produktivität führt.

2. **Simulation als Schlüsselwerkzeug**: Simulationen ermöglichen Ingenieurinnen und Ingenieuren, komplexe Prozesse oder Systeme virtuell nachzubilden, zu analysieren und zu optimieren. Programmierkenntnisse sind hier unverzichtbar, um leistungsfähige Simulationen zu entwickeln und anzupassen.

3. **Datenanalyse und Entscheidungsfindung**: In der modernen Ingenieurpraxis fallen große Mengen an Daten an. Programmierkenntnisse ermöglichen es Ingenieurinnen und Ingenieuren, diese Daten zu sammeln, zu analysieren und daraus fundierte Entscheidungen zu treffen. Dies ist besonders wichtig in Bereichen wie der Qualitätskontrolle, der Forschung und Entwicklung sowie der Prozessoptimierung.

4. **Entwicklung von maßgeschneiderten Lösungen**: Ingenieurinnen und Ingenieure stehen oft vor komplexen Problemen, die standardisierte Softwarelösungen nicht immer abdecken können. Durch Programmieren können sie individuelle Softwarelösungen entwickeln, die genau auf ihre spezifischen Anforderungen zugeschnitten sind. Mit Programmierkenntnissen können sie dabei auch auf Open-Source-Frameworks, globale Entwicklungscommunities und eine nahezu unbegrenzte Palette an modernen Tools zugreifen.

5. **Interdisziplinäre Zusammenarbeit:** Die Fähigkeit zu programmieren erleichtert die Zusammenarbeit mit anderen Fachbereichen, wie der Informatik oder der Datenwissenschaft. Ingenieurinnen und Ingenieure können durch Programmierkenntnisse besser kommunizieren und gemeinsame Projekte effizienter umsetzen. Dies fördert den interdisziplinären Austausch und die Innovationskraft.

6. **Karrierechancen und berufliche Weiterentwicklung**: Programmierkenntnisse sind in der heutigen Arbeitswelt sehr gefragt und eröffnen Ingenieurinnen und Ingenieuren vielfältige Karrierechancen. Sie können sich in Bereichen wie der Robotik, der künstlichen Intelligenz, der Automatisierungstechnik oder der Medizintechnik spezialisieren. Darüber hinaus verbessern Programmierkenntnisse die berufliche Flexibilität und die Fähigkeit, sich an neue technologische Entwicklungen anzupassen.

7. **Entscheidende Metakompetenzen**: Programmieren fördert wichtige Metakompetenzen, die Ingenieurinnen und Ingenieuren helfen, in ihrer Arbeit erfolgreich zu sein. Dazu gehören:

    - Kritisches Denken: Die Fähigkeit, Probleme systematisch zu analysieren und fundierte Entscheidungen zu treffen.

- Problemlösungsfähigkeiten: Entwickeln kreativer und effektiver Lösungen für komplexe technische Herausforderungen.
- Teamarbeit: Zusammenarbeiten in interdisziplinären Teams und effektive Kommunikation mit Kolleginnen und Kollegen.
- Anpassungsfähigkeit: Schnell auf neue Technologien und Methoden reagieren und diese in die Arbeit integrieren.
- Selbstständiges Lernen: Kontinuierliches Lernen und Weiterentwickeln der eigenen Fähigkeiten, um immer auf dem neuesten Stand der Technik zu bleiben.

Im Zusammenhang mit dem letzten Punkt ist eine Untersuchung des World Economic Forum (Link) interessant, welche versucht, die Schlüsselqualifikationen für den zukünftigen Arbeitsmarkt zu identifizieren:

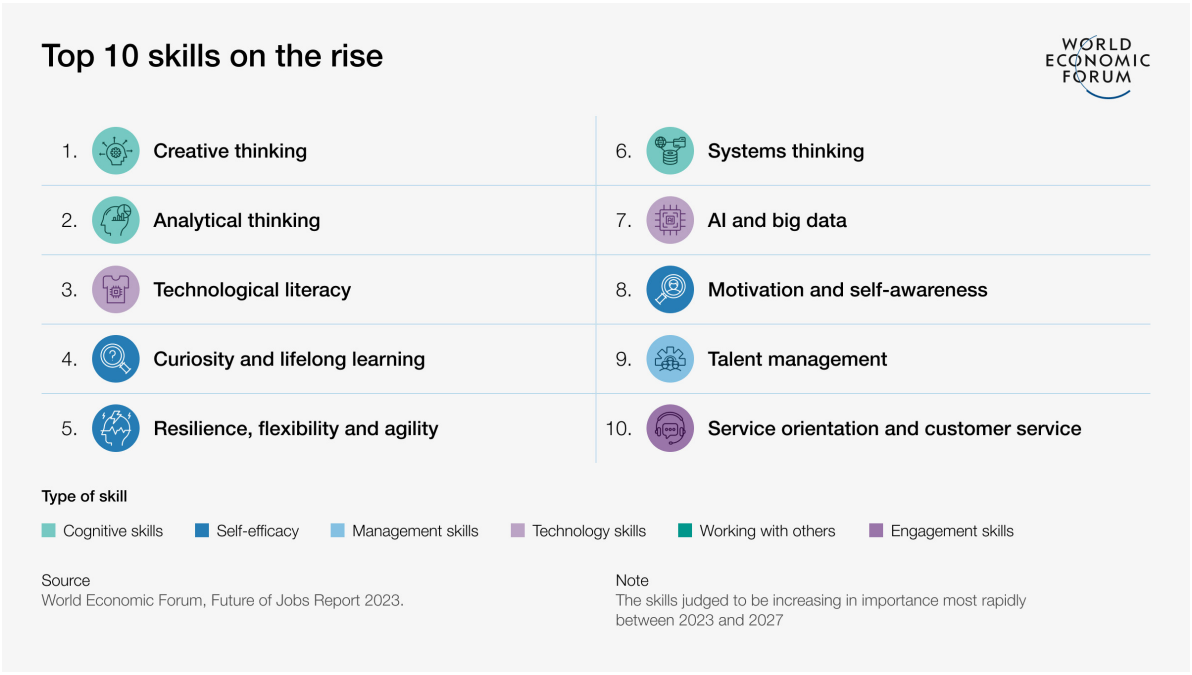


Abbildung 2.1.: Wichtigste Zukunftsskills laut World Economic Forum.

Übungen:

1. Betrachten Sie Abbildung 2.1, welche basierend auf einer Studie des World Economic Forum die wichtigsten Zukunftsskills auflistet. Welche dieser Skills könnten von einem Programmierkurs und weiteren Erfahrungen auf diesem Gebiet profitieren?

2. KI ist in den letzten Jahren immer mächtiger geworden und kann selbstständig Code schreiben. Diskutieren Sie vor diesem Hintergrund die Frage: Muss man heutzutage überhaupt noch programmieren lernen?

2.3. Grundbegriffe: Algorithmus und Programm

„In der Informatik geht es genauso wenig um Computer wie in der Astronomie um Teleskope.“ —Edsger W. Dijkstra zugeschrieben (niederländischer Informatiker und Wegbereiter der Programmierung)

Das Eingangszitat macht deutlich, dass Computer für die Informatik vor allem nützliche Werkzeuge sind. Werkzeuge wozu? Allgemein gesprochen: zur Lösung von Problemen. Hierzu werden **Algorithmen** entwickelt und in **Programmen** umgesetzt. Diese beiden Grundbegriffe möchten wir in diesem Abschnitt genauer definieren:

> **Algorithmus:** Eine präzise, endliche Abfolge von Anweisungen zur Lösung eines Problems oder zur Durchführung einer Aufgabe.

Algorithmen bilden die Grundlage der Programmierung, indem sie die Logik und die Schritte definieren, die ein Computer ausführen muss, um ein gewünschtes Ergebnis zu erzielen. Sie sind aber unabhängig von einer Programmiersprache, müssen auch gar nicht auf einem Computer ausgeführt werden, sondern können theoretisch auch von Hand durchgeführt werden. Wir alle kennen Algorithmen für bestimmte Probleme aus der Schule, z.B. den euklidischen Algorithmus, der den größten gemeinsamen Teiler (ggT) von zwei Zahlen berechnet. Eine wichtige Eigenschaft aller Algorithmen ist die *Finitheit*: Ihre Beschreibung besteht aus endlich vielen Anweisungen. Davon zu unterscheiden ist die *Terminierung*: Ein Algorithmus soll nach endlich vielen Schritten zu einem Ergebnis kommen (es können durchaus sehr viele Schritte sein, aber eben nicht unendlich viele). Beides ist nicht dasselbe: Auch eine endliche Anweisung wie „wiederhole Schritt 2, solange x größer als 0 ist“ kann endlos laufen, wenn x nie kleiner wird. Oft wird behauptet, dass Algorithmen *determiniert* seien, also bei gleicher Eingabe auch gleiche Ausgaben erzeugten. Das stimmt allerdings nicht immer, denn es gibt auch *randomisierte* Algorithmen, die bewusst mit Zufallszahlen arbeiten und bei wiederholter Durchführung andere Ergebnisse liefern können (ein Beispiel lernen wir in Ingenieurinformatik 2 kennen).

Beispiel: Bubble Sort einer Liste von Zahlen.

Problem: Gegeben ist eine Liste von Zahlen, z.B. 42, 6, -7, 100, 3. Die Zahlen sollen der Größe nach aufsteigend sortiert werden.

Ein Algorithmus für dieses Problem ist der sog. „Bubble Sort“:

- Schritt 1: Gehe zum ersten Element der Liste.
- Schritt 2: Vergleiche das aktuelle Element mit dem nächsten Element.

- Schritt 3: Wenn das aktuelle Element größer ist als das nächste Element, dann tausche die beiden.
- Schritt 4: Gehe zum nächsten Element und wiederhole die Schritte 2 und 3, bis du das Ende der Liste erreichst.
- Schritt 5: Gehe zurück zum Anfang der Liste und wiederhole die Schritte 1 bis 4, bis keine Vertauschungen mehr vorgenommen werden.

Probieren Sie den Algorithmus einmal von Hand aus. An diesem Beispiel erkennen Sie bereits die wichtigsten Elemente vieler Algorithmen: Bedingungen ("Wenn…, dann…") und Wiederholungen ("wiederhole die Schritte…, bis…"). Mit diesen Strukturen werden wir uns noch viel beschäftigen.

> **Programm:** Eine Sammlung von Anweisungen, die ein Computer ausführt, um eine bestimmte Aufgabe zu erledigen.

Programme bestehen aus einem oder mehreren Algorithmen, die in der Regel mit zusätzlichen Anweisungen kombiniert werden, z.B. für die Ausgabe der Ergebnisse. Sie sind in einer Programmiersprache geschrieben (man sagt auch "implementiert" und spricht auch von einer "Implementierung" oder "Implementation" anstelle von einem Programm). Programme können von einfachen Skripten bis hin zu komplexen Softwareanwendungen reichen. Ein und derselbe Algorithmus kann je nach Wahl der Programmiersprache unterschiedlich aussehen und unterschiedlich schwer zu implementieren sein, da nicht jede Programmiersprache dieselben Funktionalitäten bietet. Manche Programmiersprachen bieten spezifische Funktionen und Bibliotheken, die die Implementierung erleichtern, während andere Sprachen möglicherweise mehr manuelle Arbeit erfordern.

Wie würde nun eine Implementierung von Bubble Sort in Python aussehen? Hinweis: Sie müssen den folgenden Code selbstverständlich noch nicht verstehen, aber vielleicht können Sie bestimmte Parallelen zum Algorithmus bereits erkennen. Hinter dem Symbol `#` stehen übrigens Kommentare, die für Menschen zur Erläuterung gedacht sind und keine Auswirkung auf den Code haben.

```
# Python-Code, der den Algorithmus
# Bubble Sort implementiert:

def bubble_sort(numbers):
    # Vorbereitungen:
    result = numbers[:]  # Input kopieren
    n = len(numbers)  # Länge der Liste
    swapped = True # merkt sich, ob getauscht wurde

    # Es folgt eine Schleife, die
    # den Algorithmus wiederholt, bis nicht
```

```
    # mehr getauscht wurde
    while swapped:
        swapped = False

        # alle Elemente durchlaufen:
        for j in range(1, n):
            if result[j-1] > result[j]:
                # Vorgänger ist größer als Nachfolger: tauschen!
                result[j-1], result[j] = result[j], result[j-1]
                swapped = True  # es wurde getauscht

    # die sortierte Liste wird zurückgegeben:
    return result
```

Natürlich wollen wir unsere Implementierung testen:

```
bubble_sort([42,6,-7,100,3])
```

```
[-7, 3, 6, 42, 100]
```

Übungen:

1. Erinnern Sie sich an Algorithmen aus der Schulzeit und lassen Sie sich von einer KI (z.B. ChatGPT) eine Implementierung vorschlagen. Variieren Sie die Programmiersprache und vergleichen Sie!
2. Ist es bei Bubble Sort wirklich erforderlich, jedes Mal durch die gesamte Liste zu gehen? Wie könnte man den Algorithmus optimieren?

2.4. Von Assembler bis Python: Ein kurzer Überblick über Programmiersprachen

Programmiersprachen sind Werkzeuge, die es Entwicklern ermöglichen, Anweisungen zu schreiben, die ein Computer ausführen kann. Sie spielen eine zentrale Rolle in der Softwareentwicklung und haben sich in den letzten Jahrzehnten stark weiterentwickelt.

Historie der Programmiersprachen

Die Geschichte der Programmiersprachen beginnt in den 1940er Jahren mit der Entwicklung der ersten maschinennahen Sprachen. Hier sind einige wichtige Meilensteine:

- 1940er Jahre: Die ersten Computer werden direkt in *Maschinencode* programmiert, also in Folgen von Nullen und Einsen, die der Prozessor unmittelbar ausführen kann. Ende der 1940er Jahre entstehen die ersten *Assemblersprachen*: Sie ersetzen die Zahlencodes der Prozessorbefehle durch kurze, lesbare Kürzel wie `ADD` oder `MOV`, die ein Übersetzungsprogramm (ebenfalls *Assembler* genannt) eins zu eins in Maschinencode überträgt. Für Laien bleibt beides schwer verständlich.
- 1950er Jahre: Die Entwicklung von höheren Programmiersprachen beginnt, darunter Fortran (1957) und COBOL (1959). Diese Sprachen erlauben es, Programme in einer für Menschen verständlicheren Form zu schreiben.
- 1960er und 1970er Jahre: Sprachen wie ALGOL, BASIC und C werden entwickelt. C wird besonders einflussreich und bildet die Grundlage für viele moderne Sprachen.
- 1980er und 1990er Jahre: Objektorientierte Sprachen wie C++ und Java werden populär. Objektorientierung hilft, den Code besser zu strukturieren und wiederverwendbare Module zu erstellen. Python, eine vielseitige und leicht verständliche Sprache, wird 1991 veröffentlicht.
- 2000er Jahre bis heute: 2008 erscheint Python 3, das bewusst nicht vollständig mit Python 2 kompatibel ist; seit 2020 wird Python 2 nicht mehr weiterentwickelt. Stoßen Sie im Internet auf Code wie `print "Hallo"` (ohne Klammern), handelt es sich um veralteten Python-2-Code. Gleichzeitig entstehen neue Sprachen wie Go, Rust oder Kotlin, und Python entwickelt sich zu einer der meistgenutzten Programmiersprachen überhaupt – nicht zuletzt wegen seiner Bedeutung für Datenanalyse und künstliche Intelligenz.

Interpretiert oder kompiliert - das ist hier die Frage!

Programmiersprachen können grob in zwei Kategorien eingeteilt werden: interpretierte und kompilierte Sprachen.

- Interpretierte Sprachen: Diese Sprachen werden zur Laufzeit (also während das Programm läuft) interpretiert und ausgeführt. Beispiele sind Python, JavaScript und Ruby. Vorteile interpretierter Sprachen sind ihre Flexibilität und Einfachheit, da sie in der Regel keine separate Kompilierungsphase erfordern. Allerdings können die Programme langsamer sein als bei kompilierten Sprachen, da der Code während der Ausführung interpretiert werden muss. Im Hintergrund arbeitet also immer ein sog. “Interpreter”. Genau genommen übersetzt der Python-Interpreter ein Programm allerdings zunächst vollständig in einen Zwischencode (*Bytecode*) und führt erst diesen aus. Deshalb verhindert z.B. ein Syntaxfehler am Ende einer Datei, dass auch nur die erste Zeile ausgeführt wird.

- Kompilierte Sprachen: Diese Sprachen werden vor der Ausführung in Maschinencode übersetzt, den der Computer direkt ausführen kann. Beispiele sind C, C++ und Go. Kompilierte Sprachen bieten in der Regel eine höhere Ausführungsleistung, da der Code vor der Ausführung optimiert wird und nicht mehr während der Ausführung interpretiert werden muss. Sie müssen allerdings für jede Plattform eine eigene Kompilierung durchführen und erhalten jeweils ein plattformspezifisches Ergebnis: z.B. entstehen für Windows die bekannten `.exe`-Dateien, die für Linux- oder Mac-Benutzer unbrauchbar sind.

Diese Einteilung ist allerdings grob: Streng genommen ist nicht eine Sprache selbst interpretiert oder kompiliert, sondern ihre jeweilige Umsetzung. Java und C# etwa liegen dazwischen: Sie werden zunächst in einen plattformunabhängigen Zwischencode kompiliert, den dann eine virtuelle Maschine auf dem jeweiligen Rechner ausführt.

Welche Programmiersprache darf es denn sein?

Vielleicht fragen Sie sich an dieser Stelle, welche Programmiersprache denn nun *die beste* ist. Es muss doch eine geben, die alle anderen schlägt, oder? Die Antwort lautet, wie so oft im Leben: *it depends!* Und zwar hängt es von vielen Faktoren ab, sowohl technischen als auch menschlichen, welche die *beste* Programmiersprache für ein bestimmtes Projekt ist. Nennen wir einige Aspekte:

- Projektanforderungen:

  Unterschiedliche Projekte begünstigen unterschiedliche Sprachen. Webentwicklung, Datenanalyse, maschinelles Lernen, Spieleentwicklung und Systemprogrammierung haben jeweils bevorzugte Programmiersprachen. Benötigt man die höhere Leistung von kompilierten Sprachen wie C und C++ oder eher die Portabilität von interpretierten Sprachen wie Python? Auch der Umgang mit dem Speicher unterscheidet sich von Programmiersprache zu Programmiersprache und könnte ein wesentlicher Faktor sein.

- Unterstützung durch Bibliotheken und Frameworks:

  Programmiererinnen und Programmierer erfinden das Rad nicht täglich neu, sondern verwenden "Bibliotheken" und "Frameworks" für ihre Projekte. Das sind frei verfügbare Funktionen, Programme und Tools, die für spezielle Anwendungen zur Verfügung gestellt werden und die Arbeit erheblich vereinfachen und beschleunigen. Jede Programmiersprache bietet ihre eigene Palette!

  Beispiele: NumPy und pandas in Python, React und Angular in JavaScript, Spring und Hibernate in Java.

- Lernkurve, Community und persönlicher Geschmack:

  Für Anfängerinnen und Anfänger sind Sprachen wie Python und Ruby aufgrund ihrer klaren und einfachen Syntax ideal zum Lernen. Eine starke Community und umfangreiche

Dokumentation können den Lernprozess erheblich erleichtern, da es zahlreiche Ressourcen gibt (die passenden zu finden, kann dennoch eine Herausforderung sein). Nicht zu unterschätzen sind aber auch die persönlichen Präferenzen: Welche Sprache liegt mir und welche mag ich einfach nicht? Menschen sind verschieden.

Die Wahl der passenden Programmiersprache ist daher nicht immer eine einfache Entscheidung. Es kommt in der Praxis aber auch oft vor, dass man sich über diese Frage gar keinen Kopf zerbrechen muss: Man stößt z.B. neu zu einem Projekt dazu, bei dem die Sprache längst gesetzt ist. Auch ist es nicht selten, dass man älteren Code für die eigenen Zwecke anpassen muss und sich mit der dort verwendeten Sprache arrangieren muss.

Wie auch immer: Die eine one-size-fits-all-Sprache gibt es leider nicht. Wichtig ist, dass man die Grundkonzepte des Programmierens mit einer Sprache richtig gut gelernt hat, gleichzeitig aber offen gegenüber anderen Sprachen und neuen Technologien ist.

3. Vorbereitungen

Lernziele

Nach diesem Kapitel können Sie …

- eine Python-Umgebung mit Anaconda einrichten und erklären, was eine Umgebung ist
- erklären, was eine IDE ist und welche Vorteile sie bietet
- gängige Entwicklungsumgebungen (PyCharm, VS Code, Jupyter, Spyder, Colab) einordnen und eine passende auswählen
- Chancen und Risiken von KI-Assistenten beim Programmierenlernen einschätzen

In diesem Kapitel geht es um die konkreten Vorbereitungen für das Programmieren in Python: Wir legen unsere Werkzeuge bereit! Dafür installieren wir Anaconda (eine weit verbreitete Python-Distribution) und eine IDE. Alternativ können Sie auch in der Cloud programmieren, ohne etwas auf Ihrem Rechner installieren zu müssen. Hierfür empfehlen wir Google Colab (siehe unten).

Stand der Angaben

Die Angaben zu Software, Lizenzen und Angeboten in diesem Kapitel beschreiben den Stand **Oktober 2026**. Gerade bei Lizenzbedingungen und kostenlosen Kontingenten ändert sich immer wieder etwas – im Zweifel gilt die Website des jeweiligen Anbieters. Aktuell ist Python 3.15 (erschienen im Oktober 2026). Für diesen Kurs genügt jede Version ab 3.11; das Skript wurde mit Python 3.13 erstellt.

3.1. Anaconda installieren

Im Grunde bräuchten wir nur eine Python-Installation und einen Texteditor zum Programmieren. Wir gönnen uns aber etwas mehr Komfort und installieren "Anaconda", eine umfassende Python-Distribution, die viele nützliche Bibliotheken und Werkzeuge integriert. Mit Anaconda können wir Python und wichtige Pakete problemlos installieren, verwalten und aktualisieren. Die integrierte Benutzeroberfläche, der Anaconda Navigator, ermöglicht den einfachen Zugriff auf

beliebte Entwicklungsumgebungen wie Jupyter Notebooks oder Visual Studio Code (siehe unten).

Installationsanweisungen für die Plattformen Win / macOS / Linux findet man hier.

Mit der Anaconda-Installation wird auch der Anaconda Navigator installiert, der eine grafische Benutzeroberfläche zur Verfügung stellt, mit der man Conda-Umgebungen und Bibliotheken verwalten sowie Programme (z.B. IDEs) starten kann. Eine Dokumentation des Navigators findet man hier, die Bedienung ist aber intuitiv, probieren Sie es aus!

! Ist Anaconda kostenlos?

Für Studierende und Lehrende an Hochschulen: ja. Anaconda erlaubt die kostenlose Nutzung für nicht-kommerzielle Zwecke in Lehre, Studium und Forschung an akkreditierten Hochschulen (sog. *Academic Policy*). In Unternehmen mit 200 oder mehr Beschäftigten ist für die Anaconda-Paketquellen dagegen eine kostenpflichtige Lizenz nötig. Eine für jeden Zweck kostenlose Alternative ist **Miniforge**: Es enthält Python und den Paketmanager `conda`, bezieht die Pakete aber aus der freien Paketquelle *conda-forge*. Eine grafische Oberfläche wie den Navigator bringt Miniforge allerdings nicht mit.

i Was ist eine Umgebung in Anaconda?

Umgebungen (engl. *environments*) sind isolierte Arbeitsbereiche, in denen man verschiedene Versionen von Python und Paketen installieren und verwalten kann, ohne dass diese sich gegenseitig beeinflussen. Dies ist besonders nützlich, wenn man an mehreren Projekten arbeitet, die unterschiedliche Abhängigkeiten und Versionen von Bibliotheken erfordern. Die Standardumgebung heißt “base” und wir benötigen in diesem Kurs keine weiteren Umgebungen (für eigene größere Projekte empfiehlt es sich später, je Projekt eine eigene Umgebung anzulegen). Zum Experimentieren können Sie sich aber eine weitere Umgebung anlegen und dort Pakete installieren und testen, ohne Sorge zu haben, in Ihrer Standardumgebung etwas “kaputt zu machen”.

3.2. Die IDE - unsere Werkbank fürs Programmieren!

Beim Programmieren ist eine integrierte Entwicklungsumgebung (IDE, engl.: integrated development environment) das wichtigste Werkzeug – vergleichbar mit der Werkbank eines Handwerkers. Eine IDE kombiniert verschiedene Funktionen, die das Coden effizienter und angenehmer machen. Über Anaconda können wir verschiedene IDEs einfach über den Navigator installieren. Aber auch IDEs, die außerhalb des Navigators installiert werden, können in Anaconda integriert und später über den Navigator gestartet werden.

Was ist eine IDE?

Eine IDE ist eine Software, die mehrere Programmier-Tools in einer einzigen Oberfläche bündelt. Typische Bestandteile sind:

- Code-Editor mit Syntax-Highlighting und Autovervollständigung. Das ist das Herzstück: Ein Texteditor, mit dem wir unseren Quellcode schreiben.
- Debugger, um Fehler im Code zu finden. Beim "Debuggen" sucht man systematisch nach einem Fehler im Programm und nutzt hierzu verschiedene technische Hilfsmittel (z.B. "Breakpoints", die das Programm an bestimmten Stellen anhalten, um Variablen gezielt auszulesen und das gewünschte Verhalten zu überprüfen).
- Anbindung an einen Interpreter oder Compiler, um den Code auszuführen (die IDE nutzt dafür meist eine separat installierte Python-Version, z.B. aus Anaconda).
- Versionskontrolle, um Änderungen am Code zu verwalten.

Vorteile einer IDE

Warum sollte man eine IDE nutzen? Die Alternative zur IDE wäre ein einfacher Texteditor ohne spezielle Coding-Funktionalitäten. Hier sind einige Gründe, die stark für die IDE sprechen:

- ✅ Produktivität: Schnelleres Schreiben, Testen und Debuggen von Code.
- ✅ Fehlerminimierung: Automatische Code-Analyse und Debugging-Werkzeuge.
- ✅ Komfort: Bessere Übersicht und Organisation durch Projektverwaltung.

IDEs für die Python-Entwicklung

1. **PyCharm**

   PyCharm ist eine der bekanntesten IDEs für Python und wird von JetBrains entwickelt. Seit 2025 gibt es nur noch *ein* PyCharm: Die Grundfunktionen sind kostenlos – einschließlich der Unterstützung für Jupyter Notebooks –, zusätzliche Profi-Funktionen erfordern ein kostenpflichtiges Pro-Abonnement. Die frühere kostenlose Community Edition wurde damit eingestellt. Studierende erhalten die Pro-Funktionen kostenlos über das *JetBrains Student Pack* (jeweils für ein Jahr, verlängerbar; siehe PyCharm für Studierende).

   Getting Started with PyCharm: Guide.

2. **Visual Studio Code**

   Visual Studio Code (VS Code) von Microsoft ist ein beliebter, kostenloser Code-Editor, der durch Erweiterungen (Extensions) zu einer vollwertigen IDE für viele Programmiersprachen wird. Für Python installiert man die Erweiterung *Python* von Microsoft, für

Notebooks zusätzlich die Erweiterung *Jupyter* (siehe unten). Hier geht es zur Dokumentation.

3. **Jupyter Notebooks**

   Jupyter Notebook ist eine webbasierte Entwicklungsumgebung, die besonders beliebt für interaktive Datenanalyse oder maschinelles Lernen ist. Das Besondere ist die Kombination aus Markdown- und Code-Zellen. Während in Code-Zellen programmiert wird, können in Markdown-Zellen Texte formatiert werden, aber auch Bilder oder Animationen eingebunden werden (Markdown-Zellen unterstützen auch LaTeX-Befehle zur Darstellung mathematischer Formeln). So entsteht ein interaktives Skript, welches mehr als reiner Quellcode ist. Wir werden in der Vorlesung oft Jupyter Notebooks verwenden, da die Kombination aus Text, Bildern und Code für die Lehre sehr vorteilhaft ist. Auch dieses Skript ist in Jupyter Notebooks entstanden. Mehr darüber gibt es hier.

   Zur Begriffsklärung: *Notebook* bezeichnet einerseits das Dateiformat (`.ipynb`), andererseits das Programm *Jupyter Notebook*, mit dem man solche Dateien im Browser bearbeitet. Daneben gibt es **JupyterLab**, eine umfangreichere Oberfläche mit Dateibrowser und mehreren Tabs. Seit Version 7 basiert auch Jupyter Notebook technisch auf JupyterLab. Beide öffnen dieselben Dateien, und auch VS Code und PyCharm können `.ipynb`-Dateien direkt bearbeiten.

4. **Spyder**

   Spyder ist eine IDE speziell für wissenschaftliches Rechnen (aktuell Version 6) und kann direkt über den Anaconda Navigator gestartet bzw. installiert werden; es gibt sie auch als eigenständiges Installationsprogramm. Ihr *Variable Explorer* zeigt alle Variablen und Arrays übersichtlich als Tabelle an. Wer MATLAB kennt, fühlt sich hier schnell zu Hause. Mehr dazu hier.

5. **Cloud-Alternative: Google Colab**

   Google Colab (Colaboratory) ist eine cloudbasierte Entwicklungsumgebung. Mit Google Colab kann Python-Code direkt im Browser geschrieben und ausgeführt werden, ohne dass eine lokale Installation erforderlich ist. Im Prinzip können wir somit auch von einem Tablet oder Smartphone aus programmieren (ein kleiner Bildschirm und eine fehlende Tastatur können dennoch sehr herausfordernd sein beim Coden). Außerdem kann man GPUs und TPUs nutzen, ohne sich selbst Hardware anschaffen zu müssen. In der kostenlosen Version ist der Zugriff darauf allerdings stark eingeschränkt und nicht garantiert, und eine Sitzung läuft höchstens 12 Stunden. Für diesen Kurs brauchen wir keine GPU. In Google Colab arbeitet man ebenfalls mit der beliebten Notebook-Struktur aus Markdown- und Code-Zellen. Wenn man diese nicht braucht, kann man den gesamten Code in eine einzelne Zelle schreiben. Google Colab ermöglicht auch das gemeinsame Arbeiten an einem Projekt oder die Freigabe von Notebooks für andere. Zur Nutzung der Online-IDE ist ein (kostenfreies) Google-Konto erforderlich; beachten Sie, dass Ihre

Notebooks dabei in Ihrem Google Drive gespeichert werden. Zu Google Colab geht es hier.

Eine Alternative wäre **PythonAnywhere** (siehe Website). Dort lassen sich Python-Skripte und kleine Web-Anwendungen im Browser ausführen; GPUs gibt es dort allerdings nicht, auch nicht in den kostenpflichtigen Tarifen.

KI-Assistenten in der IDE

Moderne IDEs lassen sich um KI-Assistenten erweitern, die Code vorschlagen oder erklären. GitHub Copilot ist für verifizierte Studierende über GitHub Education kostenlos. Für die ersten Schritte empfehlen wir allerdings ausdrücklich, auf KI-Unterstützung zu verzichten: Die Grundlagen müssen Sie zunächst selbst beherrschen. Erst dann können Sie KI-Vorschläge sinnvoll bewerten, einordnen und in Ihren eigenen Code einbinden – und genau dafür sind KI-Assistenten später in der Praxis eine große Hilfe.

4. Einführung ins Programmieren

> **Lernziele**
>
> Nach diesem Kapitel können Sie …
>
> - ein erstes Programm schreiben und ausführen („Hello, World“)
> - Bildschirmausgaben mit `print()` erzeugen und Werte mit f-Strings formatiert ausgeben
> - Python als Taschenrechner nutzen und dabei die Rangfolge der Operatoren beachten
> - Variablen Werte zuweisen, erweiterte Zuweisungen wie `+=` nutzen und gute Variablennamen wählen
> - Benutzereingaben mit `input()` einlesen
> - Werte vergleichen und Vergleiche verketten
> - die Begriffe *Anweisung*, *Ausdruck* und *Literal* unterscheiden

Jetzt geht's los: Wir schreiben unser erstes Programm! Danach lernen wir die grundlegenden Konzepte der Programmierung kennen: arithmetische Operationen (das Rechnen), Zuweisungen (wie man Ergebnisse in Variablen speichert) und logische Ausdrücke (was ist wahr und was ist falsch?).

4.1. Das erste Programm: Hello, World

Wann immer man eine neue Programmiersprache lernt oder eine Programmierumgebung neu einrichtet (z.B. eine neue IDE installiert und ausprobiert), fängt man mit dem berühmten “Hello, World”-Programm an. Aufgabe des Programms ist die Ausgabe des Textes “Hello, World” auf dem Bildschirm. Ist dies gelungen, weiß man, dass man eine arbeitsfähige Umgebung hat, und kann sich komplizierteren Aufgaben zuwenden. Let's do it:

```
print('Hello, World')
```

```
Hello, World
```

In Python benutzen wir den Befehl `print` für eine Bildschirmausgabe. Er ist gleichzeitig unser erstes Beispiel für eine Funktion, das erkennen wir an den runden Klammern nach dem Befehl: Hier erwartet die Funktion ein Argument oder mehrere Argumente. Das kennen wir aus der Mathematik: Die Funktion $f(x)$ erwartet als Argument einen Wert x. Funktionen in der Programmierung unterscheiden sich allerdings in einigen Punkten von Funktionen in der Mathematik, wir werden z.B. gleich sehen, dass `print` mit beliebig vielen Argumenten aufgerufen werden kann, während $f(x)$ immer genau ein Argument erwartet.

Die Funktion `print(...)` wird mit dem auszugebenden Text aufgerufen. Texte werden in Python in einfache (`'`) oder doppelte (`"`) Anführungszeichen gesetzt. Wir hätten also auch diesen Befehl verwenden können:

```
print("Hello, World")
```

```
Hello, World
```

Der `print`-Befehl in Python ist ein grundlegender und oft verwendeter Befehl, der dazu dient, Texte oder andere Daten auf dem Bildschirm anzuzeigen. Das ist sehr nützlich, um Informationen während der Ausführung eines Programms zu überprüfen oder Ergebnisse von Berechnungen anzuzeigen.

Wie bereits erwähnt, kann man auch mehr als ein Argument übergeben (getrennt durch ein Komma):

```
print("Hello,", "World")
```

```
Hello, World
```

Das Ergebnis ist dasselbe, da die einzelnen Argumente standardmäßig durch ein Leerzeichen getrennt werden. Dieses Verhalten kann durch weitere Argumente geändert werden. Probieren Sie das Folgende einmal aus:

```
print("It is", "a beautiful", "day.", sep = "-")
```

Nun haben Sie schon ein sog. „Keyword-Argument“ kennengelernt: `sep`. Ein Keyword-Argument erkennen Sie daran, dass es explizit einem Namen zugewiesen wird, in unserem Beispiel durch `sep=...`. (Das Gegenstück dazu sind die sog. „Positionsargumente“: alle Argumente, die ohne Zuweisung übergeben werden. Ohne das Keyword `sep` könnte die `print`-Funktion das Argument nicht von den vorherigen unterscheiden. Auf den Unterschied zwischen Positions- und Keyword-Argumenten werden wir natürlich später noch ausführlicher eingehen. Wir sind hier noch bei unseren ersten Gehversuchen in der Welt der Programmierung.)

Übungen:

1. Experimentieren Sie mit dem `print`-Befehl. Können Sie außer Text auch andere Argumente verwenden?
2. Der `print`-Befehl kennt noch ein weiteres Keyword-Argument: `end = ...`. Finden Sie durch Experimentieren heraus, was dieses Argument bewirkt. Sie können anschließend auch im Netz recherchieren, ob Sie es richtig verstanden haben.
3. Recherchieren Sie, wie das "Hello, World"-Programm in anderen Programmiersprachen aussieht. Suchen Sie mindestens nach C, C++, Java und einer weiteren Programmiersprache Ihrer Wahl. Wo erkennen Sie Ähnlichkeiten zu unserem Beispiel in Python und wo sieht es ganz anders aus?

4.2. Python als Taschenrechner

Programme können mächtige Berechnungen und Analysen durchführen. Die Grundlage aller Berechnungen sind die Basisoperationen, die wir alle in der Schule gelernt haben. Anhand der folgenden Beispiele sehen Sie, wie wir mit Python rechnen können (es wechseln sich zeilenweise Ein- und Ausgabe ab; hinter `#` steht ein Kommentar, der nur für Menschen gedacht ist und das Programm nicht beeinflusst):

```
# Addition
3 + 2
```

```
5
```

```
# Subtraktion
6 - 2
```

```
4
```

```
# so funktioniert es auch:
-2 + 6
```

```
4
```

```
# Multiplikation
3 * 2
```

```
6
```

```
# Division
3 / 2
```

```
1.5
```

```
# Ganzzahl-Division
# (rundet auf die nächstkleinere ganze Zahl ab:
# 3 // 2 ergibt 1, aber -3 // 2 ergibt -2)
3 // 2
```

```
1
```

```
# Zweites Beispiel für
# die Ganzzahl-Division
9 // 2
```

```
4
```

```
# Modulo-Division: Division mit Rest
# Gibt den Rest zurück.
9 % 2
```

```
1
```

```
# Zweites Beispiel für Modulo
31 % 7
```

```
3
```

```
# Potenz
3 ** 2  # 3 hoch 2
```

```
9
```

Achtung bei negativen Zahlen: Die Potenz bindet stärker als das Vorzeichen. `-2**2` wird deshalb als $-(2^2)$ gelesen und ergibt `-4`. Wer $(-2)^2 = 4$ meint, muss Klammern setzen:

```
print(-2**2)    # = -(2**2)
print((-2)**2)  # = (-2)*(-2)
```

```
-4
4
```

```
# Klammern
3 * (2 + 4)
# Ohne Klammern hält sich
# Python an Punkt vor Strich!
```

```
18
```

Selbstverständlich können wir nicht nur mit ganzen Zahlen rechnen, sondern auch mit Kommazahlen, den sog. Gleitkommazahlen (engl. *floating-point numbers*, kurz *floats*). Wichtig: Diese verwenden **immer einen Punkt als Dezimalzeichen**! Ein Komma bedeutet in Python etwas ganz anderes: Es trennt Werte voneinander. `2,8 + 3,2` führt deshalb nicht zu einer Fehlermeldung. Python bildet daraus die drei Werte `2`, `8 + 3` und `2` (das `+` wird vor dem Komma ausgewertet) und gibt `(2, 11, 2)` aus – ein sog. Tupel, das wir später kennenlernen. Dieser Fehler fällt also nicht sofort auf!

```
# Gleitkommazahlen
2.8 + 3.2
```

```
6.0
```

Möglich ist auch die sog. Exponentialschreibweise:

```
# Exponentialschreibweise
# e4 bedeutet "mal 10 hoch 4":
# Komma um 4 Stellen nach rechts verschieben
2e4
```

```
20000.0
```

```
# e-4 bedeutet "mal 10 hoch -4":
# Komma um 4 Stellen nach links verschieben
2e-4
```

```
0.0002
```

Die Exponentialschreibweise ist besonders praktisch, um sehr große oder sehr kleine Zahlen kompakt darzustellen (ohne viele Nullen). Sie wird auch *wissenschaftliche Notation* genannt, da sie in wissenschaftlichen Kontexten häufig verwendet wird. Es folgen noch ein paar Beispiele:

```
5.34e5
```

```
534000.0
```

```
1.267e-4
```

```
0.0001267
```

```
8.2e9  # Erdbevölkerung 2025 (ca.)
```

```
8200000000.0
```

Übungen:

1. Machen Sie sich mit allen Operationen vertraut. Achten Sie auf die korrekte Syntax, z.B. `*` für die Multiplikation anstelle eines Punktes.
2. Führen Sie die folgenden Berechnungen in Python aus:

a) $2 \cdot (3 + 4)^2$
b) $\frac{7}{8} \cdot (1,2 - 5,3)$
c) $-3 \cdot (\frac{4}{3} - 2,1)^3$

Lösungen der Rechenaufgaben:

```
2 * (3 + 4)**2
```

```
98
```

```
7 / 8 * (1.2 - 5.3)
```

```
-3.5874999999999995
```

```
-3 * (4 / 3 - 2.1)**3
```

```
1.3518888888888898
```

4.3. Zuweisungen

In der Programmierung sind Zuweisungen ein grundlegendes Konzept, um Werte in Variablen zu speichern. Eine Zuweisung erfolgt in Python mit dem Gleichheitszeichen (=). Schauen wir uns Beispiele an:

```
# Zuweisung:
# Die Variable x erhält den Wert 5
x = 5
```

```
print(x)  # Überzeugen wir uns davon!
```

```
5
```

```
# Die Variable name erhält
# den Wert "Tina"
name = "Tina"
```

```
# Die Variable y erhält
# den Wert 1.3
y = 1.3
```

Mit Variablen können wir rechnen:

```
# Die Variable z erhält
# den Wert der Addition x + y
z = x + y
```

```
print(z)  # Überzeugen wir uns davon!
```

```
6.3
```

> **Vorsicht**
>
> Das Gleichheitszeichen ist nicht mit dem mathematischen Gleichheitszeichen zu verwechseln: In der Mathematik sind beide Seiten einer Gleichung gleichberechtigt und die Gleichung $x = 5$ ist entweder wahr oder falsch. In der Programmierung führen wir aber eine Operation durch, nämlich eine Zuweisung: Der Wert auf der rechten Seite wird der Variable auf der linken Seite zugewiesen!

Machen Sie sich an dem folgenden Beispiel klar, dass `x = y` und `y = x` nicht dasselbe sind in der Programmierung!

```
# Beispiel:
x = 5
y = 7
x = y # was würde y = x am Ergebnis ändern?
print(x, y)
```

```
7 7
```

Die bisherigen Beispiele waren sog. Einfach-Zuweisungen: Einer Variable wurde ein Wert zugewiesen. In Python kann man aber auch mehreren Variablen gleichzeitig verschiedene Werte zuweisen, indem man links mehrere Namen und rechts mehrere Werte angibt, jeweils durch Kommas getrennt (diese Schreibweise gibt es nicht in allen Programmiersprachen, in C zum Beispiel nicht):

```
# Zwei Variablen
# gleichzeitig zuweisen:
name, age = "Tina", 25
```

```
print("Ich heiße", name, "und bin", age, "Jahre alt.")
```

```
Ich heiße Tina und bin 25 Jahre alt.
```

Einschub: Die letzte Ausgabe geht deutlich leichter, wie das folgende Beispiel zeigt.

```
# Achten Sie auf das kleine f und die geschweiften
# Klammern:
print(f"Ich heiße {name} und bin {age} Jahre alt.")
# Hier verwenden wir einen sog. f-String.
```

```
# Das f steht für formatiert bzw. formatted.
# Es sagt Python, dass die Variablen in den geschweiften
# Klammern durch ihre Werte ersetzt werden sollen.
```

```
Ich heiße Tina und bin 25 Jahre alt.
```

Bei Zahlen lässt sich in f-Strings zusätzlich die **Darstellung** festlegen. Dazu schreibt man hinter den Variablennamen einen Doppelpunkt und eine Formatangabe, z.B. `.2f` für zwei Nachkommastellen oder `.1e` für die Exponentialschreibweise mit einer Nachkommastelle. Das ist besonders praktisch für Messwerte:

```
voltage = 3.14159  # Spannung in V
current = 0.0123   # Strom in A

print(f"Spannung: {voltage:.2f} V")          # zwei Nachkommastellen
print(f"Strom: {current:.1e} A")             # Exponentialschreibweise
print(f"Strom: {current * 1000:.1f} mA")     # auch Rechnungen sind möglich
```

```
Spannung: 3.14 V
Strom: 1.2e-02 A
Strom: 12.3 mA
```

Beachten Sie: Die Formatangabe ändert nur die *Darstellung* in der Ausgabe – der Wert der Variable bleibt unverändert.

Selbstverständlich kann sich der Wert einer Variable nach der ersten Zuweisung noch verändern, indem ein neuer Wert zugewiesen wird. Das demonstrieren wir anhand verschiedener Beispiele:

- **Beispiel mit einer einzelnen Variable.**

```
x = 20  # x hat den Wert 20
x = 21  # x hat den Wert 21
x = 22  # x hat den Wert 22
print(x)  # Überzeugen wir uns!
```

```
22
```

- **Beispiel mit zwei Variablen.**

```
x = 1.2  # x hat den Wert 1.2
y = 42  # y hat den Wert 42
x = y  # x bekommt den Wert von y, also 42
y = 0  # y bekommt den Wert 0, x bleibt unverändert
print(x, y)
```

```
42 0
```

- **Beispiel, welches den Unterschied zur mathematischen Gleichung noch einmal deutlich hervorhebt.**

```
x = 2
x = x + 2 # Das Ergebnis von x+2 wird der Variable x zugewiesen.
print(x)
```

```
4
```

> **Erweiterte Zuweisungen**
>
> Mit `+=`, `-=`, `*=`, `/=` können Zuweisungen wie im letzten Beispiel verkürzt werden. Dabei steht `x += 5` für `x = x + 5` (die anderen analog). Es wird also eine mathematische Operation mit der Variable durchgeführt (z.B. "addiere 5") und das Ergebnis derselben Variablen als neuer Wert zugewiesen. Es handelt sich lediglich um eine kürzere Schreibweise. Für Zahlen und Strings (Zeichenketten) ist das genau gleichwertig. Bei Listen gibt es einen kleinen, aber wichtigen Unterschied, den wir im Kapitel *Datenstrukturen* kennenlernen.

- **Beispiel mit erweiterten Zuweisungen.**

```
x, y, z = 1, 2, 3
x += 10
y -= 10
z *= 10
print(x, y, z)
```

```
11 -8 30
```

Wichtig: Python-Code wird vom Interpreter der Reihe nach abgearbeitet. Wenn Sie eine Variable z.B. erst nach einem Aufruf von `print` verändern, so bekommt der `print`-Befehl das nicht mit. Beispiel:

```
x = 5
print(x)
x += 10  # für das print darüber unerheblich!
print(x) # jetzt sieht man die Veränderung!
```

```
5
15
```

Einschub: Benutzereingaben

In Python wird die Funktion `input()` verwendet, um Benutzereingaben zu erfassen. Sie wartet, bis der Benutzer eine Eingabe macht und mit Enter bestätigt. Der eingegebene Wert wird immer als Zeichenkette (also Text) zurückgegeben, kann aber bei Bedarf in andere Datentypen umgewandelt werden (mehr dazu im nächsten Kapitel). Die Eingabe des Benutzers wird häufig einer Variable zugewiesen, damit sie weiterverarbeitet werden kann. Als Argument können Sie der Funktion eine Aufforderung oder Frage übergeben (wie man stattdessen grafische Eingabemasken baut, zeigt Anhang C):

```
name = input("Wie heißen Sie? ")  # fragt nach dem Namen; speichert die
 ↪  Eingabe in name
city = input("Wo wohnen Sie? ")  # fragt nach dem Wohnort; speichert die
 ↪  Eingabe in city
age = input("Bitte Alter eingeben: ")  # Variante mit Aufforderung statt
 ↪  Frage

print(f"Sie heißen {name}, sind {age} Jahre alt und wohnen in {city}.")
```

```
Wie heißen Sie?  John
Wo wohnen Sie?  New York
Bitte Alter eingeben:  40

Sie heißen John, sind 40 Jahre alt und wohnen in New York.
```

> **Gute Variablennamen**
>
> Obwohl Python im Vergleich zu anderen Programmiersprachen mehr Freiheiten bei der Benennung von Variablen zulässt (z.B. die Verwendung von Umlauten und anderen speziellen Buchstaben), sollte man sich auf das lateinische Alphabet beschränken. Der offizielle Python-Style-Guide (PEP 8) empfiehlt mit einem kleinen Buchstaben anzufangen und

mehrere Wörter mit einem Unterstrich (_) zu trennen (das nennt man *snake_case*). Es sind auch Ziffern erlaubt (allerdings nicht am Anfang). Wichtig ist eine gute Verständlichkeit, damit auch andere Menschen Ihren Code nachvollziehen können. Daher empfehlen sich kurze, aber aussagekräftige Namen. Es folgen ein paar Beispiele:

```
customer_name = "Markus"
user_age = 32
total_price = 3.4e3
coordinate_1, coordinate_2, coordinate_3 = 3.1, -2.0, 0.0
x_coordinate, y_coordinate, z_coordinate = -1.0, -1.0, 1.0
```

Im Style-Guide wird außerdem davor gewarnt, dass die Buchstaben 'l' (kleines L), 'O' (großes o) oder 'I' (großes i) leicht mit den Zahlen '1' bzw. '0' verwechselt werden können (in manchen Schriftarten werden sie identisch dargestellt!). Immer darauf achten, ob es aus dem Kontext eindeutig zu erkennen ist.

Viele Organisationen oder Teams definieren übrigens eigene Style-Guides, die weitere Details festlegen. In internationalen Projekten legt man z.B. häufig fest, dass die englische Sprache im Code zu verwenden ist. Andere Unternehmen verwenden deutsche Wörter. Wenn Sie Teil eines Teams werden, fragen Sie nach den internen Konventionen (explizit niedergeschrieben oder implizit von allen befolgt).

Abschließend sei an dieser Stelle darauf hingewiesen, dass Sie keine Schlüsselwörter von Python für Ihre Variablennamen verwenden dürfen, z.B. `if`, `else`, `class`, `return` usw. (Python meldet dann einen `SyntaxError`). Namen eingebauter Funktionen wie `print` oder `sum` sind dagegen erlaubt – aber gefährlich (siehe folgenden Kasten).

⚠ Fallstrick: Namen eingebauter Funktionen als Variablennamen

Python bringt viele eingebaute Funktionen mit, z.B. `print()` oder `sum()` (addiert alle Zahlen einer Folge, mehr dazu im Kapitel *Funktionen*). Python verbietet nicht, diese Namen für eigene Variablen zu verwenden – mit unangenehmen Folgen:

```
sum = 3 + 4             # die Variable heißt wie die eingebaute Funktion
 ↪  sum()
print(sum([1, 2, 3]))   # TypeError: 'int' object is not callable
```

Ab der Zuweisung zeigt `sum` nicht mehr auf die Funktion, sondern auf die Zahl 7 – und eine Zahl kann man nicht aufrufen. Der Fehler tritt oft erst viel später im Code auf und ist dann schwer zu finden. Wählen Sie deshalb einen anderen Namen, z.B. `total`. Ist es schon passiert, stellt `del sum` in Jupyter die eingebaute Funktion wieder her (oder Sie starten den Kernel – das Python im Hintergrund des Notebooks – neu).

Übungen:

1. Betrachten Sie das folgende Code-Beispiel und überlegen Sie sich, welche Werte am

Schluss ausgegeben werden. Überprüfen Sie Ihr Ergebnis anschließend, indem Sie den Code implementieren:

```
x = 5
x += 10
y = 2 * x + 10
y -= 15
x /= 2
print(x, y)
```

2. Betrachten Sie die folgenden Variablennamen und schlagen Sie bessere vor:

```
X = 10           # Anzahl der Äpfel im Korb
CLIENTnum = 1440 # Kundennummer
Zähler = 0       # zählt etwas
txt = "Hallo"    # Begrüßungstext
isLoggedIn = True  # Ob ein Benutzer angemeldet ist
lO = 10.0       # Koordinate links oben
```

4.4. Vergleiche

In Programmen wird viel gerechnet, wofür die arithmetischen Operationen, die wir bisher betrachtet haben, die Grundlage bilden. Ein weiteres grundlegendes Element von Programmen ist die **Logik**: Dabei geht es um die Entscheidung, ob etwas wahr oder falsch ist. Je nach Ergebnis kann ein Programm unterschiedliche Wege einschlagen: Ist eine Bedingung wahr, wird eine bestimmte Aktion ausgeführt. Ist sie falsch, passiert etwas anderes.

Schauen wir uns nun die grundlegenden logischen Ausdrücke an, mit denen wir solche Entscheidungen treffen können. Vergleichen wir zunächst zwei Zahlen:

```
1 < 42  # Ist 1 kleiner als 42?
```

```
True
```

```
1 > 42  # Ist 1 größer als 42?
```

```
False
```

```
2 <= 3  # Ist 2 kleiner oder gleich 3?
```

```
True
```

```
2 >= 3  # Ist 2 größer oder gleich 3?
```

```
False
```

```
2 == 3  # Ist 2 gleich 3?
```

```
False
```

```
2 != 3  # Ist 2 ungleich 3?
```

```
True
```

> **Vorsicht**
>
> Für den logischen Vergleich verwendet man das doppelte Gleichheitszeichen `==`, welches nicht mit dem Zuweisungsoperator `=` verwechselt werden darf!

Wie Sie festgestellt haben, liefern logische Vergleiche als Ergebnis einen Wahrheitswert: `True` für wahr oder `False` für falsch. Wahrheitswerte können genauso wie andere Werte einer Variable zugewiesen werden:

```
x = 3
y = 0
comparison = x > y  # Zuweisung
print(comparison)
```

```
True
```

Die Variable `comparison` speichert das Ergebnis des Vergleichs und kann an anderer Stelle im Programm verwendet werden. Es folgt ein Beispiel, welches auf die `if-else`-Verzweigung vorgreift (Sie können es sicher trotzdem schon verstehen):

```
if comparison:
    print(x, 'ist größer als', y)
else:
    print(x, 'ist kleiner oder gleich', y)
```

```
3 ist größer als 0
```

```
# Zur Erinnerung: Wir legen unsere
# Variablennamen selbst fest. Hier zur Demo dasselbe
# Beispiel erneut mit einem Quatsch-Namen:
x = 3
y = 0
dagobert_duck = x > y  # Zuweisung
print(dagobert_duck)
# Wir könnten jetzt mit dagobert_duck
# alles tun, was wir zuvor mit comparison
# getan haben.
```

```
True
```

Wahrheitswerte tauchen nicht nur bei Vergleichen von Zahlen auf, sondern auch in vielen anderen Situationen, wie zum Beispiel bei der Suche nach einem Buchstaben in einem Wort. Das würde in Python so aussehen:

```
# Suche nach einem 'a' in 'apple':
'a' in 'apple'
```

```
True
```

```
# Suche nach einem 'a' in 'lemon'
'a' in 'lemon'
```

```
False
```

```
# Achtung: Kleinbuchstaben != Großbuchstaben
'a' in 'YOGA'
```

```
False
```

Mit der Negation `not` kehren wir einen Wahrheitswert in sein Gegenteil um, wie die folgenden Beispiele illustrieren:

```
not (4 < 3)
```

```
True
```

```
not ('a' in 'apple')  # idiomatischer und gleichwertig: 'a' not in 'apple'
```

```
False
```

Einfache logische Ausdrücke können durch die Verknüpfungen `or` und `and` zu komplexeren Bedingungen kombiniert werden. Wenn zwei Bedingungen mit `and` verknüpft werden, ist die kombinierte Bedingung genau dann wahr, wenn **beide** Einzelbedingungen wahr sind:

```
5 > 4 and 4 > 3
```

```
True
```

```
5 > 4 and 4 > 6
```

```
False
```

Verknüpft man hingegen mit `or`, so genügt es, dass eine der beiden Einzelbedingungen wahr ist:

```
5 > 4 or 4 > 6 # mindestens eine Bedingung wahr.
```

```
True
```

```
5 > 4 or 4 > 3 # beide wahr
```

```
True
```

Probieren Sie eigene Beispiele mit `and`, `or` und `not` aus!

Wahrheitswerte in anderen Programmiersprachen

In vielen Programmiersprachen werden Wahrheitswerte als True (wahr) und False (falsch) dargestellt. Die Schreibweise variiert aber je nach Sprache:

- In Python: `True`, `False`
- In C++ und PHP: `true`, `false`
- In SQL: `TRUE`, `FALSE` (oder kleingeschrieben)

Intern werden Wahrheitswerte oft wie Zahlen behandelt: True entspricht dann einer 1 und False einer 0. Probieren Sie in Python Vergleiche der folgenden Art aus: `True == 1`, `False == 0`. Kleiner Vorgriff: An vielen Stellen (siehe Verzweigungen, Schleifen) werden in Python alle Zahlen ungleich 0 als True interpretiert.

Übungen:

1. Definieren Sie eine Variable `age` und weisen ihr das Alter einer Person zu. Schreiben Sie einen Vergleich, der `True` zurückgibt, wenn die Person erwachsen ist. Schreiben Sie einen weiteren Vergleich, der überprüft, ob die Person exakt 25 Jahre alt ist. Implementieren Sie einen dritten Vergleich, der überprüft, ob die Person jünger als 67 ist.
2. Recherchieren Sie, wie der BMI ('Body Mass Index') definiert ist. Legen Sie Variablen für die benötigten Parameter einer Person an und berechnen Sie den BMI. Definieren Sie anschließend eine Variable, die `True` ist, wenn der BMI ein Normalgewicht anzeigt. Tipp: Verwenden Sie `and`, um zwei Bedingungen miteinander zu verknüpfen. (Lösung siehe unten.)

```
length = 1.80  # Größe in m
weight = 75  # Gewicht in kg
bmi = weight / length**2
is_normal_weight = bmi >= 18.5 and bmi < 25  # Normalgewicht laut WHO: 18,5
 ↪  bis unter 25

# Hinweis: Wir verwenden in der Ausgabe
# round(), um das Ergebnis auf eine Stelle zu runden
print('BMI:',round(bmi,1),'Normalgewicht?', is_normal_weight)
```

Comparison Chains in Python

Es gibt in Python eine Möglichkeit, mehrere Vergleiche miteinander zu verketten, wie man es z.B. aus der Mathematik kennt. Die Vergleiche in der Kette werden einzeln ausgewertet

und mit `and` verknüpft:

Comparison Chain	Bedeutung
`18.5 <= x < 25`	`18.5 <= x and x < 25`
`x < y > z`	`x < y and y > z`
`x == y == z`	`x == y and y == z`
`x != y != z`	`x != y and y != z`

Während das erste Beispiel genauso aussieht, wie man es aus der Mathematik kennt, sieht das zweite Beispiel vielleicht etwas seltsam aus. Die Vergleiche werden aber einfach der Reihe nach abgearbeitet, unabhängig davon, welche Vergleichsoperatoren verwendet werden. Das dritte Beispiel prüft, ob alle Variablen gleich sind. Trügerisch ist das vierte Beispiel `x != y != z`, da es **nicht** feststellt, ob alle Variablen ungleich sind (`x` und `z` werden nicht miteinander verglichen!).
Tipp: Setzen Sie Comparison Chains mit Bedacht ein. Am intuitivsten sind sie, wenn die Ausdrücke so aussehen, wie man es aus der Mathematik kennt (vgl. erste Zeile in der Tabelle).

4.5. Zum Abschluss des Kapitels: ein wenig Fachsprache

Wir können bereits eine Menge tun in Python. An dieser Stelle wollen wir kurz auf das bisher Gelernte zurückschauen und dabei wichtige Fachtermini einführen, die Ihnen immer wieder begegnen werden. Es handelt sich um die Begriffe "Anweisung", "Ausdruck" und "Literal".

Anweisung (engl. statement): Ein vollständiger Befehl, der ausgeführt wird – meist eine Zeile.

Ausdruck (engl. expression): Code, der ausgewertet wird und einen Wert liefert, z.B. `5` (liefert 5), `x` (liefert den Wert von x), `x + 5` (liefert den Wert von x + 5), `x > y` (liefert True oder False) oder `len("Apfel")` (liefert 5).

Wir haben bereits verschiedene Arten von Ausdrücken kennengelernt: **arithmetische** und **logische Ausdrücke**. Arithmetische Ausdrücke rechnen mit Zahlen und liefern eine Zahl als Ergebnis. Logische Ausdrücke (auch *boolesche Ausdrücke* genannt) liefern einen Wahrheitswert und arbeiten typischerweise mit Vergleichen sowie `and`, `or` und `not`. Zuweisungen sind selbst keine Ausdrücke - sondern Anweisungen - aber sie erwarten auf ihrer rechten Seite einen Ausdruck.

```
5 + 3  # Ein arithmetischer Ausdruck. Wert: 8
x = 5 + 3 # Eine Zuweisung ist eine Anweisung; die rechte Seite ist ein
 ↪  Ausdruck.
3 * x + 4  # Ein arithmetischer Ausdruck. Wert: 28
y = 3 * x + 4 # Eine Zuweisung ist eine Anweisung; die rechte Seite ist ein
 ↪  Ausdruck.
```

In manchen Programmiersprachen sind dagegen auch Zuweisungen Ausdrücke: In C weist `x = 3` der Variablen `x` den Wert 3 zu und liefert gleichzeitig den Wert 3. Deshalb ist es in C nicht überraschend, dass `y = x = 3` funktioniert: Zuerst wird `x = 3` ausgeführt, und der gelieferte Wert 3 wird anschließend `y` zugewiesen.

In Python funktioniert `y = x = 3` ebenfalls, wird intern aber anders verarbeitet: Der Interpreter erkennt eine Zuweisung mit zwei Zielen, wertet den Ausdruck `3` einmal aus und weist ihn von links nach rechts zu, also zuerst `y = 3` und dann `x = 3`.

```
4 > 5  # Ein logischer Ausdruck. Wert: False.
1 == 1 and 5 > 4 # Ein logischer Ausdruck. Wert: True.
comparison = 99 > 42  # Eine Zuweisung ist eine Anweisung; die rechte Seite
 ↪  ist ein Ausdruck.
```

Weiterhin stellen Funktionsaufrufe einen Ausdruck dar, wie zum Beispiel:

```
len('Apfel')  # len() liefert die Länge, hier die Anzahl der Zeichen: 5
x = len('Apfel') # der Ausdruck wird als rechte Seite einer Zuweisung
 ↪  verwendet
```

Auch Variablen selbst können als Ausdruck betrachtet werden, denn sie geben ihren Wert zurück:

```
z = 100
z  # Ein Ausdruck; liefert den Wert von z, also 100.
y = z # Eine Zuweisung mit dem Ausdruck z auf der rechten Seite.
```

Literal (vom lateinischen Wort für Buchstabe): Ein Literal ist ein fester, unveränderlicher Wert (wie eine Zahl, eine Zeichenkette oder ein Wahrheitswert), der direkt im Code verwendet wird. Da Literale ausgewertet werden können, um einen Wert zu liefern, gelten sie ebenfalls als Ausdrücke.

Es folgen einige Beispiele (die Datentypen erklären wir später):

```
"Hello, World"  # String-Literal (Zeichenkette)
4.2  # Float-Literal (Gleitkommazahl)
42  # Integer-Literal (Ganzzahl)
True  # Boolean-Literal (Wahrheitswert)

# Alle diese Literale sind gleichzeitig Ausdrücke:
# Sie geben sich selbst als Wert zurück.
```

Warum sollte man diese Begriffe kennen?
Wenn Sie mit anderen Programmiererinnen und Programmierern sprechen, fallen diese Begriffe. Zum Beispiel, wenn man sich über verschiedene Programmiersprachen unterhält (“Wie sieht ein String-Literal in deiner Sprache aus?”, “Welche logischen Ausdrücke kennt deine Sprache?”). Für Sie sind das jetzt keine Fremdwörter mehr!

5. Datentypen und ihre Repräsentation

> **Lernziele**
>
> Nach diesem Kapitel können Sie …
>
> - die grundlegenden Datentypen unterscheiden und den Typ eines Werts bestimmen
> - Werte gezielt umwandeln (Casting) und typische Fallstricke dabei erkennen
> - Listen anlegen und mit ihren wichtigsten Methoden bearbeiten
> - erklären, was Instanzen und Referenzen sind, und Seiteneffekte erkennen
> - mutable und immutable Datentypen unterscheiden
> - erklären, wie Zahlen und Zeichen binär dargestellt werden und warum `0.1 + 0.2` nicht exakt `0.3` ergibt

Beim Programmieren arbeiten wir ständig mit Daten – Zahlen, Texte, Wahrheitswerte und mehr. Wie aus dieser Aufzählung hervorgeht, gibt es unterschiedliche **Datentypen**, und jeder Typ hat einen Einfluss darauf, wie Daten dargestellt, verarbeitet und gespeichert werden. In diesem Kapitel lernen wir die wichtigsten Datentypen kennen und werfen einen genaueren Blick darauf, wie diese Daten gespeichert und verwaltet werden.

Außerdem zeigen wir, wie Datentypen ineinander umgewandelt werden (Casting), und lernen mit der Liste einen ersten Datentyp kennen, der mehrere Werte aufnehmen kann.

Im Speicher des Computers liegen Daten als sog. **Instanzen**. Wie wir sehen werden, fungieren Variablen als **Referenzen** auf diese Instanzen (man könnte sagen: *Wegweiser zu den Instanzen*).

Zuletzt wollen wir noch verstehen, wie Instanzen von unterschiedlichen Datentypen eigentlich für den Computer aussehen, denn dieser versteht nur eine Sprache: das Binärsystem! Für jeden Datentyp existieren daher Kodierungen, die die jeweilige Information (ob Zahl oder Text) als Binärzahl ausdrücken.

5.1. Grundlegende Datentypen

Mit der Funktion `type()` können wir in Python den Datentyp einer Variablen oder eines Literals abfragen:

```
type(4)  # Typ einer ganzen Zahl
```

```
int
```

```
type(3.2)  # Typ einer Gleitkommazahl
```

```
float
```

```
type("Hallo")  # Typ eines Textes
```

```
str
```

```
type(True)  # Typ eines Wahrheitswertes
```

```
bool
```

```
# Beispiel mit einer Variable
x = 5.4
type(x)
```

```
float
```

Die folgende Tabelle fasst die gefundenen Datentypen zusammen:

Datentyp	Bedeutung
`int` (Integer)	ganze Zahlen
`float` (Floating-point)	Gleitkommazahlen
`str` (String)	Zeichenketten (Text)
`bool` (Boolean)	Wahrheitswert

Was sind Gleitkommazahlen?

Zahlen mit Nachkommastellen können sehr viele Stellen haben. Um sie effizient zu speichern, nutzt der Rechner intern eine Art wissenschaftliche Schreibweise. Im Dezimalsystem kennen wir sie z.B. als `2.1e-4` statt `0.00021`; der Rechner verwendet dasselbe Prinzip,

allerdings zur Basis 2 (Details folgen weiter unten). Dadurch werden führende oder nachfolgende Nullen vermieden. Statt der vollständigen Zahl speichert man nur die signifikanten Stellen (hier: `2.1`, genannt *Mantisse*) und den Exponenten (hier: `-4`) sowie das Vorzeichen der Zahl (hier: positiv).
Das wird besonders bei extrem kleinen oder großen Zahlen deutlich: `3.23e10` statt `32300000000` oder `1.2e-9` statt `0.0000000012`. Der Dezimalpunkt „gleitet" dabei entsprechend dem Exponenten, woher der Name *Gleitkommazahl (floating-point)* stammt.
Im Gegensatz dazu gibt es Festkommazahlen, bei denen der Dezimalpunkt immer an einer festen Stelle steht (z.B. Währungsbeträge wie `12.34` Euro). Diese sind in der Informatik seltener, weil sie eine feste Genauigkeit haben und weniger flexibel sind. Sie werden nur in speziellen Anwendungen genutzt.

Die genannten Datentypen gibt es auch in vielen anderen Programmiersprachen, oft mit leicht unterschiedlichen Bezeichnungen oder Eigenschaften:

- **C** arbeitet u.a. mit `int`, `float`, `double` (Gleitkommazahl mit doppelter Genauigkeit), `char` (für einzelne Zeichen; Strings werden als Zeichenarrays gespeichert, sog. *C-Strings*); `bool` kann seit C99 über einen Header eingebunden werden.
- **C++** führt u.a. eine String-Klasse ein (`std::string`) (das Wort Klasse erklären wir in der objektorientierten Programmierung).
- **Java** kennt primitive Datentypen wie `int`, `double`, `char` und `boolean` sowie die Klasse `String` für Texte.
- **JavaScript** nutzt `number` für Ganz- und Gleitkommazahlen, `boolean` für Wahrheitswerte und `string` für Texte.

Die vorgestellten Datentypen sind die Grundbausteine einer Programmiersprache, aus denen komplexere Datenstrukturen zusammengesetzt werden können. Man nennt sie daher oft auch **primitive Datentypen**. In Python sind streng genommen auch die Werte dieser einfachen Datentypen vollwertige Instanzen von Klassen (siehe Kapitel *Objektorientierte Programmierung*); man spricht deshalb in Python eher von *eingebauten* Datentypen.

Ein besonderer Datentyp, der in vielen Programmiersprachen nicht eingebaut ist, ist **`complex`** für die Darstellung komplexer Zahlen. Die imaginäre Einheit wird dabei – wie in der Elektrotechnik üblich – mit `j` geschrieben, und zwar direkt hinter einer Zahl, z.B. `2j` oder `1j` (ein `j` allein wäre ein Variablenname):

```
type(1 + 2j)  # komplexe Zahl mit Realteil 1 und Imaginärteil 2
```

```
complex
```

```
print("Das Quadrat von j ist:", (1j)**2)
```

```
Das Quadrat von j ist: (-1+0j)
```

Die letzte Ausgabe sieht auf den ersten Blick vielleicht etwas kryptisch aus, aber belegt, dass `j` die Wurzel aus `-1` ist. Es ist nun besonders einfach, mit komplexen Zahlen zu rechnen:

```
(1+2j)*(-2+4j)  # rechnen mit komplexen Zahlen
```

```
(-10+0j)
```

Statisch oder dynamisch typisiert?

Programmiersprachen unterscheiden sich darin, wie flexibel sie mit Datentypen umgehen. Man kann grundsätzlich zwischen zwei Arten unterscheiden:

Statisch typisierte Sprachen: Datentypen werden **vor der Laufzeit** durch den Programmierer oder die Programmiererin definiert (z.B. in C, Java). Typfehler (man versucht im Code etwas mit einem Datentyp zu machen, was nicht erlaubt ist) werden beim Kompilieren erkannt, was Sicherheit und Geschwindigkeit bietet, aber weniger flexibel ist.

Beispiel-Code in C: `int zahl = 42;` Bei der Deklaration der Variable `zahl` muss bereits ihr Typ angegeben werden, hier: Integer. Sie darf im gesamten Programm niemals für einen anderen Typ verwendet werden (in vielen Fällen würde die Zuweisung trotzdem gelingen, aber durch ein implizites Casting (siehe unten) ein evtl. unbeabsichtigter Datenverlust stattfinden).

Dynamisch typisierte Sprachen: Typen werden automatisch **zur Laufzeit** bestimmt (z.B. in Python, JavaScript). Das schenkt eine höhere Flexibilität: Wir können an einer Stelle `zahl = 42` und an einer anderen Stelle `zahl = "Apfel"` schreiben. Allerdings können sich so auch leichter Fehler einschleichen. Einen exemplarischen Typfehler in Python zeigt der folgende Code:

```
x = "42"
y = "2"
z = x / y # diese Zeile führt zu einem Fehler!
# Strings können nicht durcheinander dividiert werden.
# Oder könnten Sie "Auto" durch "Apfel" teilen?!
```

5.2. Casting (Typumwandlung)

Casting oder Typumwandlung bezeichnet das Konvertieren eines Wertes von einem Datentyp in einen anderen. Dies ist sowohl in statisch als auch dynamisch typisierten Sprachen nützlich, um unterschiedliche Datentypen zu kombinieren oder explizite Anforderungen zu erfüllen. Es gibt dabei implizites Casting durch die Programmiersprache und explizites Casting durch den Programmierer oder die Programmiererin.

Implizites Casting

Die Programmiersprache übernimmt die Typumwandlung automatisch bei kompatiblen Typen (z.B. Integer zu Float). Im folgenden Beispiel wird eine Integer-Zahl zu einem Float gecastet, um eine Summe zu bilden (das Ergebnis ist wieder ein Float):

```
int_number = 4
float_number = 3.2
result = int_number + float_number
type(result)
```

```
float
```

Ein zweites Beispiel demonstriert das implizite Casting von Wahrheitswerten zu Integern:

```
number = 11
boolean = True
result = number + boolean  # implizites Casting von bool zu int
print("Result:", result)
```

```
Result: 12
```

> **Vorsicht**
>
> Unbeabsichtigtes implizites Casting kann zu unerwarteten Fehlern führen!
> In C kompiliert beispielsweise folgender Code:
>
> ```
> float x = 3.99;
> int y = x;
> ```
>
> Allerdings ist der Wert von `y` nicht `3.99` und auch nicht `4`, sondern `3`. Beim impliziten Casting von float nach int gehen alle Nachkommastellen verloren!

Hinweis: Python ist beim impliziten Casting strenger als z.B. C: Automatisch umgewandelt wird nur zwischen Zahlentypen (`bool` → `int` → `float` → `complex`), und zwar so, dass keine Nachkommastellen verloren gehen (anders als im C-Beispiel oben). Zwischen Zahlen und Strings wandelt Python nie automatisch um: `"Wert: " + 5` führt zu einem `TypeError`. Eine Ausnahme stellen implizite Castings nach `bool` bei Bedingungen von Verzweigungen und Schleifen dar (beide lernen wir im nächsten Kapitel kennen). Hier wird alles, was ungleich 0 bzw. nicht-leer ist, implizit als `True` interpretiert. Im Umkehrschluss: 0 oder leer (z.B. ein leerer String) bedeutet `False`.

Explizites Casting

Beim expliziten Casting fordern wir beim Programmieren die Typumwandlung ausdrücklich an. Das ist z.B. dann erforderlich, wenn die Programmiersprache implizites Casting an einer Stelle nicht unterstützt, wir die Umwandlung aber bewusst durchführen wollen. Python bietet hierfür intuitive Funktionen an, die nach dem gewünschten Zieltyp benannt sind (`float()`, `int()`, `str()`, `bool()`). Diese Funktionen übernehmen einen Input-Wert, versuchen diesen in den gewünschten Zieltyp umzuwandeln und geben das gecastete Ergebnis zurück. Der zurückgegebene Wert kann z.B. wieder einer Variable zugewiesen werden.

Beispiele:

```
text = "Meine Glückszahl ist " + str(11) # Int -> Str
print(text)
result = "Das Messergebnis beträgt " + str(5.4e-2) + " A."
print(result)
```

```
Meine Glückszahl ist 11
Das Messergebnis beträgt 0.054 A.
```

```
pi = 3.14
pi = int(pi)  # Float -> Int
print(pi)  # alle Nachkommastellen gingen verloren!
```

```
3
```

`int()` schneidet die Nachkommastellen einfach ab, rundet also nicht. Zum Runden gibt es andere Funktionen, die sich vor allem bei negativen Zahlen unterscheiden:

Ausdruck	3.7	−3.7	2.5
`int(x)` (abschneiden)	3	−3	2
`round(x)` (runden)	4	−4	**2**
`math.floor(x)` (abrunden)	3	−4	2
`math.ceil(x)` (aufrunden)	4	−3	3

Bei `round(2.5)` überrascht das Ergebnis `2`: Liegt eine Zahl genau in der Mitte, rundet Python zur nächsten *geraden* Zahl (sog. *Banker's Rounding*), damit sich Rundungsfehler in großen Datenmengen nicht systematisch aufaddieren.

`math.floor()` und `math.ceil()` stammen aus dem Modul `math`, das vorher mit `import math` geladen werden muss.

```
number = 123
condition = bool(number)  # Int -> Bool
print(condition)
```

```
True
```

```
condition = False
number = int(condition)  # Casting Bool -> Int
print(number)
```

```
0
```

```
condition = False
"The condition is " + str(condition)  # Bool -> String
```

```
'The condition is False'
```

Wir haben bereits die `input`-Funktion kennengelernt und darauf hingewiesen, dass sie stets Strings zurückgibt. Wenn man aber mit einer User-Eingabe rechnen möchte, muss diese zunächst in einen Zahlentyp umgewandelt werden. Hierzu folgende Beispiele:

```
age = int(input("Wie alt sind Sie? "))  # Casting: str -> int
age += 10  # wäre mit str nicht möglich!
print(f"In 10 Jahren sind Sie {age} Jahre alt.")
```

```
Wie alt sind Sie?  22

In 10 Jahren sind Sie 32 Jahre alt.
```

```python
width = float(input("Breite: "))  # Casting: str -> float
height = float(input("Höhe: "))  # Casting: str -> float
area = width * height  # wäre mit str nicht möglich
print("Fläche:", area)
```

```
Breite:  12.4
Höhe:  7.8

Fläche: 96.72
```

Zwei typische Stolperfallen dabei:

- `int()` akzeptiert nur Strings, die eine ganze Zahl darstellen: `int("3.5")` führt zu einem Fehler. Wer eine Eingabe mit Nachkommastellen in eine ganze Zahl umwandeln will, schreibt `int(float("3.5"))`; die Nachkommastellen werden dabei abgeschnitten.
- Python erwartet einen Punkt als Dezimalzeichen. Gibt jemand `12,4` ein, scheitert `float("12,4")`. Abhilfe: das Komma vor der Umwandlung ersetzen, z.B. `width = float(input("Breite: ").replace(",", "."))`.

Aber auch explizites Casting ist nicht in jedem Fall möglich und kann zu Fehlern führen. Z.B. weiß der Python-Interpreter nicht, wie er die folgenden Castings durchführen soll:

```python
float("Apfel")  # Fehler! Wie soll "Apfel" als float interpretiert werden?
```

```
ValueError: could not convert string to float: 'Apfel'
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[20], line 1
----> 1 float("Apfel")  # Fehler! Wie soll "Apfel" als float interpretiert
 ↪  werden?
ValueError: could not convert string to float: 'Apfel'
```

```python
int("Haus")  # Fehler!
```

```
ValueError: invalid literal for int() with base 10: 'Haus'
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[21], line 1
----> 1 int("Haus")  # Fehler!
ValueError: invalid literal for int() with base 10: 'Haus'
```

```
# Das folgende Beispiel verwendet eine Liste,
# die wir im nächsten Abschnitt kennenlernen werden:
int([1, 2, 3, 1.2])  # Fehler! Wie soll list -> int gecastet werden?
```

```
TypeError: int() argument must be a string, a bytes-like object or a real
 ↪  number, not 'list'
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[22], line 3
      1 # Das folgende Beispiel verwendet eine Liste,
      2 # die wir im nächsten Abschnitt kennenlernen werden:
----> 3 int([1, 2, 3, 1.2])  # Fehler! Wie soll list -> int gecastet werden?
TypeError: int() argument must be a string, a bytes-like object or a real
 ↪  number, not 'list'
```

Die ersten beiden Fehler sind `ValueError`s: Der Typ (String) passt, aber der Inhalt lässt sich nicht umwandeln. Im dritten Fall passt bereits der Typ nicht, deshalb meldet Python einen `TypeError`. Wie man solche Fehler abfängt, statt das Programm abstürzen zu lassen, zeigt Anhang B.

5.3. Listen

Mit der sogenannten **Liste** bzw. dem Datentyp `list` lernen wir einen wichtigen grundlegenden Datentyp kennen, der es ermöglicht, nicht nur einzelne Werte, sondern eine Sammlung mehrerer Werte gemeinsam zu speichern und zu verarbeiten. Solche Datentypen werden auch als Datenstrukturen bezeichnet (weitere Datenstrukturen werden wir später im gleichnamigen Kapitel kennenlernen).

Die Elemente einer Liste werden – wie im folgenden Beispiel – in eckigen Klammern (`[]`) notiert und durch Kommata voneinander getrennt.

```
shopping_list = ['apples', 'bananas', 'bread']
type(shopping_list)
```

```
list
```

Die Liste ist eine **geordnete** Sammlung von Elementen, das heißt, die Elemente besitzen eine feste Reihenfolge. In unserem Beispiel ist `'apples'` das erste Element, `'bananas'` das zweite usw.

Über den sogenannten **Indexzugriff** können wir gezielt auf einzelne Elemente einer Liste zugreifen. Dabei ist unbedingt zu beachten, dass der Index stets bei Null beginnt:

```
print(shopping_list[0])  # Index 0 gehört zum ersten Element
```

```
apples
```

```
print(shopping_list[1])  # Index 1 entspricht dem zweiten Element
```

```
bananas
```

```
print(shopping_list[2])  # Index 2 entspricht dem dritten Element
```

```
bread
```

Python erlaubt auch negative Indizes: der **Index -1** entspricht immer **dem letzten Element**, -2 dem vorletzten usw. Beispiel:

```
print(shopping_list[-1])  # Index -1 liefert das letzte Element
```

```
bread
```

Listen können Elemente unterschiedlicher Datentypen enthalten und dabei auch verschiedene Datentypen miteinander kombinieren:

```
# eine Liste mit Elementen vom Typ int, float und str:
my_list = [0, 1.2, "Hallo"]
```

Auch Listen innerhalb von Listen sind möglich und kommen in der Praxis häufig vor. Dadurch entstehen sogenannte **verschachtelte Strukturen**, wie im folgenden Beispiel:

```
# in dieser Liste befinden sich Einträge vom Typ
# int, float, str und wiederum list
crazy_list = [1, 0.5, "Monty Python", [42, True]]
print(crazy_list)
```

```
[1, 0.5, 'Monty Python', [42, True]]
```

```
print(crazy_list[0])  # erstes Element: der Integer 1
```

```
1
```

```
print(crazy_list[3])  # viertes Element: die Liste [42, True]
```

```
[42, True]
```

```
print(crazy_list[3][0])  # Zugriff auf das erste Element der Liste [42, True]
```

```
42
```

Wie initialisiert man eine Liste?

Es gibt mehrere Möglichkeiten, eine neue Liste anzulegen - leer oder mit Inhalt:

- Leere Initialisierung: `my_list = []` oder `my_list = list()`
- Mit Werten: `my_list = [1, 2, 3]`
- Aus anderen Datenstrukturen: `my_list = list(x)`, wobei `x` einen String oder eine andere Datenstruktur (siehe später: Set, Tupel, Dict) sein kann. Im Falle eines Strings erhält man beispielsweise die Liste der Zeichen.

Methoden der Liste

Eine besondere Eigenschaft von Listen ist, dass sie eine Reihe von **Methoden** bereitstellen, mit denen sich die Liste direkt bearbeiten lässt. Zum Beispiel können Elemente hinzugefügt, entfernt oder sortiert werden. Methoden werden dabei stets über einen Punkt (`.`) nach dem Variablennamen aufgerufen. Im Folgenden zeigen wir einige Beispiele:

```
# 1. Beispiel: Die Methode append
my_list = ['banana', 'apple', 'orange']
my_list.append('pineapple')  # fügt ein neues Element hinzu
print(my_list)
```

```
['banana', 'apple', 'orange', 'pineapple']
```

```
# 2. Beispiel: Die Methode pop
my_list = ['banana', 'apple', 'orange']
my_list.pop()  # entfernt das letzte Element
print(my_list)
```

```
['banana', 'apple']
```

```
# 3. Beispiel: Die Methode count
my_list = ['banana', 'apple', 'orange', 'banana']
num_banana = my_list.count('banana')
print(f"Es sind {num_banana} Bananen in der Liste")
num_cherry = my_list.count('cherry')
print(f"Es sind {num_cherry} Kirschen in der Liste")
```

```
Es sind 2 Bananen in der Liste
Es sind 0 Kirschen in der Liste
```

```
# 4. Beispiel: Die Methode sort
my_list = ['banana', 'apple', 'orange', 'cherry']
my_list.sort()  # sortiert die Elemente der Liste
print(my_list)
```

```
['apple', 'banana', 'cherry', 'orange']
```

Wir geben einen Überblick über die wichtigsten Methoden der Liste:

Wichtige Methoden der Liste

Methode	Bedeutung
`.append(x)`	Fügt das Element x am Ende der Liste hinzu.
`.pop()` bzw. `.pop(i)`	Entfernt das letzte Element bzw. das Element mit Index i und gibt es zurück.
`.index(x)`	Gibt den Index des ersten Auftretens von x zurück. (Achtung: Fehler, wenn x nicht vorkommt!)
`.remove(x)`	Entfernt das erste Vorkommen von x. (Fehler, wenn x nicht vorhanden!)
`.insert(i, x)`	Fügt das Element x an Position i ein.
`.clear()`	Entfernt alle Elemente.
`.count(x)`	Zählt das Vorkommen von x in der Liste.
`.sort()`	Sortiert die Liste *in-place* (das Original wird verändert!).
`.reverse()`	Kehrt die Reihenfolge aller Elemente in-place um.

Fallstrick: Das Ergebnis von `sort()` zuweisen

```
values = [3.2, 1.5, 2.8]
values = values.sort()
print(values)   # None - die Messwerte sind weg!
```

`sort()` sortiert die Liste direkt (*in-place*) und gibt kein Ergebnis zurück, genauer: den speziellen Wert `None`, der in Python für „kein Wert“ steht. Durch die Zuweisung zeigt `values` danach auf `None` statt auf die Liste. Richtig ist einfach `values.sort()` – ohne Zuweisung. Dasselbe gilt für die meisten anderen Methoden, die die Liste in-place verändern, z.B. `append()`, `insert()` oder `reverse()` (Ausnahme: `pop()` gibt das entfernte Element zurück).

Übungen:

1. Legen Sie eine Liste an, die die folgenden Namen in der gegebenen Reihenfolge beinhaltet: Charly, Bernd, Anna, Emily, Tom, Jana, Xaver, Anton. Führen Sie die folgenden Operationen nacheinander aus.
2. Sortieren Sie die Liste.
3. Kehren Sie die Liste um.
4. Fügen Sie den Namen “Anna” am Ende dazu.

5. Entfernen Sie den Namen “Jana” aus der Liste.
6. Zählen Sie, wie oft “Anna” bzw. “Jana” in der Liste vorkommen und geben Sie die Anzahlen aus.
7. Geben Sie den Index von “Emily” aus.
8. Fügen Sie ganz vorne den Namen “Walter” ein.
9. Fügen Sie die Liste mit den Elementen 10, 20 und 30 hinten an.
10. Greifen Sie auf das Element 20 der zuletzt hinzugefügten Liste zu.
11. Leeren Sie die gesamte Liste und überzeugen Sie sich davon, dass die Liste leer ist.

Lösungen:

```
names = ["Charly", "Bernd", "Anna", "Emily", "Tom", "Jana", "Xaver", "Anton"]
 ↪  # 1.
names.sort()  # 2.
names.reverse()  # 3.
names.append("Anna")  # 4.
names.remove("Jana")  # 5.
num_anna = names.count("Anna")  # 6.
num_jana = names.count("Jana")
print("Anzahl Anna (Aufgabe 6):", num_anna, "Anzahl Jana:", num_jana)
print("Index von Emily (Aufgabe 7):", names.index("Emily"))  # 7.
names.insert(0, "Walter")  # 8.
print("Liste nach Aufgabe 8:", names)
names.append([10, 20, 30])  # 9.
print("Aufgabe 10:", names[-1][1])  # 10.
names.clear()  # 11.
print("Aufgabe 11:", names)
```

5.4. Instanzen und Referenzen

Wir wollen jetzt verstehen, wie Daten gespeichert werden und was eine Variable genau ist. Dieser Abschnitt erscheint beim ersten Lesen womöglich etwas theoretisch, sein Verständnis hilft aber später enorm bei der Vermeidung vieler Missverständnisse und Fehler.

Ein vereinfachtes Speichermodell

An dieser Stelle benötigen wir ein einfaches Modell eines Speichers: Stellen wir uns vor, unser Programm hat Zugriff auf eine Reihe von Speicherstellen, in denen es Daten ablegen kann. Die kleinste Informationsmenge, die wir speichern können, ist ein **Bit**, das entweder den Wert 0 oder 1 annehmen kann. Um jedoch komplexere Informationen zu speichern, arbeiten wir in der Regel mit Gruppen aus 8 Bits, die wir als **Byte** bezeichnen. In unserem Modell stellen

wir uns den Speicher als eine große Zahl freier Bytes vor, die alle hintereinander aufgereiht sind. Jedes Byte kann über eine eindeutige Adresse identifiziert werden (die Adressen sind in der C-Programmierung wichtig, nicht so sehr in Python). Das einfache Speichermodell ist in Abbildung 5.1 skizziert.

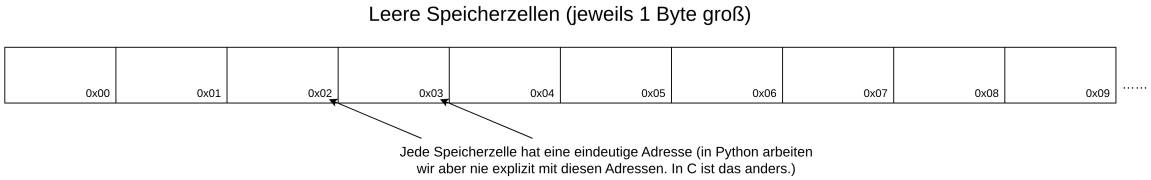


Abbildung 5.1.: Vereinfachtes Speichermodell

Instanzen

Wenn wir einen bestimmten Wert speichern wollen (z.B. eine Zahl oder einen Text), brauchen wir in der Regel mehr als eine Speicherstelle. Die genaue Größe hängt davon ab, welcher Datentyp gespeichert wird und wie viel Information dieser enthält (ein einzelner Buchstabe ist sicherlich weniger Information als ein ganzer Satz). Daten werden also je nach Datentyp und Informationsgehalt in unterschiedlich vielen Speicherzellen gespeichert. Um die Verwaltung des Speichers kümmert sich der Python-Interpreter, er ermittelt also auch, wie viele Zellen benötigt werden, und speichert die Daten. Nun aber Schritt für Schritt: Was passiert, wenn wir folgende einfache Zeile programmieren?

```
text = "Hi"
```

Zunächst ermittelt der Interpreter den Speicherbedarf für den String-Wert `"Hi"`, weist ihm einen freien Platz im Speicher zu und speichert dort die Daten ab (natürlich nicht in menschenlesbarer Form, sondern binär kodiert, wie wir später noch genauer erklären werden). Das Ergebnis nennt man eine **Instanz**: Eine Instanz ist ein konkretes Datenobjekt im Speicher, das nach den Vorgaben eines bestimmten Datentyps erzeugt und gespeichert wurde. Es kann sich dabei auch um eine Zahl handeln, z.B. `-5.25`. Wichtig an dieser Stelle: **Zu einer Instanz gehört ein bestimmter Datentyp!**

Bisher haben wir nur die rechte Seite der Zuweisung betrachtet: Das Literal erzeugt eine Instanz im Speicher. Was bewirkt die linke Seite der Zuweisung?

Referenzen

Wenn wir den String `"Hi"` keiner Variable zugeordnet hätten, wäre die erzeugte Instanz für uns nicht mehr greifbar. Sie würde nutzlos im Speicher herumliegen! In Python würde ein

Aufräummechanismus (der sog. *Garbage Collector*) diese Instanz wieder aus dem Speicher löschen. Glücklicherweise haben wir den Wert jedoch der Variable mit dem Namen `text` zugewiesen. Die Variable `text` **referenziert** die Instanz im Speicher, wodurch wir mit der Instanz arbeiten können. Eine Variable ist also eine bloße **Referenz** und von der eigentlichen Instanz (den "echten Daten") zu unterscheiden. Referenzen können Sie sich wie Hinweisschilder oder Wegweiser vorstellen: Sie zeigen auf die Instanz, sind aber nicht selbst die Instanz.

In Abbildung 5.2 wird der Unterschied zwischen Instanzen und Referenzen bildlich dargestellt.

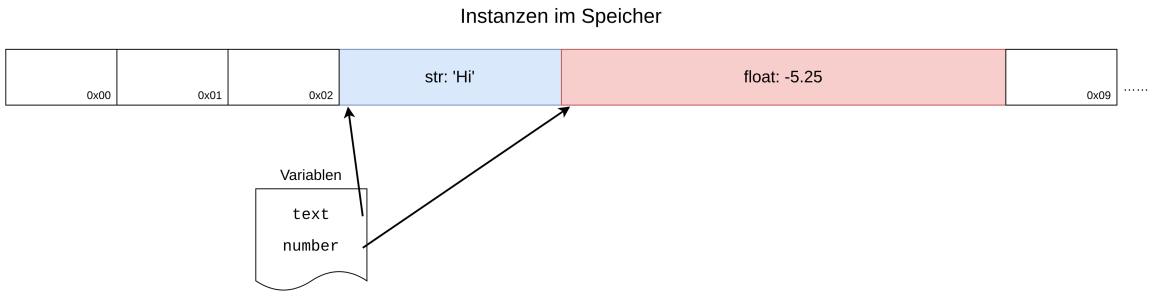


Abbildung 5.2.: Referenzen und Instanzen. Instanzen sind konkrete Datenobjekte im Speicher. Variablen verweisen auf diese Instanzen und werden deshalb auch Referenzen genannt. Im gezeigten Beispiel liegen eine String-Instanz und eine Float-Instanz im Speicher und werden jeweils von einer Variable referenziert. Eine Instanz kann aber auch mehrfach oder gar nicht referenziert werden (in letzterem Fall wird sie aber vom Garbage Collector wieder aufgeräumt)

Streng genommen ist es nicht korrekt, zu sagen, eine Variable hätte einen bestimmten Datentyp. Der Datentyp gehört immer zur referenzierten Instanz. Wenn wir in einer anderen Zeile `text = 123` schreiben, zeigt die Variable `text` plötzlich auf eine Instanz vom Typ `int`. Dennoch ist es im alltäglichen Sprachgebrauch üblich zu sagen, "Die Variable `text` hat den Typ `str`." Merken Sie sich jedoch, dass diese Formulierung ungenau ist!

Identität einer Instanz

Jede Instanz besitzt in Python eine eindeutige **Identität**, die durch die Funktion `id()` abgefragt werden kann und wie ein Fingerabdruck der Instanz verstanden werden kann (eindeutig ist die Identität allerdings nur, solange die Instanz existiert – danach kann Python dieselbe Nummer für eine neue Instanz wiederverwenden):

```
id(text)  # mit id() erhält man die Identität
```

```
2122758446064
```

Hier wurde die eindeutige ID der Instanz ausgegeben, die von `text` referenziert wird. Wenn wir die Variable auf eine andere Instanz zeigen lassen, ändert sich auch die Identität:

```
text = "Guten Tag"
id(text)
```

```
2122752622320
```

> Kurze Zusammenfassung
>
> Eine Instanz ist ein konkretes Objekt im Speicher. Sie hat drei Merkmale: eine eindeutige Identität, einen Datentyp und einen Wert. Eine Variable ist ein symbolischer Name, der eine Instanz referenziert (man sagt auch: auf eine Instanz zeigt).

Seiteneffekte – wenn mehrere Referenzen auf dieselbe Instanz zeigen

Es kann vorkommen, dass mehrere Variablen auf ein und dieselbe Instanz zeigen und dadurch unerwartete Effekte passieren. Dazu betrachten wir zunächst ein Beispiel ohne Seiteneffekte und danach ein zweites mit solchen Effekten.

Erstes Beispiel (keine Seiteneffekte):

```
a = 12
b = a  # a wird b zugewiesen
print(id(a), id(b))
```

```
140725879743752 140725879743752
```

An dieser Stelle erhalten wir identische Identitäten, d.h. `a` und `b` referenzieren dieselbe Instanz! Eine Zuweisung wie `b = a` legt nämlich niemals eine neue Instanz an – unabhängig vom Datentyp. Sie sorgt lediglich dafür, dass `b` auf dieselbe Instanz zeigt wie `a`. Wir führen unser Beispiel fort:

```
b += 10  # b zeigt danach auf eine andere Instanz (mit dem Wert 22)
print(id(a),id(b))
```

```
140725879743752 140725879744072
```

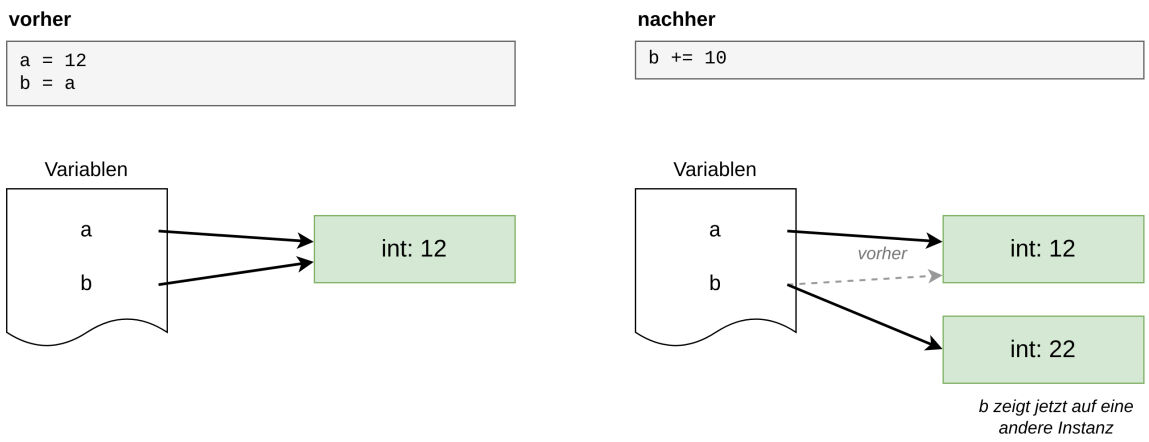


Abbildung 5.3.: Nach `b += 10` zeigt `b` auf eine andere int-Instanz (Wert 22), `a` bleibt unverändert. Der gestrichelte Pfeil zeigt die vorherige Referenz von `b`.

Wie wir sehen (Abbildung 5.3), zeigt `b` jetzt auf eine andere Instanz (eine mit Wert 22), während `a` immer noch unverändert auf die ursprüngliche verweist. In diesem Beispiel ist nichts Unerwartetes geschehen. Damit wir nicht immer die Identitäten mühsam miteinander vergleichen müssen, können wir auch den Operator `is` verwenden:

```
print(a is b)  # vergleicht die Identitäten der referenzierten Instanzen
# True heißt: a und b zeigen auf dieselbe Instanz
# False heißt: a und b zeigen auf unterschiedliche Instanzen
```

```
False
```

Zweites Beispiel (mit Seiteneffekten):

Wir wiederholen das Experiment von oben, diesmal aber mit einem anderen Datentyp (einer Liste):

```
my_shopping_list = ['apples', 'oranges', 'paper']
your_shopping_list = my_shopping_list
print(id(my_shopping_list), id(your_shopping_list))
```

```
2108606717056 2108606717056
```

Bis hierin verhalten sich die beiden Beispiele gleich. Jetzt manipulieren wir eine der Variablen:

```
your_shopping_list.append('shoes')
print(id(my_shopping_list), id(your_shopping_list))
```

```
2108606717056 2108606717056
```

Die Identitäten sind immer noch gleich! Das bestätigt auch der `is`-Operator:

```
print(your_shopping_list is my_shopping_list)
```

```
True
```

Aber was bedeutet das für die ursprüngliche Liste?

```
print(my_shopping_list)
```

```
['apples', 'oranges', 'paper', 'shoes']
```

Die ursprüngliche Liste wurde verändert, obwohl wir die Variable `my_shopping_list` gar nicht angefasst haben! Man spricht in diesem Zusammenhang von einem **Seiteneffekt** (Abbildung 5.4).

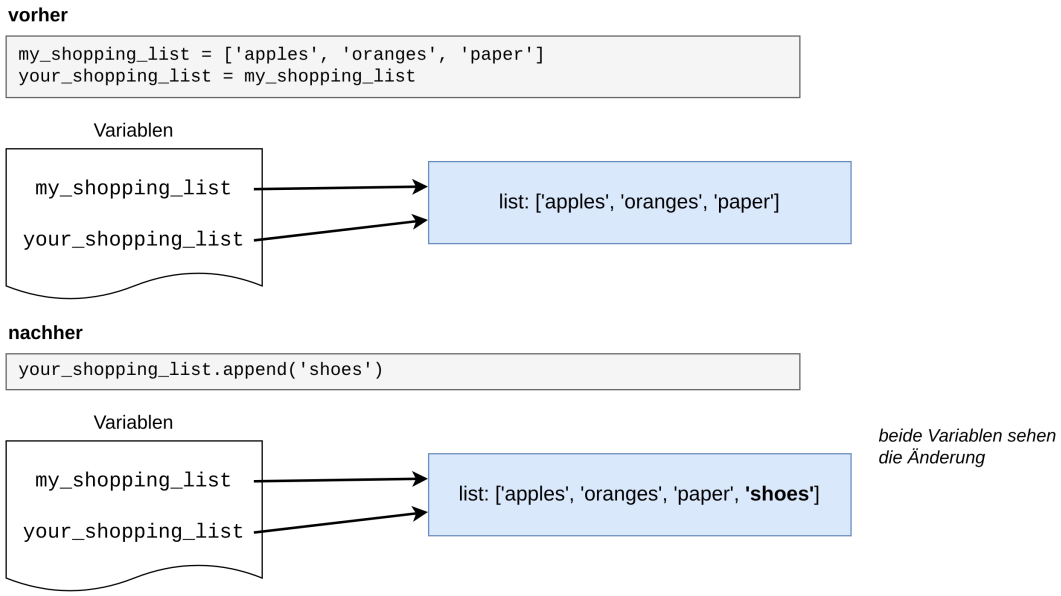


Abbildung 5.4.: Beide Variablen referenzieren dieselbe Liste. Eine Änderung über `your_shopping_list` ist deshalb auch über `my_shopping_list` sichtbar (Seiteneffekt).

Nicht verwechseln: Gleichheit und Identität

`x == y` (Gleichheit) prüft, ob die Instanzen von `x` und `y` denselben Wert haben.
`x is y` (Identität) prüft, ob `x` und `y` auf dieselbe Instanz zeigen. In diesem Fall gilt in aller Regel auch `x == y` (eine kuriose Ausnahme ist der spezielle Gleitkommawert `nan`, „not a number“, der nicht einmal mit sich selbst gleich ist), die Umkehrung ist im Allgemeinen falsch.

Mutable und immutable Datentypen

Python unterscheidet zwischen **immutable** (unveränderlichen) und **mutable** (veränderlichen) Datentypen. Diese Begriffe beziehen sich auf Instanzen eines bestimmten Datentyps:

- Wenn eine Instanz **immutable** ist, kann sich ihr Wert nie verändern! Wenn der Wert einer Variable, die auf eine immutable Instanz zeigt, sich verändert, zeigt die Variable hinterher auf eine andere Instanz! Weil sich immutable Instanzen nie ändern, darf Python sie gefahrlos mehrfach verwenden: CPython legt z.B. kleine Ganzzahlen von -5 bis 256 nur einmal an. Garantiert ist das aber nicht – zwei Variablen mit dem Wert 1000 können durchaus auf verschiedene Instanzen zeigen. Werte vergleicht man deshalb immer mit `==` und nicht mit `is`. Integers, Floats, Bools, Strings und Tupel (lernen wir später kennen) sind in Python immutable. Bei diesen Datentypen kann es **keine Seiteneffekte** geben (bei Tupeln allerdings nur, solange sie selbst keine veränderlichen Elemente wie Listen enthalten)!
- Anders bei den **mutable** Datentypen: Hier kann die Instanz verändert werden. Dies führt zu **Seiteneffekten**, wenn mehrere Variablen dieselbe Instanz referenzieren. Listen, Dictionaries (siehe später) und Sets (siehe später) sind mutable.

Das Phänomen der Seiteneffekte belegt, warum ein Verständnis von Instanzen und Referenzen so wichtig ist für angehende Programmiererinnen und Programmierer.

Abschließender Hinweis: Behalten Sie stets im Hinterkopf, dass es bei den Begriffen *mutable* bzw. *immutable* immer um die Instanz geht, nicht um die Referenz. Natürlich können Sie eine Variable, die auf eine immutable Instanz zeigt, ändern. Dann zeigt sie aber auf eine andere Instanz.

Wie ist das in anderen Sprachen?

Das Konzept der Immutability, wie wir es in Python kennengelernt haben, existiert nicht genauso in anderen Sprachen. Auch der prinzipielle Umgang mit Speicher unterscheidet sich teils grundlegend. Sprachen wie Java oder C# sind Python ähnlicher, da sie Instanzen und Referenzen verwenden und teilweise Immutability durch Sprachmechanismen unterstützen, wie z.B. immutable Strings oder immutable Klassen.

In C hingegen gibt es keine Instanzen und Referenzen im Sinne von Python und damit auch keine vergleichbare Unterscheidung zwischen mutable und immutable Datentypen. Stattdessen operiert C als Low-Level-Sprache sehr nahe am Speicher. Das Konzept der *Pointer* ist dabei entscheidend und wird Ihnen bekannt vorkommen: Pointer verweisen direkt auf Speicheradressen und mehrere Pointer können auf denselben Speicherbereich zeigen. Dadurch können – ähnlich wie in Python – Seiteneffekte entstehen. Allerdings bietet C kaum sprachseitige Schutzmechanismen gegen solche Effekte (einer der wenigen ist `const`; zum Unterschied zwischen `const` und immutable siehe Exkurs unten). Diese größere Freiheit bedeutet mehr Kontrolle, erfordert aber auch mehr Verantwortung, da Fehler schneller auftreten können.

i Exkurs für Interessierte: `const` ist nicht dasselbe wie immutable

In vielen Programmiersprachen gibt es das Schlüsselwort `const` (von *constant*). Es wird leicht mit *immutable* verwechselt, meint aber etwas anderes:

- `const` ist eine Eigenschaft der **Variable**: Ihr darf nach der ersten Zuweisung kein neuer Wert zugewiesen werden.
- *immutable* ist eine Eigenschaft des **Werts** (der Instanz): Der Wert selbst kann nicht verändert werden, egal welche Variable auf ihn zeigt.

In **C** schützt `const` eine Variable:

```c
const int max_load = 5000;
max_load = 6000;   // Fehler: max_load ist const
```

In **JavaScript** sieht man den Unterschied besonders deutlich:

```javascript
const temps = [21.5, 22.0];
temps.push(23.1);   // erlaubt! Die Liste selbst ist veränderbar
temps = [0.0];      // Fehler: Der Variable darf keine neue Liste
 ↪  zugewiesen werden
```

Python kennt kein `const`. Konstanten kennzeichnet man nur per Konvention mit Großbuchstaben, z.B. `MAX_LOAD = 5000` (siehe PEP 8); technisch kann man sie trotzdem neu zuweisen. Unveränderlich ist in Python immer der *Wert*, nie die Variable: Ein Tupel kann man nicht verändern, der Variable aber jederzeit ein anderes Tupel zuweisen.

5.5. Binäre Darstellung von Daten

Wir wissen nun, dass im Speicher eines Computers Instanzen der benötigten Daten erzeugt werden und wir über Referenzen auf diese zugreifen können. Wie aber sieht so eine Instanz im Speicher genau aus? Computer arbeiten auf einer fundamentalen Ebene mit binären Daten, also Zahlen, die nur aus den Ziffern 0 und 1 bestehen. Diese beiden Zustände repräsentieren die grundlegenden Bits (Binary Digits), mit denen alle Arten von Daten – wie Zahlen, Texte oder Bilder – gespeichert und verarbeitet werden. Mit anderen Worten: Eine Zahl (z.B. -5.25) oder ein Wort (z.B. "Apfel") muss durch eine Transformation, genannt *Kodierung*, als Binärzahl dargestellt werden können (und umgekehrt). Die Art der Kodierung hängt davon ab, welcher Datentyp dargestellt werden soll.

i Einführung ins Binärsystem

Das Binärsystem (oder Dualsystem) ist ein Zahlensystem zur Basis 2, das nur zwei Ziffern verwendet: 0 und 1. Es ist die Grundlage aller digitalen Systeme, da Computer mit diesen beiden Zuständen (z.B. Strom an/aus) arbeiten. Die natürlichen Zahlen in aufsteigender Reihenfolge sehen binär so aus: 0, 1, 10, 11, 100, 101 usw. Zur Abgrenzung von Dezimalzahlen verwendet man gelegentlich eine tiefgestellte 2 (z.B. $(100)_2$) oder das Präfix 0b (z.B. 0b100).

Umwandlung von Binär- nach Dezimalsystem

Um eine Binärzahl in eine Dezimalzahl umzuwandeln, multipliziert man jede Ziffer mit 2^{Position}, beginnend von rechts mit der Position 0, und addiert die Ergebnisse.
Beispiel: Binärzahl $(1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11$ (Dezimalzahl).

Umwandlung von Dezimal- nach Binärsystem

Eine Dezimalzahl wird durch wiederholtes Teilen durch 2 in eine Binärzahl umgewandelt. Die Reste ergeben die Binärzahl (von unten nach oben lesen).
Beispiel: Die Dezimalzahl 13 soll binär dargestellt werden.
$13 : 2 = 6,\ \text{Rest: } 1\ 6 : 2 = 3,\ \text{Rest: } 0\ 3 : 2 = 1,\ \text{Rest: } 1\ 1 : 2 = 0,\ \text{Rest: } 1$
Binärzahl (Reste von unten nach oben lesen): 1101

Stellen hinter dem Komma in binärer Schreibweise

Bei Kommazahlen entsprechen die Stellen hinter dem Komma negativen Exponenten. Wir kennen das vom Dezimalsystem: $0.243 = 2 \cdot 10^{-1} + 4 \cdot 10^{-2} + 3 \cdot 10^{-3}$.
Analog im Binärsystem: $(0.101)_2 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.625$ als Dezimalzahl.

Python-Funktion `bin()`

Mit der Funktion `bin()` erhält man die Binärdarstellung einer ganzen Zahl, z.B. `bin(4711)` oder `bin(0xAB09)` (hier ist die Zahl als Hexadezimal-Literal angegeben). Das Ergebnis ist

ein **String** wie `'0b1001001100111'`, mit dem man nicht mehr rechnen kann. Den umgekehrten Weg geht `int()` mit Angabe der Basis: `int("1011", 2)` ergibt `11`, `int("AB09", 16)` ergibt `43785`.

i Einführung ins Hexadezimalsystem

Das Hexadezimalsystem ist ein Zahlensystem zur Basis 16, das die folgenden Ziffern verwendet: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F. Zur Abgrenzung von anderen Zahlensystemen verwendet man gelegentlich eine tiefgestellte 16 (z.B. $A07_{16}$) oder das Präfix 0x (z.B. 0xA09F2).

Umwandlung von Hexadezimal- nach Dezimalsystem

Um eine Hexadezimalzahl in eine Dezimalzahl umzuwandeln, multipliziert man jede Ziffer mit 16^{Position}, beginnend von rechts mit der Position 0, und addiert die Ergebnisse.
Beispiel:
$0x4A0F = 4 \cdot 16^3 + 10 \cdot 16^2 + 0 \cdot 16^1 + 15 \cdot 16^0 = 18959$

Umwandlung von Dezimalsystem nach Hexadezimalsystem

Analog zum Algorithmus für das Binärsystem, diesmal wiederholte Division durch 16.

Umwandlung von Binärsystem nach Hexadezimalsystem

Das Hexadezimalsystem erfreut sich in der Informatik großer Beliebtheit, weil es hilft, Bytes sehr übersichtlich und kompakt auszudrücken. Vier Bits entsprechen gerade einer Hexadezimalstelle, ein Byte (8 Bits) also genau zwei Hexadezimalstellen. Man teilt das Byte in zwei Viererblöcke auf und wandelt diese jeweils ins Hexadezimalsystem um (das ist sehr einfach, da nur die Zahlen 0 - 15 vorkommen). Dann schreibt man die Hexadezimalziffern hintereinander. **Beispiel:**
0b10011110 = 0b1001 1110 = 0x9E. Hierzu haben wir 0b1001 = 0x9 und 0b1110 = 0xE verwendet.

Python-Funktion `hex()`

Mit der Funktion `hex()` können ganze Zahlen ins Hexadezimalsystem umgewandelt werden. Z.B. `hex(4711)` (von Dezimal- nach Hexadezimalsystem) oder `hex(0b11001111)` (von Binär- nach Hexadezimalsystem).

Binäre Darstellung von Ganzzahlen

Ganzzahlen (Integer) werden direkt im Binärsystem gespeichert. Dabei gibt es eine Unterscheidung:

- Positive Ganzzahlen: Sie werden als reine Binärzahlen dargestellt, z.B. ist die Dezimalzahl

5 im Binärsystem 101.

- Negative Ganzzahlen: Heute wird praktisch immer das Zweierkomplement verwendet. Das höchste Bit ist bei negativen Zahlen 1 (es wirkt also wie ein Vorzeichenbit), die übrigen Bits enthalten aber nicht einfach den Betrag. Stattdessen bildet man die negative Zahl, indem man alle Bits der positiven Zahl invertiert und 1 addiert. Mit 8 Bits wird aus $5 = (00000101)_2$ so $-5 = (11111010)_2 + 1 = (11111011)_2$. Der Vorteil: Addition und Subtraktion funktionieren für positive und negative Zahlen mit derselben Rechenschaltung. (Python selbst verwendet für `int` übrigens keine feste Bitbreite, weil Ganzzahlen beliebig groß werden können; `bin(-5)` liefert deshalb einfach `'-0b101'`.)

Integer können in Python mit der Funktion `bin()` ganz einfach ins Binärsystem umgewandelt werden:

```
print("Dezimal:", 11, "Binär:", bin(11))
print("Dezimal:", 13, "Binär:", bin(13))
print("Dezimal:", 423, "Binär:", bin(423))
```

```
Dezimal: 11 Binär: 0b1011
Dezimal: 13 Binär: 0b1101
Dezimal: 423 Binär: 0b110100111
```

Umgekehrt geht es sogar noch viel einfacher, wie die folgenden Beispiele zeigen:

```
# ein binäres Literal 0b... wird automatisch umgewandelt:
print("Binär:", "0b1011", "Dezimal:", 0b1011)
print("Binär:", "0b1101", "Dezimal:", 0b1101)
print("Binär:", "0b110100111", "Dezimal:", 0b110100111)
```

```
Binär: 0b1011 Dezimal: 11
Binär: 0b1101 Dezimal: 13
Binär: 0b110100111 Dezimal: 423
```

Binäre Darstellung von Gleitkommazahlen

Gleitkommazahlen (wie 3.14 oder -0.001) werden durch den IEEE-754-Standard dargestellt, der folgende Struktur nutzt:

- **Vorzeichen** (1 Bit): Gibt an, ob die Zahl positiv (Vorzeichenbit = 0) oder negativ (Vorzeichenbit = 1) ist.
- **Exponent**: Bestimmt die Skalierung der Zahl.

- **Mantisse**: Speichert die signifikanten Stellen der Zahl.

> **Beispiel Gleitkommazahl-Darstellung**
>
> Die Zahl -5.25 soll binär mit insgesamt 32 Bits dargestellt werden. Die Aufteilung der Bits soll so erfolgen: 1 Bit für das Vorzeichen, 8 Bits für den Exponenten und 23 Bits für die Mantisse.
> Das Vorzeichenbit ist offensichtlich 1 (für negativ), nun müssen wir noch Exponent und Mantisse kodieren. Dazu drücken wir den Betrag (das Vorzeichen interessiert uns nicht mehr) binär und wissenschaftlich notiert aus: $5.25 = (101.01)_2 = (1.0101)_2 \cdot 2^2$.
> Daraus ergibt sich:
>
> - Der Exponent ist 2, allerdings noch im Zehnersystem ausgedrückt. Mit 8 Bits binär dargestellt wäre er $(00000010)_2$. Man verwendet aber noch einen Trick, um negative Exponenten nicht wieder mit einem Vorzeichenbit unterscheiden zu müssen, und addiert 127 zum eigentlichen Exponenten: Unser Exponent 2 wird also folgendermaßen kodiert: $127 + 2 = 129 = (10000001)_2$.
> - Die Mantisse im Binärsystem ist 1.0101. Beim Kodieren wird die führende 1 weggelassen, weil sie immer da ist – ein Trick zur Speicheroptimierung. Die Stellen hinter dem Komma werden in 23 Bits abgelegt: $(01010000000000000000000)_2$
>
> Die komplette Kodierung lautet: 1 10000001 01010000000000000000000.

Das Beispiel verwendet zur besseren Übersicht 32 Bits (*einfache Genauigkeit*). Ein `float` in Python hat dagegen 64 Bits (*doppelte Genauigkeit*): 1 Bit Vorzeichen, 11 Bits Exponent und 52 Bits Mantisse. Das entspricht etwa 15 bis 16 gültigen Dezimalstellen.

Gleitkommazahlen bieten durch diese Struktur eine große Reichweite und Präzision. Allerdings lassen sich viele Dezimalzahlen im Binärsystem nicht mit endlich vielen Stellen darstellen – schon `0.1` nicht, so wie 1/3 im Dezimalsystem nicht. Es wird also bereits beim Speichern gerundet, und diese kleinen Fehler können sich beim Rechnen bemerkbar machen:

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

```
0.30000000000000004
False
```

Ein wichtiger Merksatz lautet deshalb: Vergleichen Sie **niemals** zwei Gleitkommazahlen auf exakte Gleichheit, sondern verwenden Sie immer eine kleine Toleranz. Python bietet dafür die

Funktion `isclose()` aus dem Modul `math`:[1]

```
import math
math.isclose(0.1 + 0.2, 0.3)
```

```
True
```

Für NumPy-Arrays gibt es analog die Funktionen `np.isclose()` und `np.allclose()` (NumPy lernen wir im Kapitel *Bibliotheken* kennen).

Binäre Darstellung von Strings

Strings bestehen aus einer Folge von Zeichen, die jeweils durch Zahlencodes repräsentiert werden:

- **ASCII:** Jedes Zeichen wird durch eine 7-Bit-Zahl dargestellt (z.B. A = 65 = 0b1000001). Der Code deckt eine Auswahl von druckbaren Zeichen (lateinisches Alphabet in Groß- und Kleinschreibung, unsere 10 Ziffern, Interpunktions- und Sonderzeichen) und nicht-druckbare Zeichen (z.B. Tabulator oder Protokollzeichen für Übertragungsende oder Bestätigung) ab. Für englische Texte ist der Zeichensatz völlig ausreichend, er beinhaltet aber weder deutsche Umlaute (ä, ö, ü) noch chinesische Schriftzeichen oder Emojis - um nur einige Beispiele von Zeichen zu nennen, die über ASCII hinausgehen. Aus diesem Grund gibt es erweiterte Zeichensätze. Durch Erweiterung um ein achtes Bit entstanden verschiedene Zeichensätze für unterschiedliche Sprachen (z.B. *Latin-1* für westeuropäische Sprachen, inkl. der deutschen Umlaute).
- **Unicode (UTF-8):** Unicode ist ein universeller Zeichensatz, der mehrere Sprachen mit nur einem Zeichensatz umfassen soll. Es gibt mehrere Kodierungen für Unicode, von denen UTF-8 (UTF = Unicode Transformation Format) die am weitesten verbreitete Variante darstellt. UTF-8 kodiert Zeichen mit variabler Byte-Anzahl: Dabei wird ein Unicodezeichen in 1 bis 4 Bytes kodiert (die 8 im Namen UTF-8 bezieht sich auf die kleinste Einheit beim Kodieren: 8 Bits, also 1 Byte). Die Codewerte 0 bis 127 entsprechen dem ASCII-Zeichensatz und werden in einem einzigen Byte kodiert, wobei das höchstwertige Bit stets 0 ist. Mithilfe von fest definierten Bitmustern können längere Unicode-Zeichen eingeleitet werden, die sich auf 2, 3 oder 4 Bytes erstrecken. Beispiele: A → 01000001 (1 Byte), 😊 → 11110000 10011111 10011000 10001010 (4 Bytes). Im zweiten Beispiel (der Smiley) sind 11110 vom ersten Byte und 10 von den Folge-Bytes jeweils Steuerbits.

[1] Vorsicht beim Vergleich mit 0: `math.isclose()` verwendet standardmäßig nur eine *relative* Toleranz, die sich an der Größe der verglichenen Zahlen orientiert. Gegenüber 0 ist diese Toleranz praktisch 0, sodass z.B. `math.isclose(1e-17, 0)` den Wert `False` liefert. In solchen Fällen gibt man zusätzlich eine absolute Toleranz an: `math.isclose(1e-17, 0, abs_tol=1e-12)`. Das NumPy-Pendant `np.isclose()` verwendet dagegen standardmäßig auch eine kleine absolute Toleranz – die Ergebnisse beider Funktionen können sich also unterscheiden.

11110 kündigt ein Zeichen aus insgesamt vier Bytes an, 10 leitet jeweils die Folgebytes ein. Die reine Information, die den Smiley eindeutig definiert, steckt daher in den übrigen 21 Bits: 000011111011000001010, wobei man die führenden Nullen noch streichen kann, also 11111011000001010. Diese Bits heißen auch **Codepunkt**.

- **Die Funktionen `ord()` und `chr()`**: Die Funktion `ord()` liefert den Codepunkt eines Unicode-Zeichens zurück, z.B. `ord("A")` oder `ord("😊")`. Die Funktion `chr()` ist die Umkehrung von `ord()`, d.h. `chr(0b01000001)` liefert das Zeichen zum Codepunkt `0b01000001 = 65` (das ASCII-Zeichen `"A"`).

```
# Beispiele mit ord():
print(bin(ord("A")))
print(hex(ord("😊")))
```

```
0b1000001
0x1f60a
```

```
# Beispiele mit chr():
print(chr(65))
print(chr(0xAE))
```

```
A
®
```

Abschließend stellen wir unser früheres Beispiel für Instanzen und Referenzen noch einmal dar, diesmal aber mit binär kodierten Werten (Abbildung 5.5).

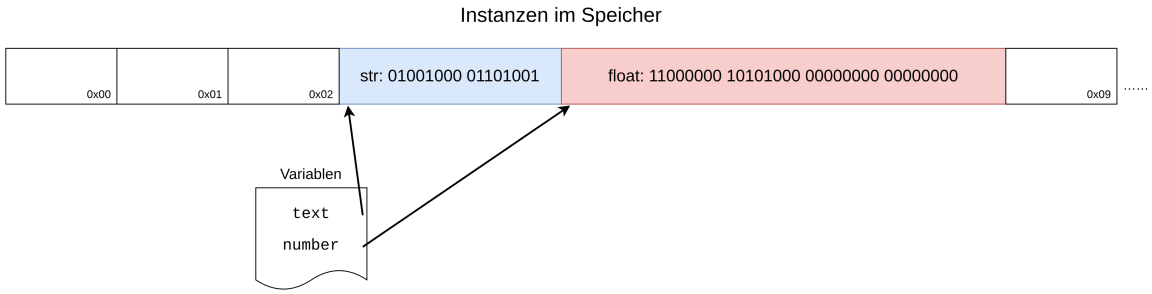


Abbildung 5.5.: Instanzen und Referenzen revisited. Diesmal mit binär kodierten Werten (den Float haben wir wie im Rechenbeispiel vereinfacht mit 32 Bit dargestellt; Python verwendet 64 Bit).

Noch ein paar Bemerkungen zum Speicherbedarf:

- In C liegen im Speicher nur die rohen Daten, daher benötigt ein Float mit der oben dargestellten Kodierung tatsächlich nur vier Bytes oder ein ASCII-Zeichen nur ein Byte.
- Python speichert viel mehr ab: Zu jedem Wert werden Metadaten wie Datentyp, Identität, Referenz-Zähler etc. abgespeichert. Außerdem verwendet Python eigene Optimierungen, z.B. im Umgang mit Strings oder kleinen Integern. Daher stimmt die Größe einer Instanz nicht mit der Größe der "Rohdaten" überein.
- Python passt den Speicherbedarf von Integern automatisch an die benötigte Informationsmenge an, so werden für eine sehr große Ganzzahl automatisch mehr Bytes verwendet als für eine kleine Ganzzahl. In C macht das niemand für Sie: Ein Datentyp stellt eine bestimmte Zahl an Bytes zur Verfügung, reicht diese nicht aus, müssen Sie selbst einen größeren Datentyp verwenden!
- Mit der Funktion `getsizeof()` aus dem Modul `sys` können Sie die Zahl der Bytes abfragen. Beispiel:

```
import sys
sys.getsizeof("42")  # Antwort: Größe in Bytes
```

```
43
```

6. Kontrollstrukturen

> **Lernziele**
>
> Nach diesem Kapitel können Sie …
>
> - Verzweigungen mit `if`, `elif` und `else` sowie Conditional Expressions formulieren
> - Bedingungen mit `and`, `or` und `not` verknüpfen
> - Wiederholungen mit `while`- und `for`-Schleifen umsetzen und den passenden Schleifentyp wählen
> - den Schleifenablauf mit `break` und `continue` steuern
> - verschachtelte Schleifen schreiben
> - einfache mathematische Verfahren (Newton-Verfahren, Vektor- und Matrixrechnung) mit Schleifen umsetzen

In der Programmierung sind Kontrollstrukturen essenziell, um den Ablauf eines Programms zu steuern. Sie ermöglichen es, Entscheidungen zu treffen, Wiederholungen durchzuführen und den Programmfluss gezielt zu lenken. Ohne sie würden Programme einfach nur von oben nach unten ausgeführt werden, ohne auf Eingaben oder Bedingungen zu reagieren.

Wir unterscheiden zwischen zwei grundsätzlichen Arten von Kontrollstrukturen: **Verzweigungen** und **Schleifen**.

6.1. Verzweigungen

Verzweigungen oder bedingte Anweisungen sind nichts anderes als Fallunterscheidungen: "Wenn eine bestimmte Bedingung erfüllt ist, tue dies, andernfalls tue das." In den meisten Programmiersprachen wird dies durch eine `if`-Anweisung realisiert (die genaue Syntax unterscheidet sich von Sprache zu Sprache). Das folgende Codebeispiel zeigt, wie es in Python funktioniert:

```
number = 42

if number > 0:
    print("Die Zahl ist positiv!")
```

```
Die Zahl ist positiv!
```

In diesem einfachen Beispiel wird die Bedingung `number > 0` geprüft. Ist die Bedingung erfüllt, werden die **eingerückten** Anweisungen ausgeführt, in diesem Beispiel der `print`-Befehl.

Die Funktionsweise einer `if`-Anweisung lässt sich mit einem Flussdiagramm (Abbildung 6.1) anschaulich darstellen.

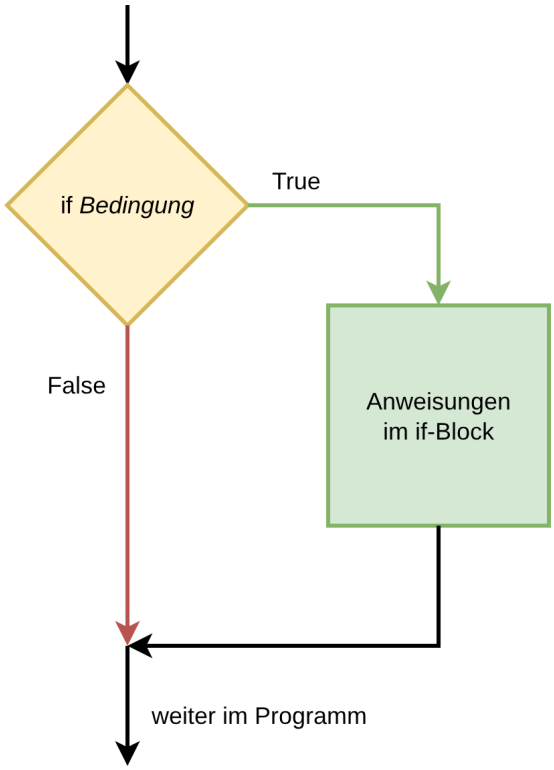


Abbildung 6.1.: Flussdiagramm zur if-Anweisung

Wichtig ist die korrekte Syntax: Die Anweisung wird mit dem Schlüsselwort `if` eingeleitet, gefolgt von der Bedingung und einem Doppelpunkt. Alle Anweisungen, die bei erfüllter Bedingung ausgeführt werden sollen, müssen darunter eingerückt sein. Wir betrachten ein zweites Beispiel:

```
max_load = 5000  # Maximale Belastung in Newton
current_load = float(input("Belastung in Newton: "))  # z.B. ein Messergebnis

if current_load > max_load:
    print("Warnung: Belastung überschritten!")
    factor = (current_load - max_load) / max_load
```

```
    print(f"Maximale Belastung um {round(100*factor, 2)} % überschritten!")
```

```
Belastung in Newton:  5200

Warnung: Belastung überschritten!
Maximale Belastung um 4.0 % überschritten!
```

Aktuell beschränken sich unsere Beispiele zu Zwecken der Illustration auf simple Ausgaben, aber natürlich gibt es keine Grenzen für die Komplexität, die in einem `if`-Block ausgeführt werden kann.

Verzweigungen mit if und else

In den meisten Fällen verwendet man erweiterte Varianten der `if`-Anweisung. Zum Beispiel möchte man auch bestimmte Anweisungen ausführen, falls die Bedingung nicht erfüllt ist. Dafür ergänzt man einen `else`-Block:

```
max_load = 5000  # Maximale Belastung in Newton
current_load = float(input("Belastung in Newton: "))

if current_load > max_load:
    print("Warnung: Belastung überschritten!")
    factor = (current_load - max_load) / max_load
    print(f"Maximale Belastung um {round(100*factor, 2)} % überschritten!")
else:
    print("Alles im grünen Bereich!")
```

```
Belastung in Newton:  4600

Alles im grünen Bereich!
```

Auch diese Variante stellen wir im Flussdiagramm dar (Abbildung 6.2).

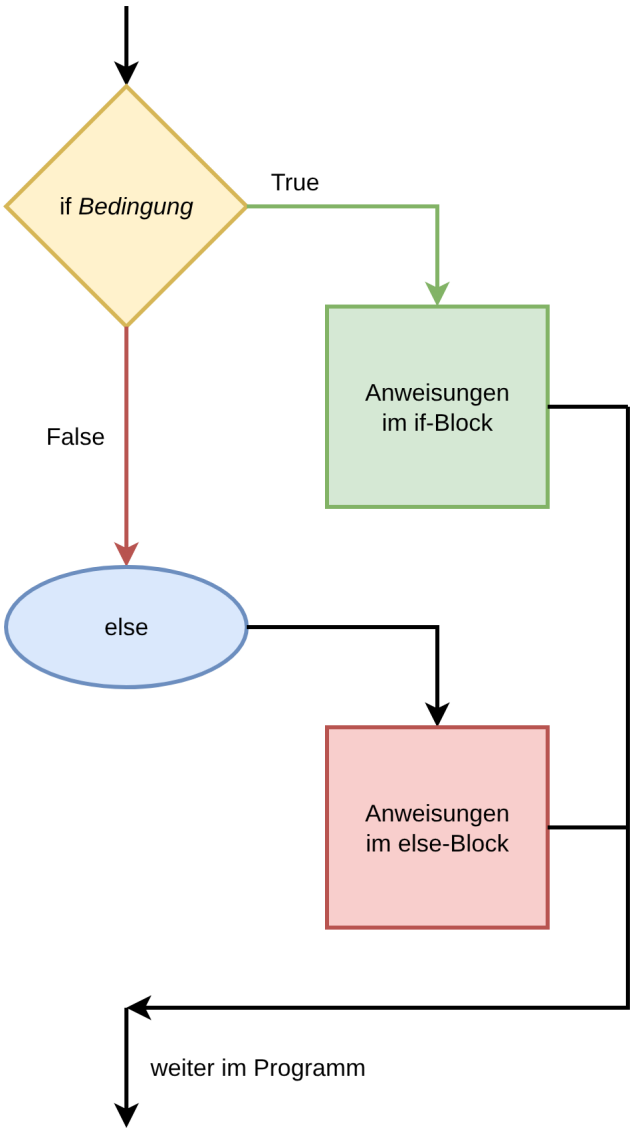


Abbildung 6.2.: Flussdiagramm zur Verzweigung mit if und else

Wir haben die Flussdiagramme bewusst so gezeichnet, dass die Einrückungen, wie sie von Python verlangt werden, auch im Diagramm ersichtlich sind: Die Schlüsselwörter `if` und `else` befinden sich untereinander auf derselben Einrückungsebene, die jeweiligen Anweisungsblöcke sind darunter eingerückt.

⚠ Fallstrick: Gleitkommazahlen mit `==` vergleichen

```
x = 0.1 + 0.2
if x == 0.3:
    print("gleich")
else:
    print("ungleich")   # wird ausgegeben!
```

Wie im Kapitel *Datentypen* gezeigt, können Gleitkommazahlen viele Dezimalzahlen nicht exakt darstellen: `0.1 + 0.2` ergibt `0.30000000000000004`. In Bedingungen sollte man Gleitkommazahlen deshalb nicht mit `==` vergleichen, sondern prüfen, ob sie *nahe genug* beieinanderliegen:

```
if abs(x - 0.3) < 1e-9:   # Abweichung kleiner als eine Toleranz
    print("gleich")
```

Alternativ verwenden Sie `math.isclose(x, 0.3)` aus dem Modul `math` (siehe Kapitel *Datentypen*).

Verzweigungen mit if, elif und else

Schließlich können wir unsere Verzweigung noch um sog. "else-if-Zweige" erweitern (das Python-Keyword heißt `elif`). Der Sinn davon: Wenn eine erste Bedingung falsch ist, möchte man vielleicht eine weitere Bedingung prüfen. Ist auch diese falsch, könnte noch eine dritte (vierte, fünfte, …) Bedingung interessant sein. In Worten: "Wenn Bedingung1 wahr ist, tue A, andernfalls prüfe Bedingung2. Ist Bedingung2 wahr, tue B, andernfalls prüfe Bedingung3 usw." Wichtig zu verstehen ist, dass die Bedingungen nacheinander geprüft werden. Wenn eine frühere Bedingung bereits erfüllt ist, werden spätere Bedingungen gar nicht mehr angeschaut. Ein (optionales) finales `else` fängt den Fall auf, wenn keine Bedingung erfüllt ist. Das Flussdiagramm Abbildung 6.3 stellt die möglichen Fälle dar. Es können beliebig viele `elif`-Blöcke verwendet werden und das finale `else` kann auch weggelassen werden. Damit steht uns eine sehr flexible Möglichkeit für Verzweigungen zur Verfügung.

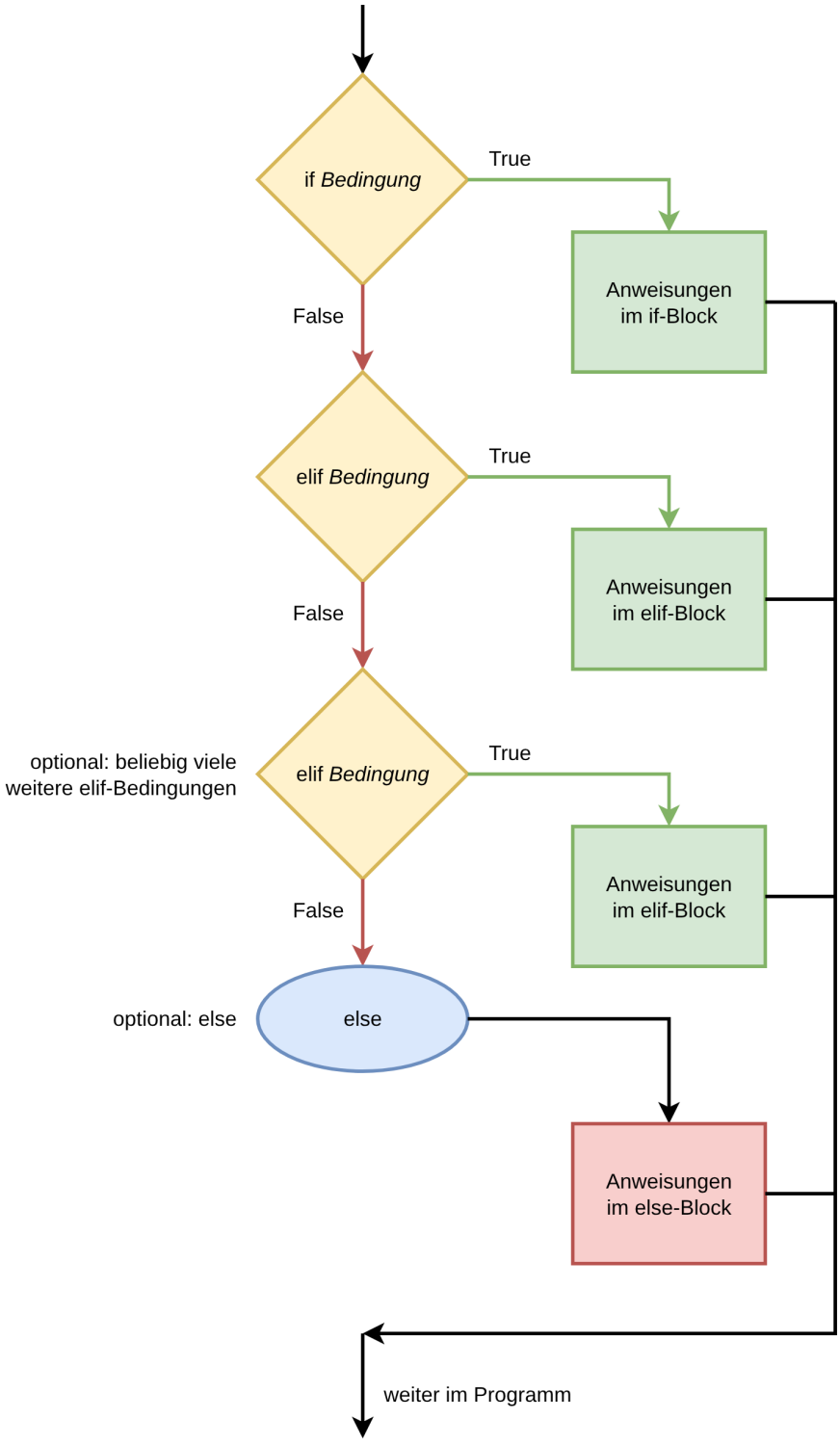


Abbildung 6.3.: Flussdiagramm zur Verzweigung mit if, elif und else

```python
# Grenzwerte für die Motorüberwachung (in °C)
normal_temp = 80  # normale Betriebstemperatur
warning_temp = 100  # Warnschwelle
danger_temp = 120  # kritische Temperatur

# Eingabe: gemessene Motortemperatur
temperature = float(input("Aktuelle Motortemperatur in °C: "))

# Bewertung der Temperatur
if temperature < 0:
    print("Fehler: Ungültige Temperatur. Bitte Sensor prüfen.")
elif temperature <= normal_temp:
    print("Temperatur im sicheren Bereich.")
elif temperature <= warning_temp:
    print("Achtung: Temperatur erhöht. Kühlung überprüfen.")
elif temperature <= danger_temp:
    print("Kritische Warnung! Motor überhitzt!")
    #hier folgt Code, der eine zusätzliche Kühlung aktiviert
else:
    print("Motor wird abgeschaltet!")
    #hier folgt Code, der den Motor sofort abschaltet
```

```
Aktuelle Motortemperatur in °C:  110

Kritische Warnung! Motor überhitzt!
```

Die if-Anweisung in anderen Programmiersprachen

Die `if`-Anweisung ist ein universelles Werkzeug der Programmierung, das Sie in vielen Programmiersprachen wiederfinden werden. Beim Wechsel zu einer anderen Programmiersprache müssen Sie sich mit der jeweiligen Syntax vertraut machen. Python verpflichtet Sie z.B. zu Einrückungen, was in anderen Programmiersprachen eher unüblich ist (zu Zwecken der Übersichtlichkeit aber trotzdem gemacht wird). In C verwendet man geschweifte Klammern, um die Blöcke voneinander abzugrenzen. Hier ein Beispiel:

```c
    int temperatur = 25;

    if (temperatur > 30) {
        printf("Es ist heiß!\n");
    } else if (temperatur > 20) {
        printf("Angenehmes Wetter.\n");
    } else {
        printf("Es ist kühl.\n");
    }
```

Sie erkennen sicherlich die große Ähnlichkeit zu dem, was Sie in Python kennengelernt haben, aber auch Unterschiede. Zum Beispiel schreibt man in C `else if` statt des verkürzten `elif`.

Conditional Expressions

Python bietet außerdem die sog. Conditional Expressions (in C und verwandten Sprachen gibt es dafür den Operator `bedingung ? wert1 : wert2`). Sie sind eine elegante Art, Code kompakt darzustellen, wenn eine Zuweisung abhängig von einer `if`-Bedingung stattfinden soll. Dazu betrachten wir zunächst ein Beispiel als traditionelle Verzweigung mit `if` und `else`:

```python
if voltage > 1000:
    classification = "Hochspannung" # nach DIN VDE 0100: 1000 V AC
else:
    classification = "Niederspannung"
```

Die kompakte Schreibweise als Conditional Expression sieht dagegen so aus:

```python
classification = "Hochspannung" if voltage > 1000 else "Niederspannung"
```

Die Conditional Expression ist leicht zu verstehen, da sie wie ein englischer Satz gelesen werden kann. Man kann den gesamten Ausdruck zusätzlich in Klammern setzen, nötig ist das aber nicht.

Übungen:

1. Implementieren Sie einen Türsteher vor einem Club. Er fragt die Besucher nach dem Alter und reagiert entsprechend.
2. Schreiben Sie Code, der den BMI einer Person berechnet und professionelles Feedback gibt.

3. Schreiben Sie ein Programm, das einen primitiven Taschenrechner darstellt: Der Benutzer gibt zwei Zahlen ein, wählt eine Grundrechenart aus und erhält das gewünschte Ergebnis.

Zusammengesetzte Bedingungen mit *and*, *or* und *not*

Wir wiederholen: Mit den Operatoren `and`, `or` und `not` können komplexere logische Bedingungen aufgebaut werden. Die Bedeutung der Operatoren ist aus der Aussagenlogik bekannt:

- `and`: "A und B" ist wahr, wenn beide Bedingungen wahr sind.

- `or`: "A oder B" ist wahr, wenn mindestens eine der Bedingungen wahr ist.
- `not`: "nicht A" kehrt den Wahrheitswert einer Bedingung um (wahr wird falsch und falsch wird wahr).

```
age = 25
has_license = True

# "Führerschein und Fahrzeugpapiere bitte!"
if age >= 18 and has_license:
    print("Sie dürfen fahren!")
else:
    print("Steigen Sie mal bitte aus...")
```

```
Sie dürfen fahren!
```

Bei der Verknüpfung von mehr als zwei Bedingungen empfiehlt sich die Verwendung von Klammern. Ohne Klammern gelten die Prioritätsregeln:

- `not` hat die höchste Priorität, es bindet also am stärksten.
- `and` hat eine mittlere Priorität: `a or b and c` bedeutet `a or (b and c)`.
- `or` hat die niedrigste Priorität.

Die Priorität legt fest, wie Python einen Ausdruck gedanklich klammert, nicht in welcher Reihenfolge ausgewertet wird. Ausgewertet wird von links nach rechts, und Python hört auf, sobald das Ergebnis feststeht (Kurzschlussauswertung): Bei `a or b` wird `b` gar nicht mehr betrachtet, wenn `a` bereits wahr ist.

Im folgenden Beispiel bräuchte man die Klammern nicht, aber sie unterstützen die Lesbarkeit und beugen Missverständnissen vor:

```python
temperature = 25
is_sunny = True
is_holiday = False

# and bindet stärker als or
if (temperature > 20 and is_sunny) or is_holiday:
    print("Ich bin happy!")
else:
    print("Ein Tag wie jeder andere.")
```

```
Ich bin happy!
```

Im folgenden Beispiel bezieht sich `not` nur auf `is_guest`, weil es am stärksten bindet:

```python
is_guest = False
is_admin = False
data_updated = True

if not is_guest and (is_admin or data_updated):
    print("Zugriff gewährt!")
else:
    print("Zugriff verweigert!")
```

```
Zugriff gewährt!
```

Zugriff wird hier nur gewährt, wenn es sich **nicht** um einen Gast handelt. Darüber hinaus muss es ein Admin oder ein User mit aktualisierten Daten sein.

6.2. Schleifen

Mit Schleifen können wir Anweisungen wiederholen, ohne sie erneut hinschreiben zu müssen. Die Anweisungen, die wiederholt ausgeführt werden sollen, stehen in einem eingerückten *Schleifenkörper* (so wie die Anweisungen in `if`- oder `else`-Blöcken). In Python gibt es zwei grundlegende Schleifentypen, die es in ähnlicher Weise auch in vielen anderen Programmiersprachen gibt.

While-Schleife

Bei der `while`-Schleife (engl. while-loop) legen wir eine Bedingung fest. Nur wenn diese Bedingung erfüllt ist, werden die Anweisungen im Schleifenkörper ausgeführt. Nach jedem Durchlauf wird die Bedingung erneut geprüft und der Schleifenkörper ggf. ein weiteres Mal durchlaufen. Sie ahnen vielleicht schon die Gefahr dieses Schleifentyps: Wenn die Bedingung immer wahr ist, kommt das Programm nie mehr aus der Schleife raus! Sie haben dann eine sog. *Endlosschleife* erschaffen, ein gefürchteter Schrecken in der Programmierung. Denken Sie daher gut über Ihre Schleifenbedingungen nach!

Das geschilderte Verhalten können Sie auch im Flussdiagramm der `while`-Schleife nachvollziehen (Abbildung 6.4).

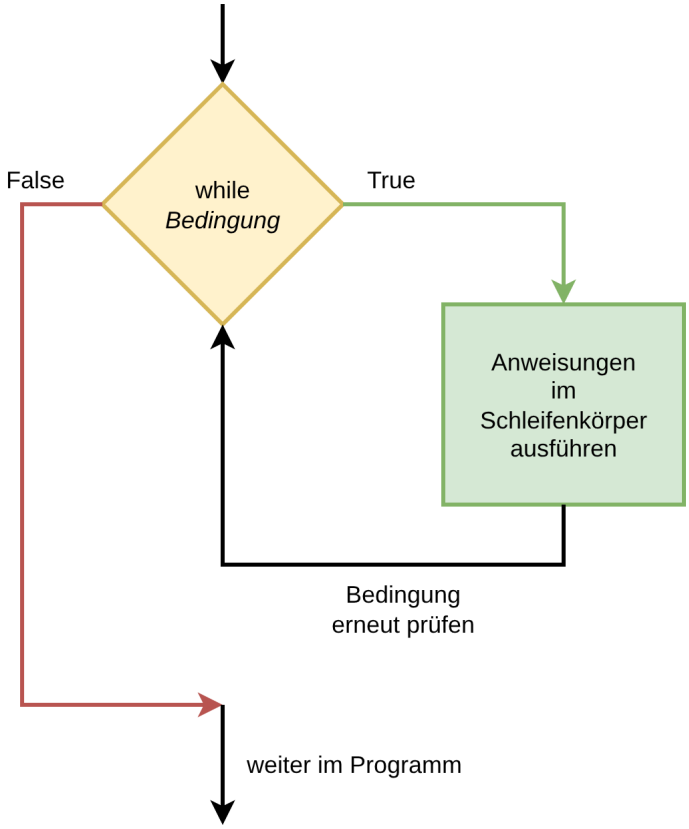


Abbildung 6.4.: Flussdiagramm zur while-Schleife

Betrachten wir ein Beispiel, an dem Sie Syntax und Funktionsweise der `while`-Schleife lernen können:

```python
water_level = 0  # Startfüllstand in Liter
max_water_level = 80  # Kapazität des Wassertanks in Liter
flow_rate = 10  # Befüllung pro Schritt in Liter

while water_level < max_water_level:
    water_level += flow_rate
    print("Wassertank wird befüllt... Aktueller Stand:", water_level)


print("Der Tank ist voll! Befüllung gestoppt.")
```

```
Wassertank wird befüllt... Aktueller Stand: 10
Wassertank wird befüllt... Aktueller Stand: 20
Wassertank wird befüllt... Aktueller Stand: 30
Wassertank wird befüllt... Aktueller Stand: 40
Wassertank wird befüllt... Aktueller Stand: 50
Wassertank wird befüllt... Aktueller Stand: 60
Wassertank wird befüllt... Aktueller Stand: 70
Wassertank wird befüllt... Aktueller Stand: 80
Der Tank ist voll! Befüllung gestoppt.
```

For-Schleife

Die `for`-Schleife hat einen anderen Charakter als die `while`-Schleife: Sie wird verwendet, um eine Sequenz (z.B. eine Liste, eine Zeichenkette oder eine Zahlenreihe) Schritt für Schritt zu durchlaufen. Für jedes Element der Sequenz wird der Schleifenkörper abgearbeitet, bis schließlich das Ende der Sequenz erreicht ist und die Schleife verlassen wird. Als Faustregel gilt: Steht vorher fest, worüber bzw. wie oft wiederholt wird, nimmt man `for`; soll wiederholt werden, bis eine Bedingung eintritt (z.B. bis eine gesuchte Zahl erraten wurde), nimmt man `while`.

Die grundlegende Syntax sieht so aus:

```python
for var in sequence:
    # Codeblock, der für jedes Element der Sequenz ausgeführt wird.
    #
    # var ist beim ersten Durchlauf das erste Element,
    # beim zweiten Durchlauf das zweite Element,
    # ...
    # beim letzten Durchlauf das letzte Element.
```

Bei einer for-Schleife weist Python der Schleifenvariable (`var` im Syntaxblock oben) **automatisch** das nächste Element der Sequenz zu: Beim ersten Eintritt in die Schleife ist dies das erste Element (*Initialisierung*), nach jedem Durchlauf des Schleifenkörpers das jeweils folgende Element. Jeden dieser Durchläufe nennt man eine *Iteration*. Das Flussdiagramm (Abbildung 6.5) ähnelt deshalb dem der while-Schleife (Abbildung 6.4): An die Stelle einer selbst formulierten Bedingung tritt die Frage, ob noch ein Element übrig ist. Neu ist der Kasten *var = nächstes Element*: Diesen Schritt führt Python vor jedem Durchlauf des Schleifenkörpers automatisch aus.

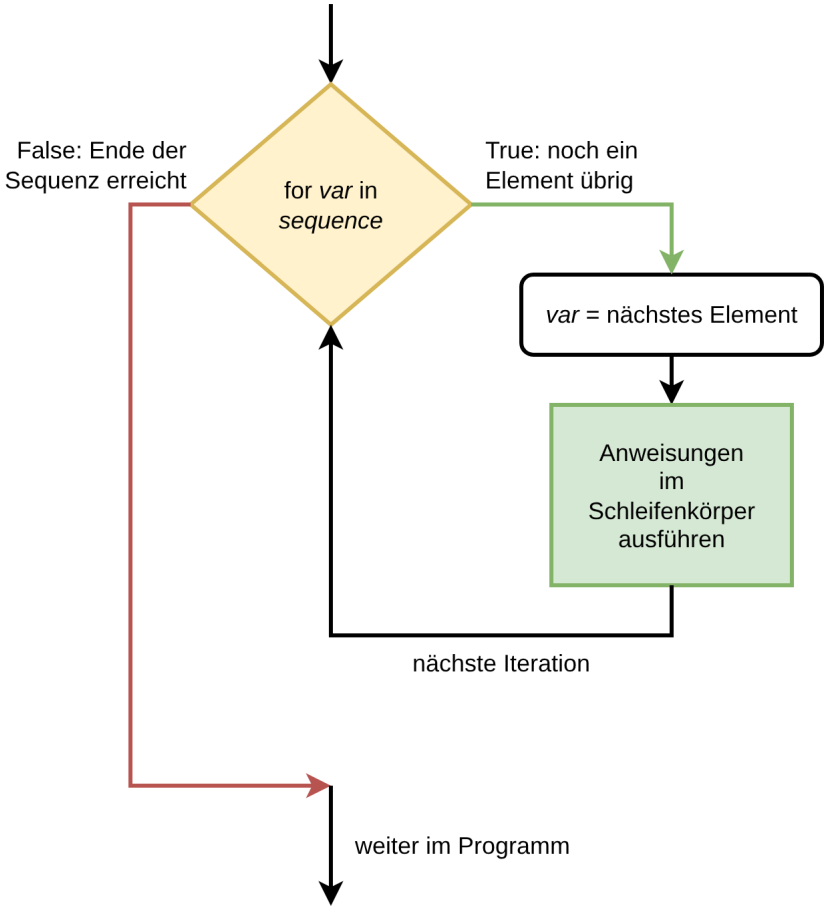


Abbildung 6.5.: Flussdiagramm zur for-Schleife

Am besten wir betrachten ein paar Beispiele:

```python
# Wir buchstabieren ein Wort:
# "Apfel" ist hierbei die Sequenz
for letter in "Apfel":
    print(letter)
```

```
A
p
f
e
l
```

```python
# Wir gehen eine Liste durch und prüfen eine Bedingung
sensor_values = [12.4, 15.7, 19.3, 21.2, 13.5, 7.8, 23.2]
critical_threshold = 20.0

for value in sensor_values:
    if value > critical_threshold:
        print("Kritischer Messwert gefunden:", value, "V")
```

```
Kritischer Messwert gefunden: 21.2 V
Kritischer Messwert gefunden: 23.2 V
```

Eine häufige Anwendung von `for`-Schleifen sind sog. Zählschleifen, bei denen eine feste Anzahl an Wiederholungen vorgegeben wird. Dies erreicht man in Python durch die Funktion `range()`:

```python
# range(5) erzeugt die Sequenz 0 - 4
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

Beachten Sie, dass die von `range(n)` erzeugte Sequenz bei 0 beginnt und bei n-1 endet. Die Buchstaben `i` und `j` werden gerne als Zählvariablen in Kombination mit `range` verwendet. Die `range()`-Funktion bietet verschiedene Optionen, um die erzeugte Sequenz zu beeinflussen. So können Sie neben dem End-Wert auch einen Start-Wert angeben:

```
# range(3,10) erzeugt die Sequenz 3 - 9
for i in range(3, 10):
    print(i)
```

```
3
4
5
6
7
8
9
```

Und was macht der folgende Code?

```
for i in range(3, 10, 2):
    print(i)
```

```
3
5
7
9
```

Wir fassen die drei Varianten von `range()` zusammen:

- `range(stop)` Sequenz von `0` bis `stop-1`
- `range(start, stop)` Sequenz von `start` bis `stop-1`
- `range(start, stop, step)` Sequenz von `start` bis maximal `stop-1` mit der Schrittweite `step`.

In der dritten Variante steht "bis maximal `stop-1`", weil nicht mehr garantiert werden kann, dass `stop-1` angenommen wird: Es wird jeweils `step` hinzuaddiert und damit könnte auch `stop-1` übersprungen werden. Genauer: Bei positiver Schrittweite endet die Sequenz vor dem ersten Wert, der `>= stop` ist, bei negativer Schrittweite (siehe unten) vor dem ersten Wert, der `<= stop` ist. Der Wert `stop` selbst ist also nie enthalten.

Übrigens kann man mit `range()` auch rückwärts zählen:

```
for i in range(10, 0, -3):
    print(i)
```

```
10
7
4
1
```

Wir wollen noch kurz untersuchen, welchen Datentyp die Funktion `range` zurückgibt:

```
print(type(range(10)))
```

```
<class 'range'>
```

Wie wir sehen, erzeugt `range` keine Liste, sondern einen eigenen Datentyp, der ebenfalls `range` heißt. Dieser Datentyp ist für die Zählschleife optimiert: Im Gegensatz zu einer Liste speichert er nicht alle Werte ab, sondern erzeugt diese *on-the-fly*, also dann, wenn sie gebraucht werden. Eine Liste mit 1 Million Zahlen würde viel Speicher brauchen; `range` dagegen speichert nur drei Werte (`start`, `stop` und `step`). Die Berechnung der jeweils nächsten Zahl erfolgt in C und ist daher sehr schnell!

For-Schleifen in anderen Programmiersprachen

In den meisten anderen Programmiersprachen ähnelt die for-Schleife von vornherein eher einer Zählschleife. In C sieht das z.B. so aus:

```
for (int i = 0; i < 10; i++) {
    // Schleifenkörper für
    // i von 0 bis 9
}
```

In der runden Klammer erkennt man drei Bestandteile:

- `int i = 0` initialisiert die Zählvariable `i`
- `i < 10` definiert die Bedingung, wie lange die Schleife ausgeführt werden soll
- `i++` bedeutet, dass die Zählvariable nach jedem Durchgang um eins erhöht wird.

Das Python-Analogon wäre also:

```
for i in range(10):
```

Übungen: Gegeben sei eine Liste mit Messwerten `data=[0.2, -1.3, 2.7, 0.6, 9.9, 8.2]`.

1. Implementieren Sie eine for-Schleife, die die Summe aller Messwerte berechnet.
2. Schreiben Sie Code, der den Durchschnitt aller Messwerte berechnet.
3. Schreiben Sie Code, der das Maximum der Messwerte findet.

Lösung (alle drei Aufgaben in einem Code):

```
data = [0.2, -1.3, 2.7, 0.6, 9.9, 8.2]
sum_result = 0  # Summe initialisieren mit Null
max_value = data[0]  # Maximum initialisieren mit erstem Element

for value in data:
    sum_result += value  # aufaddieren
    if value > max_value:  # Maximum suchen
        max_value = value

# Für den Durchschnitt brauchen wir noch
# die Anzahl der Elemente. Diese erhält
# man mit len(data):
average = sum_result / len(data)

print("Summe: ", sum_result,"Durchschnitt: ", average, "Maximum: ",
 ↪  max_value)
```

Den Schleifenfluss kontrollieren mit *break* und *continue*

break – Schleife vorzeitig beenden

Sobald in einer Schleife das Keyword `break` verwendet wird, bricht die Schleife an Ort und Stelle ab. Das funktioniert sowohl in `while`- als auch in `for`-Schleifen. Betrachten Sie das folgende Ratespiel - können Sie den Nutzen von `break` nachvollziehen?

```
import random
secret_number = random.randint(0,100) #erzeugt eine Zufallszahl

guess = -1  # guess wird auf -1 initialisiert

while guess != secret_number:

    guess = int(input("Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für
 ↪  Abbruch): "))
```

```
    if guess == -1:
        print("Abbruch.")
        break  # Die Schleife wird abgebrochen

    # Falls das Spiel fortgesetzt wird, geben wir Tipps:
    if secret_number < guess:
        print("Zu groß!")
    elif secret_number > guess:
        print("Zu klein!")
    else:
        print("Gratulation!")  # Geheime Zahl erraten!
```

```
Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch):  34

Zu klein!

Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch):  78

Zu klein!

Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch):  89

Zu klein!

Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch):  99

Zu groß!

Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch):  91

Zu klein!

Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch):  102

Zu groß!

Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch):  -3
```

```
Zu klein!

Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch):  -1

Abbruch.
```

Ein Beispiel für `break` in einer `for`-Schleife:

```python
bag = ["Schlüssel", "Handy", "Brille", "Geldbörse", "Stift"]  # Dinge in der
↪ Tasche
search_item = "Geldbörse"  # Der Gegenstand, den wir suchen

for item in bag:
    print("Suche nach ", search_item, " läuft. Aktueller Gegenstand: ", item)

    if item == search_item:
        # Die Schleife wird abgebrochen,
        # da der Gegenstand gefunden wurde:
        print(search_item, "gefunden!")
        break
```

```
Suche nach  Geldbörse  läuft. Aktueller Gegenstand:  Schlüssel
Suche nach  Geldbörse  läuft. Aktueller Gegenstand:  Handy
Suche nach  Geldbörse  läuft. Aktueller Gegenstand:  Brille
Suche nach  Geldbörse  läuft. Aktueller Gegenstand:  Geldbörse
Geldbörse gefunden!
```

continue - Aktuellen Schleifendurchlauf vorzeitig abbrechen, weiter mit dem nächsten!

Das Keyword `continue` bricht nur den aktuellen Schleifendurchlauf an Ort und Stelle ab, aber nicht die ganze Schleife. Es geht einfach direkt mit dem nächsten Durchlauf weiter. Wir erweitern das Ratespiel von oben durch ein `continue` bei ungültiger Eingabe:

```python
import random
secret_number = random.randint(0,100)  # erzeugt eine Zufallszahl
guess = -1

while guess != secret_number:

    guess = int(input("Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für
↪ Abbruch): "))
```

```
    if guess > 100 or guess < -1:
        # Eingabe außerhalb gültiger Grenzen
        print("Ungültig!")
        # Durchgang abbrechen und nächste Runde:
        continue

    if guess == -1:
        # Die Schleife wird abgebrochen
        print("Abbruch.")
        break

    # Falls das Spiel fortgesetzt wird, geben wir Tipps:
    if secret_number < guess:
        print("Zu groß!")
    elif secret_number > guess:
        print("Zu klein!")
    else:
        print("Gratulation!")  # Geheime Zahl erraten!
```

```
Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch): 102
Ungültig!
Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch): -3
Ungültig!
Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch): 23
Zu klein!
Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch): 54
Zu groß!
Raten Sie die geheime Zahl zwischen 0 und 100 (-1 für Abbruch): -1
Abbruch.
```

Auch hierzu ein Beispiel mit `for` (es illustriert eine einfache Datenbereinigung):

```
user_inputs = ["23", "abc", "45", "-1", "67", "150", "", "89", "0", "xyz"]
cleaned_data = []  # Liste für bereinigte Eingaben

for input_str in user_inputs:

    # 1. Überspringe nicht-numerische Eingabe
    # (Achtung: isdigit() erkennt nur Ziffern - auch "-1" oder "3.5" werden
    ↪ übersprungen!)
```

```
    if not input_str.isdigit():
        continue

    # 2. Wandle die Eingabe in eine Zahl um
    input_number = int(input_str)

    # 3. Überspringe Werte außerhalb gültiger Grenzen
    if input_number < 10 or input_number > 100:
        continue

    # Füge gültige Eingabe der Liste hinzu:
    cleaned_data.append(input_number)

print("Bereinigte Eingaben:", cleaned_data)
```

```
Bereinigte Eingaben: [23, 45, 67, 89]
```

Verschachtelte Schleifen

Verschachtelte Schleifen, also **Schleifen innerhalb anderer Schleifen**, sind ein wesentliches Konstrukt in der Programmierung. Sie werden verwendet, um mehrdimensionale Strukturen (z.B. Matrizen, Tabellen, Raster) zu durchlaufen oder kombinatorische Probleme (z.B. Permutationen) zu lösen. Typische Anwendungen finden sich in der Datenanalyse, Bildbearbeitung oder Mathematik.

Python erlaubt beliebig tiefe Schachtelungen von Schleifen. Jede innere Schleife wird bei jedem Durchlauf der äußeren Schleife vollständig ausgeführt. Das entspricht der ganz normalen Funktionsweise von Schleifen: Der Schleifenkörper wird für jede Iteration abgearbeitet, ganz egal, was drin steht (im Fall von verschachtelten Schleifen eine weitere Schleife).

```
for i in range(3):       # äußere Schleife
    for j in range(2):   # innere Schleife
        print(f"i={i}, j={j}")
```

```
i=0, j=0
i=0, j=1
i=1, j=0
i=1, j=1
i=2, j=0
i=2, j=1
```

Die innere Schleife wird für jeden Wert der äußeren Schleife komplett durchlaufen (dadurch kommt es zu insgesamt 3 * 2 = 6 Aufrufen von `print`). Dieses Schema könnte zum Beispiel genutzt werden, um eine Tabelle, bestehend aus drei Zeilen (Index `i`) und zwei Spalten (Index `j`), einzulesen oder zu bearbeiten.

> **Vorsicht**
>
> `break` und `continue` beziehen sich immer auf die innerste Schleife, in der sie stehen!

Beispiele zu `break` und `continue` bei verschachtelten Schleifen:

```
for i in range(3):
    for j in range(5):
        if j == 2:
            break  # verlässt nur die innere Schleife
        print(f"i={i}, j={j}")
```

```
i=0, j=0
i=0, j=1
i=1, j=0
i=1, j=1
i=2, j=0
i=2, j=1
```

```
for i in range(3):
    for j in range(3):
        if j == 1:
            continue  # überspringt j==1
        print(f"i={i}, j={j}")
```

```
i=0, j=0
i=0, j=2
i=1, j=0
i=1, j=2
i=2, j=0
i=2, j=2
```

Übung: In eine Namensliste haben sich manche Namen doppelt eingeschlichen. Diese sollen mithilfe von verschachtelten Schleifen gefunden werden. Gewünschte Ausgabe: Doppelte

Namen mit ihren Indizes. Sie können voraussetzen, dass kein Name häufiger als zweimal vorkommt! Testdaten: `names = ["Anna", "Chris", "Fabiola", "Clara", "Chris", "Ben", "Dora", "Ben", "Anna"]`.

Tipp, falls Sie Schwierigkeiten mit den korrekten Schleifen haben:

```
names = ["Anna", "Chris", "Fabiola", "Clara", "Chris", "Ben", "Dora", "Ben",
 ↪  "Anna"]

for i in range(len(names)):
    for j in range(i + 1, len(names)):
        if ...:  # Bedingung ergänzen
            # Ausgabe ergänzen
            break  # Keine weiteren Vergleiche für diesen Namen nötig
```

6.3. Anwendungen in der Mathematik

Mit den Werkzeugen, die uns jetzt zur Verfügung stehen, können wir bereits interessante Berechnungen durchführen und praktische Algorithmen implementieren. Wir illustrieren dies mit zwei Anwendungsfällen:

- Das **Newton-Verfahren** ist ein Algorithmus aus der numerischen Mathematik, um Nullstellen einer Funktion zu finden. Es wird auch angewendet, um Gleichungen zu lösen (dafür werden die Gleichungen so umformuliert, dass ein Nullstellenproblem vorliegt) oder Extremstellen zu identifizieren (das Verfahren sucht nach den Nullstellen der ersten Ableitung).
- **Lineare Algebra** arbeitet mit Vektoren und Matrizen. Der Datentyp `list` eignet sich zur Modellierung dieser Objekte. Mithilfe von Schleifen können alle elementaren Operationen (z.B. Skalarprodukt oder Matrix-mal-Vektor) implementiert werden.

Das Newton-Verfahren

Aufgabe: Implementieren Sie das Newton-Verfahren (siehe Wikipedia), welches nach einer Nullstelle einer Funktion $f(x)$ sucht. Testen Sie es für die Funktion $f(x) = x^3 + x^2 - 2\,x - 1$ und probieren Sie dabei verschiedene Startwerte aus.

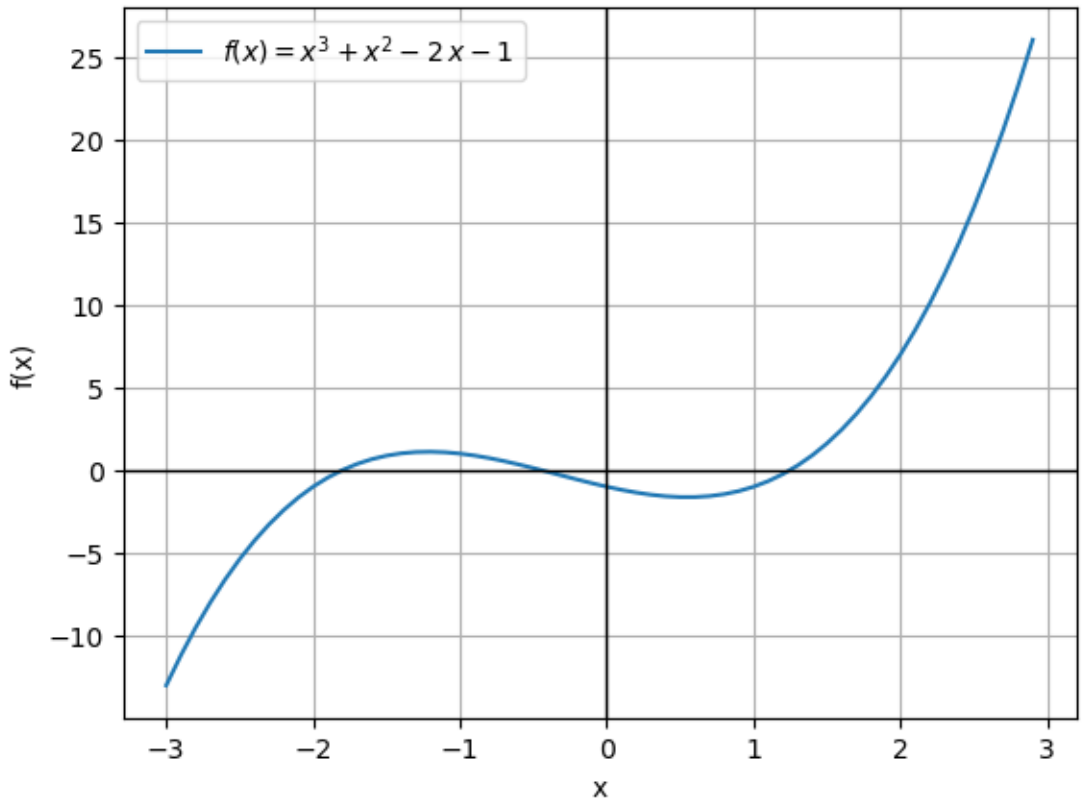
Hinweis: Da das Verfahren nicht immer konvergiert, sollten Sie eine maximale Zahl an Iterationen festlegen und danach das Verfahren abbrechen.

Wir erzeugen zunächst einen Plot der Funktion (den Code hierzu müssen Sie noch nicht nachvollziehen können):

```
import matplotlib.pyplot as plt
import numpy as np
xs = np.arange(-3, 3, 0.1)
ys = xs**3 + xs**2 - 2 * xs-1
plt.plot(xs, ys, label=r"$f(x)=x^3+x^2-2\,x-1$")

plt.axhline(0, color="black", linewidth=1)  # x-Achse
plt.axvline(0, color="black", linewidth=1)  # y-Achse
plt.xlabel("x")
plt.ylabel("f(x)")
plt.legend()
plt.grid(True)

plt.show()
```



Jetzt implementieren wir das Verfahren:

```
max_iter = 100  # maximale Zahl der Iterationen
x = 1.0  # Startwert
tol = 1e-6  # Toleranz

for i in range(max_iter):

    # Funktion:
    f = x**3 + x**2 - 2 * x - 1

    # Ableitung:
    fp = 3 * x**2 + 2 * x - 2

    if abs(f) < tol:
        # Funktionswert ist nahe genug bei Null
        print("Konvergiert nach ", i, "Iterationen.")
        break

    x = x - f / fp  # Iteration des Newton-Verfahrens

if abs(f) >= tol:
    print("Keine Konvergenz nach", max_iter, "Iterationen!")

print("x=", x, "f(x)=", f)
```

Bemerkung: Eine alternative Abbruchbedingung betrachtet die Änderung in x von Schritt zu Schritt. Sobald sich x nicht mehr signifikant ändert, bricht das Verfahren ab. Wir gehen an dieser Stelle nicht auf die Vor- und Nachteile der jeweiligen Abbruchbedingung ein.

Lineare Algebra mit Listen

Vektoren und Matrizen können mit Python-Listen dargestellt werden. Mithilfe von Schleifen können elementare Operationen der Linearen Algebra programmiert werden. Wir beginnen mit zwei Vektoren:

```
v = [1.0, -2.0, 1.5] # Vektor v
w = [-1.0, 1.0, 0.5] # Vektor w
```

Vektoraddition

```
v_plus_w = [0,0,0] # Initialisierung

for i in range(3):
    v_plus_w[i] = v[i] + w[i]

print(v_plus_w)
```

```
[0.0, -1.0, 2.0]
```

Was wäre eigentlich geschehen, wenn wir `v + w` ausprobiert hätten? Vergessen Sie nicht, dass `v` und `w` Listen sind und als solche keine natürliche Addition kennen. Stattdessen konkateniert der Plus-Operator die beiden Listen:

```
v_concat_w = v + w
print(v_concat_w)  # offensichtlich keine Vektoraddition
```

```
[1.0, -2.0, 1.5, -1.0, 1.0, 0.5]
```

Skalarprodukt

Das Skalarprodukt ist eine wichtige Operation in der Linearen Algebra, mit der z.B. überprüft werden kann, ob zwei Vektoren senkrecht aufeinander stehen (Skalarprodukt gleich Null). Implementierung:

```
v_dot_w = 0  # Initialisierung

for i in range(3):
    v_dot_w += v[i] * w[i]

print(v_dot_w)
```

```
-2.25
```

Solche Schleifen über zwei Listen mit gemeinsamem Index lassen sich mit der Funktion `zip()` noch eleganter schreiben – siehe Abschnitt 8.9 im Kapitel *Datenstrukturen*.

Matrizen

Eine Matrix kann als verschachtelte Liste dargestellt werden. Man kann sich das so vorstellen, dass die Elemente der äußeren Liste die Zeilen der Matrix sind (jede Zeile ist wiederum eine Liste ihrer Elemente):

```
# innerhalb der eckigen Klammern sind
# Zeilenumbrüche erlaubt, was die
# Lesbarkeit als Matrix erhöht:
M = [
    [1.0, 0.0, 1.0],   # erste Zeile
    [0.0, -1.0, 0.0],  # zweite Zeile
    [2.0, 0.0, 0.0]    # dritte Zeile
]
print(M)
```

```
[[1.0, 0.0, 1.0], [0.0, -1.0, 0.0], [2.0, 0.0, 0.0]]
```

Auf die Elemente der Matrix kann über Indizes zugegriffen werden:

```
M[0]  # ist die erste Zeile
M[0][0]  # erste Zeile, erste Spalte
M[0][1]  # erste Zeile, zweite Spalte
M[0][2]  # erste Zeile, dritte Spalte

print(M[2][0])  # dritte Zeile, erste Spalte
```

```
2.0
```

Matrixaddition

```
M = [
    [1.0, 0.0, 1.0],   # erste Zeile
    [0.0, -1.0, 0.0],  # zweite Zeile
    [2.0, 0.0, 0.0]    # dritte Zeile
]

N = [
    [1.0, 0.5, 1.0],   # erste Zeile
```

```
    [2.0, 1.0, 2.0],  # zweite Zeile
    [-1.0, -1.0, 1.0]    # dritte Zeile
]
```

```
M_plus_N = [           # Initialisierung
    [0.0, 0.0, 0.0],   # erste Zeile
    [0.0, 0.0, 0.0],   # zweite Zeile
    [0.0, 0.0, 0.0]    # dritte Zeile
]

for i in range(3):     # i iteriert über die Zeilen
    for j in range(3): # j iteriert über die Spalten
        M_plus_N[i][j] = M[i][j] + N[i][j]

print(M_plus_N)  # überprüfen Sie das Ergebnis
```

```
[[2.0, 0.5, 2.0], [2.0, 0.0, 2.0], [1.0, -1.0, 1.0]]
```

Machen Sie sich die Funktionsweise der verschachtelten for-Schleifen klar: Der *Zeilen*index `i` iteriert von 0 bis 2. Für jede Zeile, also jeden Wert von `i`, startet der *Spalten*index `j` jeweils wieder bei 0 und iteriert bis 2. So wird jede Kombination von `i` und `j` durchlaufen.

Matrix mal Vektor

```
M_dot_v = [0, 0, 0]  # Initialisierung

for i in range(3):
    for j in range(3):
        M_dot_v[i] += M[i][j] * v[j]

print(M_dot_v)  # rechnen Sie nach!
```

```
[2.5, 2.0, 2.0]
```

Übungen:

1. Implementieren Sie die Multiplikation von zwei Matrizen.
2. Implementieren Sie eine Schleife, die die Transponierte einer Matrix berechnet.

7. Funktionen

Lernziele

Nach diesem Kapitel können Sie …

- erklären, was eine Funktion ist, und wichtige Built-in Functions gezielt einsetzen
- eigene Funktionen mit Parametern, optionalen Parametern und Rückgabewerten schreiben
- Funktionen mit Positions- und Keyword-Argumenten aufrufen und Funktionen mit beliebig vielen Parametern (`*args`) schreiben
- Risiken der dynamischen Typisierung erkennen und Typen mit `isinstance` prüfen
- den Gültigkeitsbereich (Scope) von Variablen erklären
- vorhersagen, wie sich mutable Argumente in einer Funktion verhalten
- Funktionen mit Docstrings und Typ-Annotationen dokumentieren

Funktionen sind ein zentrales Konzept der Programmierung, das es ermöglicht, Code in wiederverwendbare Einheiten zu gliedern. Sie tragen dazu bei, Programme übersichtlicher und wartbarer zu gestalten, und vermeiden insbesondere redundanten Code. Wann immer man geneigt ist, eine Codezeile oder einen Block zu kopieren, sollte man darüber nachdenken, ob eine Funktion sinnvoll ist. In diesem Kapitel werden Funktionen allgemein und ihre Anwendung in Python im Speziellen vorgestellt.

7.1. Prozedurale Programmierung

Bisher haben wir ausschließlich in der sog. **imperativen Programmierung** gearbeitet: Dabei wird der Programmablauf durch eine Abfolge von Befehlen gesteuert, die der Reihe nach abgearbeitet werden (der Kontrollfluss wird insbesondere durch Verzweigungen und Schleifen beeinflusst).

Die **prozedurale Programmierung** ist ein Ansatz innerhalb der imperativen Programmierung, bei dem das Programm als eine Reihe von Prozeduren oder Funktionen organisiert wird. Durch diese Aufteilung können komplexe Abläufe in einzelne, handhabbare Teile zerlegt werden, was zu einem klareren und besser wartbaren Code führt.

Was ist eine Funktion?

Eine Funktion ist ein in sich geschlossener Block von Anweisungen, der eine bestimmte Aufgabe erfüllt. Vergleichen Sie eine Funktion mit einem Kochrezept: Das Rezept beschreibt eine schrittweise Anleitung zur Zubereitung eines Gerichts. Wann immer Sie das Gericht kochen, verwenden Sie dasselbe Rezept wieder und wieder. In der Programmierung würden wir sagen: "Sie rufen dieselbe Funktion immer wieder auf." Von dem Rezept benötigen Sie aber nur ein einziges Exemplar (ganz egal, wie oft Sie das Gericht kochen werden). So ist es auch mit dem Code, der in einer Funktion definiert wird: Sie schreiben ihn ein einziges Mal und verwenden ihn dann beliebig oft.

Manchmal wird auch eine Analogie zu mathematischen Funktionen $f(x)$ gezogen. Diese Analogie hat aber einige Schwächen: Die mathematische Funktion bekommt immer einen *Input* (hier x) und gibt immer einen Wert als *Output* ($y = f(x)$) zurück. Das muss bei Funktionen in der Programmierung nicht sein! Eine Funktion in der Programmierung kommt auch ohne Input und Output aus, wie wir gleich sehen werden. Es liegt ganz in Ihrer Freiheit, ob Ihre Funktionen mit Inputs und Outputs arbeiten oder nicht. Sie können sogar eine Funktion programmieren, die beliebig viele Inputs verarbeiten kann (eine solche Funktion haben wir bereits kennengelernt, wissen Sie, welche?).

Bevor wir konkret werden, fassen wir die Vorteile von Funktionen zusammen:

- **Modularisierung:** Zerlegung des Codes in kleine, verständliche Einheiten.
- **Wiederverwendbarkeit:** Einmal definierte Funktionen können mehrfach eingesetzt werden.
- **Lesbarkeit und Wartbarkeit:** Klar strukturierter Code erleichtert das Verständnis und die Fehlersuche.
- **Vermeidung von Redundanz:** Gleicher Code muss nicht mehrfach geschrieben werden, sondern kann über Funktionsaufrufe wiederverwendet werden. Insbesondere von Vorteil, wenn Sie etwas ändern müssen: Sie müssen nur eine Stelle korrigieren, egal wie oft die Funktion aufgerufen wird.

7.2. Built-in Functions

In Python gibt es eine Vielzahl von *Built-in Functions* (etwa 70 Funktionen, je nach Version). Das sind Funktionen, die ohne zusätzliche Importe direkt verwendet werden können. Viele davon haben wir bereits kennengelernt und wollen sie in diesem Abschnitt einmal übersichtlich darstellen (außerdem ergänzen wir ein paar nützliche Funktionen, die wir noch nicht behandelt haben):

Funktionen für Ein- und Ausgabe

`print(*args)` – erzeugt eine Konsolenausgabe (das Symbol `*` zeigt an, dass beliebig viele Argumente erlaubt sind, wie wir später noch sehen werden).

`input(prompt)` – zeigt eine Aufforderung (`prompt`) an und liest die Benutzereingabe als **String** ein.

Funktionen für Typumwandlungen (Casting)

`int(x)`, `float(x)`, `str(x)`, … – konvertiert in den entsprechenden Datentyp.

Diese Funktionen können auch genutzt werden, um leere Instanzen zu erzeugen, z.B. `str()` (ein String ohne Zeichen) oder `list()` (eine Liste ohne Elemente).

Allgemeine Informationsfunktionen

`type(x)` – gibt den Datentyp einer Instanz zurück.

`id(x)` – liefert die eindeutige Identität einer Instanz.

`help(x)` – zeigt die Hilfeseite an (kann mit Objekten oder Funktionen verwendet werden).

```
# Beispiele zur Verwendung von help:
print("Hilfe für die Funktion len:")
help(len)
print("\nHilfe für die Funktion bin:")
help(bin)
```

```
Hilfe für die Funktion len:
Help on built-in function len in module builtins:

len(obj, /)
    Return the number of items in a container.


Hilfe für die Funktion bin:
Help on built-in function bin in module builtins:

bin(number, /)
    Return the binary representation of an integer.
```

```
>>> bin(2796202)
'0b1010101010101010101010'
```

Numerische Funktionen

`abs(x)` – Betrag einer Zahl (funktioniert auch mit `complex`).

`round(x, n)` – rundet eine Zahl auf `n` Nachkommastellen. Ohne `n` entsteht ein `int`. Achtung: Genau in der Mitte rundet Python zur geraden Zahl, `round(2.5)` ergibt `2` (siehe Kapitel *Datentypen*).

```
# Beispiele zur Verwendung von abs und round:
print(abs(-7.8234))
print(abs(1+1j))  # Betrag einer komplexen Zahl
print(round(-7.8234, 2))
print(round(-7.8234, 1))
print(round(-7.8234)) # ohne Angabe von n
```

```
7.8234
1.4142135623730951
-7.82
-7.8
-8
```

Nützliche Funktionen für Sequenzen

`range(start, stop, step)` – erzeugt eine Zahlenfolge.

`len(x)` – gibt die Länge einer Sequenz zurück, z.B. Anzahl der Zeichen von einem String oder Anzahl der Elemente in einer Liste.

`sum(x)` – summiert alle Werte in einer Sequenz.

`max(x)` und `min(x)` – gibt das Maximum bzw. Minimum in einer Sequenz zurück.

`sorted(x)` – erzeugt eine sortierte Liste mit den Elementen der Sequenz und gibt diese zurück. (Hinweise: Das Original bleibt unverändert! Auch wenn man `sorted()` auf ein `set` oder `tuple` anwendet, erhält man eine Liste.)

```python
# Beispiele zur Verwendung von len, sum, max, min und sorted:
my_list = [0.5, 1, 0.25, -0.5, 2]
print("len:", len(my_list))
print("sum:", sum(my_list))
print("max:", max(my_list))
print("min:", min(my_list))
print("sorted:", sorted(my_list))
```

```
len: 5
sum: 3.25
max: 2
min: -0.5
sorted: [-0.5, 0.25, 0.5, 1, 2]
```

Funktionen für verschiedene Zahlensysteme

`bin(x)` – wandelt eine ganze Zahl in eine Binärdarstellung (z.B. `bin(10) → '0b1010'`)

`oct(x)` – wandelt eine ganze Zahl in eine Oktaldarstellung (z.B. `oct(10) → '0o12'`)

`hex(x)` – wandelt eine ganze Zahl in eine Hexadezimaldarstellung (z.B. `hex(255) → '0xff'`)

Hinweis: Diese Funktionen liefern **Strings** zurück.

Funktionen für Zeichencodierung

`ord(char)` – gibt den Unicode-Codepunkt eines Zeichens zurück, z.B. `ord('A') → 65`, `ord('ß') → 223`.

`chr(code)` – gibt das Zeichen zum angegebenen Codepunkt zurück, z.B. `chr(65) → 'A'`, `chr(223) → 'ß'`

7.3. Eigene Funktionen programmieren

Python bietet eine einfache und flexible Syntax zur Definition von Funktionen. Eine Funktion wird mit dem Schlüsselwort `def` eingeleitet, gefolgt vom Funktionsnamen (den Sie selbst festlegen). In runden Klammern werden optional Eingangsparameter definiert, dann folgt ein Doppelpunkt. Der eigentliche Funktionsblock muss eingerückt werden (das kennen Sie schon von Code-Blöcken in Verzweigungen und Schleifen). Hier ein einfaches Beispiel ohne Eingangsparameter:

```
def greeting():
    print("Hello, world!")
```

Wir haben eine sehr primitive Funktion definiert, die nichts anderes tut, als `"Hello, world!"` auszugeben, wann immer wir sie aufrufen:

```
greeting()
```

```
Hello, world!
```

Beachten Sie beim Aufruf der Funktion die runden Klammern, die gleich mehr Sinn ergeben werden.

Parameter und Argumente

Wie bereits erwähnt, können Funktionen **Parameter** entgegennehmen, um mit variablen Eingabewerten zu arbeiten. Zum Beispiel können wir unsere Begrüßungsfunktion so erweitern, dass sie einen Namen als Parameter entgegennimmt und diesen für eine personalisierte Begrüßung verwendet:

```
def greeting(name):
    print("Hallo,",name)
```

```
greeting("Anna")
```

```
Hallo, Anna
```

```
greeting("Tom")
```

```
Hallo, Tom
```

> **Argumente**
>
> Den konkreten Wert, den man an eine Funktion übergibt, nennt man **Argument**. Wir haben in unseren Beispielen bereits verschiedene Argumente (“Anna”,“Tom”) für den Parameter `name` verwendet.

Was jetzt nicht mehr funktioniert: Wir können die Funktion nicht mehr ohne ein Argument für den Parameter `name` aufrufen! Probieren Sie `greeting()` und Python wird sich beschweren, dass ein Argument fehlt! Eine besondere Möglichkeit, dieses Problem zu beheben, ist es, sog. **Standardargumente** für Parameter festzulegen. Diese werden verwendet, wenn der Anwender der Funktion kein Argument übergibt (ob das sinnvoll ist oder nicht, hängt ganz von Ihrer Anwendung ab). Wir demonstrieren die Begrüßungsfunktion mit einem Standardargument:

```
def greeting(name = "Welt"):
    print("Hallo,",name)
```

```
greeting("Lisa")
```

```
Hallo, Lisa
```

```
greeting()
```

```
Hallo, Welt
```

Der Parameter `name` wurde durch die Definition eines Standardarguments zu einem **optionalen Parameter**: Es ist nicht mehr zwingend erforderlich, ihn beim Funktionsaufruf festzulegen.

Mehrere Parameter

Selbstverständlich können wir mehr als einen Parameter verwenden:

```
def greeting(name, age):
    print("Hallo,", name)
    if age < 18:
        print("Sie sind noch minderjährig!")
    else:
        print("Willkommen im Club!")
```

```
greeting("Tom", 16)
greeting("Lisa", 21)
```

```
Hallo, Tom
Sie sind noch minderjährig!
Hallo, Lisa
Willkommen im Club!
```

Jetzt muss man allerdings aufpassen, wenn man Standardargumente verwenden will, denn es gibt eine wichtige Regel:

> Optionale Parameter immer hinter nicht-optionalen Parametern!
>
> Folgende Definition ist korrekt:
>
> ```
> def greeting(name, age = 20):
>     # Funktionsblock
>     ...
> ```
>
> Folgender Code ist **falsch** und wird nicht funktionieren:
>
> ```
> def greeting(name="Peter", age):
>     # Funktionsblock
>     ...
> ```

Rückgabewerte

Funktionen können einen Wert zurückgeben. Hierfür verwendet man (wie in C oder C++) das Keyword `return`:

```
def square(x):
    return x * x

result = square(4)
print(result)
```

```
16
```

Wir können die Funktion wieder beliebig oft mit verschiedenen Argumenten aufrufen:

```
print(square(3))
print(square(-2))
print(square(7.3))
```

```
9
4
53.29
```

Und was gibt eine Funktion ohne `return` zurück, z.B. unsere Funktion `greeting` von oben? Den speziellen Wert `None` („nichts“). Daraus ergibt sich ein typischer Anfängerfehler: Man gibt ein Ergebnis in der Funktion mit `print` aus, statt es mit `return` zurückzugeben. Das folgende Beispiel zeigt **absichtlich, wie man es nicht machen sollte**:

```
# Negativbeispiel: print statt return!
def square_print(x):
    print(x * x)  # gibt das Ergebnis nur auf dem Bildschirm aus

result = square_print(4)
print("Rückgabewert:", result)
```

```
16
Rückgabewert: None
```

Die Funktion zeigt zwar `16` an, ihr Rückgabewert ist aber `None` – mit dem Ergebnis kann man also nicht weiterrechnen. Richtig ist die Variante `square` von oben mit `return`.

> ❗ Merke: `return` beendet sofort die Funktion!
>
> Innerhalb einer Funktion können wir den Kontrollfluss mit allen uns bekannten Mitteln beeinflussen, z.B. mit Verzweigungen. Wichtig ist aber, dass die Funktion sofort verlassen wird, wenn der Interpreter auf ein `return` trifft!

Eine Besonderheit in Python ist, dass wir sogar mehr als einen Wert zurückgeben können! Das geht in vielen anderen Programmiersprachen nicht ohne Weiteres, in C oder C++ würde man dafür z.B. Datenstrukturen (als Container für mehrere Werte) oder Zeiger verwenden (die auf einen Bereich im Speicher verweisen, wo die Werte hintereinander abgelegt werden). In Python geht das ganz einfach, wie das folgende Beispiel zeigt:

```
def int_division(a, b):
    # Division mit Rest
    # zwei Rückgabewerte durch Komma getrennt:
    return a // b, a % b

a = 13
b = 3

quotient, remainder = int_division(a, b)

print(a, "geteilt durch", b ,"ergibt:")
print(quotient, "mit Rest", remainder)
```

```
13 geteilt durch 3 ergibt:
4 mit Rest 1
```

Genau genommen gibt die Funktion dabei einen einzigen Wert zurück, nämlich das Tupel `(4, 1)`, das beim Aufruf in die beiden Variablen *entpackt* wird (mehr zu Tupeln im Kapitel *Datenstrukturen*). Die Reihenfolge der Rückgabewerte ist natürlich zu beachten, sonst geschehen leicht Fehler (würde man versehentlich `remainder, quotient = int_division(a, b)` schreiben, stünde in `quotient` der Rest und in `remainder` der Quotient).

Übrigens gibt es genau diese Funktion in Python bereits eingebaut: `divmod(13, 3)` liefert `(4, 1)`. Unsere eigene Funktion haben wir bewusst anders genannt, denn eine Funktion namens `divmod` würde die eingebaute Funktion überdecken.

Keyword-Argumente

Python erlaubt uns, Funktionen mit sog. **Keyword-Argumenten** aufzurufen. Damit ist gemeint, dass die Argumente explizit benannt werden, anstatt sie anhand ihrer Reihenfolge zu identifizieren. Betrachten wir zunächst folgendes Beispiel, in dem wir eine Funktion wie gewohnt aufrufen:

```
def print_personal_details(name, age, city):
    print("Name:", name)
    print("Alter:", age)
    print("Wohnort:", city)

print_personal_details("Alice", 23, "London") # wie gehabt
```

```
Name: Alice
Alter: 23
Wohnort: London
```

Jetzt zeigen wir Varianten mit Keyword-Argumenten:

```
# mit Verwendung von Keyword-Argumenten:
print_personal_details(name = "John", city = "Birmingham", age = 32)
```

```
Name: John
Alter: 32
Wohnort: Birmingham
```

```
# teilweise mit Keyword-Argumenten:
print_personal_details("Marc", city = "Hamburg", age = 19)
```

```
Name: Marc
Alter: 19
Wohnort: Hamburg
```

Wie Sie sehen, können Sie die Parameternamen explizit verwenden, um ein Argument zu definieren, und dabei sogar von der Reihenfolge im Funktionskopf abweichen. Eine Regel gibt es allerdings auch hier wieder: Wenn Sie nur teilweise Keyword-Argumente verwenden (wie im Beispiel `"Marc"` oben), kommen die Keyword-Argumente immer am Schluss! Zur namentlichen Abgrenzung von den Keyword-Argumenten bezeichnet man die Argumente, die anhand ihrer Reihenfolge identifiziert werden, als **Positionsargumente** (engl.: *positional arguments*). Halten wir also fest:

> **Positionsargumente zuerst, dann Keyword-Argumente!**
>
> Python weist zunächst die Argumente entsprechend ihrer Reihenfolge den Parametern zu (Positionsargumente). Danach können verbleibende Parameter explizit über Schlüsselwörter (Keyword-Argumente) gesetzt werden.

Übungen:

1. Schreiben Sie eine Funktion, die einen persönlichen Begrüßungstext in verschiedenen Sprachen ausgeben kann. Die Standardsprache soll Deutsch sein, bieten Sie mindestens zwei weitere Sprachen an. Übergeben Sie neben der Sprache weitere Argumente, die Sie für eine personalisierte Begrüßung benötigen.

2. Ein Körper werde aus seiner Ruhelage beschleunigt. Implementieren Sie eine Funktion `accelerated_motion(a, t)`, die die Beschleunigung `a` und die Zeit `t` als Parameter erwartet und die zurückgelegte Strecke $s(t) = \frac{1}{2} a\, t^2$ und die Geschwindigkeit $v(t) = a \cdot t$ zurückgibt.
3. Erweitern Sie die Funktion `accelerated_motion` um zwei optionale Parameter `s0` und `v0`, die eine Anfangsposition bzw. eine Anfangsgeschwindigkeit zum Zeitpunkt $t = 0$ darstellen. Übergibt man keine Argumente, soll die Bewegung wie gehabt aus der Ruhelage beginnen.

7.4. Dynamische Typisierung und mögliche Gefahren

Wir kennen bereits die dynamische Typisierung als besondere Eigenschaft von Python. Im Zusammenhang mit Funktionen erlaubt sie uns, dieselbe Funktion mit unterschiedlichen Da-

tentypen aufzurufen (wir haben das bereits oben einmal getan, finden Sie das Beispiel?). In anderen Programmiersprachen muss man dafür die Funktion in der Regel mehrfach implementieren (eine Variante für jeden Datentyp), man sagt auch: Die Funktion wird **überladen**. Bei überladenen Funktionen in C++ (in C gibt es kein Überladen) erkennt der Compiler anhand der übergebenen Datentypen, welche Version der Funktion gemeint ist.

Nicht so in Python, wo der Datentyp der Parameter und Rückgabewerte nicht vom Programmierer bzw. der Programmiererin festgelegt wird. Das kann sehr nützlich sein, aber auch zu Fehlern führen, wenn die Funktion mit inkompatiblen Datentypen aufgerufen wird. Hierzu ein Beispiel:

```
def multiply(x, y):
    return x * y
```

Eine sehr einfache Funktion, die zwei Parameter miteinander multipliziert. Wir rufen sie jetzt mit unterschiedlichen Datentypen auf:

```
print(multiply(3, 4)) # zwei Integer
print(multiply(2, 3.2)) # Integer und Float
print(multiply(2+2j, 3-1j)) # zwei komplexe Zahlen
print(multiply("hi", 3)) # String und Integer
```

```
12
6.4
(8+4j)
hihihi
```

Hätten Sie geahnt, was alles mit dieser kleinen, unscheinbaren Funktion möglich ist?

Es scheint fast alles möglich zu sein, aber eben nur *fast*! Rufen Sie die Funktion mit zwei Strings auf und Ihr Programm stürzt ab (wie sollen zwei Strings miteinander multipliziert werden?). Natürlich sind auch viele weitere Kombinationen nicht möglich (versuchen Sie z.B. zwei Listen miteinander zu multiplizieren oder eine Liste mit einem String). Manchmal ist es sogar noch schlimmer, wenn das Programm zwar nicht abstürzt, aber das Ergebnis völlig unerwartet ist (hätten Sie z.B. das Ergebnis der Multiplikation von String mit Integer in der Form erwartet?). Dann entstehen an einer anderen Stelle im Code womöglich überraschend Probleme und Sie müssen langwierig debuggen, bis Sie die Ursache finden.

Daran erkennen Sie wieder: Python bietet große Flexibilität, aber Sie müssen damit verantwortungsbewusst umgehen!

Übung: Schreiben Sie eine Funktion `a_plus_b(a, b)`, die die Summe von zwei Parametern `a` und `b` zurückgibt.

a) Finden Sie mindestens fünf verschiedene Kombinationen von Datentypen, für die sich die Funktion ausführen lässt.

b) Finden Sie mindestens fünf verschiedene Kombinationen von Datentypen, die zu einem Absturz führen!

Ergänzung: Mit `isinstance` den Typ von Parametern prüfen

Mit der Funktion `isinstance` kann bei Bedarf der Typ von Übergabeparametern geprüft werden. Es ist somit möglich, eine Funktion gegen die Verwendung mit inkompatiblen Typen abzusichern. Im Beispiel sieht dies so aus:

```
def add_and_sort_lists(a, b):
    if isinstance(a, list) and isinstance(b, list):
        c = a + b
        c.sort()
        return c
    else:
        print("Inkompatible Typen")
```

```
add_and_sort_lists([1, 5, 2], [4, 3])
```

```
[1, 2, 3, 4, 5]
```

```
add_and_sort_lists([1, 2], 3.4)
```

```
Inkompatible Typen
```

Hinweis: Eine Abfrage mit `type()` (z.B. `type(x) == int`) wird für diesen Zweck nicht empfohlen, da `isinstance()` im Gegensatz zu `type()` auch abgeleitete Datentypen erkennt (siehe Vererbung / objektorientierte Programmierung: Wenn eine Klasse `Dog` von einer Klasse `Animal` erbt, würde `isinstance` einen `Dog` auch als `Animal` erkennen).

7.5. Funktionen mit beliebig vielen Parametern

Wir kennen seit Beginn des Kurses eine Funktion, die beliebig viele Argumente entgegennehmen kann:

```
print()  # 0 Argumente
print("Hallo")  # 1 Argument
print("Hallo,", "Welt")  # 2 Argumente usw.
```

```
Hallo
Hallo, Welt
```

Mit einem einfachen Trick können Sie selbst solche Funktionen definieren: Stellen Sie vor einen Parameter ein *. Dann können beliebig viele Argumente übergeben werden; innerhalb der Funktion stehen sie als Tupel zur Verfügung, über das Sie z.B. mit der bekannten for-Schleife iterieren können:

```
def sum_numbers(*numbers):
    result = 0

    for number in numbers:
        result += number

    return result

print(sum_numbers(-2))
print(sum_numbers(5, 10))
print(sum_numbers(1, 2, 3, 4))
```

```
-2
15
10
```

7.6. Scope

In Python – und generell in der Programmierung – bezieht sich der Scope (Gültigkeitsbereich) auf den Bereich des Codes, in dem eine Variable sichtbar und zugänglich ist. Es gibt dabei zwei Hauptarten von Scope:

- **Lokaler Scope**: Eine Variable, die innerhalb einer Funktion definiert wird, ist nur in dieser Funktion sichtbar und außerhalb nicht zugänglich.

- **Globaler Scope:** Eine Variable, die außerhalb einer Funktion definiert wird, ist nach ihrer Definition im gesamten Programm sichtbar (auch innerhalb einer Funktion). Python schützt uns aber davor, globale Variablen unbeabsichtigt im Inneren einer Funktion neu zuzuweisen, es sei denn, wir machen unsere Absicht explizit deutlich. Dafür gibt es in Python das Keyword `global`.

Beginnen wir mit einem Beispiel zum lokalen Scope:

```
def my_function(a, b):
    c = a + b
    return c**2

my_function(2,3)
print(c) # FEHLER!!!! c ist hier nicht bekannt.
```

```
NameError: name 'c' is not defined
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[234], line 6
      3     return c**2
      5 my_function(2,3)
----> 6 print(c)
NameError: name 'c' is not defined
```

Auf die Variable `c` kann außerhalb der Funktion nicht zugegriffen werden, sie existiert nur im lokalen Scope der Funktion.

Nun versuchen wir umgekehrt innerhalb der Funktion auf eine Variable zuzugreifen, die außerhalb definiert wurde:

```
def greeting():
    print("Hallo,", name)  # name existiert im globalen Scope

name = "Peter"
greeting()
```

```
Hallo, Peter
```

Das Beispiel ist geglückt! Allerdings handelt es sich hierbei nicht um guten Programmierstil. Vielmehr sollte man `name` als Parameter der Funktion definieren. Was geschieht eigentlich, wenn es `name` im globalen und im lokalen Scope gibt?

```
def greeting(name):
    print("Hallo,", name)

name = "Peter"
greeting("Tina")
```

```
Hallo, Tina
```

Der lokale Scope geht vor! Die Funktion sucht immer zuerst in ihrem lokalen Scope nach der Variable. Nur wenn sie sie dort nicht findet, schaut sie im globalen Scope nach. Diese Regel nennt man auch LEGB-Rule (Abbildung 7.1): Python sucht nach Variablennamen *von innen nach außen* über diese Stufen: L = local scope, E = enclosing scope (umschließende Funktionen), G = global scope, B = built-in scope (eingebaute Python-Bezeichner).

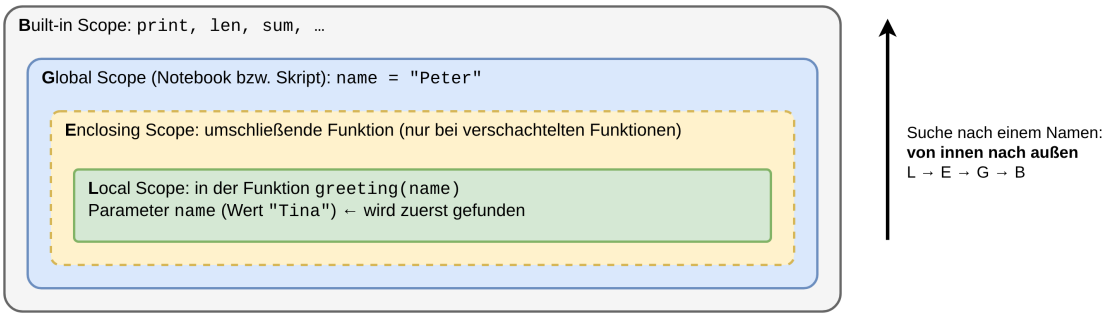


Abbildung 7.1.: Python sucht einen Namen von innen nach außen: Local → Enclosing → Global → Built-in. Im Beispiel wird der lokale Parameter `name` (Wert `"Tina"`) zuerst gefunden (einen Enclosing Scope gibt es hier nicht).

Das Keyword `global` in Python

Wie bereits erwähnt, schützt uns Python davor, unbeabsichtigt eine Variable aus dem globalen Scope neu zuzuweisen, wenn wir uns innerhalb einer Funktion befinden. Das wird im folgenden Beispiel illustriert:

```
def change_name(new_name):
    name = new_name

name = "Peter"
change_name("John")
print(name)
```

```
Peter
```

Hätten Sie die Ausgabe "John" erwartet? Python schützt uns: Beim Versuch, die globale Variable innerhalb einer Funktion neu zuzuweisen, hat Python stattdessen eine neue Variable im lokalen Scope der Funktion angelegt - mit demselben Bezeichner: `name`! Diese lokale Variable bekam den Wert "John", aber die Variable im globalen Scope wurde nicht angefasst. Machen wir das ruhig explizit:

```
def change_name(new_name):
    name = new_name
    print("name innerhalb der Funktion:", name)

name = "Peter"
change_name("John")
print("name außerhalb:", name)
```

```
name innerhalb der Funktion: John
name außerhalb: Peter
```

Wir haben nun bereits mehrfach gesehen, dass Variablen mit demselben Namen in unterschiedlichen Scopes existieren können. Python speichert die Variablennamen für jeden Scope in einer eigenen Zuordnungstabelle (Name → Instanz), dem sog. Namespace, der zu diesem Scope gehört. Im obigen Beispiel existiert also eine Variable `name` im globalen Namespace und eine Variable `name` im Namespace der Funktion `change_name`. Obwohl diese beiden Variablen gleich heißen, handelt es sich doch um getrennte Variablen, die in der Regel unterschiedliche Instanzen referenzieren. Beim Aufruf von `print` innerhalb der Funktion wird gemäß der LEGB-Rule die lokale Variable zuerst gefunden und verwendet. Bei der Verwendung von `print` außerhalb der Funktion findet Python stattdessen die globale Variable (Reminder: Python sucht immer *von innen nach außen*).

Eine Feinheit führt immer wieder zu Verwirrung: Ob ein Name in einer Funktion lokal ist, entscheidet Python nicht erst, wenn die Zeile ausgeführt wird, sondern bereits beim Einlesen der Funktion. Sobald einem Namen **irgendwo** in der Funktion ein Wert zugewiesen wird, gilt er in der **gesamten** Funktion als lokal – auch in den Zeilen davor:

```
counter = 0

def increment():
    counter += 1  # FEHLER! counter gilt hier als lokale Variable ohne Wert

increment()
```

```
UnboundLocalError: cannot access local variable 'counter' where it is not
 ↪  associated with a value
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
Cell In[1], line 6
      2
      3 def increment():
      4     counter += 1  # FEHLER! counter gilt hier als lokale Variable
 ↪  ohne Wert
      5
----> 6 increment()
Cell In[1], line 4, in increment()
      3 def increment():
----> 4     counter += 1  # FEHLER! counter gilt hier als lokale Variable
 ↪  ohne Wert
UnboundLocalError: cannot access local variable 'counter' where it is not
 ↪  associated with a value
```

`counter += 1` ist eine Zuweisung an `counter`. Deshalb behandelt Python `counter` in der ganzen Funktion als lokale Variable – und die hat beim Lesen noch keinen Wert. Abhilfe schafft das Keyword `global` (siehe gleich) oder, besser, ein Parameter und ein Rückgabewert.

Zurück zu unserem Beispiel `change_name`: Den vorgestellten Sicherheitsmechanismus kann man mit dem Keyword `global` bewusst aushebeln!

```python
def change_name(new_name):
    global name  # hey Python, wir meinen die globale Variable!
    name = new_name

name = "Peter"
change_name("John")
print(name)
```

```
John
```

Mit der Zeile `global name` weisen wir Python explizit an, die Variable `name` aus dem globalen Scope zu verwenden! In diesem Beispiel wird **keine** lokale Variante von `name` angelegt. Die Variable `name` existiert nur im globalen Namespace, kann aber in der Funktion frei verwendet werden.

Verwendung globaler Variablen in Funktionen

Finden Sie das alles noch ein wenig verwirrend? Gut so, denn Sie sollten von der Verwendung globaler Variablen in Funktionen sowieso besser absehen. Versuchen Sie stattdessen saubere Schnittstellen mithilfe von Parametern und Rückgabewerten zu definieren. Wann immer Sie geneigt sind, lesend auf einen globalen Wert zuzugreifen, übergeben Sie diesen lieber als Argument. Wann immer Sie geneigt sind, schreibend auf einen globalen Wert zuzugreifen, arbeiten Sie lieber mit einem Rückgabewert, den Sie im Hauptprogramm zuweisen können.
Beispiel: Eine Zahl im globalen Scope soll durch ihre Quadratzahl ersetzt werden.

```
def square(x):
    return x**2 # die Funktion führt die Rechnung durch

a = 42
a = square(a) # die eigentliche Veränderung geschieht außerhalb der
 ↪ Funktion
```

7.7. Funktionsaufrufe mit mutable Datentypen

Wir haben bereits den Unterschied zwischen mutable und immutable Datentypen kennengelernt, insbesondere das Phänomen von Seiteneffekten, wenn mehrere Referenzen auf eine veränderliche Instanz zeigen. Seiteneffekte können auch durch den Aufruf von Funktionen auftreten: Wir übergeben an Funktionen eine Referenz auf eine Instanz. Wenn die Instanz mutable ist, kann die Funktion die Instanz verändern! Hierzu eine Demo:

```
def append_a(l, a):
    l.append(a)

my_list = [1, 2, 3]
append_a(my_list, 100)

print(my_list)  # my_list wurde verändert
```

```
[1, 2, 3, 100]
```

In diesem Beispiel zeigen innerhalb der Funktion `append_a` zwei Referenzen auf dieselbe mutable Instanz: einmal die Variable `my_list` aus dem globalen Namespace und einmal die Variable `l` im lokalen Namespace der Funktion. Die Veränderung, die `l` durch den Aufruf von `append`

bewirkt, sieht deshalb auch die Variable `my_list`. Vorsicht beim Operator `+=`: Schreibt man in der Funktion `l += [a]`, wird die Liste des Aufrufers ebenfalls verändert (wie bei `append`). Mit `l = l + [a]` entsteht dagegen eine neue Liste, die nur innerhalb der Funktion bekannt ist – `my_list` bleibt dann unverändert (mehr dazu im Kapitel *Datenstrukturen*).

Besonders überraschend ist vielleicht das folgende Beispiel:

```
def append_a_to_global_list(a):
    my_list.append(a) # my_list wird über die LEGB-Rule im globalen Namespace
 ↪  gefunden

my_list = [1, 2, 3]
append_a_to_global_list(100)

print(my_list)  # my_list wurde verändert
```

```
[1, 2, 3, 100]
```

Wundern Sie sich, dass wir das Keyword `global` nicht gebraucht haben? Der Grund ist einfach: In der Funktion wurde keine neue Zuweisung an `my_list` vorgenommen. Stattdessen haben wir lediglich die bereits existierende Instanz verändert – und das funktioniert nur bei mutable Datentypen! Das Keyword `global` wäre nur dann erforderlich gewesen, wenn wir `my_list` innerhalb der Funktion eine neue Instanz zugewiesen hätten, also z.B. mit `my_list = [4, 5, 6]`. In diesem Fall müssten wir – wie bekannt – mit `global my_list` deutlich machen, dass sich die Zuweisung auf die globale Variable beziehen soll (und nicht nur auf eine lokale im Namespace der Funktion). Achtung: Auch `my_list += [4]` zählt als Zuweisung und führt ohne `global` zum `UnboundLocalError` (siehe oben).

Es sei darauf hingewiesen, dass die Funktion aus dem letzten Beispiel schlechter Programmierstil ist: Viel verständlicher ist es, wenn die Liste, die verändert werden soll, als Parameter übergeben wird und nicht erst über die LEGB-Rule gefunden werden muss. Außerdem funktioniert der Code nicht mehr, wenn die Variable im globalen Namespace umbenannt wird und die Namensänderung innerhalb der Funktion vergessen wird.

⚠ Fallstrick: Eine Liste als Standardwert eines Parameters

```
def add_measurement(value, measurements=[]):
    measurements.append(value)
    return measurements

print(add_measurement(4.8))   # [4.8]
print(add_measurement(5.1))   # [4.8, 5.1] - der erste Wert ist noch da!
```

Der Standardwert `[]` wird nur **ein einziges Mal** erzeugt, nämlich wenn Python die `def`-Zeile ausführt – nicht bei jedem Aufruf neu. Alle Aufrufe ohne zweites Argument teilen sich deshalb dieselbe Liste, und weil Listen mutable sind, sammeln sich die Werte darin an. Richtig ist es, `None` als Standardwert zu verwenden und die Liste erst in der Funktion zu erzeugen:

```
def add_measurement(value, measurements=None):
    if measurements is None:
        measurements = []     # bei jedem Aufruf eine neue Liste
    measurements.append(value)
    return measurements
```

7.8. Docstrings und Typ-Annotationen: Meine Funktion für andere dokumentieren

In großen Software-Projekten sind Sie nicht die einzige Person, die Ihre Funktionen benutzt. Sie können durch verständliche Dokumentation die Nutzbarkeit Ihrer Funktionen enorm erhöhen. Eine einfache Möglichkeit sind sog. Docstrings: Dabei handelt es sich um eine spezielle Kommentierung, direkt nach der definierenden Zeile:

```
def bending_stress(moment, y, inertia):
    """
    Berechnet die Biegespannung in einem Balken.

    Formel: sigma = (M * y) / I

    Args:
        moment (float): Biegemoment in Newton-Meter (Nm).
        y (float): Abstand von der Neutralachse in Meter (m).
        inertia (float): Flächenträgheitsmoment in m^4.

    Returns:
        float: Biegespannung in Pascal (Pa).
    """

    return moment * y / inertia
```

Der Docstring wird mit `"""` eingeleitet, es folgt eine kurze Erklärung der Funktion, anschließend eine Erklärung der Argumente und der Rückgabewerte, und zum Abschluss wieder `"""`. Wie

genau der Docstring formatiert wird, hängt von der gewählten Konvention ab (wir haben hier den Google-Style verwendet). Viele Editoren greifen Docstrings in unterstützenden Funktionen auf (Tooltips oder Autovervollständigungen). Insbesondere kann nun aber jeder Entwickler und jede Entwicklerin über die Funktion `help()` eine Hilfe zu Ihrer Funktion aufrufen:

```
help(bending_stress)
```

```
Help on function bending_stress in module __main__:

bending_stress(moment, y, inertia)
    Berechnet die Biegespannung in einem Balken.

    Formel: sigma = (M * y) / I

    Args:
        moment (float): Biegemoment in Newton-Meter (Nm).
        y (float): Abstand von der Neutralachse in Meter (m).
        inertia (float): Flächenträgheitsmoment in m^4.

    Returns:
        float: Biegespannung in Pascal (Pa).
```

Eine weitere Unterstützung bieten **Typ-Annotationen**, mit denen wir andere Entwickler bzw. Entwicklerinnen darüber informieren, welche Datentypen unsere Funktion erwartet bzw. zurückgibt. Aber Vorsicht: Diese Angaben sind rein informativ! Sie verhindern nicht, dass die Funktion mit anderen Datentypen aufgerufen wird! Unser Beispiel mit Typ-Annotationen:

```
def bending_stress(moment: float, y: float, inertia: float) -> float:
    """Berechnet die Biegespannung in einem Balken (Docstring wie oben)."""
    return moment * y / inertia
```

Die Typ-Annotationen geben an: Wir erwarten den Datentyp `float` für die Parameter `moment`, `y` und `inertia` und geben auch einen Float als Ergebnis zurück (`-> float`).

Übung:

Schreiben Sie eine Funktion `ohms_law(R, U, I)`: Der Anwender übergibt für jeweils zwei der drei Parameter Zahlenwerte und für den dritten den String `"*"`. Daraufhin berechnet die Funktion den fehlenden Wert mithilfe des ohmschen Gesetzes und gibt ihn zurück. Dokumentieren Sie die Funktion mit einem Docstring, der die Benutzung erklärt.

```
def ohms_law(R, U, I):
    """
    Berechnet Widerstand, Spannung oder Strom mithilfe des ohmschen Gesetzes.
 ↪
    Der fehlende Wert wird mit dem Argument "*" markiert; für die anderen
 ↪  beiden müssen Zahlen übergeben werden.
    Args:
        R (float oder str): El. Widerstand (Ohm) oder "*".
        U (float oder str): El. Spannung (Volt) oder "*".
        I (float oder str): El. Strom (A) oder "*".
    Returns:
        float: fehlender Wert
    """

    if R == "*":
        return U / I
    elif U == "*":
        return R * I
    elif I == "*":
        return U / R
    else:
        print("Ungültige Eingabe! Siehe Hilfe: ")
        help(ohms_law)
```

8. Datenstrukturen in Python

> **Lernziele**
>
> Nach diesem Kapitel können Sie …
>
> - Liste, Set, Dictionary und Tupel unterscheiden und für eine Aufgabe die passende Struktur wählen
> - per Index und Slicing auf Listenelemente zugreifen und Listen sortieren
> - mit Sets eindeutige Werte ermitteln und Mengenoperationen ausführen
> - Daten mit (verschachtelten) Dictionaries strukturieren
> - Listen sicher kopieren und flache von tiefer Kopie unterscheiden
> - List-, Set- und Dictionary-Comprehensions schreiben
> - mit `enumerate()` und `zip()` übersichtlich über Positionen und parallele Sequenzen iterieren

Datenstrukturen und **Datentypen** sind zentrale Begriffe in der Informatik, die eng miteinander verbunden, aber dennoch voneinander zu unterscheiden sind:

- Datentypen beschreiben die Art und den Wertebereich von Daten, wie Zahlen (z.B. int, float), Zeichenketten (z.B. str) oder Wahrheitswerte (bool). Sie definieren die grundlegenden Eigenschaften eines Werts und wie dieser vom Computer verarbeitet wird. Damit haben wir uns bereits ausführlich beschäftigt.
- Datenstrukturen hingegen organisieren mehrere Werte in einer bestimmten Weise, sodass sie effizient gespeichert und verarbeitet werden können. Sie kombinieren oft mehrere Datentypen, um komplexe Informationen abzubilden. Beispiele hierfür sind Listen, Dictionaries oder Sets in Python.

Während Datentypen also die Bausteine sind, definieren Datenstrukturen den Bauplan, wie diese Bausteine miteinander in Beziehung stehen und verarbeitet werden. In diesem Kapitel fokussieren wir uns auf grundlegende Datenstrukturen und deren Anwendung in Python anhand eines praxisnahen Beispiels: der Organisation und Analyse von Sensordaten. Das Beispiel wird im Verlauf des Kapitels sukzessive komplexer und damit auch realistischer.

Sie sollen einen Eindruck davon gewinnen, wie verschiedene Datenstrukturen für bestimmte Fragestellungen und Anwendungen ihre Stärken ausspielen.

Am Ende des Kapitels gehen wir noch auf wichtige Eigenschaften wie Mutability von Datenstrukturen ein und stellen mit den sogenannten List Comprehensions ein elegantes Werkzeug zur Erzeugung von Listen vor und zeigen, wie man mit `enumerate()` und `zip()` übersichtlicher über Listen iteriert.

8.1. Ein Sensor speichert stündlich Temperaturdaten – Anwendung der Liste

Ein Sensor misst stündlich die Temperatur (z.B. in einem Raum oder einer Maschine). Diese Daten speichern wir der Reihe nach in einer Liste, einer der einfachsten Datenstrukturen, die wir bereits im Kapitel über Datentypen kennengelernt haben. Die Liste ist ideal für lineares Speichern (Messwert für Messwert hintereinander). Sie ermöglicht den einfachen Zugriff auf Elemente anhand ihrer Position, stellt aber auch nützliche Funktionen zum Arbeiten mit den Daten bereit. Im Folgenden wiederholen wir einige wesentliche Eigenschaften von Listen und lernen weitere Möglichkeiten der Datenverarbeitung kennen.

Speichern der Daten

```
# eine Liste kann mit Werten initialisiert werden
temperatures = [22.5, 23.1, 21.8]

# es kann aber auch nach und nach ein Wert hinten angefügt werden:
temperatures.append(24.0)
temperatures.append(22.5)
temperatures.append(21.8)

print(temperatures)
```

```
[22.5, 23.1, 21.8, 24.0, 22.5, 21.8]
```

In der Praxis würde an einer bestimmten Stelle im Code (einmal pro Stunde) mithilfe von `append` der aktuelle Messwert in der Liste gespeichert werden.

Sortieren der Messungen

Sortieren hilft, die Werte geordnet darzustellen und z.B. das Finden des höchsten oder niedrigsten Messwerts zu erleichtern. Auch hierfür steht eine Methode bereit:

```
# Sortieren der Liste
temperatures.sort()
print("Sortierte Temperaturen:", temperatures)
```

```
Sortierte Temperaturen: [21.8, 21.8, 22.5, 22.5, 23.1, 24.0]
```

Wir können nun direkt ablesen, dass der Wertebereich zwischen 21.8 und 24.0 liegt. Allerdings haben wir unsere Originalliste durch das Sortieren verändert! Das war womöglich gar nicht unsere Absicht, denn wir wollen weiterhin die Information besitzen, wann welcher Wert erhoben wurde. Für diesen Zweck gibt es eine Lösung: Anstelle der Methode `.sort()` verwenden wir die bereits bekannte Built-in Function `sorted()` (siehe Kapitel *Funktionen*), die eine sortierte Liste zurückgibt, **ohne das Original zu verändern**:

```
# Wir stellen zunächst das Original wieder her
temperatures = [22.5, 23.1, 21.8, 24.0, 22.5, 21.8]

sorted_data = sorted(temperatures)
print("Sortierte Liste: ", sorted_data)
print("Original: ", temperatures)
```

```
Sortierte Liste:  [21.8, 21.8, 22.5, 22.5, 23.1, 24.0]
Original:  [22.5, 23.1, 21.8, 24.0, 22.5, 21.8]
```

Die implementierten Sortieralgorithmen sind übrigens sehr effizient (schneller als der früher vorgestellte Bubble Sort).

> **!** Wichtig
>
> Beachten Sie unbedingt den Unterschied zwischen `.sort()` und `sorted()`:
> `my_list.sort()  # Original verändert!`
> `sorted_list = sorted(my_list)  # Original unverändert!`

Werte extrahieren – Indexzugriff und List Slicing

Wie bereits bekannt, kann man über Indizes auf die einzelnen Elemente zugreifen:

```
print("1. Messwert: " , temperatures[0])
print("Letzter Messwert: " , temperatures[-1])
print("Min: " , sorted_data[0])
print("Max: " , sorted_data[-1])
print("5. Messwert: " , temperatures[4])
print("Vorletzter Messwert: " , temperatures[-2])
```

```
1. Messwert:  22.5
Letzter Messwert:  21.8
Min:  21.8
Max:  24.0
5. Messwert:  22.5
Vorletzter Messwert:  22.5
```

Oft interessiert man sich für ganze Bereiche von Messwerten und kann diese in Python sehr effizient mit sog. "List Slicing" *ausschneiden*: Hierfür verwendet man einen Start- und Endindex, getrennt durch ein `:`. Zu beachten ist, dass der Anfangsindex inkludiert wird, das Slicing aber einen Schritt vor dem Endindex aufhört (Abbildung 8.1):

```
# Slicing: Werte gesucht ...

# ... mit Indizes: 0,1,2
print(temperatures[0:3])

# ... mit Indizes: 2,3,4
print(temperatures[2:5])

# ... mit Indizes: 1,2,3,...,vorletztes
print(temperatures[1:-1])
```

```
[22.5, 23.1, 21.8]
[21.8, 24.0, 22.5]
[23.1, 21.8, 24.0, 22.5]
```

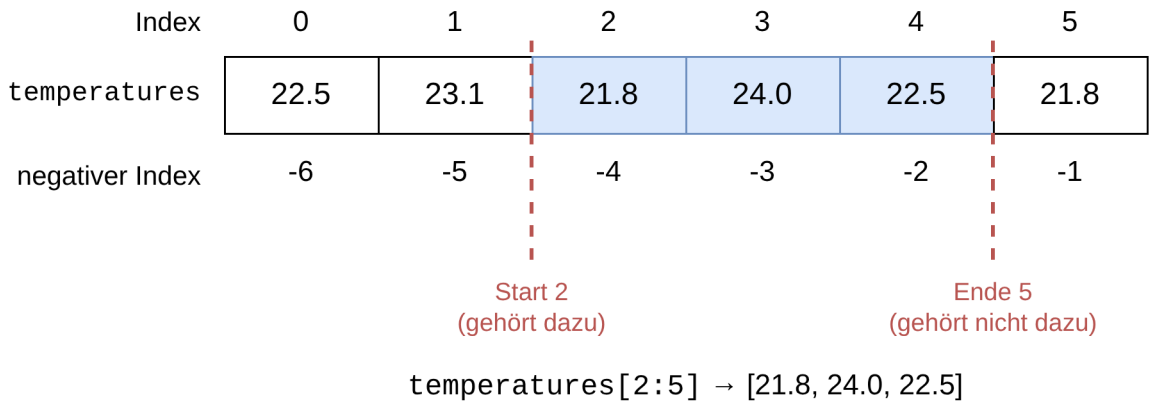


Abbildung 8.1.: Positive und negative Indizes der Liste `temperatures`. Beim Slicing gehört der Startindex dazu, der Endindex nicht.

Man kann auch Start- und Endindex weglassen und meint damit "von Anfang" oder "bis Ende":

```
# Slicing: Werte gesucht ...

# ... von Anfang bis Index 2
print(temperatures[:3])

# ... von Index 4 bis Schluss
print(temperatures[4:])

# Die ganze Liste als großer Slice:
print(temperatures[:])
```

```
[22.5, 23.1, 21.8]
[22.5, 21.8]
[22.5, 23.1, 21.8, 24.0, 22.5, 21.8]
```

Python erlaubt sogar noch präzisere Analysen, indem es Schrittweiten beim Slicing zulässt. Die Schrittweite ist standardmäßig `1`, aber kann als dritter Parameter hinter einem weiteren `:` verändert werden:

```
# von Index 0 bis 4 in 2er-Schritten
print(temperatures[0:5:2])
```

```
# von Index 1 bis Schluss in 2er-Schritten
print(temperatures[1::2])

# jeder dritte Messwert
print(temperatures[::3])

# auch rückwärts funktioniert!
print(temperatures[4:1:-1])

# komplette Liste rückwärts
print(temperatures[::-1])
```

```
[22.5, 21.8, 22.5]
[23.1, 24.0, 21.8]
[22.5, 24.0]
[22.5, 24.0, 21.8]
[21.8, 22.5, 24.0, 21.8, 23.1, 22.5]
```

Man kann auch mit dem Operator `in` fragen, ob ein spezifischer Wert in der Liste enthalten ist:

```
23.1 in temperatures
```

```
True
```

Dabei muss intern aber jedes Element einzeln abgefragt werden, mit Pech also die gesamte Liste durchgegangen werden (die Datentypen `set` und `dict` sind hier deutlich effizienter).

Darüber hinaus sind die Built-in Functions `max()`, `min()`, `len()` oder `sum()` nützlich.

```
# Beispiel: Verwendung von Built-in Functions min, max, len
print("Min: ", min(temperatures))
print("Max: ", max(temperatures))
print("Anzahl der Elemente: ", len(temperatures))
```

```
Min:  21.8
Max:  24.0
Anzahl der Elemente:  6
```

8.2. Analyse der verschiedenen Messwerte – Anwendung des Sets

Wir möchten wissen, welche verschiedenen Temperaturwerte vorkommen. Hierfür ist ein Set ideal, da es automatisch doppelte Einträge entfernt. Würden wir weiter mit einer Liste arbeiten, müssten wir selbst die Daten filtern, z.B. mit den uns bekannten Schleifen. Das wäre allerdings deutlich ineffizienter. Das Set entspricht einer mathematischen Menge, in der jedes Element nur einmal vorkommen darf. Für die Umwandlung führen wir ein explizites Casting durch:

```
unique_values = set(temperatures)
print(unique_values) # keine doppelten Werte mehr
```

```
{24.0, 21.8, 22.5, 23.1}
```

Syntaktisch erkennen Sie das Set an den geschweiften Klammern `{}`. Beachten Sie: Ein Set ist ungeordnet. Die Reihenfolge der Ausgabe kann von der Eingabe abweichen, und einen Indexzugriff wie bei Listen gibt es nicht. Ein Set erlaubt alle bekannten Mengenoperationen, die Sie aus der Mathematik kennen (z.B. Schnittmenge oder Vereinigung). Nehmen wir an, wir wollen überprüfen, ob unser Sensor mit einer anderen Messreihe gemeinsame Werte hat. Hierfür können wir ganz einfach die Schnittmenge aus beiden Mengen untersuchen:

```
second_temperatures = [22.0, 23.1, 25.0, 22.4, 20.8]
second_set = set(second_temperatures)
intersection = unique_values.intersection(second_set)
print("Gemeinsame Werte: ", intersection)
```

```
Gemeinsame Werte:  {23.1}
```

Die Abfrage mit dem Operator `in` ist übrigens bei Sets viel effizienter als bei Listen. Das liegt an einem Trick: Zu jedem Element eines Sets berechnet Python einen sog. “Hash-Wert”, der angibt, wo das Element im Speicher liegt. Fragt man, ob ein Element im Set liegt, muss Python nur den Hash-Wert für dieses Element berechnen und checken, ob der entsprechende Speicherplatz zum Set gehört. Daher ist der Aufwand - anders als bei Listen - unabhängig von der Größe des Sets! Deshalb dürfen Sets allerdings nur Elemente enthalten, für die sich ein Hash-Wert berechnen lässt (*hashable*). Das sind die unveränderlichen (immutable) Datentypen wie Zahlen, Strings oder Tupel (sofern das Tupel selbst keine Liste o. Ä. enthält); eine Liste als Element führt zu einem `TypeError`. Dasselbe gilt später für die Schlüssel von Dictionaries.

Wie initialisiert man ein Set?

Es gibt mehrere Möglichkeiten, ein neues Set anzulegen - leer oder mit Inhalt:

- Leere Initialisierung: `my_set = set()` (**Achtung: leere geschweifte Klammern initialisieren ein Dictionary und kein Set!**)
- Mit Werten: `my_set = {1, 2, 3}`
- Aus anderen Datenstrukturen: `my_set = set(x)`, wobei `x` einen String, eine Liste, ein Tupel oder ein Dictionary referenzieren kann. Im Falle des Strings erhält man beispielsweise die Menge der Zeichen (Duplikate werden entfernt!), im Falle des Dictionaries die Menge der Schlüssel.

Wichtige Methoden des Sets

Methode	Bedeutung
`.add(x)`	Fügt das Element x hinzu.
`.remove(x)`	Entfernt x aus dem Set. (Achtung: Fehler, wenn x nicht vorhanden!)
`.clear()`	Entfernt alle Elemente.
`.union(*sets)`	Gibt die Vereinigungsmenge zurück (A ∪ B).
`.intersection(*sets)`	Gibt die Schnittmenge zurück (A ∩ B).
`.difference(*sets)`	Gibt die Differenzmenge zurück (A \ B).

Erinnerung: Darüber hinaus sind die Built-in Functions `max()`, `min()`, `len()`, `sum()`, `sorted()` (gibt eine sortierte Liste zurück) und der `in`-Operator nützlich!

Übungen:

1. Erstellen Sie eine eigene Liste mit fünf Messwerten. Erweitern Sie Ihre Liste durch `append()` auf insgesamt 8 Elemente.
2. Geben Sie mithilfe des Slicing-Operators folgende Messwerte aus: a) die ersten drei, b) jeden zweiten, c) alle zwischen dem 3. und 7. Messwert (ohne den 3. und ohne den 7. Messwert selbst), d) die letzten drei.
3. Finden Sie den größten und den kleinsten Wert Ihrer Messreihe.
4. Sortieren Sie alle Messwerte der Größe nach von klein nach groß. Finden Sie auch heraus, wie Sie die umgekehrte Sortierung erhalten (also vom größten zum kleinsten Wert).
5. Reduzieren Sie Ihre Datenreihe auf ein Set, welches jeden Messwert nur einmal enthält.

Lösungsvarianten für die umgekehrte Sortierung in Aufgabe 4:

```
# Variante 1
data = [1,3,5,2,3,4,0,-6]
data.sort(reverse=True)
print(data)

# Variante 2
data = [1,3,5,2,3,4,0,-6]
sorted_data = sorted(data, reverse=True)
print(sorted_data)

# Variante 3
data = [1,3,5,2,3,4,0,-6]
sorted_data = sorted(data)
sorted_data.reverse()
print(sorted_data)

# Variante 4
data = [1,3,5,2,3,4,0,-6]
reverse_sorted_data = sorted(data)[::-1]
print(reverse_sorted_data)
```

8.3. Hinzufügen von Zeitstempeln – Anwendung des Dictionaries

Für eine präzisere Darstellung ergänzen wir die Daten um Zeitstempel. Wir wären gerne in der Lage, folgende Frage zu beantworten: “Wie war die Temperatur am Tag xy zum Zeitpunkt z?”.

Hierzu speichern wir die Temperaturen als **Werte (values)** und die Zeitpunkte als **Schlüssel (keys)** in einem **Dictionary**. Bei einem Dictionary greift man über einen eindeutigen Schlüssel auf das entsprechende Element zu. Es bedarf keiner komplexen Suche nach dem passenden Element etc. Wir demonstrieren die Anwendung:

```
# Dictionary für Zeitstempel und Temperaturen
data = {
    "2025-03-23 08:00": 22.5,
    "2025-03-23 09:00": 23.1,
    "2025-03-23 10:00": 21.8,
    "2025-03-23 11:00": 24.0,
    "2025-03-23 12:00": 22.5,
```

```
}

# neue Werte können jederzeit einzeln hinzugefügt werden:
data["2025-03-23 13:00"] = 21.8

print(data)
```

```
{'2025-03-23 08:00': 22.5, '2025-03-23 09:00': 23.1, '2025-03-23 10:00': 21.8, '2025-
03-23 11:00': 24.0, '2025-03-23 12:00': 22.5, '2025-03-23 13:00': 21.8}
```

Achten Sie erneut auf die Syntax: Geschweifte Klammern (wie beim Set), aber darin befinden sich Auflistungen von `Key:Value`. Der Zugriff auf einzelne Elemente ähnelt syntaktisch dem Index-Zugriff bei einer Liste, aber anstelle eines Index verwendet man einen Schlüssel:

```
# Zugriff auf bestimmte Temperatur
print(data["2025-03-23 09:00"])
```

```
23.1
```

Eine Gefahr besteht darin, dass man auf Schlüssel zugreifen möchte, die es nicht gibt. Dann würde das Programm abstürzen! Daher gibt es noch eine sicherere Zugriffsmethode über die Methode `.get()`:

```
print(data.get("2025-03-23 09:00"))
print(data.get("2025-03-23 22:00")) # Key existiert nicht
print(data.get("2025-03-23 22:00", "Datensatz fehlt")) # mit Alternativtext
```

```
23.1
None
Datensatz fehlt
```

Wie initialisiert man ein Dictionary?

Es gibt mehrere Möglichkeiten, ein neues Dictionary anzulegen - leer oder mit Inhalt:

- Leere Initialisierung: `my_dict = {}` oder `my_dict = dict()`
- Mit Schlüssel-Wert-Paaren: `my_dict = {"a": 1, "b": 2}`
- Aus einer Liste von Tupeln: `my_dict = dict([("a", 1), ("b", 2)])`

Wichtige Methoden des Dictionaries

Methode	Bedeutung
`.clear()`	Entfernt alle Elemente.
`.get(key, default)`	Gibt den Value zu `key` zurück; falls nicht vorhanden: `default`.
`.pop(key, default)`	Entfernt Eintrag zu `key` und gibt dessen Wert zurück; falls nicht vorhanden: `default`.
`.keys()` und `.values()`	Zum Iterieren über alle Keys bzw. Values.
`.items()`	Zum Iterieren über alle (key, value)-Paare.

Erinnerung: Darüber hinaus sind die Built-in Functions `max()`, `min()`, `len()`, `sum()`, `sorted()` und der `in`-Operator nützlich, aber sie operieren standardmäßig auf den Keys des Dictionaries! Variante: `max(my_dict.values())`.

```
# Beispiel zum Iterieren über alle Einträge:
for time, temp in data.items():
    print(f"{time}: {temp}°C")
```

```
2025-03-23 08:00: 22.5°C
2025-03-23 09:00: 23.1°C
2025-03-23 10:00: 21.8°C
2025-03-23 11:00: 24.0°C
2025-03-23 12:00: 22.5°C
2025-03-23 13:00: 21.8°C
```

Das letzte Beispiel sieht etwas ungewohnt aus, da wir zwei Iterationsvariablen in einer `for`-Schleife gemeinsam verwenden: `time` und `temp`. In Wahrheit geschieht hier ein Entpacken von Tupeln (das Tupel wird im nächsten Abschnitt ausführlicher behandelt): Mit `.items()` iterieren wir über die Tupel `(key, value)` zu jedem Eintrag. Dieses Tupel haben wir aufgelöst in zwei Variablen, da wir wissen, dass die Keys den Zeitstempeln und die Values den Temperaturen entsprechen. Aber auch folgender Code funktioniert:

```
for t in data.items():
    print(f"key: {t[0]}, value: {t[1]}")
```

```
key: 2025-03-23 08:00, value: 22.5
key: 2025-03-23 09:00, value: 23.1
```

```
key: 2025-03-23 10:00, value: 21.8
key: 2025-03-23 11:00, value: 24.0
key: 2025-03-23 12:00, value: 22.5
key: 2025-03-23 13:00, value: 21.8
```

Übungen:

1. Erstellen Sie ein Dictionary, das ein einfaches Telefonbuch darstellt. Fügen Sie einige Namen und Telefonnummern hinzu. Greifen Sie auf einige Einträge zu, überschreiben Sie einen Eintrag und versuchen Sie auch auf einen Eintrag zuzugreifen, den es nicht gibt (einmal mit und einmal ohne `.get()`).
2. Erstellen Sie ein beliebiges Dictionary und wandeln Sie es anschließend in eine Liste um. Welche Einträge finden Sie in der Liste?
3. Recherchieren Sie, wie man in Python echte Zeitstempel erzeugt (Tipp: Es gibt eine Bibliothek dafür). Schreiben Sie Code, der alle zwei Minuten einen echten Zeitstempel erzeugt und mit einem simulierten Sensorwert (z.B. einer Zufallszahl) in ein Dictionary schreibt. Programmieren Sie eine Abbruchbedingung nach 10 Minuten und untersuchen Sie das entstandene Dictionary (Tipp: Zum Testen kann man zunächst kürzere Zeitintervalle wählen).

8.4. Speichern einer zweiten Messgröße – Anwendung des Tupels (engl.: tuple)

Neben der Temperatur misst der Sensor nun auch die relative Luftfeuchtigkeit. Wir speichern beide Messwerte als **Tupel**. Ein Tupel ist ähnlich wie die Liste eine geordnete Kollektion von Elementen, aber mit einem wichtigen Unterschied: Das Tupel ist im Gegensatz zur Liste **unveränderlich**. Man kann weder Elemente hinzufügen noch entfernen oder überschreiben! Diese Einschränkungen machen das Tupel zu einem sicheren Datenformat, welches außerdem weniger Speicher verbraucht als die Liste. Syntaktisch erkennt man das Tupel meistens an den runden Klammern `()`:

```
measurement = (23.4, 40) # Tupel mit zwei Werten
print(measurement)
```

```
(23.4, 40)
```

Die Klammern können in diesem Beispiel aber auch weggelassen werden:

```
measurement = 23.4, 40 # ebenfalls ein Tupel mit zwei Werten (Merke: "das
 ↪  Komma macht das Tupel")
print(measurement)
```

```
(23.4, 40)
```

Eine Funktion mit mehreren Rückgabewerten liefert ebenfalls ein Tupel:

```
def my_function(x, y):
    return x + y, x - y, x * y

result = my_function(5, 3)  # Ergebnisse als Tupel speichern
print(result)
```

```
(8, 2, 15)
```

```
# alternativ: Ergebnisse einzeln speichern
a, b, c = my_function(5, 3)
print(a, b, c)
```

```
8 2 15
```

Man kann die Werte eines Tupels über den Indexzugriff auslesen (aber nicht überschreiben!):

```
print("Temperatur: ", measurement[0])
print("Luftfeuchtigkeit: ", measurement[1])

#measurement[0] = 25.0 ist nicht erlaubt! Ein Tupel ist unveränderlich!
```

```
Temperatur:  23.4
Luftfeuchtigkeit:  40
```

Unser neues Dictionary sieht jetzt so aus:

```python
# Erweiterung: Values = Tupel aus Temperatur und Luftfeuchtigkeit
data = {
    "2025-03-23 08:00": (22.5, 60),
    "2025-03-23 09:00": (23.1, 55),
    "2025-03-23 10:00": (21.8, 58),
    "2025-03-23 11:00": (24.0, 61),
    "2025-03-23 12:00": (22.5, 62)
}

# neuen Wert hinzufügen
data["2025-03-23 13:00"] = (21.8, 63)

# einen Zeitpunkt abfragen
print(data.get("2025-03-23 10:00"))
```

```
(21.8, 58)
```

Tatsächlich könnte man die beiden Messwerte natürlich auch als Liste mit zwei Elementen zusammenfassen, aber das Tupel hat in diesem Fall klare Vorteile:

- Semantik: Ein Tupel repräsentiert eine feste Kombination von Werten, z.B. Temperatur und Luftfeuchtigkeit, die logisch zusammengehören.
- Unveränderbarkeit: Tupel verhindern unbeabsichtigte Änderungen. (Will man bewusst einen Wert verändern, kann man in eine Liste umwandeln.)
- Effizienz: Tupel benötigen weniger Speicherplatz und werden (etwas) schneller verarbeitet als Listen.

Wie initialisiert man ein Tupel?

Es gibt mehrere Möglichkeiten, ein neues Tupel anzulegen - leer oder mit Inhalt:

- Leere Initialisierung: `my_tuple = ()` oder `my_tuple = tuple()`
- Mit Werten: `my_tuple = (1, 2, 3)`
- Ohne Klammern: `my_tuple = 1, 2, 3` wird automatisch als Tupel interpretiert.
- Achtung Spezialfall - Tupel mit exakt einem Wert: `t = (42,)` oder `t = 42,` (Merke: *Das Komma macht das Tupel, nicht die Klammern!*)
- Aus anderen Datenstrukturen: `my_tuple = tuple(x)`, wobei `x` einen String, eine Liste, ein Set oder ein Dictionary referenzieren kann. Im Falle des Strings erhält man beispielsweise das Tupel der Zeichen, im Falle des Dictionaries das Tupel der

Schlüssel.

Methoden des Tupels

Da das Tupel unveränderlich ist, stellt es nur Informationsmethoden zur Verfügung:

Methode	Bedeutung
`.count(x)`	Zählt das Vorkommen von x.
`.index(x)`	Gibt den Index des ersten Vorkommens von x zurück.

Erinnerung: Darüber hinaus sind die Built-in Functions `max()`, `min()`, `len()`, `sum()`, `sorted()` und der `in`-Operator nützlich!

Übung:

Erstellen Sie ein Dictionary, das eine Kontaktliste darstellt: Zu jedem Namen speichert es ein Tupel aus Telefonnummer, Geburtsdatum und Adresse. Testen Sie Ihr Dictionary mit einigen Dummy-Einträgen. Lesen Sie dabei die drei Attribute eines Eintrags einzeln aus (wenn Sie beispielsweise nur am Geburtsdatum interessiert sind, wollen Sie nicht die Telefonnummer oder die Adresse ausgeben).

Lösung:

```
contacts_dict = {}
# Tupel-Einträge für 3 Dummy-Kontakte:
contacts_dict["Harry"] = "0111-222", "1.1.1995", "Hogwarts 1"
contacts_dict["Hermine"] = "0222-1234", "3.2.1992", "Hogwarts 2"
contacts_dict["Ron"] ="0311-4321", "4.5.1994", "Hogwarts 3"
#  Hinweis: Auf der rechten Seite können runde Klammern benutzt werden,
# das ist aber nicht erforderlich. "Das Komma macht das Tupel!"


# Zugriffe auf die 3 Attribute eines Eintrags erfolgen
# über den korrekten Index des Tupels:
print("Rons Geburtstag:", contacts_dict["Ron"][1])
print("Rons Adresse:", contacts_dict["Ron"][2])
print("Rons Telefonnummer:", contacts_dict["Ron"][0])
```

8.5. Daten mehrerer Sensoren – verschachtelte Dictionaries

Stellen Sie sich vor, wir hätten drei anstelle von einem Sensor und wollten die Daten aller Sensoren strukturiert erfassen. Hierzu können wir zwei Dictionaries ineinander verschachteln: Der erste Key ist der Name des Sensors, zugehöriger Value ist ein Dictionary, das wie bisher die Messwerte dieses einen Sensors sammelt (also Zeitstempel als Key, Messwert als Value). Im Code sieht das so aus:

```
# Verschachtelte Dictionaries für drei Sensoren
data = {
    "Sensor1": {
        "2025-03-23 08:00": (22.5, 60),
        "2025-03-23 09:00": (23.1, 55)
    },
    "Sensor2": {
        "2025-03-23 08:00": (21.8, 57),
        "2025-03-23 09:00": (22.0, 67)
    },
    "Sensor3": {
        "2025-03-23 08:00": (24.0, 58),
        "2025-03-23 09:00": (23.8, 57)
    }
}

# So würde man Messwerte hinzufügen:
data["Sensor3"]["2025-03-23 10:00"] = (25.2, 62) # Neuer Wert für Sensor 3
```

Übung:

Erstellen Sie ein verschachteltes Dictionary, das eine Kontaktliste darstellt: Zu jedem Namen speichert es ein weiteres Dictionary mit den Keys `"Telefon"`, `"Geburtsdatum"` und `"Adresse"`. Testen Sie Ihr Dictionary mit einigen Dummy-Einträgen. Greifen Sie für einen Namen auf jedes der drei Attribute einzeln zu.

Lösung:

```
contacts_dict = {}
# Einträge für 3 Dummy-Kontakte:
contacts_dict["Harry"] = {"Telefon":"0111-222", "Geburtsdatum":"1.1.1995",
 ↪  "Adresse":"Hogwarts 1"}
contacts_dict["Hermine"] = {"Telefon":"0222-1234", "Geburtsdatum":"3.2.1992",
 ↪  "Adresse":"Hogwarts 2"}
contacts_dict["Ron"] ={"Telefon":"0311-4321", "Geburtsdatum":"4.5.1994",
 ↪  "Adresse":"Hogwarts 3"}
```

```
# Zugriffe:
print("Hermines Geburtsdatum:", contacts_dict["Hermine"]["Geburtsdatum"])
print("Hermines Adresse:", contacts_dict["Hermine"]["Adresse"])
print("Hermines Telefonnummer:", contacts_dict["Hermine"]["Telefon"])
```

8.6. Mutable und immutable Datenstrukturen

Die Unterscheidung zwischen *mutable* und *immutable* Datentypen haben wir bereits in früheren Kapiteln kennengelernt. Es ist bei der Programmierung wichtig, jederzeit zu wissen, ob man mit einem mutable oder immutable Datentyp arbeitet. Bei den grundlegenden Datenstrukturen in Python lässt sich das zum Glück leicht merken: **Nur das Tupel ist unveränderlich (immutable)**, während Listen, Sets und Dictionaries mutable sind. Achtung: Enthält ein Tupel selbst veränderliche Elemente, z.B. eine Liste, kann sich deren Inhalt trotzdem ändern.

Alle mutable Datenstrukturen können potenziell Seiteneffekte verursachen. Im Folgenden illustrieren wir dies anhand von zwei Beispielen:

```
# Beispiel: Seiteneffekte bei Sets
first_set = {"yellow", "red", "green"}
second_set = first_set
print("Identitäten gleich?", first_set is second_set)
second_set.add("blue")
print("First:", first_set, "Second:", second_set)
```

```
Identitäten gleich? True
First: {'green', 'red', 'yellow', 'blue'} Second: {'green', 'red', 'yellow', 'blue'}
```

```
# Beispiel: Seiteneffekte bei Dicts
first_dict = {"a": 1, "b" : 2}
second_dict = first_dict
second_dict["a"] = 42
print("First:",first_dict,"Second:",second_dict)
```

```
First: {'a': 42, 'b': 2} Second: {'a': 42, 'b': 2}
```

Und ein Tupel, das eine Liste enthält? Das Tupel selbst lässt sich nicht verändern, die Liste darin aber schon – zum Beispiel über eine zweite Referenz auf dieselbe Liste:

```
lst = [1, 2]
t = (lst, 3)    # das Tupel enthält eine Referenz auf die Liste
lst.append(99)  # die Liste wird über die zweite Referenz lst verändert ...
print(t)        # ... und das Tupel "sieht" die Änderung
```

```
([1, 2, 99], 3)
```

! Wichtig

Listen, Sets und Dictionaries sind mutable, Seiteneffekte können auftreten!
Tupel sind immutable und verursachen selbst keine Seiteneffekte – solange sie keine veränderlichen Elemente wie Listen enthalten.

Übung (Was gibt der Code aus?): Überlegen Sie *vor* dem Ausführen, welche Werte `a` und `b` am Ende haben:

```
a = [1, 2]
b = a
a += [3]
a = a + [4]
print(a, b)
```

Auflösung

Ausgabe: `[1, 2, 3, 4] [1, 2, 3]`. Überrascht? Bei Listen verändert `+=` die bestehende Liste direkt – ähnlich wie `append()`. Deshalb sieht auch `b` die 3. `a = a + [4]` erzeugt dagegen eine **neue** Liste und lässt nur `a` darauf zeigen; `b` zeigt weiter auf die alte. Bei unveränderlichen Datentypen wie Zahlen, Strings oder Tupeln gibt es diesen Unterschied nicht, denn sie können gar nicht „an Ort und Stelle“ verändert werden.
Merke: `liste += …` verändert die Liste, `liste = liste + …` erzeugt eine neue.
Für Fortgeschrittene: Technisch steckt dahinter, dass Python bei `a += b` zuerst prüft, ob der Datentyp von `a` eine spezielle Methode `__iadd__` (*in-place add*) besitzt. Listen haben eine solche Methode, die die Liste direkt verändert. Zahlen, Strings und Tupel haben sie nicht – dann führt Python stattdessen `a = a + b` aus. Solche Methoden mit doppelten Unterstrichen heißen *Magic Methods*; mehr dazu im Kapitel *Objektorientierte Programmierung*.

8.7. Sicheres Kopieren von Listen

Seien Sie immer auf der Hut, wenn Sie eine Liste kopieren wollen. Da sie zu den mutable Datentypen gehört, erzeugen Sie schnell unbeabsichtigt nur eine zweite Referenz auf das Original mit den genannten Seiteneffekten als Gefahr. Eine einfache Möglichkeit, dies zu umgehen, ist eine Kopie mithilfe des Slicing-Operators:

```
original = ["apples", "oranges", "books"]

# Zuweisung erzeugt nur eine zweite Referenz:
second_reference = original  # keine Kopie!

# mit Slicing eine Kopie erzeugen:
my_copy = original[:]

# Wir zeigen, dass my_copy keine Seiteneffekte hat:
my_copy.append("bananas")
print("Original: ", original) # unverändert
print("Kopie: ", my_copy)
```

```
Original:  ['apples', 'oranges', 'books']
Kopie:  ['apples', 'oranges', 'books', 'bananas']
```

Es gibt allerdings noch eine Diffizilität: Der Trick mit dem Slicing-Operator funktioniert nur so lange, wie die Elemente der Liste immutable sind. Sollten sich darunter aber mutable Datentypen befinden, kann es immer noch zu Seiteneffekten kommen. Wir haben zwar eine Kopie erstellt, allerdings nur eine sog. “flache Kopie”. Um alle Mehrfachreferenzen aufzulösen und eine völlig unabhängige Kopie zu erhalten, braucht es eine sog. “tiefe Kopie”. Hierfür gibt es bestimmte Algorithmen aus dem Modul `copy`:

```
import copy
original = [[1,2],3,4] # Liste mit einem mutable Element

copy1 = original[:] # flache Kopie
copy2 = copy.deepcopy(original) # tiefe Kopie

# Demo der Seiteneffekte:
# Das mutable erste Element wird in den Kopien verändert
copy1[0].append(3)
copy2[0].append(100)
```

```
print("Original: ", original)
print("flache Kopie: ", copy1)
print("tiefe Kopie: ", copy2)
```

```
Original:  [[1, 2, 3], 3, 4]
flache Kopie:  [[1, 2, 3], 3, 4]
tiefe Kopie:  [[1, 2, 100], 3, 4]
```

Die Änderungen an der flachen Kopie hatten noch Auswirkungen auf das Original, weil beide die Referenz auf das erste Element teilten, welches mutable war. Nur bei der tiefen Kopie handelt es sich um eine völlig autarke Kopie, die nichts mehr mit dem Original gemein hat (Abbildung 8.2).

```
import copy
original = [[1, 2], 3, 4]
copy1 = original[:]              # flache Kopie
copy2 = copy.deepcopy(original)  # tiefe Kopie
copy1[0].append(3)
copy2[0].append(100)
```

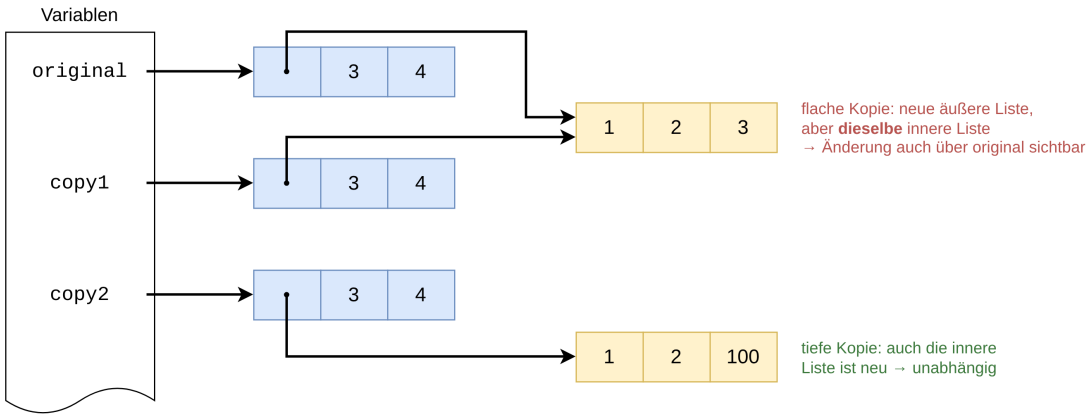


Abbildung 8.2.: Flache Kopie: Die äußere Liste ist neu, die innere Liste wird aber geteilt. Tiefe Kopie: Auch die innere Liste wird kopiert.

Vier Wege zur flachen Kopie

Die folgenden vier Varianten erzeugen alle eine **flache Kopie**:

1. Slicing: `my_copy = original[:]`
2. `list`-Funktion: `my_copy = list(original)`
3. Methode `.copy()`: `my_copy = original.copy()`

4. Funktion `copy()` aus Modul `copy`: `my_copy = copy.copy(original)`

Die tiefe Kopie (`copy.deepcopy`) ist nicht als Built-in Function verfügbar, sondern muss aus dem Modul `copy` importiert werden. Das passt gut zur Python-Philosophie *„Explicit is better than implicit“*: Eine tiefe Kopie sollte nur dann eingesetzt werden, wenn man sie wirklich benötigt. Da die Operation rekursiv alle enthaltenen Objekte dupliziert, ist sie vergleichsweise aufwendig in Bezug auf Laufzeit und Speicherverbrauch. In vielen Fällen ist es daher sinnvoller, benötigte Datenstrukturen gezielt neu aufzubauen oder nur Teilstrukturen zu kopieren, anstatt pauschal eine tiefe Kopie der gesamten Struktur zu erzeugen.

8.8. List Comprehensions

In Python gibt es eine elegante und kompakte Möglichkeit, Listen zu erzeugen: die sogenannte **List Comprehension**.

Statt mit einer Schleife umständlich Elemente einer Liste hinzuzufügen, können wir in einer einzigen Zeile eine neue Liste erstellen. Das macht den Code kürzer, lesbarer und oft schneller.

Syntax

Die allgemeine Syntax sieht so aus:

```
[ Ausdruck for Element in Sequenz ]
```

- **Sequenz**: Etwas, über das iteriert werden kann (z.B. eine Liste, ein String, ein Bereich `range()`), wie wir es schon von der for-Schleife kennen.
- **Element**: Eine Variable, die nacheinander die Werte aus der Sequenz annimmt.
- **Ausdruck**: Ein Ausdruck, der für jedes Element ausgewertet wird. Dieser wird dann der neuen Liste hinzugefügt.

Der hintere Teil der List Comprehension verhält sich ganz analog zur for-Schleife, neu ist der Ausdruck. Ein Beispiel macht alles verständlicher:

```
# Beispiel: Liste aller Quadratzahlen der Zahlen von 0 bis 9
squares = [x**2 for x in range(10)]
print(squares)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Analysieren wir das Beispiel:

- **Sequenz**: `range(10)`.
- **Element**: `x`, eine eigene Bezeichnung für die Elemente der Sequenz.
- **Ausdruck**: `x**2`, jedes `x` aus der genannten Sequenz soll quadriert werden.

Die Syntax kann noch um eine `if`-Bedingung am Ende erweitert werden:

```
[ Ausdruck for Element in Sequenz if Bedingung ]
```

- **Bedingung** am Ende der List Comprehension: Nur Elemente, die die Bedingung erfüllen, werden berücksichtigt.

```
# Ein Beispiel: Liste aller Quadratzahlen der geraden Zahlen von 0 bis 9
squares2 = [x**2 for x in range(10) if x % 2 == 0]
print(squares2)
```

```
[0, 4, 16, 36, 64]
```

Darüber hinaus können auch `if`- und `else`-Bedingungen vor dem `for` verwendet werden, um alternative Ausdrücke zu verwenden (vor dem `for` ist das `else` Pflicht; ein reiner Filter ohne `else` gehört ans Ende):

```
# Mit if-else vor dem for
result = [x if x % 2 == 0 else -x for x in range(10)]
print(result)
```

```
[0, -1, 2, -3, 4, -5, 6, -7, 8, -9]
```

Je komplexer eine List Comprehension wird, desto schwieriger wird sie allerdings zu lesen und zu verstehen – und das kann schnell die Vorteile zunichtemachen, die man eigentlich durch die kürzere Schreibweise erreichen wollte.

Beispiele aus der Ingenieurpraxis

List Comprehensions sind sehr praktisch, um Messwerte von einer Einheit in eine andere umzurechnen:

```
values_in_cm = [100, 250, 430, 125]
values_in_m = [x / 100 for x in values_in_cm]
print(values_in_m)
```

```
[1.0, 2.5, 4.3, 1.25]
```

```
temperatures_celsius = [0, 20, 37, 100]
temperatures_fahrenheit = [t * 9/5 + 32 for t in temperatures_celsius]
print(temperatures_fahrenheit)
```

```
[32.0, 68.0, 98.6, 212.0]
```

List Comprehensions werden auch gerne zum Filtern von Messwerten verwendet, wie das folgende Beispiel zeigt:

```
voltages = [3.3, 5.1, 2.5, 4.8, 2.1, 7.2, 0.7]
high_voltages = [u for u in voltages if u > 5]  # 5 V ist hier ein oberer
↪ Grenzwert
low_voltages = [u for u in voltages if u <= 5]
print("Oberhalb des Grenzwertes:", high_voltages)
print("Gültige Messwerte:", low_voltages)
```

```
Oberhalb des Grenzwertes: [5.1, 7.2]
Gültige Messwerte: [3.3, 2.5, 4.8, 2.1, 0.7]
```

```
# Alternativ könnte man die zu hohen Werte "abschneiden":
voltages = [3.3, 5.1, 2.5, 4.8, 2.1, 7.2, 0.7]
limited_voltages = [u if u <= 5 else 5 for u in voltages]  # 5 V ist hier ein
↪ oberer Grenzwert
print("Bereinigte Messwerte:", limited_voltages)
```

```
Bereinigte Messwerte: [3.3, 5, 2.5, 4.8, 2.1, 5, 0.7]
```

⚠ Fallstrick: Elemente entfernen, während man über die Liste iteriert

```
voltages = [4.8, 5.3, 5.6, 4.9]
for u in voltages:
    if u > 5.0:
        voltages.remove(u)
print(voltages)   # [4.8, 5.6, 4.9] - 5.6 wurde übersehen!
```

Die Schleife merkt sich intern, an welcher Position sie gerade ist. Nach dem Entfernen von 5.3 rutscht 5.6 auf dessen Platz nach. Die Schleife macht aber an der nächsten Position

weiter und überspringt 5.6 dadurch.
Lösung: Die Schleife wird **komplett ersetzt**. Statt Elemente aus der alten Liste zu entfernen, baut man eine **neue** Liste, die nur die gewünschten Elemente enthält:

```
voltages = [4.8, 5.3, 5.6, 4.9]
voltages = [u for u in voltages if u <= 5.0]   # ersetzt die ganze
 ↪ Schleife
print(voltages)   # [4.8, 4.9]
```

Ausgeschrieben als Schleife sieht dasselbe so aus:

```
voltages = [4.8, 5.3, 5.6, 4.9]
valid = []
for u in voltages:          # über die alte Liste iterieren ...
    if u <= 5.0:
        valid.append(u)     # ... aber nur die neue Liste verändern
voltages = valid
```

⚠ Fallstrick: Verschachtelte Listen mit `*` erzeugen

```
matrix = [[0] * 3] * 3   # sieht aus wie eine 3x3-Matrix ...
matrix[0][0] = 1
print(matrix)
# Ausgabe: [[1, 0, 0], [1, 0, 0], [1, 0, 0]] - alle Zeilen geändert!
```

Was passiert hier? Python wertet den inneren Ausdruck `[0] * 3` **genau einmal** aus. Dabei entsteht eine einzige Liste `[0, 0, 0]`. Das äußere `* 3` kopiert diese Liste nicht, sondern vervielfältigt nur die **Referenz** auf sie. Das ist wie bei `b = a`, das ja auch nie kopiert. Die Zeile bedeutet also dasselbe wie:

```
row = [0] * 3
matrix = [row, row, row]       # dreimal dieselbe Liste
print(matrix[0] is matrix[1])  # True
```

Richtig: `matrix = [[0] * 3 for i in range(3)]`
Der Unterschied (Abbildung 8.3): Die List Comprehension führt den Ausdruck `[0] * 3` in **jedem Durchlauf neu** aus. So entstehen drei verschiedene Listen, genau wie bei dieser Schleife:

```python
matrix = []
for i in range(3):
    matrix.append([0] * 3)      # in jedem Durchlauf eine neue Liste

print(matrix[0] is matrix[1])  # False
matrix[0][0] = 1
print(matrix)                  # [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

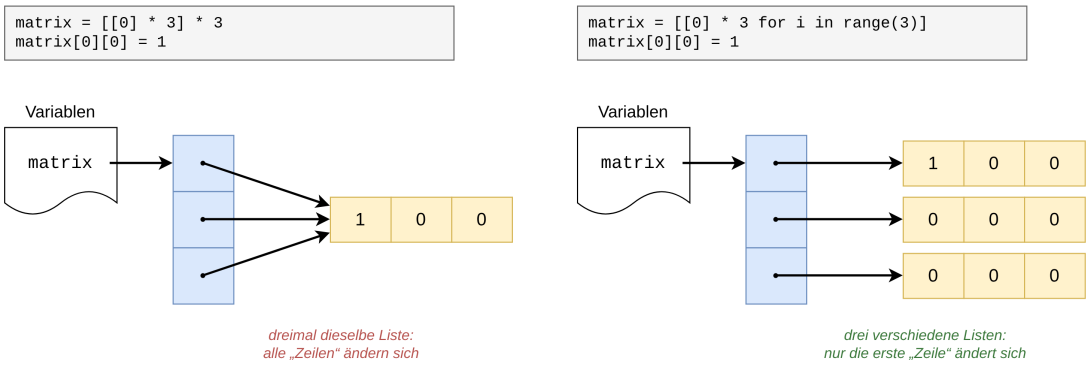


Abbildung 8.3.: Links: `* 3` vervielfältigt nur die Referenz auf eine einzige Zeile. Rechts: Die List Comprehension erzeugt drei verschiedene Listen.

Auch `[0] * 3` vervielfältigt übrigens nur Referenzen, nämlich auf die Zahl 0. Das ist aber harmlos: `int` ist immutable, und `matrix[0][0] = 1` lässt nur diesen einen Platz auf ein neues Objekt zeigen.

Übungen: Gegeben ist eine Liste, die Geschwindigkeitsmessungen in der Einheit km/h beinhaltet: `speeds = [30.0, 50.7, 45.3, 77.4, 100.9]`

1. Erstellen Sie mit einer List Comprehension eine Liste, die die Geschwindigkeiten in der Einheit m/s, gerundet auf zwei Nachkommastellen, beinhaltet.
2. Passen Sie Ihre List Comprehension so an, dass nur Werte oberhalb von 50 km/h umgerechnet und in die neue Liste aufgenommen werden.

Gegeben ist eine Liste von Fahrten, jeweils als Tupel (Strecke in km, Zeit in Minuten): `trips = [(10, 30), (25, 60), (20, 20), (5, 10)]`.

3. Erstellen Sie mit einer List Comprehension eine Liste der jeweiligen Durchschnittsgeschwindigkeiten in der Einheit km/h.
4. Erstellen Sie mit einer List Comprehension eine Liste aus Strings mit den Labeln “langsam” (für Fahrten mit einer Durchschnittsgeschwindigkeit unter 50 km/h) und “schnell” (für alle anderen).

```python
speeds = [30.0, 50.7, 45.3, 77.4, 100.9]

# Übung 1: Einheit umrechnen nach m/s
# Umrechnungsfaktor ist *1000/3600 (oder dividieren durch 3.6)
# runden auf 2 Nachkommastellen: mit round(..., 2)
speeds_ms = [round(kmh * 1000/3600, 2) for kmh in speeds]
print("Ergebnis Übung 1:", speeds_ms)

# Übung 2: nur für solche mit kmh > 50
fast_speeds_ms = [round(kmh * 1000/3600, 2) for kmh in speeds if kmh > 50]
print("Ergebnis Übung 2:", fast_speeds_ms)

# Übung 3
trips = [(10, 30), (25, 60), (20, 20), (5, 10)]
# gesuchte Durchschnittsgeschwindigkeit: Strecke in km / Zeit in h
# Die Zeiten müssen von min in h umgerechnet werden: durch 60 dividieren
average_speeds = [trip[0]/(trip[1]/60) for trip in trips]
print("Ergebnis Übung 3:", average_speeds)

# Übung 4
trip_labels = ["langsam" if v < 50 else "schnell" for v in average_speeds]
print("Ergebnis Übung 4:", trip_labels)
```

Performance

List Comprehensions sind in der Regel schneller als eine klassische Schleife mit `append()`, weil Python dabei intern einige Schritte einsparen kann, vor allem den wiederholten Aufruf der Methode `append()`. Eine kleine Demo:

```python
import time  # zum Messen von Zeiten

# Klassische For-Loop mit append
start = time.time()

result = []
for i in range(10000000):
    if i % 2 == 0:
        result.append(i * 2)
end = time.time()

print(f"For-Loop Dauer: {round(end - start,2)} Sekunden")
```

```python
# List Comprehension mit demselben Ergebnis

start = time.time()

result = [i * 2 for i in range(10000000) if i % 2 == 0]

end = time.time()

print(f"List Comprehension Dauer: {round(end - start,2)} Sekunden")
```

```
For-Loop Dauer: 2.12 Sekunden
List Comprehension Dauer: 1.01 Sekunden
```

Die Zeiten variieren natürlich und werden auf Ihrem System andere Werte annehmen. Wie groß der Unterschied ausfällt, hängt von Python-Version und Rechner ab; oft ist er geringer als hier. Probieren Sie es aus!

Ob ein Code, wie in diesem Beispiel, eine Sekunde oder zwei Sekunden benötigt, spielt natürlich keine große Rolle. Der Performance-Unterschied wird nur relevant, wenn Sie enorm große Mengen an Daten verarbeiten müssen.

Fazit für die Praxis

List Comprehensions sind oft eine gute Wahl, wenn eine neue Liste erzeugt werden soll. Sie führen in der Regel zu kürzerem und verständlicherem Code. Allerdings sollte man sie nur verwenden, wenn Bedingungen und Ausdrücke einfach genug sind, um die Lesbarkeit nicht zu gefährden. Bei komplexerer Logik ist eine klassische Schleife oft die bessere Wahl, um die Wartbarkeit zu sichern. Sollte Performance eine entscheidende Rolle spielen, kann es sich lohnen, größere Schleifen in eine List Comprehension umzuschreiben – vorausgesetzt, die Lesbarkeit bleibt erhalten.

> **Set- und Dictionary-Comprehensions**
>
> Auch mit geschweiften Klammern lassen sich Comprehensions erzeugen, probieren Sie es aus:
> `my_set = {i**2 for i in range(10)}  # Set-Comprehension`
> `my_dict = {x: x**2 for x in range(10)} # Dict-Comprehension`
> **Hinweis:** Wenn man stattdessen runde Klammern verwendet, entsteht **kein Tupel**, wie man erwarten könnte, sondern ein *Generator-Objekt*. Generatoren gehen über den Rahmen dieser Einführung hinaus. Hier sei nur erwähnt, dass das Generator-Objekt an

die Funktion `tuple()` übergeben werden kann, um auf diese Weise ein Tupel zu erzeugen: `my_tuple = tuple((i**2 for i in range(10)))  # Generator-Objekt wird an tuple() übergeben`
So erhält man funktional eine *Tuple-Comprehension*, obwohl es dafür in Python keine direkte Syntax gibt.

8.9. Eleganter iterieren: `enumerate()` und `zip()`

In Schleifen braucht man oft mehr als nur das aktuelle Element: seine **Position** in der Liste oder das **passende Element einer zweiten Liste**, etwa den Zeitpunkt zu einem Messwert. Mit `range(len(...))` und Indexzugriffen geht das zwar, Python bietet dafür aber zwei eingebaute Funktionen, die den Code kürzer und lesbarer machen.

`enumerate()` liefert bei jedem Schleifendurchlauf ein Tupel aus **Index und Element**, das wir – wie bei `.items()` – direkt in zwei Variablen entpacken:

```
temperatures = [21.4, 23.9, 27.5, 31.2, 36.8]

for i, temp in enumerate(temperatures):
    print(f"Messung {i}: {temp} °C")
```

```
Messung 0: 21.4 °C
Messung 1: 23.9 °C
Messung 2: 27.5 °C
Messung 3: 31.2 °C
Messung 4: 36.8 °C
```

`zip()` fügt mehrere Sequenzen zusammen wie ein Reißverschluss – daher der Name: Bei jedem Durchlauf erhalten wir ein Tupel mit den Elementen, die in den Sequenzen an derselben Position stehen:

```
times_min = [0, 5, 10, 15, 20]

for t, temp in zip(times_min, temperatures):
    print(f"t = {t} min: {temp} °C")
```

```
t = 0 min: 21.4 °C
t = 5 min: 23.9 °C
t = 10 min: 27.5 °C
```

```
t = 15 min: 31.2 °C
t = 20 min: 36.8 °C
```

Was beide Funktionen leisten – und wo die Falle lauert –, erleben Sie am besten selbst:

Übungen:

1. Gegeben ist die Liste `sensors = ["T_Einlass", "T_Lager", "T_Auslass"]`. Geben Sie die Sensoren nummeriert aus, beginnend bei 1 (`1: T_Einlass` usw.) – einmal mit `range(len(sensors))` und Indexzugriff, einmal mit `enumerate(sensors, start=1)`. Welche Variante ist lesbarer?
2. Im Kapitel *Kontrollstrukturen* haben wir das Skalarprodukt der Vektoren `v = [1.0, -2.0, 1.5]` und `w = [-1.0, 1.0, 0.5]` mit einer Indexschleife berechnet. Berechnen Sie es nun in einer einzigen Zeile mit `zip()`, einer List Comprehension und der eingebauten Funktion `sum()`. Funktioniert Ihre Lösung ohne Änderung auch für Vektoren mit vier Komponenten?
3. Zu den Sensoren aus Übung 1 gibt es die aktuellen Messwerte `readings = [24.3, 41.7, 30.2]`. Erzeugen Sie mit `zip()` und `dict()` ein Dictionary, das jedem Sensornamen seinen Messwert zuordnet, und geben Sie die Temperatur von `"T_Lager"` aus. (Tipp: Wie erzeugt man ein Dictionary aus einer Liste von Tupeln?)
4. Bei einer Messung ist ein Wert verloren gegangen: `times_min = [0, 5, 10, 15, 20]`, aber `temperatures = [21.4, 23.9, 27.5, 31.2]`. Was gibt `list(zip(times_min, temperatures))` aus? Was fällt Ihnen auf, und warum ist das bei Messdaten gefährlich? Wiederholen Sie den Aufruf mit `zip(times_min, temperatures, strict=True)`.

```python
# Übung 1
sensors = ["T_Einlass", "T_Lager", "T_Auslass"]

for i in range(len(sensors)):  # mit Index: das "+ 1" vergisst man leicht
    print(f"{i + 1}: {sensors[i]}")

for number, name in enumerate(sensors, start=1):  # mit enumerate()
    print(f"{number}: {name}")

# Übung 2
v = [1.0, -2.0, 1.5]
w = [-1.0, 1.0, 0.5]
v_dot_w = sum([vi * wi for vi, wi in zip(v, w)])
print(v_dot_w)  # -2.25, wie im Kapitel Kontrollstrukturen
# Anders als range(3) passt sich zip() automatisch an die Länge der Vektoren
 ↪  an.

# Übung 3
```

```
readings = [24.3, 41.7, 30.2]
sensors_dict = dict(zip(sensors, readings))
print(sensors_dict)
print("Lagertemperatur:", sensors_dict["T_Lager"], "°C")

# Übung 4
times_min = [0, 5, 10, 15, 20]
temperatures = [21.4, 23.9, 27.5, 31.2]
print(list(zip(times_min, temperatures)))
# Der Zeitpunkt 20 fehlt einfach - ohne jede Warnung!
# Schlimmer noch: zip() weiß nicht, WELCHER Wert fehlt. Fehlte z.B. der Wert
 ↪  bei 10 min,
# wären alle folgenden Temperaturen stillschweigend dem falschen Zeitpunkt
 ↪  zugeordnet.
# Mit strict=True gibt es stattdessen eine Fehlermeldung:
# list(zip(times_min, temperatures, strict=True))
# -> ValueError: zip() argument 2 is shorter than argument 1
```

! `zip()` bei unterschiedlich langen Sequenzen

`zip()` hört auf, sobald die **kürzeste** Sequenz erschöpft ist – überzählige Elemente werden stillschweigend ignoriert. Bei Messdaten verwenden Sie deshalb besser `zip(a, b, strict=True)`: Sind die Sequenzen unterschiedlich lang, bricht das Programm mit einem `ValueError` ab, statt unbemerkt mit unvollständigen Daten weiterzurechnen.

9. Die Verwendung von Bibliotheken

> **Lernziele**
>
> Nach diesem Kapitel können Sie …
>
> - Module importieren, Aliase verwenden, fehlende Pakete installieren und die Begriffe Modul, Paket und Bibliothek unterscheiden
> - NumPy-Arrays erzeugen und für Matrixoperationen, Vektorisierung und Broadcasting nutzen
> - Funktionen und Messwerte mit Matplotlib darstellen (Linien-, Streu- und Balkendiagramm) und eine Ausgleichskurve anpassen
> - Messdaten mit pandas einlesen, sich einen Überblick verschaffen und neue Größen berechnen

In diesem Kapitel führen wir kurz in die Verwendung von Bibliotheken ein. Es gibt unzählige praktische Bibliotheken, die Entwicklerinnen und Entwicklern auf der ganzen Welt das Leben leichter machen. Natürlich können wir im Rahmen dieser Einführung nur einen kurzen Einblick geben: Wir erklären allgemein, was eine Bibliothek ist, wie man sie verwendet und installiert, und werfen dann einen exemplarischen Blick auf bekannte Bibliotheken, die Ihnen mit hoher Wahrscheinlichkeit in Ihren Projekten begegnen und helfen werden.

9.1. Einführung

Eine **Bibliothek** ist eine Sammlung von fertigem Code, die Sie in Ihrem Programm nutzen können. So müssen Sie das Rad nicht immer wieder neu erfinden! Außerdem sind Bibliotheken oft hochoptimiert, teilweise "unter der Haube" in anderen Programmiersprachen geschrieben, um eine bessere Performance zu erzielen (z.B. in C oder C++).

Bevor wir eine Bibliothek in unserem Programm verwenden können, müssen wir sie **importieren**:

```
import math  # importiert die Bibliothek math

# jetzt kann math genutzt werden und das geht so:
```

```
print(math.sqrt(256))  # Wurzel ziehen
print(math.cos(2*math.pi))  # Trigonometrie
print(math.factorial(5))  # Fakultät

# vieles mehr ist möglich mit math ...
```

```
16.0
1.0
120
```

Wie Sie am Beispiel erkannt haben, wird der Name der Bibliothek den Funktionen vorangestellt. Über den `.` greifen Sie auf die Inhalte der Bibliothek zu. Man kann auch nur ausgewählte Elemente der Bibliothek importieren:

```
from math import cos, sin, pi  # nur die genannten werden importiert

print(cos(2*pi) + sin(2*pi))  # das Präfix math fällt weg
```

```
0.9999999999999998
```

Tipp: Ich bevorzuge es, zu sehen, aus welcher Bibliothek die Funktionen stammen. So werden auch potenzielle Konflikte mit eigenen Bezeichnungen vermieden. Daher nutze ich meistens die obere Variante.

Aliase

Es ist möglich und auch üblich, einer Bibliothek beim Importieren einen Kurznamen (sog. *Alias*) zu geben. Wir könnten z.B. die `math`-Bibliothek mit `m` abkürzen:

```
import math as m  # importiert math mit Alias m

# Verwendung:
print(m.sqrt(225), m.cos(m.pi), m.factorial(3))
```

```
15.0 -1.0 6
```

Häufig verwendete Bibliotheken haben mittlerweile Standard-Aliase, die so gut wie immer verwendet werden. Wir listen einige berühmte Beispiele auf:

Bibliothek	Typischer Alias	Import	Anwendungsbereich
numpy	np	import numpy as np	Numerische Mathematik
matplotlib.pyplot	plt	import matplotlib.pyplot as plt	Visualisierung / Diagramme
pandas	pd	import pandas as pd	Datenanalyse und Tabellenverarbeitung
seaborn	sns	import seaborn as sns	Statistische Datenvisualisierung
tensorflow	tf	import tensorflow as tf	Maschinelles Lernen / Neuronale Netze

Installation

Nicht alle Bibliotheken sind auf unserem System vorinstalliert; manche müssen bei Bedarf erst heruntergeladen und installiert werden. Das geht zum Glück mithilfe von Paketverwaltungssoftware wie *pip* oder *conda* sehr einfach. Nehmen wir an, wir möchten die Bibliothek `requests` (für HTTP-Requests) installieren:

- **Mit pip (für Standard-Python ohne Anaconda)**

  Im Terminal oder in der Konsole eingeben:

  ```
  pip install requests
  ```

- **Mit Anaconda**

  Starten Sie **Anaconda Prompt** und tippen Sie ein:

  ```
  conda install requests
  ```

- **Über die grafische Oberfläche von Anaconda**

  1. Starten Sie **Anaconda Navigator**.
  2. Wählen Sie „Environments“ und setzen Sie den Filter auf „not installed“ oder „all“.
  3. Verwenden Sie „Search Packages“, um das gewünschte Paket zu finden (z.B. requests).
  4. Häkchen setzen und „Apply“ klicken.

Kurze Begriffsklärung: Modul, Paket, Bibliothek

Diese drei Begriffe gehen manchmal durcheinander und werden nicht immer sauber voneinander getrennt:

- **Modul:**
  Eine einzelne Datei mit Python-Code (.py), die Funktionen, Klassen oder Variablen enthält.
- **Paket:** Ein strukturierter Ordner mit mehreren Modulen.
- **Bibliothek:** Ein allgemeiner Begriff für Module und Pakete, die bestimmte Aufgaben lösen (z.B. Datenanalyse).

Bei der praktischen Verwendung von Bibliotheken ist die Unterscheidung in der Regel nicht so wichtig: Wir importieren `math` (ein Modul) oder `pandas` (ein Paket), um Probleme zu lösen, die uns in unseren Projekten begegnen. Wenn wir dagegen selbst Module oder Bibliotheken für die Community entwickeln möchten, werden die Details natürlich wichtig.

9.2. NumPy

Überblick

NumPy (Numerical Python) ist eine der zentralen Bibliotheken im wissenschaftlichen Rechnen mit Python. Sie bietet leistungsfähige Datenstrukturen, insbesondere das n-dimensionale Array, und eine breite Palette mathematischer Funktionen zur effizienten Verarbeitung großer Datenmengen. Für Ingenieurinnen und Ingenieure ist NumPy besonders wertvoll, da es numerische Simulationen, Datenanalyse und Algorithmenentwicklung deutlich vereinfacht und beschleunigt. Damit es keine Missverständnisse gibt: Numerisch bedeutet, dass wir in NumPy immer mit konkreten Zahlen rechnen und solche als Ergebnisse erhalten. Das Gegenteil wären symbolische Berechnungen, bei denen es darum geht, eine Formel als Lösung für ein Problem zu finden. Hierfür gibt es übrigens auch eine Python-Bibliothek: *SymPy (Symbolic Python)*.

Drei Highlights von NumPy für die Ingenieurpraxis:

- Effiziente Matrixoperationen: Lineare Algebra ist das Rückgrat vieler ingenieurtechnischer Berechnungen, z.B. in der Statik, Regelungstechnik oder Signalverarbeitung. NumPy ermöglicht schnelle und speichereffiziente Operationen auf Matrizen – inklusive Inversion, Eigenwertberechnung oder Lösung linearer Gleichungssysteme.
- Schnelle Datenverarbeitung: Dank einer optimierten Implementierung (der rechenintensive Kern ist in C geschrieben) ist NumPy um ein Vielfaches schneller als Standard-Python-Schleifen. Das ist ideal für Messdatenanalysen oder Simulationen, bei denen Zeit eine kritische Rolle spielt.

- Nahtlose Integration mit anderen Tools: NumPy bildet die Grundlage für viele andere wissenschaftliche Bibliotheken wie SciPy, pandas oder Matplotlib. So lässt es sich leicht in bestehende Arbeitsabläufe integrieren – vom Datenimport bis zur Visualisierung der Ergebnisse.

NumPy wird üblicherweise mit dem Alias `np` importiert:

```
import numpy as np
```

Arrays

In NumPy arbeitet man mit dem optimierten Datentyp `ndarray`. Wir sagen auch vereinfacht nur "Array" oder "NumPy-Array". Das "nd" steht für "n-dimensional", denn ein NumPy-Array kann genau wie die Liste als Vektor, Matrix oder höherdimensionaler Tensor (Verallgemeinerung einer Matrix auf mehr Dimensionen) angelegt werden. Es gibt einen wichtigen Unterschied zur Liste: **Alle Elemente eines NumPy-Arrays müssen denselben Datentyp haben!**

Erzeugung von Arrays

Arrays werden typischerweise aus Listen erzeugt oder mit speziellen Funktionen:

```
# 1-dimensionales Array aus einer Python-Liste
vector = np.array([1, 2, 3, 4])
print(vector)
```

```
[1 2 3 4]
```

```
print(type(vector))
```

```
<class 'numpy.ndarray'>
```

```
# 2-dimensionales Array aus einer Python-Liste
matrix = np.array([[1, 2],[3, 4]])
print(matrix)  # erkennen Sie den Unterschied zu der Ausgabe einer
 ↪  verschachtelten Liste?
```

```
[[1 2]
 [3 4]]
```

```
print(type(matrix))
```

```
<class 'numpy.ndarray'>
```

```
# Mit np.zeros und np.ones
zeros = np.zeros((2, 3))  # 2 x 3 Matrix mit Nullen
print(zeros)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

```
ones = np.ones((3, 4))  # 3 x 4 Matrix mit Einsen
print(ones)
```

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

```
# Mit np.arange(start, stop, step)
# verhält sich analog zu range(start, stop, step), aber step kann auch ein
 ↪  Float sein!
arange_test = np.arange(0, 10, 2)
print(arange_test)
```

```
[0 2 4 6 8]
```

```
arange_test2 = np.arange(0, 5, 0.5)  # step ist hier ein Float
# Vorsicht: Bei Float-Schrittweiten können Rundungsfehler ein Element zu viel
 ↪  oder zu wenig
# erzeugen - für eine feste Anzahl Punkte besser np.linspace verwenden (siehe
 ↪  unten).
print(arange_test2)
```

```
[0.  0.5 1.  1.5 2.  2.5 3.  3.5 4.  4.5]
```

```
# Mit np.linspace(start, stop, num)
# erzeugt man ein Array mit äquidistanten Punkten zwischen
# den Grenzen start und stop (standardmäßig beide eingeschlossen!).
# Es werden insgesamt num Punkte erzeugt.

linspace_test = np.linspace(0, 1, 5)

# Es werden exakt 5 äquidistante Werte von 0 bis 1 erzeugt
print(linspace_test)
```

```
[0.   0.25 0.5  0.75 1.  ]
```

```
# Ein zweites Beispiel:
linspace_test2 = np.linspace(-2, 3, 10)

# Es werden exakt 10 äquidistante Werte von -2 bis 3 erzeugt
print(linspace_test2)
```

```
[-2.         -1.44444444 -0.88888889 -0.33333333  0.22222222  0.77777778
  1.33333333  1.88888889  2.44444444  3.        ]
```

> Der Parameter `endpoint` von `np.linspace`
>
> Bei `np.linspace(start, stop, num)` sind Anfangs- **und** Endwert standardmäßig inkludiert! Dies ist ein wichtiger Unterschied zu `np.arange(start, stop, step)` oder `range(start, stop, step)`.
> Möchte man den Endwert bei `np.linspace` bewusst exkludieren, kann dies über das Keyword-Argument `endpoint` erfolgen: `np.linspace(start, stop, num, endpoint = False) # Endpunkt ausgeschlossen`

Dimensionen und Shape

Jedes Array hat die Attribute `.ndim` (Anzahl Dimensionen), `.shape` (Größe pro Dimension) und `.size` (Gesamtzahl der Elemente):

```
my_array = np.array([[1, 2, 3], [4, 5, 6]])
print("Array:")
print(my_array)
print("Dimensionen:", my_array.ndim)  # Matrix, also 2-dimensional
```

```
print("Shape:", my_array.shape)  # 2 x 3 Matrix
print("Anzahl Elemente:", my_array.size)  # 6 Elemente
```

```
Array:
[[1 2 3]
 [4 5 6]]
Dimensionen: 2
Shape: (2, 3)
Anzahl Elemente: 6
```

```
my_array = np.array([1, 2, 3, 4])
print("Array:")
print(my_array)
print("Dimensionen:", my_array.ndim)  # Vektor, also 1-dimensional
print("Shape:", my_array.shape)  # Vektor mit Länge 4
print("Anzahl Elemente:", my_array.size)  # 4 Elemente
```

```
Array:
[1 2 3 4]
Dimensionen: 1
Shape: (4,)
Anzahl Elemente: 4
```

Beachten Sie, dass `shape` immer vom Typ `tuple` ist!

Indexzugriff und Slicing

Der Zugriff auf Elemente erfolgt ähnlich wie bei Python-Listen. Bei mehrdimensionalen Arrays werden die Indizes jedoch **durch Kommata getrennt** innerhalb eines einzigen Paares von eckigen Klammern.

```
my_array = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
print("Array:")
print(my_array)

# Einzelne Elemente
print("my_array[0, 0] =", my_array[0, 0])
print("my_array[2, 1] =", my_array[2, 1])
```

```
Array:
[[10 20 30]
 [40 50 60]
 [70 80 90]]
my_array[0, 0] = 10
my_array[2, 1] = 80
```

```python
# Slicing (Logik wie bei Python-Listen)
print("Komplette Matrix:\n", my_array)
print("Erste Zeile:", my_array[0, :])
print("Zweite Spalte:", my_array[:, 1])
print("Teilarray:\n", my_array[1:, :])
```

```
Komplette Matrix:
 [[10 20 30]
 [40 50 60]
 [70 80 90]]
Erste Zeile: [10 20 30]
Zweite Spalte: [20 50 80]
Teilarray:
 [[40 50 60]
 [70 80 90]]
```

Ein wichtiger Unterschied zu Python-Listen betrifft das Slicing. Bei einer Liste erzeugt ein Slice wie `liste[0:2]` eine (flache) **Kopie** der Liste (siehe Kapitel *Datenstrukturen*). Bei einem NumPy-Array entsteht dagegen **keine Kopie**, sondern eine sog. **View** (Ansicht): ein neues Array-Objekt, das auf dieselben Daten im Speicher zeigt. NumPy spart sich so das Kopieren, was bei großen Arrays viel Zeit und Speicher spart. Die Kehrseite: Ändert man die View, ändert sich auch das ursprüngliche Array – ein Seiteneffekt, wie wir ihn von Listen kennen, wenn zwei Variablen auf dieselbe Liste zeigen:

```python
data = np.array([[1, 2, 3],
                 [4, 5, 6]])

row = data[0, :]   # View auf die erste Zeile (keine Kopie!)
row[0] = 99        # wir ändern die View ...
print(data)        # ... und das Original hat sich mitgeändert!
```

```
[[99  2  3]
 [ 4  5  6]]
```

Zum Vergleich dasselbe Vorgehen mit einer Liste. Hier ist der Slice eine Kopie, das Original bleibt unverändert:

```
numbers = [1, 2, 3]
part = numbers[0:2]  # Kopie der ersten beiden Elemente
part[0] = 99
print(numbers)       # unverändert
```

```
[1, 2, 3]
```

Wenn Sie mit einem Ausschnitt eines Arrays weiterarbeiten wollen, ohne das Original zu verändern, erzeugen Sie mit `.copy()` ausdrücklich eine Kopie:

```
row_copy = data[1, :].copy()  # echte Kopie der zweiten Zeile
row_copy[0] = -1
print(row_copy)
print(data)                   # unverändert
```

```
[-1  5  6]
[[99  2  3]
 [ 4  5  6]]
```

Demo: Matrixoperationen mit Arrays

```
import numpy as np
# 3x3 Matrix als np.array
A = np.array([[2, 1, 3],
              [0, -1, 4],
              [5, 2, 0]])

# ein Vektor
b = np.array([1, 2, 3])

# Matrix mal Vektor (Matrix-Vektor-Produkt):
v = np.dot(A, b)

# WARNUNG: Es ist wichtig, np.dot zu verwenden und nicht *
# Letzteres führt kein Matrix-Vektor-Produkt, sondern eine
# elementweise Multiplikation durch!
```

```
print("np.dot(A, b) =", v)
```

```
np.dot(A, b) = [13 10  9]
```

Es gibt auch eine Kurzschreibweise für `np.dot(..)` mit dem `@`-Operator:

```
print("A @ b =", A @ b)
```

```
A @ b = [13 10  9]
```

Wir demonstrieren einmal, was passiert, wenn man mit dem `*`-Operator multipliziert:

```
print(A * b) # elementweise Multiplikation! Das ist NICHT das Matrixprodukt!
```

```
[[ 2  2  9]
 [ 0 -2 12]
 [ 5  4  0]]
```

> Skalar- und Matrixprodukte immer mit `np.dot(...)` oder dem `@`-Operator!

Auch das **Kreuzprodukt** ist in NumPy verfügbar:

```
v = np.array([1, 2, 3])
w = np.array([0, -1, 1])
z = np.cross(v, w)
print(z)
```

```
[ 5 -1 -1]
```

Wir können viele Operationen aus der linearen Algebra direkt ausführen:

```
A.T  # liefert die transponierte Matrix
# alternativ: np.transpose(A)
```

```
array([[ 2,  0,  5],
       [ 1, -1,  2],
       [ 3,  4,  0]])
```

```
np.linalg.eigvals(A)  # liefert die Eigenwerte von A
```

```
array([ 5.83891641, -0.80708431, -4.0318321 ])
```

```
np.linalg.eig(A)  # liefert Eigenwerte und Eigenvektoren von A
```

```
EigResult(eigenvalues=array([ 5.83891641, -0.80708431, -4.0318321 ]), eigenvectors=array([[-
0.62757479, -0.37722073, -0.16597401],
       [-0.39307122,  0.92504816, -0.78589196],
       [-0.67204531,  0.04461408,  0.59567311]]))
```

Lineare Gleichungssysteme mit NumPy lösen

Gesucht ist die Lösung eines linearen Gleichungssystems:

$$\begin{aligned} 2\,x_1 + x_2 + 3\,x_3 &= 1 \\ -x_2 + 4\,x_3 &= 2 \\ 5\,x_1 + 2\,x_2 &= 3 \end{aligned}$$

In Matrixschreibweise:

$$\begin{pmatrix} 2 & 1 & 3 \\ 0 & -1 & 4 \\ 5 & 2 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

In NumPy legen wir die Koeffizientenmatrix und den Vektor auf der rechten Seite als Arrays an:

```
A = np.array([[2., 1., 3.],
              [0., -1., 4.],
              [5., 2., 0.]])

b = np.array([1., 2., 3.])
```

Der Aufruf `np.linalg.solve(A, b)` löst das lineare Gleichungssystem:

```
x = np.linalg.solve(A, b)  # finde x, so dass A x = b
print(x)
```

```
[ 1.31578947 -1.78947368  0.05263158]
```

Überprüfen wir das Ergebnis, indem wir `A x` berechnen und mit `b` vergleichen:

```
print(A @ x, "Zum Vergleich: ", b)
```

```
[1. 2. 3.] Zum Vergleich:  [1. 2. 3.]
```

Übung: Lineare Gleichungssysteme begegnen uns bei der Lösung verschiedener technischer Probleme, z.B. bei statischen Berechnungen oder der Bestimmung von Strömen und Spannungen in Schaltkreisen. Als Beispiel sei das folgende System für drei unbekannte Ströme gegeben, das mithilfe der kirchhoffschen Regeln für Gleichstromnetze hergeleitet wurde [Komplette Aufgabe "Stromkreis mit drei Maschen"]:

$$\begin{aligned} I_1 - I_2 - I_3 &= 0 \\ -10.8\,V + I_1 \cdot 6\,\Omega + I_2 \cdot 8\,\Omega &= 0 \\ 3.2\,V + I_3 \cdot 4\,\Omega - I_2 \cdot 8\,\Omega &= 0 \end{aligned}$$

Bestimmen Sie die drei unbekannten Ströme I_1, I_2, I_3 mithilfe von NumPy!

```
import numpy as np
# Die Lösung erfolgt in 3 Schritten:

# 1. Zunächst bringen wir das Gleichungssystem in die richtige Form!
# Da nur SI-Einheiten verwendet werden, können wir alle Einheiten weglassen
 ↪  und wissen, dass das Ergebnis in Ampere sein wird.
#1 I_1 - 1 I_2 - 1 I_3 = 0
#6 I_1 + 8 I_2 + 0 I_3 = 10.8
#0 I_1 - 8 I_2 + 4 I_3 = -3.2

# 2. Jetzt legen wir Matrix und rechte Seite als Arrays an:
A = np.array([[1., -1., -1.], [6., 8., 0.], [0., -8., 4]])
b = np.array([0., 10.8, -3.2])

# 3. Jetzt lösen wir das System:
I = np.linalg.solve(A, b)
```

```
print("Die drei unbekannten Stromstärken sind:")
print(f"I_1 = {I[0]: .2f} A")
print(f"I_2 = {I[1]: .2f} A")
print(f"I_3 = {I[2]: .2f} A")
```

Demo: Vektorisierung

NumPy bietet vektorisierten Code, d. h. mathematische Operationen wie Addition, Subtraktion, Multiplikation auf Arrays werden auf alle Elemente angewendet, ohne dass wir eine Schleife schreiben müssen (die Schleife läuft intern in schnellem, kompiliertem Code). Dies ist viel schneller als manuelle Schleifen über Listen!

Wir erstellen zunächst ein Array mit x-Werten, an denen wir eine Funktion auswerten wollen:

```
x_values = np.array(range(10))  # Array mit den x-Werten 0, 1, 2, ..., 9
print(x_values)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

An diesen Stellen sollen Funktionswerte der folgenden Funktion berechnet werden: $f(x) = 3\,x^3 - x^2 + 2\,x + 1$. Vektorisierung bedeutet, dass wir die Rechenoperationen mit dem Array `x_values` durchführen können und NumPy diese elementweise durchführt. Das ist kinderleicht:

```
f_values = 3 * x_values**3 - x_values**2 + 2 * x_values + 1  # berechnet f(x)
 ↪  für jeden x-Wert
print(f_values) # das Ergebnis ist wieder ein Array!
```

```
[   1    5   25   79  185  361  625  995 1489 2125]
```

Wir benötigen keine Schleife und die Operationen werden sogar optimiert, also sehr schnell, ausgeführt. In der numerischen Mathematik kommen solche Operationen sehr häufig vor. Im Abschnitt zu Matplotlib werden wir sehen, wie man Vektorisierung verwendet, um mit wenigen Zeilen einen Funktionsgraphen zu zeichnen.

Wir können natürlich auch komplexere mathematische Funktionen berechnen. Das wollen wir an $g(x) = \frac{1}{2}\sqrt{x}$ und $h(x) = x\,e^{x^2}$ demonstrieren:

```
g_values = 1/2 * np.sqrt(x_values)  # berechnet Funktionswerte von g an den
 ↪  x-Stellen
print(g_values)
```

```
[0.         0.5        0.70710678 0.8660254  1.         1.11803399
 1.22474487 1.32287566 1.41421356 1.5       ]
```

```
h_values = x_values * np.exp(x_values**2) # berechnet Funktionswerte von h an
 ↪  den x-Stellen
print(h_values)
```

```
[0.00000000e+00 2.71828183e+00 1.09196300e+02 2.43092518e+04
 3.55444421e+07 3.60024497e+11 2.58673893e+16 1.33514260e+22
 4.98811926e+28 1.35548758e+36]
```

Wichtig ist, dass wir in den beiden letzten Beispielen die Mathe-Funktionen aus NumPy verwendet haben, also `np.sqrt(...)` bzw. `np.exp(...)`. Andere Mathebibliotheken (z.B. `math`) sind nicht für vektorisierte Operationen auf NumPy-Arrays ausgelegt.

Die beiden Funktionen `np.linspace(...)` und `np.arange(...)` werden oft zusammen mit Vektorisierung verwendet. Zum Beispiel erstellt man mit `np.linspace(...)` ein Array von x-Werten und berechnet dann mithilfe von Vektorisierung die zugehörigen y-Werte einer Funktion. Das könnte so aussehen wie im folgenden Beispiel:

```
xs = np.linspace(-np.pi, np.pi, 100)  # 100 Punkte zwischen -pi und pi
ys = np.sin(xs)  # Funktionswerte von sin(x) an den zuvor berechneten Stellen
print("Auswertung an einer Beispielstelle:")
print("x =", round(xs[20],2), ", sin(x)=", round(ys[20],2))

# Teaser: Matplotlib kann mit diesen Werten einen Funktionsgraphen zeichnen!
```

```
Auswertung an einer Beispielstelle:
x = -1.87 , sin(x)= -0.95
```

Demo: Broadcasting

NumPy unterstützt sog. “Broadcasting”, sodass auch Arrays unterschiedlicher Formen miteinander rechnen können, ohne dass wir diese mühsam aneinander anpassen müssen.

Ein Beispiel zu Broadcasting:

```
A = np.array([[1, 0, 0],
              [0, 2, 0],
              [0, 0, 3]])

v = np.array([1, 2, 3])

B = A + v  # das passt eigentlich nicht!
```

Die letzte Zeile führt überraschenderweise nicht zu einem Programmfehler, obwohl man A und v mathematisch gar nicht addieren kann. In solchen Fällen geschieht Broadcasting: "Was nicht passt, wird passend gemacht (*falls möglich*)!". Schauen wir uns das Ergebnis genauer an:

```
print(B)
```

```
[[2 2 3]
 [1 4 3]
 [1 2 6]]
```

Wir sehen: Der Vektor v wurde zu jeder Zeile von A addiert – so, als hätte man ihn dreimal untereinander geschrieben. Die Regel dahinter: NumPy vergleicht die Formen (*shapes*) der Arrays von hinten. Zwei Dimensionen passen zusammen, wenn sie gleich sind oder eine davon 1 ist bzw. fehlt. Entlang einer solchen Dimension mit Größe 1 (bzw. einer fehlenden) wird das Array gedanklich vervielfältigt – hier wird aus der Form (3,) die Form (3, 3). Passen die Formen nicht zusammen, meldet NumPy einen `ValueError`. Man muss aufpassen, dass man kein unbeabsichtigtes Broadcasting durchführt und unerwartete Ergebnisse erhält.

Übungen:

1. Erzeugen Sie 100 x-Werte zwischen -2 und 5 und berechnen Sie zu jedem dieser Werte den Funktionswert von $f(x) = \frac{e^{-x}}{1+x^2}$. Geben Sie exemplarisch den Funktionswert für ein zufällig ausgewähltes x aus (auf zwei Nachkommastellen gerundet)!
2. Erzeugen Sie ein Array, das 1000 Zeitpunkte zwischen t=0 s und t= 100 s beinhaltet. Ein Körper befinde sich zum Zeitpunkt t=0 an der Position x(0)=0 und bewege sich gleichmäßig mit der Geschwindigkeit v= 2.5 m/s in positive x-Richtung. Erstellen Sie ein Array, das die x-Position zu jedem Zeitpunkt t angibt. Geben Sie die Position zum Zeitpunkt t=100 s aus.
3. Multiplizieren Sie zwei 3x3 Matrizen miteinander.
4. Wie können Sie Broadcasting und Transponieren kombinieren, um einen Vektor v zu jeder Spalte einer Matrix zu addieren?

Lösungen:

```python
import numpy as np
# Übung 1
xs = np.linspace(-2, 5, 100)
ys = np.exp(-xs)/(1 + xs**2)
print(f"Übung 1: f({round(xs[30],2)}) = {round(ys[30],2)}")

# Übung 2
ts = np.linspace(0,100, 1000)
xs = 2.5 * ts  # x = velocity * time
print("\nÜbung 2:\nx(t = 100s) =", xs[-1])

# Übung 3
A = np.array([[1, 2, 3],
     [1, 2, 3],
     [1, 2, 3]])

B = np.ones((3, 3))

C = np.dot(A, B)  # oder C = A @ B

print("\nÜbung 3:", "A =", A, "B = ",B, "A @ B = ", C, sep="\n")

# Übung 4
A = np.array([[1, 2, 3],
              [1, 2, 3],
              [1, 2, 3]])
v = np.array([-1, 0, -2])  # soll zu jeder Spalte addiert werden

# A wird transponiert und v addiert,
# dann wird das Ergebnis zurücktransponiert:

result = (A.T + v).T

print("\nÜbung 4:","A =", A,"v =",v, "A + v (spaltenweise)", result,
 ↪  sep="\n")
```

9.3. Matplotlib

Matplotlib ist die Standard-Bibliothek in Python zum Erstellen von Diagrammen. Der am häufigsten verwendete Teil ist das Untermodul **pyplot** (eine Schnittstelle, über die das Modul leicht zu bedienen ist), das man fast immer mit dem Alias `plt` importiert.

```
import matplotlib.pyplot as plt  # Standard-Import für matplotlib
```

Mit Matplotlib kann man die unterschiedlichsten Arten von Diagrammen erzeugen. Auch hier können wir nur ein paar exemplarische Beispiele demonstrieren.

Eine Funktion plotten

Um eine Funktion zu plotten, arbeiten wir mit Matplotlib und NumPy zusammen: NumPy erzeugt die x- und y-Werte, Matplotlib stellt diese in einem einfachen Linienplot dar. Linienplot heißt, dass gerade Linien von Punkt zu Punkt gezogen werden. Wenn es ausreichend viele Punkte sind, entsteht der Eindruck einer glatten Kurve. Los geht's:

```
import matplotlib.pyplot as plt
import numpy as np

# Punkte erzeugen mit NumPy

x_values = np.linspace(0, 50, 100) # 100 Punkte zwischen 0 und 50
y_values = x_values**2 # Vektorisierung: so einfach rechnet man mit
 ↪  NumPy-Arrays!

# Liniendiagramm erzeugen mit Matplotlib

plt.plot(x_values, y_values)

# Es folgen Einstellungen zur Darstellung (z.B. Beschriftung)

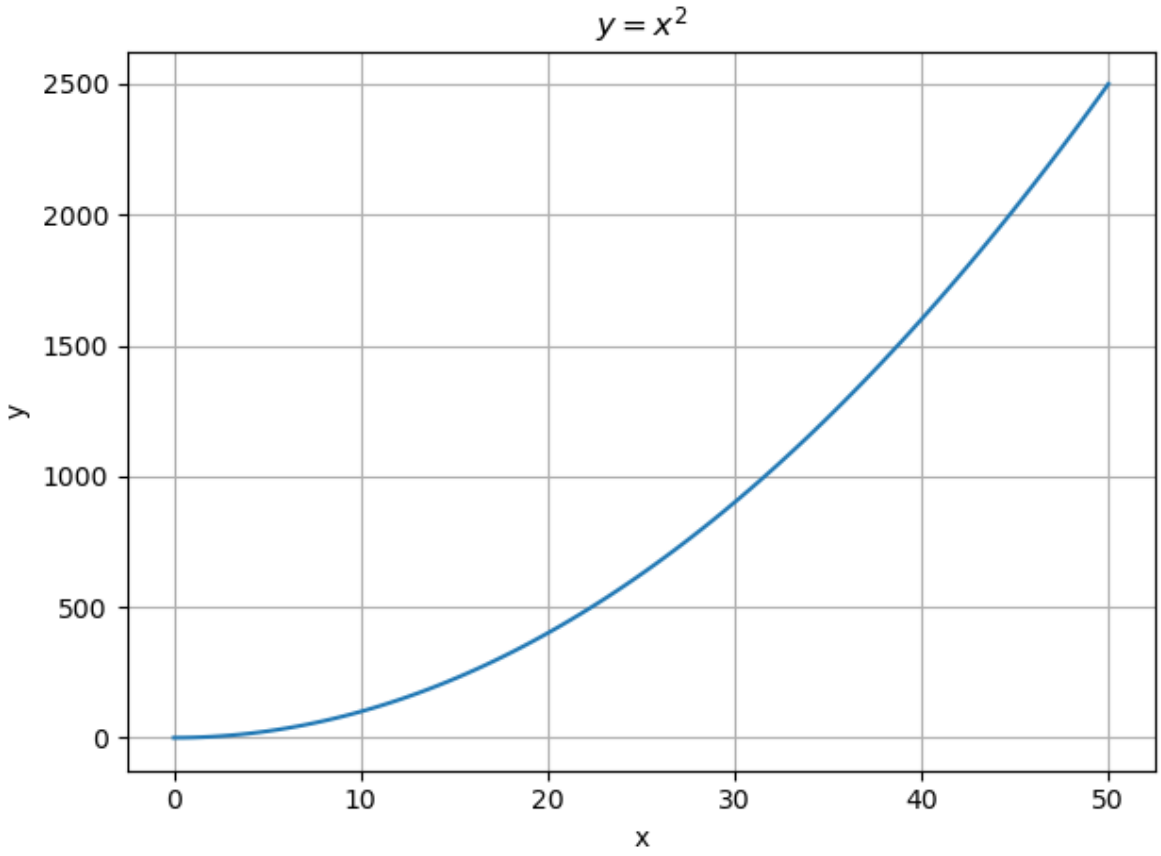
plt.title(r"$y = x^2$")  # Titel
plt.xlabel("x")  # Beschriftung x-Achse
plt.ylabel("y")  # Beschriftung y-Achse
plt.tight_layout() # optimiert bei Bedarf den Zeichenbereich, damit nichts
 ↪  abgeschnitten wird
plt.grid(True) # zeigt ein Gitter zum einfacheren Ablesen der Werte

# Finally:
# Der eigentliche Befehl zum Rendern (= Zeichnen).

plt.show()  # öffnet ein Fenster (oder rendert im Notebook) und zeigt den
 ↪  Plot.

# PS: Der Befehl plt.show() wird von Jupyter oft automatisch aufgerufen, an
 ↪  anderer Stelle braucht man
```

```
# ihn aber explizit. Wir schreiben ihn daher immer hin.
```



Verwendung von LaTeX-Befehlen in Strings

Beim Plotten von Funktionen verwendet man häufig LaTeX-Befehle, um Achsen, Titel oder Labels zu beschriften. Beispielsweise kann die Funktion
$f(x) = \frac{1}{1+x^2}$
in Matplotlib so als String dargestellt werden:
`"$\frac{1}{1+x^2}$"`
In Python hat der Backslash \ jedoch die Bedeutung eines *Escape-Zeichens*, z.B. für Zeilenumbrüche (`\n`) oder Tabulatoren (`\t`). Damit Python den Backslash in LaTeX-Befehlen **wörtlich** interpretiert, verwendet man **Raw Strings**, indem man ein kleines **r** vor das String-Literal setzt:
`r"$\frac{1}{1+x^2}$"`
`r"$\sqrt{x}$"`
Dieses r teilt dem Python-Interpreter mit, dass alle Backslashes in diesem String unverän-

dert bleiben, wodurch LaTeX-Befehle korrekt dargestellt werden.

Messwerte in einem Streudiagramm darstellen

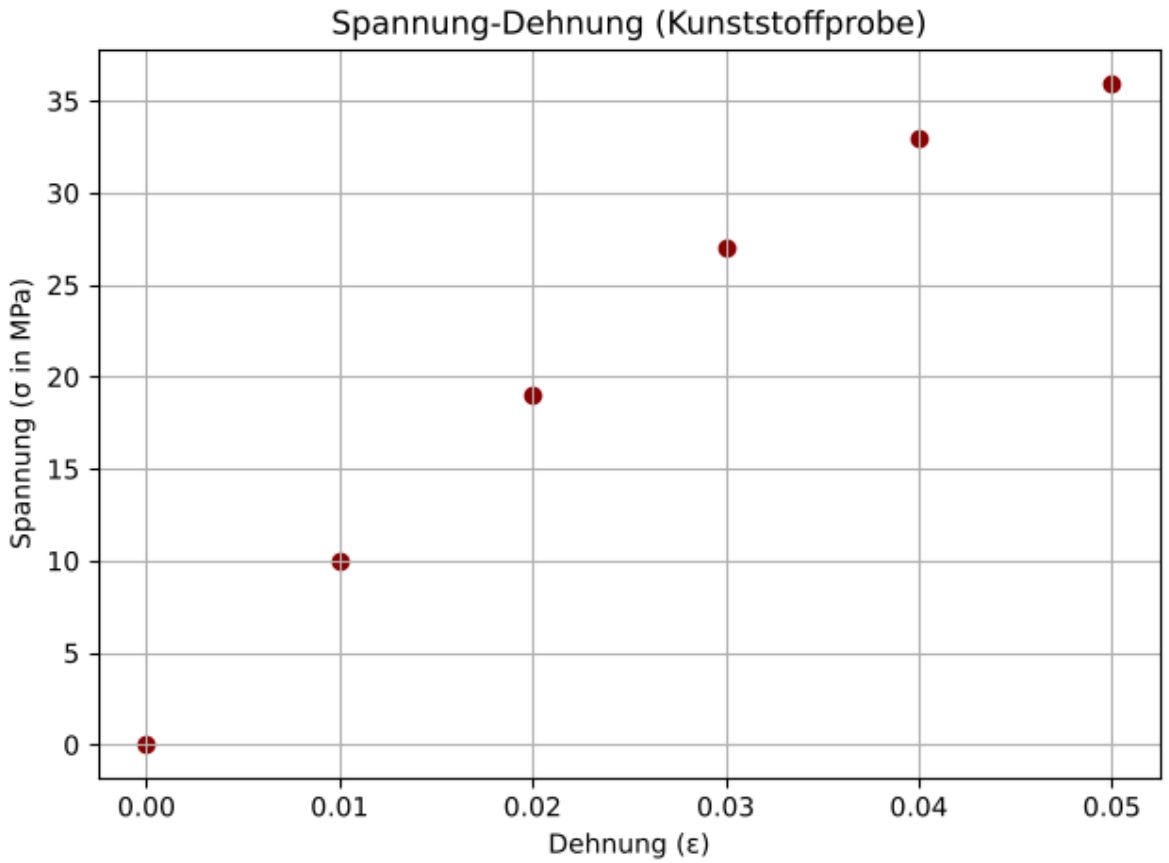
```
import matplotlib.pyplot as plt

# Messwerte
dehnung = [0.0, 0.01, 0.02, 0.03, 0.04, 0.05]
spannung = [0, 10, 19, 27, 33, 36]  # in MPa

# Streudiagramm mit Matplotlib erzeugen
plt.scatter(dehnung, spannung, color='darkred')

# Einstellungen für die Darstellung:
plt.title("Spannung-Dehnung (Kunststoffprobe)")
plt.xlabel("Dehnung (ε)")
plt.ylabel("Spannung (σ in MPa)")
plt.grid(True)
plt.tight_layout()

# Finally: Rendern
plt.show()
```



Haben Sie auch sofort den Gedanken: *"Kann ich da eine Kurve durchlegen?"*

Sie können! Allerdings muss die Kurve zunächst berechnet werden, Matplotlib ist nur für die visuelle Darstellung verantwortlich. Eine einfache Möglichkeit bietet NumPy mit polynomialer Regression: Ein Polynom vorgegebenen Grades wird an die Messwerte angepasst. Das sieht im Ergebnis zum Beispiel so aus:

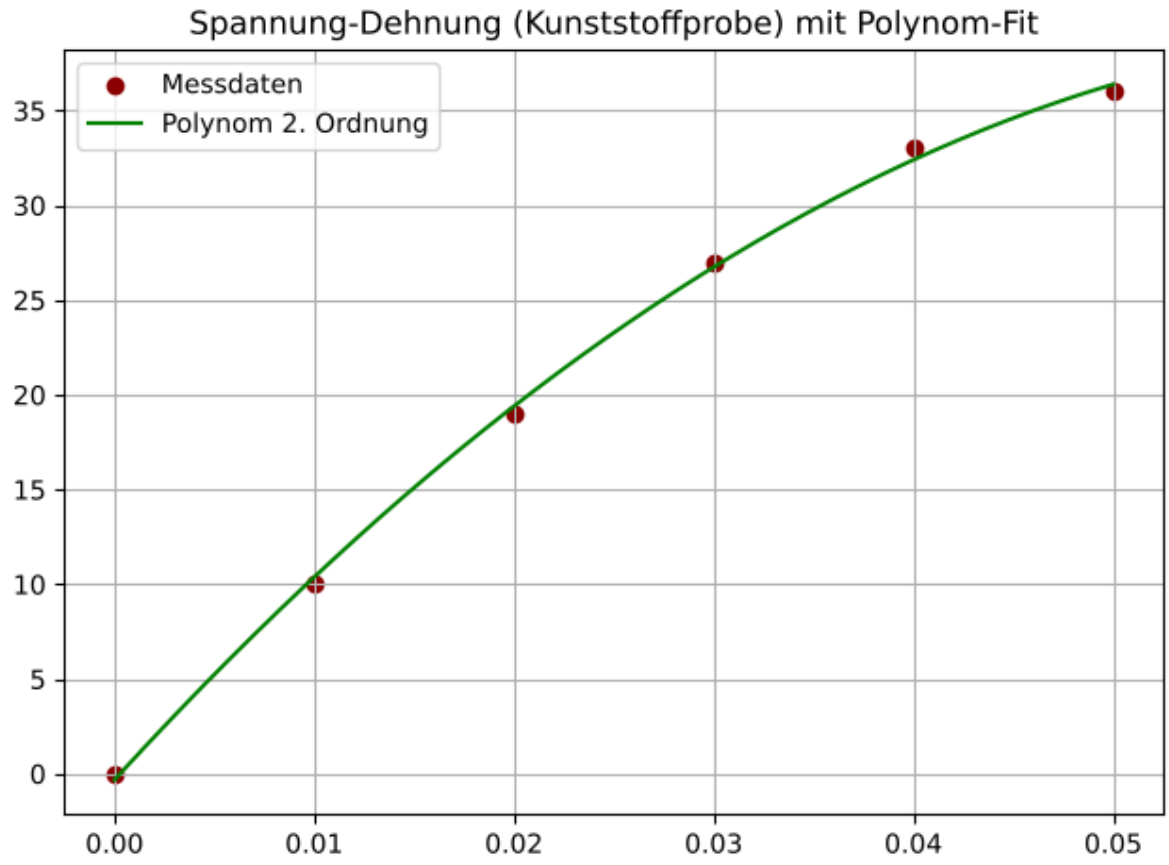
```
# Polynom 2. Ordnung fitten
from numpy.polynomial import Polynomial
polynom = Polynomial.fit(dehnung, spannung, deg=2)
x_poly = np.linspace(min(dehnung), max(dehnung), 200) # x-Werte für das
 ↪  Polynom

# Plotten
plt.scatter(dehnung, spannung, label="Messdaten", color='darkred')
plt.plot(x_poly, polynom(x_poly), label="Polynom 2. Ordnung", color="green")

# Einstellungen zur Darstellung
```

```
plt.title("Spannung-Dehnung (Kunststoffprobe) mit Polynom-Fit")
plt.legend()
plt.grid(True)
plt.tight_layout()

# Finally: Rendern
plt.show()
```



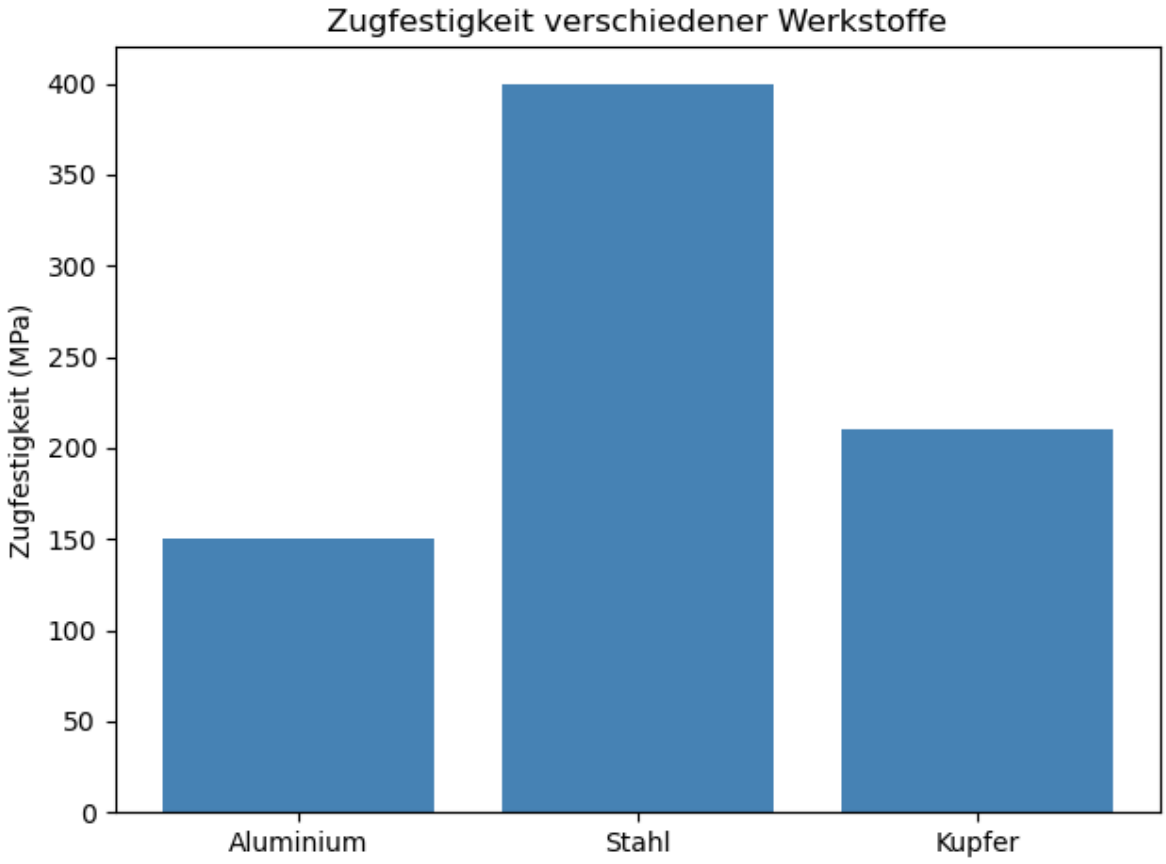
Ein einfaches Balkendiagramm

```
# Daten
werkstoffe = ["Aluminium", "Stahl", "Kupfer"]
festigkeit = [150, 400, 210]  # in MPa
```

```
# Plotten
plt.bar(werkstoffe, festigkeit, color='steelblue')

# Einstellungen zur Darstellung
plt.title("Zugfestigkeit verschiedener Werkstoffe")
plt.ylabel("Zugfestigkeit (MPa)")
plt.tight_layout()

# Finally: Rendern
plt.show()
```



Übungen:

1. Plotten Sie die Funktion $f(x) = 1 + 3\sqrt{x}$ im Bereich von 0 bis 10. Tipp: `np.sqrt` für die Wurzel.

2. Gegeben sind mittlere Tagestemperaturen in Grad Celsius für die Tage einer Woche: `temperatures = np.array([15, 17, 14, 20, 22, 19, 16])`. Stellen Sie den Verlauf visuell dar.
3. Gegeben sind die Verkaufszahlen von vier verschiedenen Produkten im letzten Monat: `sales_numbers = [50, 75, 30, 90]`. Stellen Sie die Daten in einem Balkendiagramm dar.

Lösungen:

```
import matplotlib.pyplot as plt
import numpy as np

# Übung 1
xs = np.linspace(0, 10, 500)
ys = 1 + 3 * np.sqrt(xs)

plt.plot(xs, ys)

plt.title(r"Ü1: $f(x)= 1 + 3\sqrt{x}$")  # das kleine r: raw string, damit
 ↪  der Backslash \ als solcher erkannt wird
plt.xlabel("x")
plt.ylabel("y")
plt.tight_layout()
plt.grid(True)

plt.show()

# Übung 2

days = ['Mo', 'Di', 'Mi', 'Do', 'Fr', 'Sa', 'So']
temperatures = np.array([15, 17, 14, 20, 22, 19, 16])

plt.plot(days, temperatures, marker='o') # mit marker='o' markieren wir die
 ↪  Einzelpunkte
# scatter-Plot wäre auch möglich


plt.title("Ü2: Temperaturverlauf")
plt.xlabel("Tag")
plt.ylabel("mittlere Temperatur [°C]")
plt.tight_layout()
plt.grid(True)
```

```
plt.show()

# Übung 3

products = ["A", "B", "C", "D"]
sales_numbers = [50, 75, 30, 90]

plt.bar(products, sales_numbers)

plt.title("Ü3: Verkaufszahlen im letzten Monat")
plt.ylabel("Verkäufe")
plt.tight_layout()

plt.show()
```

9.4. pandas

pandas ist eine Python-Bibliothek zur Datenanalyse und -verarbeitung. Sie macht es einfach, mit tabellarischen Daten zu arbeiten – wie man es von Excel oder MATLAB kennt, nur viel mächtiger! Sie ist ideal für CSV-Dateien, Zeitreihen, Sensorlogs usw. (Wie man Text- und CSV-Dateien auch ohne pandas liest und schreibt, zeigt Anhang A.)

Stellen Sie sich vor, Sie hätten eine CSV-Datei `pruefstand.csv`, die Messdaten von einem Elektromotor-Prüfstand auflistet, evtl. mit Tausenden oder Zehntausenden an Einträgen (die folgenden Beispieldaten sind simuliert, orientieren sich aber an einem typischen 5,5-kW-Asynchronmotor):

```
zeit_s,drehzahl_rpm,drehmoment_Nm,temperatur_C,strom_A,spannung_V
0,1467.2924821489682,33.81031628726315,24.628518648371458,10.215086869820023,400.39745115695
2,1461.4533926626523,35.36085692530741,25.32622786824618,10.628453711233902,400.836376917262
4,1459.7331276504278,32.1555104756014,25.657537051768216,9.701252783384483,400.415662806061
6,1453.9484770091583,37.09365756516304,25.88773307868301,11.153416913010403,400.690401682028
8,1455.7907345141277,35.957442111392325,25.807826294286944,10.844172930105008,398.828541765
10,1461.721999729195,34.56192877217096,25.848570214283473,10.556782004465514,400.447806846483
...
```

Mit pandas ist es ein Leichtes, die Daten zu analysieren! Wir zeigen nur ein paar wenige der vielen Features, die pandas anbietet.

```python
import pandas as pd

# CSV einlesen (statt eines Dateipfads ist auch eine URL möglich)
df = pd.read_csv(r"./data/pruefstand.csv")

# Excel-Dateien liest man analog mit pd.read_excel("datei.xlsx") ein
# (benötigt das Paket openpyxl, in Anaconda bereits enthalten)

# Die Abkürzung df ist sehr gebräuchlich und steht für 'Data Frame' = eine
 ↪  Tabelle

# Kurze Übersicht der Daten
df.head()  # Zeigt die ersten 5 Zeilen
```

	zeit_s	drehzahl_rpm	drehmoment_Nm	temperatur_C	strom_A	spannung_V
0	0	1467.292482	33.810316	24.628519	10.215087	400.397451
1	2	1461.453393	35.360857	25.326228	10.628454	400.836377
2	4	1459.733128	32.155510	25.657537	9.701253	400.415663
3	6	1453.948477	37.093658	25.887733	11.153417	400.690402
4	8	1455.790735	35.957442	25.807826	10.844173	398.828542

```python
df.describe()  # liefert statistische Kennzahlen aller Größen
```

	zeit_s	drehzahl_rpm	drehmoment_Nm	temperatur_C	strom_A	spannung_V
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000
mean	499.000000	1459.727502	35.047270	51.742815	10.586593	399.899417
std	288.963666	3.537586	1.571831	10.276739	0.479168	0.963109
min	0.000000	1448.249747	29.378879	24.628519	8.965309	396.941205
25%	249.500000	1457.498059	33.962797	45.445497	10.220183	399.231993
50%	499.000000	1459.804317	35.094876	55.402018	10.580069	399.918036
75%	748.500000	1462.151006	36.080189	60.201944	10.890667	400.562544
max	998.000000	1471.917075	39.726362	63.147595	12.028734	403.333527

Der `describe`-Befehl ist sehr nützlich: Wir sehen auf einen Blick, dass unsere Datei 500 Zeilen beinhaltet und erhalten Mittelwerte, Standardabweichungen, Wertebereiche und Quantile aller Messgrößen. Ein erster aufschlussreicher Überblick über unsere Daten!

```
# Mittelwerte oder andere Kennzahlen können
# ganz einfach auch einzeln abgefragt werden:

mean_torque = df["drehmoment_Nm"].mean()
print(f"Mittleres Drehmoment: {round(mean_torque, 2)} Nm")

mean_temperature = df["temperatur_C"].mean()
print(f"Mittlere Temperatur: {round(mean_temperature, 2)} °C")

# Wie diese Beispiele zeigen, greift man auf einzelne Spalten
# eines Data Frames über einen Indexzugriff zu, ganz analog zu
# Dictionaries.
```

```
Mittleres Drehmoment: 35.05 Nm
Mittlere Temperatur: 51.74 °C
```

Sie können auch einfach neue Spalten erzeugen, wie das folgende Beispiel zeigt. Wir berechnen aus dem Drehmoment M (in Nm) und der Drehzahl n (in Umdrehungen pro Minute) die mechanische Leistung $P = M \cdot 2\pi n/60$ in Watt:

```
# neue Spalte: mechanische Leistung P = M · ω mit ω = 2π · n / 60
df["p_mech_W"] = df["drehmoment_Nm"] * 2 * np.pi * df["drehzahl_rpm"] / 60
df.head()  # achten Sie auf die neue Spalte.
```

	zeit_s	drehzahl_rpm	drehmoment_Nm	temperatur_C	strom_A	spannung_V	p_mech_W
0	0	1467.292482	33.810316	24.628519	10.215087	400.397451	5195.107562
1	2	1461.453393	35.360857	25.326228	10.628454	400.836377	5411.733090
2	4	1459.733128	32.155510	25.657537	9.701253	400.415663	4915.384443
3	6	1453.948477	37.093658	25.887733	11.153417	400.690402	5647.773785
4	8	1455.790735	35.957442	25.807826	10.844173	398.828542	5481.713820

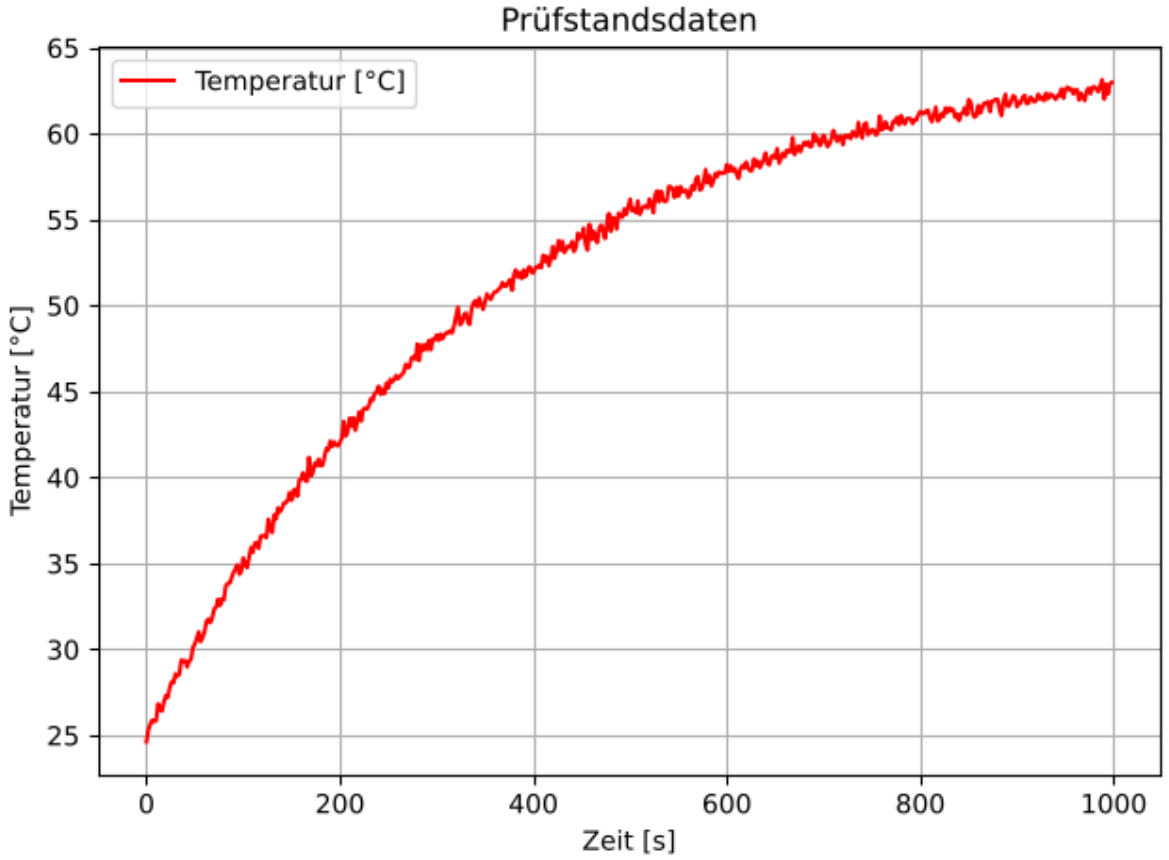
Selbstverständlich können wir nun auch Matplotlib in Kombination mit pandas einsetzen, um Daten zu visualisieren:

```
import matplotlib.pyplot as plt

# Plot: Temperatur über Zeit
plt.plot(df["zeit_s"], df["temperatur_C"], label="Temperatur [°C]",
 ↪  color="red")
```

```
plt.xlabel("Zeit [s]")
plt.ylabel("Temperatur [°C]")
plt.title("Prüfstandsdaten")
plt.legend()
plt.grid(True)
plt.tight_layout()

plt.show()
```



Übung: Der Motor am Prüfstand ist ein Drehstrommotor. Seine aufgenommene elektrische Wirkleistung berechnet sich aus der Leiterspannung U, dem Strom I und dem Leistungsfaktor $\cos\varphi$ (laut Typenschild 0,83) zu $P_{el} = \sqrt{3} \cdot U \cdot I \cdot \cos\varphi$.

1. Fügen Sie dem Data Frame eine Spalte `p_el_W` mit der elektrischen Leistung hinzu.
2. Berechnen Sie für jeden Zeitpunkt den Wirkungsgrad $\eta = P_{mech}/P_{el}$ als neue Spalte und geben Sie den mittleren Wirkungsgrad aus.

3. Ist das Ergebnis plausibel? (Typische Wirkungsgrade von Asynchronmotoren dieser Größe liegen bei 85 bis 90 %.)

Lösung:

```
# 1. Elektrische Wirkleistung (Drehstrom)
cos_phi = 0.83
df["p_el_W"] = np.sqrt(3) * df["spannung_V"] * df["strom_A"] * cos_phi

# 2. Wirkungsgrad je Zeitpunkt und Mittelwert
df["wirkungsgrad"] = df["p_mech_W"] / df["p_el_W"]
print(f"Mittlerer Wirkungsgrad: {df['wirkungsgrad'].mean():.1%}")

# 3. Mit rund 88 % liegt der Wirkungsgrad im typischen Bereich - die
 ↪  Messdaten sind plausibel.
```

Ein Data Frame kann auch aus einem Dictionary erzeugt werden:

```
import pandas as pd

data = {
    'Produkt': ['Produkt A', 'Produkt B', 'Produkt C', 'Produkt D'],
    'Preis': [20, 35, 50, 15],
    'Verkaufte Menge': [100, 150, 80, 200]
}

df = pd.DataFrame(data)
df.head()
```

	Produkt	Preis	Verkaufte Menge
0	Produkt A	20	100
1	Produkt B	35	150
2	Produkt C	50	80
3	Produkt D	15	200

Übungen: Erzeugen Sie das Data Frame, wie im letzten Beispiel gezeigt.

1. Fügen Sie eine Spalte Umsatz hinzu (Umsatz = Preis * Verkaufte Menge).
2. Finden Sie das Produkt mit dem höchsten Umsatz.
3. Erstellen Sie ein Balkendiagramm mit Matplotlib, das den Umsatz jedes Produkts zeigt.

4. Wie groß sind Mittelwert und Standardabweichung des Umsatzes?

Lösung:

```
import pandas as pd
import matplotlib.pyplot as plt

# Daten erstellen
data = {
    'Produkt': ['Produkt A', 'Produkt B', 'Produkt C', 'Produkt D'],
    'Preis': [20, 35, 50, 15],
    'Verkaufte Menge': [100, 150, 80, 200]
}

df = pd.DataFrame(data)

# 1. Umsatz berechnen und als Spalte hinzufügen
df['Umsatz'] = df['Preis'] * df['Verkaufte Menge']

# 2. Kann durch einfaches print gelöst werden:
print(df)  # Produkt B hat den höchsten Umsatz!

# 3. Balkendiagramm erstellen
plt.bar(df['Produkt'], df['Umsatz'], color=['red', 'blue', 'green',
 ↪  'purple'])
plt.title('Umsatz der Produkte')
plt.xlabel('Produkt')
plt.ylabel('Umsatz')

plt.show()

# 4. Statistische Kennzahlen
print("Mittlerer Umsatz: ", df['Umsatz'].mean(), "Standardabweichung: ",
 ↪  df['Umsatz'].std())
# Hinweis: pandas teilt bei .std() durch n-1
 ↪  (Stichproben-Standardabweichung), np.std() dagegen
# standardmäßig durch n. Bei großen Datenmengen spielt das kaum eine Rolle,
 ↪  bei nur 4 Werten wie
# hier ist der Unterschied aber deutlich (np.std() ergäbe ca. 1204 statt
 ↪  1390).
# Kann auch über describe abgelesen werden:
df.describe()
```

10. Objektorientierte Programmierung (OOP)

Lernziele

Nach diesem Kapitel können Sie …

- die Grundidee der OOP und ihre Prinzipien (Abstraktion, Kapselung, Vererbung, Polymorphie) erklären
- eigene Klassen mit Attributen und Methoden schreiben und Objekte daraus erzeugen
- UML-Klassendiagramme lesen und erstellen
- Vererbung nutzen und mit `super()` auf die Basisklasse zugreifen
- Zugriffsrechte (public, protected, private) einsetzen und Getter/Setter schreiben
- Operatoren mit Magic Methods überladen und abstrakte Klassen einsetzen

10.1. Grundidee und Prinzipien

Die objektorientierte Programmierung (OOP) ist ein Programmierparadigma, das Software in Form von **Objekten** strukturiert. Diese Objekte kombinieren **Daten (Attribute)** und **Verhalten (Methoden)** in einer einzigen Einheit. Das Ziel ist es, die reale Welt besser im Code abzubilden, indem Software-Entitäten geschaffen werden, die Eigenschaften und Fähigkeiten besitzen. Dadurch können sehr komplexe Systeme übersichtlich modelliert werden. OOP fördert dadurch eine modularere und wartungsfreundlichere Softwareentwicklung, die näher an der Wirklichkeit ist.

Vier zentrale Prinzipien prägen die objektorientierte Programmierung:

- **Kapselung (Encapsulation)**

  Daten und Funktionen, die zusammengehören, werden in **Klassen** zusammengefasst. Der Zugriff auf sensible Daten wird kontrolliert – häufig durch private Attribute und öffentliche Schnittstellen (Methoden), die genau festlegen, wie die Attribute verändert oder gelesen werden dürfen. Eine Klasse ist die abstrakte Beschreibung einer Gesamtheit: Sie definiert, was Objekte dieser Art auszeichnet. Beispielsweise beschreibt die Klasse *Person* abstrakt Eigenschaften wie Name, Geburtsdatum oder Wohnort. Eine konkrete Instanz, wie etwa Max Mustermann, ist dann ein Objekt dieser Klasse.

- **Vererbung (Inheritance)**

  Klassen können Eigenschaften und Verhalten von anderen Klassen übernehmen, um Redundanzen zu vermeiden und Code-Wiederverwendung zu ermöglichen. Man kann sich Klassen vorstellen, die verschiedene Tierarten beschreiben, etwa *Dog*, *Cat* oder *Mouse*. Obwohl sich diese Klassen unterscheiden, teilen sie gemeinsame Merkmale, die in einer Basisklasse *Animal* zusammengefasst werden können. Die abgeleiteten Klassen spezifizieren dann die Besonderheiten jeder Art. Man sagt: "*Dog* erbt von *Animal*" oder "*Dog* ist abgeleitet von *Animal*".

- **Polymorphie oder Polymorphismus (Polymorphism)**

  Objekte verschiedener Klassen können über die gleiche Schnittstelle angesprochen werden. Dadurch können Funktionen allgemeiner und flexibler gestaltet werden. Im obigen Beispiel kann eine Funktion, die ein Objekt der Klasse *Animal* erwartet, gleichermaßen mit einem *Dog*, einer *Cat* oder einer *Mouse* arbeiten. Für die Funktion ist nur relevant, dass es sich um ein Animal-Objekt handelt, nicht um dessen genaue Art. Polymorphismus unterstützt dadurch die Erweiterbarkeit und Anpassbarkeit von Programmen. Beispiel: In einer Tierarztpraxis werden alle Patienten in einer Datenbank verwaltet. Dabei ist es egal, ob es sich um Hunde, Katzen oder Mäuse handelt – alle Tiere werden als Instanzen der allgemeinen Klasse *Animal* gespeichert. Die spezifischen Eigenschaften (z.B. Hunderasse oder Lieblingsfutter der Katze) werden in abgeleiteten Klassen ergänzt.

- **Abstraktion (Abstraction)**

  Nur die wesentlichen Merkmale eines Objekts werden dargestellt; unwichtige Details werden ausgeblendet. Dies erleichtert die Handhabung komplexer Systeme. Dieses Prinzip entspricht dem allgemeinen Konzept der Modellbildung: Ein Modell sollte so präzise wie nötig, aber so einfach wie möglich sein. Eine vollständige Abbildung der Realität wäre genauso komplex wie die Realität selbst und damit unpraktisch. Daher ist Abstraktion immer ein bewusster Kompromiss – zugunsten besserer Verständlichkeit und Lösbarkeit von Problemen. In der objektorientierten Programmierung kann man sogar sog. abstrakte Klassen programmieren, die zwar selbst nicht instanziiert werden können, aber die Struktur aller abgeleiteten Klassen festlegen.

i Paradigmen der Programmierung

In der Softwareentwicklung bezeichnet der Begriff Paradigma einen grundlegenden Ansatz oder eine Methode, wie Probleme durch Programmieren gelöst werden. Jedes Paradigma hat seine eigenen Prinzipien und Konzepte, die eine bestimmte Herangehensweise an die Problemstellung fördern. Die Wahl des Paradigmas beeinflusst die Struktur des Codes, die Art der Probleme, die gelöst werden können, und wie effizient der Entwicklungsprozess verläuft. Jedes Programmierparadigma hat seine Stärken und eignet sich für bestimmte Arten von Aufgaben. In der Praxis wird häufig eine Kombination verschiedener Paradigmen verwendet.

Wichtige Programmierparadigmen:

1. **Imperative Programmierung**:

   In der imperativen Programmierung wird die Ausführung einer Software durch eine Reihe von Anweisungen beschrieben, die den Zustand des Programms verändern. Der Fokus liegt darauf, wie ein Problem gelöst wird, also auf den einzelnen Schritten und deren Reihenfolge. Prozedurale Programmierung erweitert die imperative Programmierung um die Möglichkeit, Code in wiederverwendbare Teilaufgaben zu zerlegen. Praktisch alle aktuellen imperativen Programmiersprachen unterstützen den prozeduralen Ansatz.

   Hauptprinzipien: Sequentielle Anweisungen, Zustandsänderung, Schleifen und Bedingungen.

   Beispiele: Sprachen wie C, Python und Java unterstützen dieses Paradigma.

2. **Objektorientierte Programmierung (OOP)**:

   In der objektorientierten Programmierung werden Daten und Funktionen, die auf diese Daten operieren, in Objekten organisiert. OOP fördert die Wiederverwendbarkeit und Strukturierung von Code durch die Konzepte von Klassen, Objekten, Vererbung und Polymorphismus.

   Hauptprinzipien: Kapselung, Vererbung, Polymorphismus, Abstraktion.

   Beispiele: Java, Python, C++.

3. **Funktionale Programmierung**:

   Funktionale Programmierung basiert auf der Verwendung von Funktionen als grundlegende Bausteine des Programms. Sie betont die Unveränderlichkeit von Daten und vermeidet Zustandsänderungen und Seiteneffekte. Funktionen werden als *First-Class-Objekte* behandelt, was bedeutet, dass sie als Argumente übergeben und als Rückgabewerte genutzt werden können.

   Hauptprinzipien: Unveränderliche Daten, Funktionen als Objekte, keine Seiteneffekte.

   Beispiele: Haskell, Scala.

4. **Logische Programmierung**:

   Bei der logischen Programmierung wird ein Problem in Form von Fakten und Regeln beschrieben, die zusammen die Lösung des Problems definieren. Während der Programmierung gibt man an, was wahr ist, und der Computer zieht Schlussfolgerungen, um zu einer Lösung zu gelangen.

   Hauptprinzipien: Fakten, Regeln, Schlussfolgerungen.

   Beispiel: Prolog.

Kurze Historie

Die Grundlagen der objektorientierten Programmierung entstanden bereits in den 1960er Jahren: Simula 67 (entwickelt von Ole-Johan Dahl und Kristen Nygaard) gilt als die erste Programmiersprache mit objektorientierten Konzepten. In den 1980ern wurde OOP populärer, insbesondere durch die Sprache Smalltalk. Mit der Verbreitung von C++ und später Java wurde OOP zum dominanten Paradigma in der Softwareentwicklung.

Heute wird OOP in vielen modernen Programmiersprachen wie Python, C# oder Ruby eingesetzt und ist ein fester Bestandteil der Informatikausbildung. Sie findet in zahlreichen Bereichen Anwendung, etwa in der Anwendungsentwicklung, der Spieleprogrammierung oder der Systementwicklung.

OOP ermöglicht es, große Softwareprojekte effizient zu entwickeln, indem die Komplexität in kleinere, logisch verbundene Einheiten zerlegt wird. In der Praxis werden dazu von Software-Architekt:innen zunächst die benötigten Klassen und deren Schnittstellen abstrakt definiert – häufig mithilfe von UML-Diagrammen –, bevor die konkrete Implementierung durch Programmiererinnen und Programmierer erfolgt.

10.2. Bestandteile einer Klasse: Methoden und Attribute

Ohne genauer darauf hinzuweisen, haben wir bereits sehr oft mit Objekten gearbeitet: In Python 3 sind nämlich alle Datentypen in Wahrheit Klassen. Besonders auffällig ist dies bei der Verwendung von Datenstrukturen, wie Listen:

```
my_list = [3, 1, 2]  # erzeugt ein Objekt der Klasse "list"
print(type(my_list))
```

```
<class 'list'>
```

Die Klasse `list` bietet eine Vielzahl an **Methoden** an, das sind alle Funktionen, die innerhalb der Klasse definiert sind, z.B.:

```
my_list.sort()  # sort-Methode der Klasse list
my_list.reverse()  # reverse-Methode
my_list.append(10)  # append-Methode
```

Wir betrachten nun als weiteres Beispiel die Klasse `datetime` aus dem gleichnamigen Modul:

```
from datetime import datetime  # importiert die Klasse

# wir erzeugen eine Instanz von datetime
# mit dem heutigen Datum:
today = datetime.now()
```

Über die Referenz `today` können wir nun die erzeugte Instanz untersuchen und auf ihre Eigenschaften - die sogenannten **Attribute** - zugreifen:

```
today.year  # Zugriff auf das Attribut year
```

```
2025
```

```
today.month  # Zugriff auf das Attribut month
```

```
4
```

Die Klasse bietet aber auch Methoden an, z.B. `toordinal()`, eine Funktion, die das Datum als Integer ausgibt, der die Tage fortlaufend nummeriert, wobei der 1. Januar des Jahres 1 n. Chr. die Nummer 1 hat:

```
today.toordinal()
```

```
739369
```

Ein zweites Beispiel ist die Methode `strftime`, mit der ein Datum als formatierter String ausgegeben wird:

```
today.strftime("%d.%m.%Y")  # das Format ist beliebig anpassbar
```

```
'28.04.2025'
```

Als wir unsere Instanz oben erzeugt haben, verwendeten wir die Methode `now()`. Diese Methode konnten wir ohne konkretes Objekt verwenden: Sie wurde direkt über den Klassennamen `datetime` aufgerufen und liefert ein neues `datetime`-Objekt. Methoden, die zur Klasse selbst gehören und keine Instanz benötigen, heißen in vielen Programmiersprachen **statische Methoden**. Python unterscheidet hier genauer zwischen **Klassenmethoden** (sie erhalten die

Klasse als ersten Parameter; `now()` ist eine solche) und **statischen Methoden** (sie erhalten weder Instanz noch Klasse). Entsprechend gibt es **Klassenattribute**, die zur Klasse gehören und von allen Instanzen geteilt werden.

Fassen wir die wichtigsten Begriffe zusammen:

Attribut
Ein Attribut ist eine Variable innerhalb einer Klasse, die den Zustand oder eine Eigenschaft eines Objekts beschreibt.
Beispiel: `today.year`, wobei `today` ein Objekt der Klasse `datetime` referenziert.
Methode
Eine Methode ist eine Funktion innerhalb einer Klasse, die das Verhalten eines Objekts definiert und auf dessen Attribute zugreifen oder sie verändern kann.
Beispiel: `today.toordinal()`, wobei `today` ein Objekt der Klasse `datetime` referenziert.
Statisch
Statisch bezeichnet Elemente, die zur Klasse selbst gehören und nicht an einzelne Objekte gebunden sind. Sie werden unabhängig von Instanzen verwendet. In Python spricht man genauer von Klassenattributen sowie von Klassenmethoden und statischen Methoden.
Beispiel: `datetime.now()` (in Python eine Klassenmethode)
Members
Der Begriff **Members** fasst alle Bestandteile einer Klasse zusammen; alle Attribute und alle Methoden (auch statische) sind Members der Klasse.

10.3. Erste eigene Klassen

Wir programmieren erste kleine Klassen, um uns an das Konzept und die Syntax zu gewöhnen.

```
class Animal:

    # mit der Methode __init__ - dem "Konstruktor" - fängt alles an:
    def __init__(self, name, age):
        self.name = name  # Attribut name wird angelegt
        self.age = age  # Attribut age wird angelegt

    # eine weitere Methode:
    def speak(self):
        print(f"{self.name} macht ein Geräusch.")

    # eine weitere Methode:
    def birthday(self):
        print(f"Herzlichen Glückwunsch zum Geburtstag, {self.name}!")
```

```
        self.age += 1
```

Wir erkennen die allgemeine Syntax zum Definieren einer Klasse:

```
class Klassenname:
    # eingerückter Code
    # Definition aller Methoden
    ...
```

Der Klassenname soll nach den offiziellen Empfehlungen mit einem Großbuchstaben beginnen und bei mehreren Wörtern in CapWords-Schreibweise (auch PascalCase genannt, siehe PEP 8) geschrieben werden (z.B. `GameController` oder `WindowManager`). Im obigen Beispiel haben wir die Klasse `Animal` definiert und drei Methoden angelegt: `__init__(...)`, `speak(...)` und `birthday(...)`. Auffällig ist, dass alle Methoden unserer Klasse als erstes Argument den Parameter `self` verwenden. Dieser Parameter verweist auf das aktuelle Objekt der Klasse. Das bedeutet, dass `self` die Instanz der Klasse repräsentiert, die die Methode gerade aufruft. Der Parameter wird beim Aufruf automatisch gesetzt: `my_dog.speak()` (siehe unten) ist gleichbedeutend mit `Animal.speak(my_dog)`, `self` ist dann also das Objekt `my_dog`.

Ein weiteres auffälliges Merkmal ist die Methode `__init__`, die durch die doppelten Unterstriche vor und nach ihrem Namen hervorsticht. Methoden dieser Art werden in Python als **Magic Methods** bezeichnet, da sie eine spezielle Bedeutung für den Python-Interpreter haben. In diesem Fall ist `__init__` der **Konstruktor** einer Klasse. Der Konstruktor wird automatisch aufgerufen, wenn ein neues Objekt dieser Klasse erstellt wird. Neben dem obligatorischen `self` erhält der Konstruktor die Argumente `name` und `age`, die dann über die Zuweisungen `self.name = name` und `self.age = age` als Attribute in der Instanz gespeichert werden. Die Übergabeparameter des Konstruktors müssen dabei nicht zwangsläufig genauso heißen wie die Attribute, allerdings ist das eine weit verbreitete Praxis, um Verwechslungen bei der Zuweisung zu vermeiden.

Die Methoden `speak` und `birthday` sind relativ selbsterklärend: `speak` gibt eine allgemeine Meldung des Tieres auf dem Bildschirm aus (mit `print`; einen Rückgabewert hat die Methode nicht), während `birthday` zum Geburtstag gratuliert und das Alter um eins erhöht. Testen wir nun unsere Klasse:

```
my_dog = Animal("Bello", 5)  # ruft den Konstruktor auf
my_dog.speak()
```

```
Bello macht ein Geräusch.
```

```
my_dog.age  # greift auf das Attribut age zu
```

```
5
```

```
my_dog.birthday()
```

```
Herzlichen Glückwunsch zum Geburtstag, Bello!
```

```
my_dog.age
```

```
6
```

```
# Natürlich können wir beliebig viele
# Animals erzeugen, hier ein zweites:
my_cat = Animal("Liz", 1)
my_cat.speak()
```

```
Liz macht ein Geräusch.
```

Die `speak`-Methode ist noch sehr unzufriedenstellend. Ein Hund sollte doch bellen, eine Katze miauen. Allerdings ist `Animal` eine relativ abstrakte Basisklasse für alle Tiere und deshalb an dieser Stelle zwangsläufig unspezifisch. Wir können nun das Prinzip der Vererbung verwenden, um spezifische Tierklassen anzulegen:

```
# Vererbung: Dog erbt von Animal
class Dog(Animal):
    def speak(self):
        print(f"{self.name} bellt.")

# Vererbung: Cat erbt von Animal
class Cat(Animal):
    def speak(self):
        print(f"{self.name} miaut.")
```

Die Basisklasse (hier: `Animal`) wird bei der Klassendefinition in Klammern angegeben. Das Anlegen der Klassen `Dog` und `Cat` ist sehr einfach, denn wir spezifizieren nur die Unterschiede zur Basisklasse, also die individuellen Realisierungen der Methode `speak`. Alles andere wird von der Basisklasse `Animal` übernommen (die abgeleiteten Klassen werden deshalb auch "Subklassen" oder "Kindklassen" genannt). Probieren wir unsere neuen Klassen aus:

```
my_dog = Dog("Bello", 6)
my_cat = Cat("Liz", 1)
my_turtle = Animal("Einstein", 100)

my_dog.speak()
my_cat.speak()
my_turtle.speak()
```

```
Bello bellt.
Liz miaut.
Einstein macht ein Geräusch.
```

Man sagt auch: Die Methode `speak` wurde in den Klassen `Dog` und `Cat` *überschrieben*. Mit Vererbung könnten wir noch deutlich spezifischere Klassen anlegen, z.B. konkrete Hunderassen (hierzu würden wir von `Dog` ableiten, z.B. `class Husky(Dog):`). Es entstehen beliebig komplexe Klassen-Hierarchien. An unserem Beispiel erkennen wir auch, dass die Basisklasse davon unbeschadet ist und weiterhin verwendet werden kann (`my_turtle` referenziert eine allgemeine `Animal`-Instanz).

Alle Subklassen von `Animal` werden weiterhin als `Animal` betrachtet und können auch als solche verwendet werden. Das bedeutet, dass eine Funktion, die ein `Animal` erwartet, auch mit einem `Dog` oder einer `Cat` funktioniert. Dies ist ein Beispiel für Polymorphismus. Wichtig: Obwohl alle Instanzen Tiere sind, sind sie nicht alle gleich - die gleiche Methode (hier `speak()`) wird je nach konkretem Typ unterschiedlich ausgeführt.

Umgekehrt muss man vorsichtiger sein: Eine Subklasse kann zusätzliche Methoden oder Attribute haben, die in der Basisklasse nicht existieren. Wenn eine Funktion speziell auf diese neuen Methoden zugreifen möchte, muss sie den spezifischen Typ des Objekts kennen, anstatt nur die Basisklasse zu verwenden. Andernfalls könnte der Zugriff auf diese zusätzlichen Eigenschaften oder Methoden zu einem Fehler führen, falls das Objekt tatsächlich eine Instanz der Basisklasse und nicht der Subklasse ist. Beispielhaft erweitern wir unsere Klasse `Dog` um eine Methode `fetch()`:

```
class Dog(Animal):
    def speak(self):
        print(f"{self.name} bellt.")

    def fetch(self):
        print(f"{self.name} fängt den Ball.")
```

```python
my_dog = Dog("Bello", 6)
my_dog.fetch()

my_turtle = Animal("Einstein", 100)
# my_turtle.fetch() FEHLER! Die Methode existiert nicht in der Basisklasse!

my_cat = Cat("Liz", 1)
# my_cat.fetch() FEHLER! In der Klasse Cat existiert sie auch nicht!
```

```
Bello fängt den Ball.
```

10.4. UML-Klassendiagramme

Wir wollen uns nun mit dem systematischen Design von Klassen befassen. Häufig wird dazu die **Unified Modeling Language (UML)** verwendet. Es handelt sich dabei um eine standardisierte grafische Sprache zur Modellierung und Visualisierung von Software-Systemen. UML ermöglicht es Entwickler:innen, die Struktur, das Verhalten und die Interaktionen eines Systems darzustellen. Ein besonders wichtiger Bestandteil von UML ist das Klassendiagramm. Es stellt die statische Struktur eines Systems dar, indem es die Klassen eines Systems sowie deren Beziehungen zueinander visualisiert. UML ist sehr mächtig und kann hier nur in kleinen Auszügen dargestellt werden.

Grundstruktur eines Klassendiagramms

Eine Klasse wird durch ein Rechteck dargestellt, welches in drei übereinanderliegende Felder unterteilt wird (siehe Abbildung 10.1):

- Der Name der Klasse wird in das oberste Feld geschrieben.
- Im zweiten Feld werden alle Attribute aufgeführt.
- Im dritten Feld folgen die Methoden.

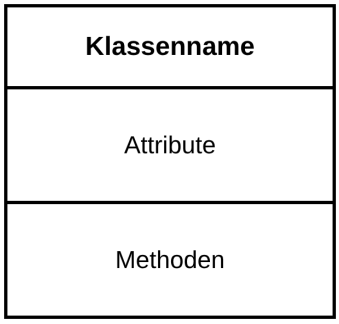


Abbildung 10.1.: Aufbau eines UML-Klassendiagramms.

Konkret sieht es für unsere `Animal`-Klasse aus dem vorigen Abschnitt so aus wie in Abbildung 10.2.

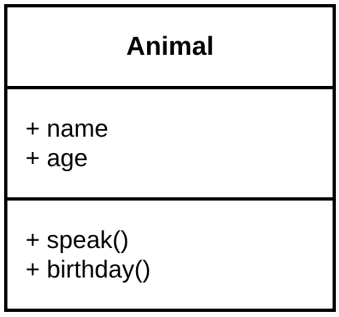


Abbildung 10.2.: UML-Klassendiagramm für die Klasse `Animal`.

Vererbung wird im UML-Diagramm durch einen Pfeil mit nicht ausgefüllter Dreiecksspitze dargestellt, der **von der abgeleiteten Klasse zur Basisklasse** zeigt. In der abgeleiteten Klasse werden Attribute und Methoden, die unverändert von der Basisklasse übernommen werden, nicht aufgeführt. In Abbildung 10.3 sehen wir dies für die Klassen `Dog` und `Animal`.

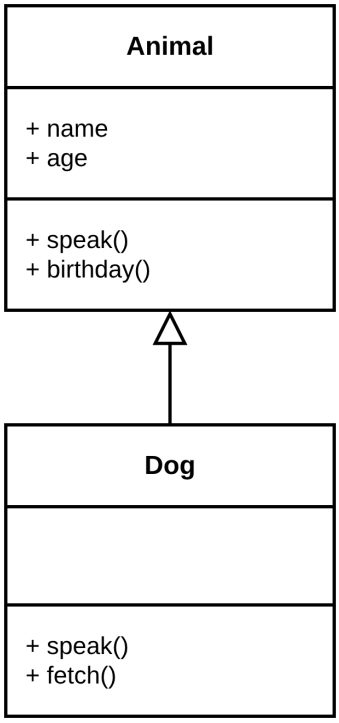


Abbildung 10.3.: UML-Klassendiagramm mit Vererbung: `Dog` erbt von `Animal`.

Zugriffsmodifikatoren

Die +-Zeichen in den bisherigen Diagrammen sind nicht beliebig, sondern stellen sog. **Zugriffsmodifikatoren** dar. Sie geben an, wer über die gekennzeichneten Attribute und Methoden verfügen darf. Man unterscheidet drei Stufen:

- **Public (+)**: Öffentliche Elemente, die von überall und jedem aufgerufen werden können.
- **Private (-)**: Private Elemente, die nur innerhalb der Klasse verwendet werden können. Außenstehende sollen diese Elemente nicht verwenden. Auch abgeleitete Klassen sollen keinen Zugriff auf sie haben.
- **Protected (#)**: Geschützte Elemente, die in der Klasse und ihren Unterklassen verwendet werden können. Für Außenstehende sollen sie aber nicht direkt verfügbar sein.

In Python gibt es keine Möglichkeit, die Zugriffsrechte eins zu eins umzusetzen. Das ist ein großer Unterschied zu anderen Programmiersprachen, wie z.B. C++. Dennoch gibt es Mechanismen und Konventionen, wie in Python mit den unterschiedlichen Zugriffsrechten umgegangen wird. Darauf gehen wir in einem späteren Abschnitt genauer ein.

Beziehungen zwischen Klassen

In UML wird eine ungerichtete Verbindungslinie verwendet, um auszudrücken, dass zwei Klassen miteinander in Beziehung stehen. Diese Beziehung wird als *Assoziation* bezeichnet. Ein Beispiel hierfür ist die Beziehung zwischen den Klassen `Person` und `Animal` in Abbildung 10.4, wobei eine Person Haustiere besitzen kann. In UML wird in diesem Fall häufig die **Multiplizität** angegeben, um zu definieren, wie viele Instanzen der einen Klasse mit wie vielen Instanzen der anderen Klasse in Beziehung stehen können.

In unserem Beispiel kann eine Person 0 oder beliebig viele Tiere besitzen, während jedes Tier exakt eine Person als Besitzer bzw. Besitzerin hat (eine vereinfachte Annahme). Dies verdeutlicht, dass zu jeder Assoziation immer zwei Multiplizitäten gehören, die direkt an den Enden der Verbindungslinie notiert werden. Die gängigsten Bezeichner für Multiplizitäten sind:

1: genau eine Instanz.

0..1: keine oder genau eine Instanz.

0..*: keine oder beliebig viele Instanzen.

1..*: eine oder beliebig viele Instanzen.

m..n: mindestens m Instanzen, maximal n Instanzen (z.B. 3..10).

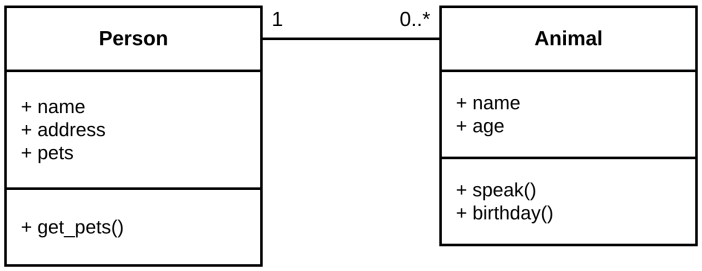


Abbildung 10.4.: UML-Klassendiagramm mit Assoziation: Beziehung zwischen `Person` und `Animal`.

Ergänzungen

- Im UML-Klassendiagramm kann außerdem der Datentyp von Attributen mitangegeben werden. Dazu verwendet man einen Doppelpunkt hinter dem Attribut gefolgt von der Typangabe, z.B. `name: str`.

- Bei den Methoden werden in den Klammern alle Parameter angegeben, allerdings ohne den Parameter `self`. Dieser muss in Python zwar in jeder Methodendefinition ausdrücklich als erster Parameter stehen (Ausnahmen: Klassenmethoden erhalten stattdessen die Klasse, statische Methoden weder Instanz noch Klasse – siehe oben), wird beim Aufruf aber automatisch übergeben und ist eine Besonderheit von Python (UML-Diagramme sind nicht auf eine bestimmte Programmiersprache beschränkt).

10.5. Beispiel: Bibliotheksverwaltung

In diesem Abschnitt wollen wir eine einfache Bibliotheksverwaltung programmieren. Das geplante System wird im folgenden UML-Klassendiagramm dargestellt:

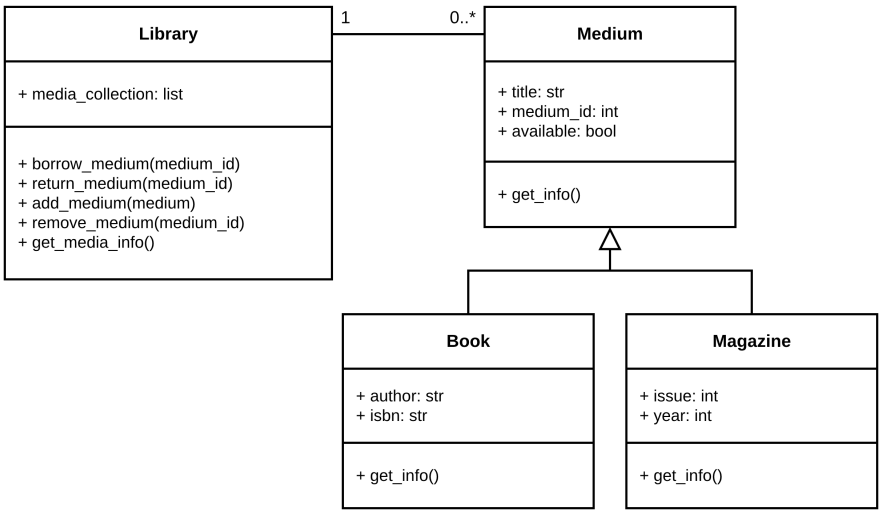


Abbildung 10.5.: UML-Klassendiagramm für die Bibliotheksverwaltung.

Die Klasse `Library`

Das einzige Attribut der Klasse `Library` ist `media_collection` und stellt eine Liste aller Medien im Bestand dar. Über die Methoden der Klasse können Medien entliehen (`borrow_medium(medium_id)`) bzw. zurückgegeben (`return_medium(medium_id)`), dem Bestand hinzugefügt (`add_medium(medium)`) bzw. aus ihm entfernt (`remove_medium(medium_id)`) werden. Schließlich soll `get_media_info()` eine Übersicht über alle Medien der Bibliothek liefern.

Die Klasse `Medium` und ihre Kindklassen: `Book` und `Magazine`

Die Klasse `Medium` auf der anderen Seite stellt eine Basisklasse für alle in der Bibliothek verfügbaren Medien dar (in unserem Beispiel Bücher und Magazine). Alle Medien besitzen einen Titel und eine eindeutige ID. Außerdem kennzeichnet ein boolesches Attribut, ob sie zurzeit verfügbar sind oder nicht (`available`). Die Methode `get_info()` soll die wesentlichen Informationen über das Medium und den aktuellen Ausleih-Status auf dem Bildschirm ausgeben. Die Klassen `Book` und `Magazine` erben von `Medium` und erweitern die Basisklasse um spezifische Attribute. Die Methode `get_info()` soll jeweils überschrieben werden.

Die Medien-ID hat eine wichtige Bedeutung: Wie wir an den Methoden zum Ausleihen und Zurückgeben erkennen, arbeitet die Bibliothek intern ausschließlich mit dieser eindeutigen Kennzeichnung und nicht etwa mit dem Titel oder anderen Attributen. Sie muss auf jeden Fall eindeutig sein! Es stellt sich noch die Frage, wann und durch wen ein Medium seine ID erhält. In der realen Welt sicherlich durch die Bibliothek, vermutlich in dem Moment, in dem es in den Bestand der Bibliothek aufgenommen wird. Wir wollen das Beispiel aber zunächst einfacher halten und weisen jedem Medium bei seiner Erzeugung händisch eine ID zu.

Mit `super()` auf die Basisklasse zugreifen

Bevor wir mit dem Programmieren beginnen, möchten wir noch auf eine wichtige Besonderheit hinweisen: Da die abgeleiteten Klassen diesmal eigene zusätzliche Attribute besitzen, reicht es nicht aus, einfach den Konstruktor der Basisklasse zu übernehmen. Stattdessen müssen wir den Konstruktor in den Unterklassen selbst definieren. Dabei können wir jedoch weiterhin den bereits bestehenden Konstruktor der Basisklasse nutzen, um deren Attribute korrekt zu initialisieren. Dafür stellt Python die Funktion `super()` bereit. Mit ihrer Hilfe können wir gezielt Methoden der übergeordneten Klasse aufrufen. Konkret bedeutet das: Mit `super().__init__(...)` rufen wir den Konstruktor der Basisklasse auf und können ihm die entsprechenden Werte übergeben.

Die Implementierung

Jetzt sind wir bereit für die Implementierung, wir beginnen auf der rechten Seite mit den Medien:

```
class Medium:

    def __init__(self, title, medium_id):
        self.title = title
        self.medium_id = medium_id
        self.available = True  # wird auf True initialisiert
```

```
    def get_info(self):
        status = "verfügbar" if self.available else "entliehen"
        print(f"ID: {self.medium_id}, Titel: {self.title}, Status: {status}")

class Book(Medium):

    def __init__(self, title, medium_id, author, isbn):
        # Konstruktor der Basisklasse aufrufen:
        super().__init__(title, medium_id)
        # zusätzliche Attribute zuweisen:
        self.author = author
        self.isbn = isbn

    def get_info(self):
        # Wir könnten auch hier super() verwenden,
        # allerdings könnten wir dann die neuen Attribute
        # nur darunter anfügen:
        # super().get_info()
        # print(f"Autor: {self.author}, ISBN: {self.isbn}")

        status = "verfügbar" if self.available else "entliehen"
        print(f"ID: {self.medium_id}, Titel: {self.title}, Autor:
         ↪  {self.author}, ISBN: {self.isbn}, Status: {status}")

class Magazine(Medium):

    def __init__(self, title, medium_id, issue, year):
        # Konstruktor der Basisklasse aufrufen:
        super().__init__(title, medium_id)
        # zusätzliche Attribute zuweisen:
        self.issue = issue
        self.year = year

    def get_info(self):
        status = "verfügbar" if self.available else "entliehen"
        print(f"ID: {self.medium_id}, Titel: {self.title}, Ausgabe:
         ↪  {self.issue}, Jahr: {self.year}, Status: {status}")
```

Legen wir ein paar Beispiel-Medien an:

```
book1 = Book("Herr der Ringe", 1, "Tolkien, J. R. R.", "978-3-16-148410-0")
magazine1 = Magazine("Nature", 2, 5, 2004)
```

```
book1.get_info()
magazine1.get_info()
```

```
ID: 1, Titel: Herr der Ringe, Autor: Tolkien, J. R. R., ISBN: 978-3-16-
148410-0, Status: verfügbar
ID: 2, Titel: Nature, Ausgabe: 5, Jahr: 2004, Status: verfügbar
```

```
class Library:

    def __init__(self):
        self.media_collection = []

    def borrow_medium(self, medium_id):

        # Bestand wird durchsucht
        for medium in self.media_collection:

            if medium.medium_id == medium_id:

                # Das Medium wurde im Bestand gefunden

                if medium.available:
                    medium.available = False
                    print(f"{medium.title} wurde entliehen.")
                else:
                    print(f"AUSLEIHEN NICHT MÖGLICH! {medium.title} ist
                     ↪  bereits entliehen.")

                return

        print("Das gewünschte Medium befindet sich nicht im Bestand.")

    def return_medium(self, medium_id):

        for medium in self.media_collection:

            if medium.medium_id == medium_id:

                if not medium.available:
                    medium.available = True
                    print(f"{medium.title} wurde zurückgegeben.")
```

```
                else:
                    print(f"Fehler: {medium.title} wurde noch nicht
                     ↪  entliehen.")

                return

        print("Das Medium gehört nicht zum Bestand der Bibliothek.")

    def add_medium(self, medium):

        medium.available = True
        self.media_collection.append(medium)

    def remove_medium(self, medium_id):

        for medium in self.media_collection:

            if medium.medium_id == medium_id:
                self.media_collection.remove(medium)
                print(f"{medium.title} wurde aus dem Bestand entfernt.")
                return

        print("Das Medium gehört nicht zum Bestand der Bibliothek.")

    def get_media_info(self):
        print("Bibliotheks-Bestand:")
        for medium in self.media_collection:
            medium.get_info()
```

```
# Es folgen ein paar Tests unseres Systems
library = Library()
library.add_medium(book1)
library.add_medium(magazine1)
library.get_media_info()
```

```
Bibliotheks-Bestand:
ID: 1, Titel: Herr der Ringe, Autor: Tolkien, J. R. R., ISBN: 978-3-16-
148410-0, Status: verfügbar
ID: 2, Titel: Nature, Ausgabe: 5, Jahr: 2004, Status: verfügbar
```

```
library.borrow_medium(1)
library.get_media_info()
```

```
Herr der Ringe wurde entliehen.
Bibliotheks-Bestand:
ID: 1, Titel: Herr der Ringe, Autor: Tolkien, J. R. R., ISBN: 978-3-16-
148410-0, Status: entliehen
ID: 2, Titel: Nature, Ausgabe: 5, Jahr: 2004, Status: verfügbar
```

```
library.borrow_medium(1)
library.borrow_medium(2)
library.return_medium(1)
library.get_media_info()
```

```
AUSLEIHEN NICHT MÖGLICH! Herr der Ringe ist bereits entliehen.
Nature wurde entliehen.
Herr der Ringe wurde zurückgegeben.
Bibliotheks-Bestand:
ID: 1, Titel: Herr der Ringe, Autor: Tolkien, J. R. R., ISBN: 978-3-16-
148410-0, Status: verfügbar
ID: 2, Titel: Nature, Ausgabe: 5, Jahr: 2004, Status: entliehen
```

Übungen:

1. Implementieren Sie eine Klasse, die einen weiteren Medien-Typ darstellt (z.B. DVD oder CD). Wählen Sie selbst geeignete Attribute und überschreiben Sie `get_info()`. Fügen Sie der Bibliothek ein paar Exemplare des neuen Typs hinzu.
2. Leiten Sie eine Klasse `EBook` von `Book` ab. Zusätzliche Attribute: Dateiformat und Dateigröße in MB. Fügen Sie der Bibliothek ein paar EBooks hinzu.
3. Implementieren Sie in der Klasse `Library` eine Suchfunktion für Benutzer, die ein Medium anhand des Titels findet.
4. Recherchieren Sie, was UUIDs sind und wie man diese in Python erzeugen kann. Jedes Medium soll bei der Aufnahme in den Bibliotheksbestand eine UUID als `medium_id` zugewiesen bekommen. Passen Sie den Konstruktor von `Medium` entsprechend an (initialisieren Sie `medium_id` z.B. mit -1).
5. Ändern Sie die Klasse `Library` ab: Ersetzen Sie die Liste `media_collection` durch ein geeignetes Dictionary.

10.6. Public, Protected, Private

In objektorientierten Programmiersprachen wie Java oder C++ ist die Sichtbarkeit von Attributen und Methoden streng durch Zugriffsmodifikatoren geregelt: public, protected und private. Python kennt diese Schlüsselwörter nicht in derselben Weise, bietet aber eigene **Konventionen**, um gewünschte Zugriffseinschränkungen auf Klassenbestandteile auszudrücken.

public - der Standard

In Python ist alles standardmäßig public. Das bedeutet, dass auf Methoden und Attribute von außerhalb der Klasse direkt zugegriffen werden kann. In den bisherigen Beispielen konnte man sowohl auf Attribute (z.B. `issue` in unserer `Magazine`-Klasse) als auch auf Methoden (z.B. `get_info()`) uneingeschränkt zugreifen. Das ist in anderen Programmiersprachen eher die Ausnahme als die Regel.

protected - nur für den internen Gebrauch

Python verwendet einen führenden Unterstrich (_), um zu signalisieren: Dieses Attribut oder diese Methode sollte von außen nicht direkt verwendet werden, sondern nur innerhalb der Klasse und ihrer abgeleiteten Klassen. Technisch ist der Zugriff aber **weiterhin möglich** – es handelt sich nur um eine Konvention, kein Verbot. Hier wird klar, dass Python stark auf die Verantwortung der Programmiererinnen und Programmierer setzt, statt auf strenge Zugriffsverbote. Ein Beispiel:

```
class Vehicle:
    def __init__(self):
        self._fuel_level = 100  # protected: nur für interne Verwendung
         ↪ gedacht

class Car(Vehicle):
    def show_fuel(self):
        print(f"Aktueller Tankstand: {self._fuel_level}%")
    def refuel(self):
        self._fuel_level = 100
    def drive(self, velocity, distance):
        pass  # Platzhalter für die Verbrauchsberechnung während einer Fahrt
         ↪
```

private - Gehört nur einer einzigen Klasse!

Private bedeutet, dass dieses Attribut oder diese Methode nur zu genau einer Klasse gehören soll und nicht zu davon abgeleiteten Klassen. Sie werden durch zwei führende Unterstriche (`__`) gekennzeichnet. Hier handelt es sich um mehr als nur eine Konvention: Python reagiert auf die beiden führenden Unterstriche mit *Name Mangling*: Das heißt, Python verändert intern den Namen und stellt den Namen der *besitzenden* Klasse voran (wiederum mit einem führenden Unterstrich). Kennt man den Mechanismus, könnte man über den neuen Namen wie gewohnt auf das Attribut oder die Methode zugreifen (auch von außerhalb oder innerhalb abgeleiteter Klassen), das ist aber nicht erwünscht. Beispiel:

```
class Engine:
    def __init__(self):
        self.__power = 150  # private: gehört nur der Klasse Engine

    def show_power(self):
        print(f"Leistung: {self.__power} PS")
```

```
test_engine = Engine()
# So ist es gedacht:
test_engine.show_power()

# So nicht und es gibt einen Fehler:
# print(test_engine.__power)
# Grund: Intern heißt das Attribut nicht mehr __power, sondern
 ↪  _Engine__power.

print(test_engine._Engine__power)  # funktioniert - aber unsauber!
```

```
Leistung: 150 PS
150
```

Wichtig: `__power` ist nicht wirklich privat. Es wird intern als `_Engine__power` gespeichert. Über diesen Namen kann man darauf zugreifen, sollte es aber nicht tun. Gedacht ist der doppelte Unterstrich vor allem dafür, Namenskonflikte in abgeleiteten Klassen zu vermeiden. In der Praxis kennzeichnet man nicht-öffentliche Attribute meist mit einem einfachen Unterstrich. In anderen Sprachen heißt `private` auch wirklich `private`: kein Zugriff von außerhalb der Klasse ist möglich (auch nicht aus abgeleiteten Klassen heraus)!

Getter und Setter

Getter- und Setter-Methoden sind einfache Funktionen, die den kontrollierten Zugriff auf protected (`_attribute`) oder private (`__attribute`) Attribute ermöglichen. Die Idee: Über diese Funktionen wird eine öffentliche, kontrollierte Schnittstelle zu den geschützten Attributen angeboten.

- **Getter** liefern den aktuellen Wert eines Attributs zurück.
- **Setter** erlauben das Ändern eines Attributs, oft mit Prüfungen oder zusätzlicher Logik, um den gültigen Wertebereich einzuhalten.

```
class Vehicle:
    def __init__(self):
        self._fuel_level = 100  # protected Attribut: nur intern direkt
         ↪  verwenden!

    def get_fuel_level(self):  # Getter: zum Abrufen des Attributs von außen.
        return self._fuel_level

    def set_fuel_level(self, value):  # Setter mit Validierung: zum
     ↪  kontrollierten Setzen des Attributs.
        if 0 <= value <= 100:
            self._fuel_level = value
```

Getter und Setter unterstützen das Prinzip der Kapselung und sind in der objektorientierten Programmierung stark verbreitet.

10.7. Magic Methods

Magic Methods – auch bekannt als *dunder methods* (wegen der doppelten Unterstriche: double underscore) – sind spezielle Methoden einer Klasse, die eine ganz bestimmte Bedeutung haben. Sie ermöglichen es selbstprogrammierten Objekten, sich wie eingebaute Python-Objekte zu verhalten, z.B. bei Vergleichen, Rechenoperationen oder der Darstellung im Print-Output. *Magisch* ist daran, dass diese Methoden automatisch von Python aufgerufen werden, wenn bestimmte Operationen auf einem Objekt durchgeführt werden – man muss sie nicht explizit aufrufen. Ein Beispiel ist der Konstruktor, der automatisch bei der Objekt-Erzeugung aufgerufen wird. Es gibt eine Vielzahl von Magic Methods, wir stellen einige vor:

Methode	Wird aufgerufen bei …	Zweck
`__init__`	Objekt-Erzeugung	Initialisierung des Objekts

Methode	Wird aufgerufen bei …	Zweck
`__str__`	`str(obj)` oder `print(obj)`	String-Repräsentation für Ausgaben
`__eq__`	`obj1 == obj2`	Vergleich auf Gleichheit
`__lt__`	`obj1 < obj2`	Kleiner-Vergleich
`__add__`	`obj1 + obj2`	Addition mit benutzerdefinierter Logik
`__len__`	`len(obj)`	Länge eines Objekts
`__int__`	`int(obj)`	Integer-Repräsentation des Objekts (analog für `float`)

Übrigens steckt auch hinter `+=` eine Magic Method: `__iadd__`. Ist sie definiert, verändert `obj += x` das Objekt direkt – so bei Listen (siehe Kapitel *Datenstrukturen*). Fehlt sie, führt Python stattdessen `obj = obj + x` aus und verwendet dafür `__add__`.

Wir erweitern unsere Klasse `Book`, um einige Magic Methods exemplarisch vorzustellen:

```
class Book(Medium):
    def __init__(self, title, medium_id, author, isbn):
        super().__init__(title, medium_id)
        self.author = author
        self.isbn = isbn
    def get_info(self):

        status = "verfügbar" if self.available else "entliehen"

        print(f"ID: {self.medium_id}, Titel: {self.title}, Autor:
         ↪  {self.author}, ISBN: {self.isbn}, Status: {status}")

    # der neue Teil: #
    #----------------#

    # wir definieren eine String-Repräsentation für Books:
    def __str__(self):
        return f"'{self.title}' von {self.author}"

    # wir definieren den Gleichheitsoperator für Books:
    def __eq__(self, other):
        if not isinstance(other, Book):
            return False  # ein Book ist nie gleich einem Objekt anderen Typs
        return self.isbn == other.isbn

    # wir definieren eine Integer-Repräsentation für Books:
    def __int__(self):
```

```
        return self.medium_id
```

```
book1 = Book("Herr der Ringe", 1, "Tolkien, J. R. R.", "978-3-16-148410-0")
book2 = Book("Harry Potter und der Feuerkelch", 2, "Rowling, J. K.",
 ↪  "978-3-16-148411-7")
```

```
print(book1)  # String-Repräsentation wird ausgegeben
```

```
'Herr der Ringe' von Tolkien, J. R. R.
```

```
print(book2)
```

```
'Harry Potter und der Feuerkelch' von Rowling, J. K.
```

```
int(book1)  # Casting nach int: wir erhalten die ID
```

```
1
```

```
int(book2)
```

```
2
```

```
print(book1 == book2)  # ISBNs werden verglichen
```

```
False
```

```
print(book1 != book2)
```

```
True
```

```
# book1 < book2  # Diese Operation haben wir nicht definiert, sie führt
 ↪  deshalb zum Fehler!
```

Operatoren überladen

Ein **Operator** ist eine Vorschrift, die aus einer Reihe von **Operanden** einen neuen Wert berechnet. Typische Beispiele sind Rechenoperatoren, wie `+` oder `*`, aber auch Vergleichsoperatoren wie `<` oder `==` (bei Letzteren ist das Ergebnis ein Wahrheitswert). Mit Magic Methods kann man in Python **Operatoren überladen** – also ihnen eine benutzerdefinierte Bedeutung für eigene Klassen geben.
Wichtige Magic Methods für Rechenoperationen:

Operator	Magic Method	Beschreibung
`+`	`__add__(self, other)`	Addition
`-`	`__sub__(self, other)`	Subtraktion
`*`	`__mul__(self, other)`	Multiplikation
`/`	`__truediv__(self, other)`	Division (Gleitkomma)
`//`	`__floordiv__(self, other)`	Ganzzahl-Division
`%`	`__mod__(self, other)`	Modulo (Rest)
`**`	`__pow__(self, other)`	Potenz
`abs(obj)`	`__abs__(self)`	Absolutwert / Betrag

Wichtige Magic Methods für Vergleichsoperationen:

Operator	Magic Method	Beschreibung
`==`	`__eq__(self, other)`	Gleichheit
`!=`	`__ne__(self, other)`	Ungleichheit
`<`	`__lt__(self, other)`	Kleiner als
`<=`	`__le__(self, other)`	Kleiner oder gleich
`>`	`__gt__(self, other)`	Größer als
`>=`	`__ge__(self, other)`	Größer oder gleich

Hinweis: Wenn `__eq__(...)` implementiert wird, nicht aber `__ne__(...)`, funktioniert `!=` automatisch als Negation von `__eq__(...)`.

Übung:

Implementieren Sie eine Klasse `Vector`, die Vektoren beliebiger Dimension repräsentiert und grundlegende Operationen der linearen Algebra mithilfe von Magic Methods unterstützt. Prüfen Sie bei allen Vektoroperationen, ob die Dimensionen übereinstimmen.

Anforderungen:

- Der Konstruktor nimmt eine Liste von Zahlen entgegen (z.B. `v = Vector([1, 2, 3])`).

- Implementieren Sie eine String-Repräsentation, so dass `print(v)` aufgerufen werden kann.
- `+` und `-` sollen Vektoren komponentenweise addieren bzw. subtrahieren.
- `*` soll ein Skalarprodukt zwischen zwei Vektoren realisieren.
- `abs()` soll die euklidische Norm des Vektors, also seine Länge, zurückgeben.
- Die Operatoren `==` und `!=` sollen überprüfen, ob zwei Vektoren gleich sind, d.h. ob alle Komponenten übereinstimmen.
- Die anderen Vergleichsoperatoren sollen sich auf die Länge der Vektoren beziehen.

10.8. Abstrakte Klassen

Abstrakte Klassen sind spezielle Klassen, die man nicht direkt benutzen kann, um Objekte zu erstellen. Stattdessen dienen sie als Vorlage für andere Klassen. Sie definieren Methoden, die alle abgeleiteten Klassen verpflichtend selbst umsetzen müssen. Man könnte sagen: Die abstrakte Klasse gibt die Struktur für alle konkreten Klassen vor. Allerdings handelt es sich dabei um eine Art *Basisstruktur*, jede abgeleitete Klasse kann darüber hinaus weitere Attribute und Methoden anlegen.

Im UML-Klassendiagramm kennzeichnen wir eine abstrakte Klasse durch ein `<<abstract>>` über dem Klassennamen (alternativ kann eine kursive Schrift für den Klassennamen verwendet werden, was aber leichter übersehen werden kann). Abstrakte Methoden werden kursiv gesetzt. Das folgende Beispiel illustriert das Konzept der abstrakten Klassen am Beispiel von geometrischen Figuren: `Shape` ist eine abstrakte Basisklasse mit zwei abstrakten Methoden, nämlich `area()` und `perimeter()` (Flächen- und Umfangsberechnung). Alle konkreten geometrischen Formen werden von dieser Basisklasse abgeleitet und müssen verpflichtend `area()` und `perimeter()` überschreiben. Dadurch wird ein einheitliches Design aller geometrischen Formen sichergestellt. Eine andere Sichtweise ist: Die abstrakte Klasse legt das Interface für die Benutzung der Klassen fest.

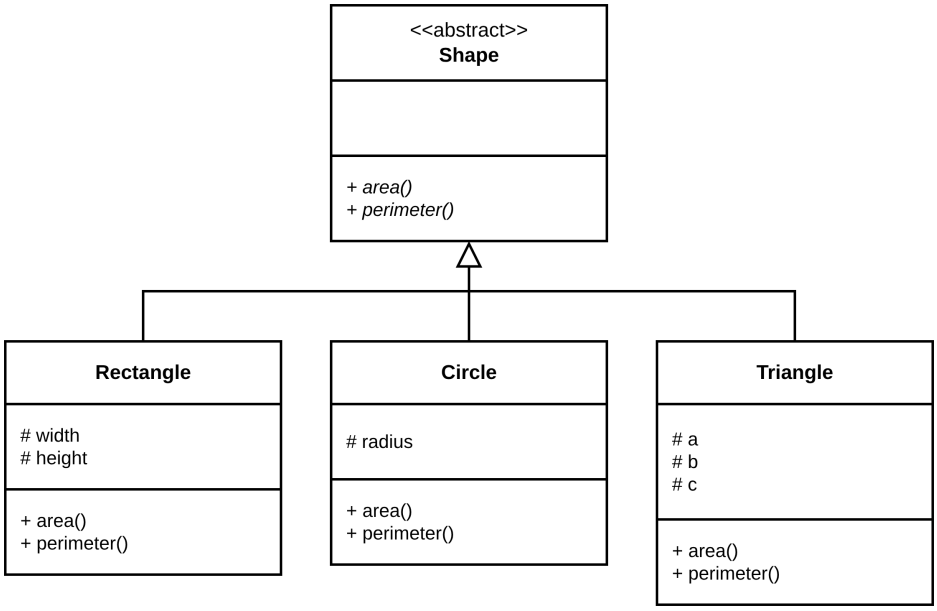


Abbildung 10.6.: UML-Klassendiagramm mit abstrakter Basisklasse

In Python nutzt man für die Umsetzung im Code das Modul **abc** (*abstract base classes*, abstrakte Basisklassen). Im folgenden Codebeispiel sehen wir, wie dies funktioniert:

- Die abstrakte Klasse wird von `ABC` abgeleitet.
- Ihre Methoden werden mit einem sog. Decorator, `@abstractmethod`, als abstrakte Methoden markiert.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        """Flächeninhalt berechnen"""
        pass

    @abstractmethod
    def perimeter(self):
        """Umfang berechnen"""
        pass
```

Die abstrakte Klasse `Shape` kann natürlich nichts berechnen (Welche Fläche?! Welchen Umfang?!), sie dient nur als eine Art “Bauplan” für alle konkreten Klassen, die nun von ihr abgeleitet werden:

```
import math
class Rectangle(Shape):
    def __init__(self, width, height):
        self._width = width
        self._height = height

    def area(self):
        return self._width * self._height

    def perimeter(self):
        return 2 * (self._width + self._height)

class Circle(Shape):
    def __init__(self, radius):
        self._radius = radius

    def area(self):
        return math.pi * self._radius ** 2

    def perimeter(self):
        return 2 * math.pi * self._radius

class Triangle(Shape):
    def __init__(self, a, b, c):
        self._a = a
        self._b = b
        self._c = c

    def area(self):
        # heronsche Formel
        s = self.perimeter() / 2
        return math.sqrt(s * (s - self._a) * (s - self._b) * (s - self._c))

    def perimeter(self):
        return self._a + self._b + self._c

# Test

shapes = [
    Rectangle(4, 5),
    Circle(3),
    Triangle(3, 4, 5)
]
```

```
for shape in shapes:
    print(f"{shape.__class__.__name__}: Fläche = {shape.area():.2f}, Umfang =
     ↪ {shape.perimeter():.2f}")
```

```
Rectangle: Fläche = 20.00, Umfang = 18.00
Circle: Fläche = 28.27, Umfang = 18.85
Triangle: Fläche = 6.00, Umfang = 12.00
```

Durch die Verwendung des Moduls `abc` garantieren wir, dass die abstrakten Methoden vollständig implementiert werden müssen. Im Folgenden verletzen wir diese Regel und definieren eine unvollständige Klasse `Hexagon`, die die abstrakte Methode `perimeter` nicht implementiert:

```
class Hexagon(Shape):
    def __init__(self, side_length):
        self._side_length = side_length

    def area(self):
        return (3 * math.sqrt(3) / 2) * self._side_length ** 2
```

Beim Versuch, ein `Hexagon` zu instanziieren, erhalten wir folgerichtig eine Fehlermeldung:

```
hexagon = Hexagon(3.0)
```

```
TypeError: Can't instantiate abstract class Hexagon without an implementation
 ↪ for abstract method 'perimeter'
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[77], line 1
----> 1 hexagon = Hexagon(3.0)
TypeError: Can't instantiate abstract class Hexagon without an implementation
 ↪ for abstract method 'perimeter'
```

Übungen:

a) Korrigieren Sie die Klasse `Hexagon` und testen Sie sie.

b) Leiten Sie von `Rectangle` eine Klasse `Square` ab. Müssen Sie die abstrakten Methoden erneut implementieren?

c) Recherchieren Sie zu *Decorators*: Wie kann man mit einem Decorator eine statische Methode erstellen? Welche anderen Decorators gibt es?

A. Dateien lesen und schreiben

Bisher standen die Daten unserer Programme fest im Code oder wurden per `input()` eingegeben; nach Programmende waren alle Ergebnisse verloren. In der Praxis kommen Messwerte aber oft aus Dateien, die ein Datenlogger oder Prüfstand geschrieben hat, und Auswertungen sollen gespeichert und weitergegeben werden.

In diesem Anhang lernen Sie,

- wie man Textdateien schreibt und liest,
- wie man Pfade mit dem Modul `pathlib` sauber angibt und
- wie man CSV-Dateien mit dem Modul `csv` verarbeitet.

Im Kapitel *Bibliotheken* haben Sie mit `pd.read_csv()` bereits eine sehr bequeme Möglichkeit kennengelernt, Tabellen einzulesen. Hier schauen wir, was darunter passiert.

A.1. Dateien, Pfade und das Arbeitsverzeichnis

Eine Datei wird über ihren **Pfad** angesprochen. **Absolute Pfade** gehen vom Laufwerk bzw. Wurzelverzeichnis aus (z.B. `C:\Users\weber\Messungen\lauf1.txt`), **relative Pfade** vom aktuellen **Arbeitsverzeichnis** (z.B. `data/lauf1.txt`). Bei einem Jupyter Notebook ist das normalerweise der Ordner, in dem das Notebook liegt. Wir verwenden relative Pfade in den Unterordner `data`, in dem auch die Prüfstandsdaten aus dem Kapitel *Bibliotheken* liegen.

Für Pfade empfiehlt sich das Modul `pathlib` aus der Standardbibliothek. Ein `Path`-Objekt wird mit dem Operator `/` zusammengesetzt – unabhängig davon, ob das Betriebssystem `/` oder `\` als Trennzeichen verwendet:

```
from pathlib import Path

print("Arbeitsverzeichnis:", Path.cwd().name)  # .name: nur der letzte Teil
 ↪  des Pfads

data_dir = Path("data")
protocol_file = data_dir / "messprotokoll.txt"  # Pfade werden mit /
 ↪  zusammengesetzt
```

```
print(protocol_file)
print("Dateiname:", protocol_file.name)
print("Endung:", protocol_file.suffix)
print("Ordner existiert?", data_dir.exists())
```

```
Arbeitsverzeichnis: notebooks
data/messprotokoll.txt
Dateiname: messprotokoll.txt
Endung: .txt
Ordner existiert? True
```

> **!** Windows-Pfade als String
>
> Wer Pfade als normale Strings schreibt, stolpert unter Windows über den Backslash: In `"C:\Daten\neu.txt"` wird `\n` als Zeilenumbruch interpretiert! Abhilfe schaffen ein **Raw String** `r"C:\Daten\neu.txt"`, normale Schrägstriche `"C:/Daten/neu.txt"` (funktionieren auch unter Windows) oder eben `Path`.

A.2. Textdateien schreiben

Eine Datei wird mit der eingebauten Funktion `open()` geöffnet. Neben dem Pfad gibt man den **Modus** an:

Modus	Bedeutung
`"r"`	lesen (read) – Standard; Fehler, wenn die Datei nicht existiert
`"w"`	schreiben (write) – legt die Datei an, **überschreibt** eine bestehende Datei!
`"a"`	anhängen (append) – schreibt ans Ende einer bestehenden Datei

Die `with`-Anweisung schließt die Datei automatisch, sobald der eingerückte Block verlassen wird – auch dann, wenn darin ein Fehler auftritt. Ohne Schließen landen Daten unter Umständen nicht vollständig auf der Festplatte.

Wir schreiben ein kleines Messprotokoll eines Temperatursensors:

```
times_min = [0, 5, 10, 15, 20, 25, 30]
temperatures = [21.4, 23.9, 27.5, 31.2, 34.8, 36.1, 35.7]

with open(protocol_file, "w", encoding="utf-8") as f:
    f.write("# Messprotokoll Temperatursensor T1\n")  # \n: Zeilenumbruch
 ↪  selbst einfügen!
    f.write("# Zeit [min], Temperatur [°C]\n")
    for t, temp in zip(times_min, temperatures):  # zip(): siehe Kapitel
     ↪  Datenstrukturen
        f.write(f"{t},{temp}\n")

print("Geschrieben:", protocol_file)
```

```
Geschrieben: data/messprotokoll.txt
```

Einige Details:

- `write()` schreibt einen String in die Datei – im Gegensatz zu `print()` **ohne** automatischen Zeilenumbruch. Zahlen wandelt man vorher in Strings um, am bequemsten mit f-Strings (z.B. `f"{temp:.1f}"`).
- `encoding="utf-8"` legt fest, wie Zeichen wie `°` oder `ä` gespeichert werden (siehe Kapitel *Datentypen*). Geben Sie es immer an, sonst verwendet Python das Standard-Encoding des Betriebssystems, und die Datei sieht auf einem anderen Rechner kaputt aus („Â°C“).

Mit dem Modus `"a"` hängen wir weitere Messwerte an, ohne die bisherigen zu verlieren – typisch für einen Datenlogger:

```
with open(protocol_file, "a", encoding="utf-8") as f:
    f.write("35,35.2\n")
    f.write("40,34.6\n")
```

A.3. Textdateien lesen

Mit `f.read()` erhalten Sie den gesamten Inhalt einer geöffneten Datei als einen einzigen String.

Meist möchte man eine Datei aber **zeilenweise** verarbeiten. Dafür iteriert man einfach mit einer `for`-Schleife über das Dateiobjekt. Jede Zeile ist ein String, der noch das Zeilenende `\n` enthält; dieses entfernt man mit `strip()`. Anschließend zerlegt `split(",")` die Zeile in ihre Bestandteile – die aber immer noch Strings sind und erst in Zahlen umgewandelt werden müssen:

```
times = []
values = []

with open(protocol_file, encoding="utf-8") as f:  # "r" ist der Standard und
 ↪  darf entfallen
    for line in f:
        line = line.strip()
        if line == "" or line.startswith("#"):  # Leerzeilen und Kommentare
         ↪  überspringen
            continue
        t_str, temp_str = line.split(",")
        times.append(int(t_str))
        values.append(float(temp_str))

print("Zeiten:", times)
print("Temperaturen:", values)
print(f"Maximum: {max(values)} °C nach {times[values.index(max(values))]}
 ↪  min")
```

```
Zeiten: [0, 5, 10, 15, 20, 25, 30, 35, 40]
Temperaturen: [21.4, 23.9, 27.5, 31.2, 34.8, 36.1, 35.7, 35.2, 34.6]
Maximum: 36.1 °C nach 25 min
```

> **Lesen, verarbeiten, schreiben**
>
> Das Muster *Datei öffnen → zeilenweise lesen → Zeile bereinigen (`strip`) → zerlegen (`split`) → umwandeln (`int`, `float`) → in Listen speichern* begegnet Ihnen immer wieder.

Für kleine Dateien bietet `Path` die Kurzformen `read_text()` und `write_text()`, die Öffnen und Schließen selbst erledigen.

Übungen:

1. Schreiben Sie die Zahlen 1 bis 10 und ihre Quadrate in eine Datei `data/quadrate.txt`, eine Zeile pro Zahl im Format `3 -> 9`.
2. Lesen Sie die Datei wieder ein und berechnen Sie die Summe aller Quadratzahlen.

```
# Übung 1
with open("data/quadrate.txt", "w", encoding="utf-8") as f:
    for n in range(1, 11):
        f.write(f"{n} -> {n**2}\n")
```

```
# Übung 2
total = 0
with open("data/quadrate.txt", encoding="utf-8") as f:
    for line in f:
        left, right = line.strip().split(" -> ")
        total += int(right)
print("Summe der Quadrate:", total)  # 385
```

A.4. CSV-Dateien mit dem Modul `csv`

CSV (*comma-separated values*) ist das wohl verbreitetste Austauschformat für Messdaten: eine Textdatei, in der jede Zeile einen Datensatz enthält und die Werte durch ein Trennzeichen getrennt sind. Die Prüfstandsdaten aus dem Kapitel *Bibliotheken* liegen in diesem Format vor:

```
csv_file = data_dir / "pruefstand.csv"

with open(csv_file, encoding="utf-8") as f:
    for i in range(3):
        print(f.readline().strip())  # readline(): genau eine Zeile lesen
```

```
zeit_s,drehzahl_rpm,drehmoment_Nm,temperatur_C,strom_A,spannung_V
0,1467.2924821489682,33.81031628726315,24.628518648371458,10.215086869820023,400.39745115695
2,1461.4533926626523,35.36085692530741,25.32622786824618,10.628453711233902,400.836376917621
```

Im Prinzip könnte man solche Dateien mit `split(",")` zerlegen. Das bricht aber, sobald ein Wert selbst ein Komma enthält (z.B. ein Kommentar in Anführungszeichen). Robuster ist das Modul `csv` aus der Standardbibliothek. Besonders praktisch ist der `DictReader`: Er liest die Kopfzeile und liefert jede weitere Zeile als Dictionary mit den Spaltennamen als Schlüsseln. CSV-Dateien öffnet man dabei immer mit `newline=""`, weil das Modul `csv` die Zeilenenden selbst behandelt.

```
import csv

max_temp = None
max_time = None
n_rows = 0

with open(csv_file, encoding="utf-8", newline="") as f:
```

```
    reader = csv.DictReader(f)
    for row in reader:
        n_rows += 1
        temp = float(row["temperatur_C"])  # auch csv liefert nur Strings!
        if max_temp is None or temp > max_temp:
            max_temp = temp
            max_time = float(row["zeit_s"])

print(f"{n_rows} Datensätze gelesen")
print(f"Höchste Temperatur: {max_temp:.1f} °C bei t = {max_time:.0f} s")
```

```
500 Datensätze gelesen
Höchste Temperatur: 63.1 °C bei t = 988 s
```

Mit pandas genügt dafür eine Zeile (`df.loc[df["temperatur_C"].idxmax()]`); für tabellarische Messdaten ist pandas meist die bessere Wahl. Das Modul `csv` lohnt sich, wenn man ohne Zusatzpaket auskommen, Zeile für Zeile verarbeiten oder ein genau vorgegebenes Format schreiben muss.

! Typische Fehler beim Arbeiten mit Dateien

- **Falsches Arbeitsverzeichnis:** `FileNotFoundError`, obwohl die Datei „doch da ist“ – der relative Pfad bezieht sich auf einen anderen Ordner. Prüfen Sie mit `Path.cwd()`, wo Sie sich befinden.
- **Modus `"w"` statt `"a"`:** Die Datei wird ohne Rückfrage überschrieben.
- **Zahlen nicht umgewandelt:** Aus Dateien gelesene Werte sind immer Strings; `"21.4" + "1"` ergibt `"21.41"`.
- **CSV für das deutsche Excel:** Deutsches Excel erwartet Semikolon als Trennzeichen und Dezimalkomma, sonst landet alles in Spalte A. Abhilfe beim Schreiben: `csv.writer(f, delimiter=";")` zusammen mit `f"{x:.1f}".replace(".", ",")` und `encoding="utf-8-sig"`; beim Einlesen mit pandas `pd.read_csv(datei, sep=";", decimal=",")`.

Wie man auf Fehler wie eine fehlende Datei reagiert, statt dass das Programm abstürzt, zeigt Anhang B (Übung 3).

Übung:

3. Lesen Sie `data/pruefstand.csv` mit dem `csv.DictReader` ein und berechnen Sie für jeden Datensatz die mechanische Leistung $P = M \cdot 2\pi n/60$ (in W, mit M in Nm und n in 1/min). Geben Sie die mittlere Leistung in kW aus.

```
import csv, math
from pathlib import Path

data_dir = Path("data")

# Übung 3
powers_W = []
with open(data_dir / "pruefstand.csv", encoding="utf-8", newline="") as f:
    for row in csv.DictReader(f):
        n = float(row["drehzahl_rpm"])
        M = float(row["drehmoment_Nm"])
        powers_W.append(M * 2 * math.pi * n / 60)
print(f"Mittlere mechanische Leistung: {sum(powers_W) / len(powers_W) /
 ↪  1000:.2f} kW")
```

B. Fehlerbehandlung

Ein Programm, das bei der ersten unerwarteten Eingabe abstürzt, ist in der Praxis unbrauchbar: Ein Messprogramm soll nicht abbrechen, weil eine Zeile der Messdatei beschädigt ist, und ein Berechnungstool soll bei einem Tippfehler eine verständliche Meldung ausgeben statt eines Tracebacks.

In diesem Anhang lernen Sie,

- welche Arten von Fehlern es gibt und wie man eine Fehlermeldung liest,
- wie man Laufzeitfehler mit `try` und `except` abfängt,
- wie man in eigenen Funktionen selbst Fehler mit `raise` auslöst und
- welche typischen Fehler man bei der Fehlerbehandlung vermeiden sollte.

B.1. Arten von Fehlern

1. **Syntaxfehler** (`SyntaxError`, `IndentationError`): Der Code ist kein gültiges Python, z.B. weil eine Klammer oder ein Doppelpunkt fehlt. Er läuft gar nicht erst los und muss korrigiert werden.
2. **Laufzeitfehler** (*Exceptions*, „Ausnahmen“): Der Code ist korrekt, aber bei der Ausführung tritt ein Problem auf, etwa eine Division durch null oder eine fehlende Datei. Um diese Fehler geht es in diesem Anhang.
3. **Logische Fehler**: Das Programm läuft durch, liefert aber ein falsches Ergebnis, z.B. weil Grad und Bogenmaß verwechselt wurden. Hier helfen nur Tests und Plausibilitätsprüfungen.

B.2. Eine Fehlermeldung lesen

Wenn eine Exception auftritt und nicht behandelt wird, bricht Python ab und gibt einen **Traceback** aus. Ein Beispiel: Ein Messwert wurde mit deutschem Dezimalkomma eingegeben.

```
def mean_value(values_as_text):
    numbers = [float(v) for v in values_as_text]
    return sum(numbers) / len(numbers)
```

```
mean_value(["21.4", "22,8", "23.1"])  # FEHLER: "22,8" ist keine gültige Zahl
```

```
ValueError: could not convert string to float: '22,8'
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[1], line 5
      1 def mean_value(values_as_text):
      2     numbers = [float(v) for v in values_as_text]
      3     return sum(numbers) / len(numbers)
      4
----> 5 mean_value(["21.4", "22,8", "23.1"])  # FEHLER: "22,8" ist keine
 ↪  gültige Zahl
Cell In[1], line 2, in mean_value(values_as_text)
      1 def mean_value(values_as_text):
----> 2     numbers = [float(v) for v in values_as_text]
      3     return sum(numbers) / len(numbers)
ValueError: could not convert string to float: '22,8'
```

Ein Traceback wird am besten **von unten nach oben** gelesen: Die **letzte Zeile** nennt den Fehlertyp (`ValueError`) und eine Beschreibung – damit ist meist schon klar, was passiert ist. Darüber steht der **Weg** zur Fehlerstelle, hier vom Aufruf `mean_value(...)` bis zur Zeile mit `float(v)`. Die Fehlerstelle liegt oft tief in einer Funktion, die eigentliche Ursache aber beim Aufruf (hier: falsche Daten).

Die wichtigsten eingebauten Exceptions:

Exception	Typische Ursache	Beispiel
`ValueError`	richtiger Typ, aber unbrauchbarer Wert	`float("22,8")`, `int("3.5")`
`TypeError`	Operation passt nicht zum Typ	`"Wert: " + 5`, `len(42)`
`ZeroDivisionError`	Division durch null	`1 / 0`
`IndexError`	Index außerhalb einer Sequenz	`[1, 2, 3][5]`
`KeyError`	Schlüssel nicht im Dictionary	`{"a": 1}["b"]`
`NameError`	Name nicht definiert (oft ein Tippfehler)	`pirnt("Hallo")`
`AttributeError`	Objekt hat das Attribut/die Methode nicht	`[1, 2].add(3)`
`FileNotFoundError`	Datei existiert nicht	`open("gibtsnicht.txt")`

B.3. Fehler abfangen mit `try` und `except`

Code, bei dem ein Fehler zu erwarten ist, schreibt man in einen `try`-Block. Tritt darin eine Exception auf, springt Python sofort in den passenden `except`-Block, statt abzubrechen:

```
text = "22,8"

try:
    value = float(text)
    print("Umgewandelt:", value)  # wird übersprungen, wenn float() scheitert
except ValueError:
    print(f"'{text}' ist keine gültige Zahl.")

print("Das Programm läuft weiter.")
```

```
'22,8' ist keine gültige Zahl.
Das Programm läuft weiter.
```

Ablauf im Detail:

- Python führt den `try`-Block Zeile für Zeile aus.
- Tritt **kein** Fehler auf, wird der `except`-Block übersprungen.
- Tritt ein Fehler auf, wird der Rest des `try`-Blocks übersprungen. Passt der Fehlertyp zu `except`, wird dieser Block ausgeführt und das Programm danach normal fortgesetzt. Passt er nicht, bricht das Programm wie gewohnt ab.

Mehrere Fehlertypen und die Fehlermeldung selbst

Ein `try`-Block kann mehrere `except`-Blöcke haben, die nacheinander geprüft werden. Mehrere Fehlertypen lassen sich auch als Tupel zusammenfassen, z.B. `except (ValueError, TypeError) as e:`. Mit `as e` erhalten Sie das Fehlerobjekt selbst und können z.B. die Originalmeldung ausgeben:

```
def resistance(voltage_V, current_A):
    return voltage_V / current_A

measurements = [(12.0, 0.5), (5.0, 0.0), ("230", 2.0)]

for u, i in measurements:
    try:
        r = resistance(u, i)
```

```
        print(f"U = {u} V, I = {i} A  ->  R = {r:.1f} Ohm")
    except ZeroDivisionError:
        print(f"U = {u} V, I = {i} A  ->  kein Strom, Widerstand nicht
         ↪  bestimmbar")
    except TypeError as e:
        print(f"U = {u!r}  ->  ungültige Daten ({e})")
```

```
U = 12.0 V, I = 0.5 A  ->  R = 24.0 Ohm
U = 5.0 V, I = 0.0 A  ->  kein Strom, Widerstand nicht bestimmbar
U = '230'  ->  ungültige Daten (unsupported operand type(s) for /: 'str' and 'float')
```

`else` und `finally`

Zwei optionale Blöcke ergänzen `try`/`except`: `else` wird nur ausgeführt, wenn im `try`-Block **kein** Fehler aufgetreten ist; `finally` wird **immer** ausgeführt, typischerweise für Aufräumarbeiten wie das Abschalten eines Geräts. Beim Arbeiten mit Dateien übernimmt das die `with`-Anweisung aus Anhang A, die man einem eigenen `finally` vorziehen sollte.

B.4. Fehler selbst auslösen mit `raise`

Bisher haben wir auf Fehler reagiert, die Python ausgelöst hat. Mindestens ebenso wichtig ist es, in **eigenen Funktionen** Fehler auszulösen, wenn diese mit unsinnigen Werten aufgerufen werden. Dafür gibt es die Anweisung `raise`.

Betrachten wir die Biegespannung in einem Balken mit Rechteckquerschnitt (Breite b, Höhe h): Mit dem Flächenträgheitsmoment $I = bh^3/12$ und dem Randfaserabstand $y = h/2$ gilt $\sigma = My/I = 6M/(bh^2)$.

```
def bending_stress_rect(moment_Nmm, width_mm, height_mm):
    """Biegespannung in N/mm² (MPa) für einen Rechteckquerschnitt."""
    if width_mm <= 0 or height_mm <= 0:
        raise ValueError(f"Breite und Höhe müssen positiv sein (b={width_mm},
         ↪  h={height_mm}).")
    return 6 * moment_Nmm / (width_mm * height_mm**2)

print(f"{bending_stress_rect(2.0e6, 40, 80):.1f} MPa")
```

```
46.9 MPa
```

```
bending_stress_rect(2.0e6, 40, -80)  # FEHLER: absichtlich ausgelöst
```

```
ValueError: Breite und Höhe müssen positiv sein (b=40, h=-80).
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[7], line 1
----> 1 bending_stress_rect(2.0e6, 40, -80)  # FEHLER: absichtlich ausgelöst
Cell In[6], line 4, in bending_stress_rect(moment_Nmm, width_mm, height_mm)
      1 def bending_stress_rect(moment_Nmm, width_mm, height_mm):
      2     """Biegespannung in N/mm² (MPa) für einen Rechteckquerschnitt."""
      3     if width_mm <= 0 or height_mm <= 0:
----> 4         raise ValueError(f"Breite und Höhe müssen positiv sein
 ↪  (b=width_mm, h=height_mm).")
      5     return 6 * moment_Nmm / (width_mm * height_mm**2)
ValueError: Breite und Höhe müssen positiv sein (b=40, h=-80).
```

Warum nicht einfach `print("Fehler")` und `return None`? Weil die aufrufende Stelle dann mit `None` weiterrechnet und der Fehler erst viel später – an ganz anderer Stelle – als rätselhafter `TypeError` auftaucht. Eine Exception stoppt das Programm dort, wo das Problem erkannt wurde, beschreibt es mit einer klaren Meldung und lässt dem Aufrufer die Wahl, sie mit `try`/`except` zu behandeln. Wählen Sie einen passenden Fehlertyp: `ValueError` für unzulässige Werte, `TypeError` für falsche Typen.

B.5. Typische Fehler bei der Fehlerbehandlung

! Fehler nicht verschlucken

Die folgende Konstruktion sieht man leider häufig:

```
try:
    result = compute(data)
except:
    pass
```

Ein `except` ohne Fehlertyp fängt **alle** Fehler ab – auch Tippfehler im eigenen Code (`NameError`) und sogar den Programmabbruch per Strg+C. Zusammen mit `pass` verschwindet jede Spur des Problems, und `result` existiert anschließend womöglich gar nicht. Deshalb:

- Fangen Sie nur Fehlertypen ab, die Sie erwarten und sinnvoll behandeln können (`except ValueError:`).

- Halten Sie den `try`-Block so klein wie möglich, damit Sie nicht versehentlich Fehler aus ganz anderen Zeilen abfangen.
- Wenn Sie einen Fehler abfangen, tun Sie etwas Sinnvolles: einen Ersatzwert verwenden, die Eingabe erneut abfragen oder zumindest eine aussagekräftige Meldung ausgeben.

Übungen:

1. Schreiben Sie eine Funktion `safe_divide(a, b)`, die `a / b` zurückgibt und bei einer Division durch null `float("inf")` liefert.
2. Ergänzen Sie die Funktion `bending_stress_rect` so, dass bei einem Biegemoment, das kein `int` oder `float` ist, ein `TypeError` ausgelöst wird. Rufen Sie die Funktion anschließend in einer Schleife mit den Querschnitten `[(40, 80), (30, 0), (20, 60)]` auf und fangen Sie den Fehler so ab, dass für jeden Querschnitt entweder die Spannung oder eine Fehlermeldung ausgegeben wird. Testen Sie den `TypeError` zusätzlich mit einem Biegemoment als String, z.B. `"2e6"`.
3. Schreiben Sie eine Funktion `read_values(path)`, die eine Datei mit einer Zahl pro Zeile einliest und eine Liste von floats zurückgibt. Existiert die Datei nicht, soll eine leere Liste zurückgegeben und eine Meldung ausgegeben werden; ungültige Zeilen sollen übersprungen werden.

```
# Übung 1
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return float("inf")

print(safe_divide(1, 4), safe_divide(1, 0))

# Übung 2
def bending_stress_rect(moment_Nmm, width_mm, height_mm):
    """Biegespannung in N/mm² (MPa) für einen Rechteckquerschnitt."""
    if not isinstance(moment_Nmm, (int, float)):
        raise TypeError(f"Biegemoment muss eine Zahl sein, nicht
        ↪ {type(moment_Nmm).__name__}.")
    if width_mm <= 0 or height_mm <= 0:
        raise ValueError(f"Breite und Höhe müssen positiv sein (b={width_mm},
        ↪ h={height_mm}).")
    return 6 * moment_Nmm / (width_mm * height_mm**2)

for b, h in [(40, 80), (30, 0), (20, 60)]:
```

```
    try:
        print(f"b={b}, h={h}: {bending_stress_rect(2.0e6, b, h):.1f} MPa")
    except (ValueError, TypeError) as e:
        print(f"b={b}, h={h}: {e}")

try:
    bending_stress_rect("2e6", 40, 80)  # Biegemoment als String
except TypeError as e:
    print("Fehler:", e)

# Übung 3
def read_values(path):
    values = []
    try:
        with open(path, encoding="utf-8") as f:
            for line in f:
                try:
                    values.append(float(line.strip().replace(",", ".")))
                except ValueError:
                    pass  # ungültige Zeile bewusst überspringen (z.B.
                     ↪ Kopfzeile)
    except FileNotFoundError:
        print(f"Datei {path} nicht gefunden.")
    return values

print(read_values("data/gibtsnicht.txt"))
```

C. Interaktivität und grafische Oberflächen

Alle Programme in diesem Skript kommunizieren über die Konsole: Eingaben mit `input()`, Ausgaben mit `print()` oder als Diagramm. Sobald aber andere Personen ein Werkzeug nutzen sollen – etwa ein Prüfstandsbediener oder Studierende im Labor –, ist eine **grafische Benutzeroberfläche** (*Graphical User Interface*, GUI) mit Eingabefeldern, Schiebereglern und Buttons komfortabler und weniger fehleranfällig.

In diesem Anhang lernen Sie zwei Wege kennen:

- **ipywidgets**: interaktive Bedienelemente direkt im Jupyter Notebook, ideal für Parameterstudien („Was passiert, wenn ich die Dämpfung erhöhe¿‘),
- **tkinter**: eigenständige Desktop-Fenster mit der Standardbibliothek von Python, ideal für kleine Rechen- und Hilfswerkzeuge.

C.1. Ereignisgesteuerte Programmierung

Ein Konsolenprogramm läuft **linear** von oben nach unten ab: Es fragt eine Eingabe ab, rechnet, gibt das Ergebnis aus und endet. Bei einer GUI bestimmt dagegen die **Benutzerin**, was als Nächstes passiert: Sie füllt Felder in beliebiger Reihenfolge aus, verschiebt einen Regler oder klickt einen Button. Das Programm baut deshalb zunächst die Oberfläche auf und wartet dann in einer **Ereignisschleife** (*event loop*) auf Ereignisse. Für jedes Ereignis hinterlegen wir eine Funktion, einen **Callback** („Rückruf“): Wir rufen ihn nicht selbst auf, sondern übergeben ihn der GUI, die ihn im passenden Moment aufruft.

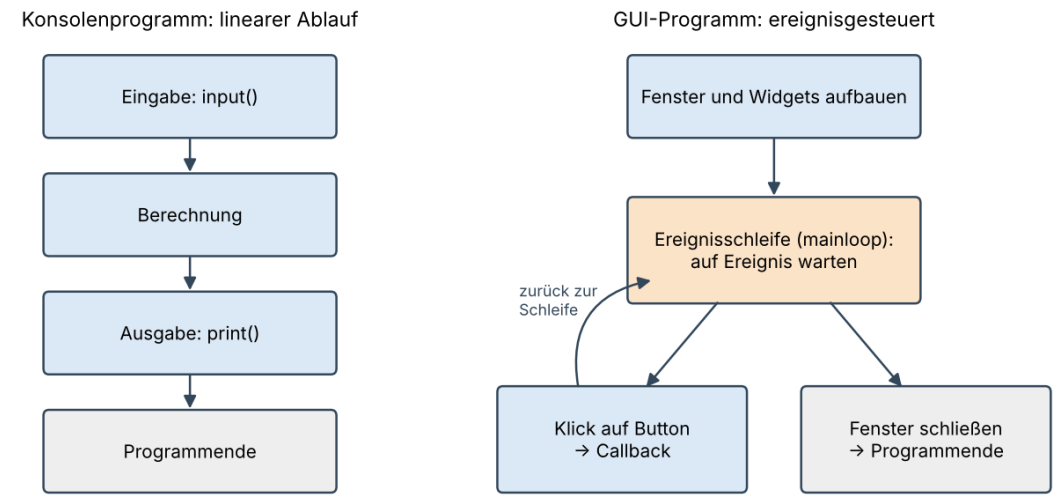


Abbildung C.1.: Linearer Ablauf eines Konsolenprogramms (links) und ereignisgesteuerter Ablauf eines GUI-Programms (rechts).

Es lohnt sich, **Berechnung und Oberfläche zu trennen**: Die Rechenfunktion weiß nichts von Fenstern und Buttons und kann unabhängig getestet werden; die Oberfläche kümmert sich nur um Ein- und Ausgabe.

C.2. Interaktive Notebooks mit ipywidgets

Das Paket `ipywidgets` stellt Bedienelemente für Jupyter Notebooks bereit. Es ist in Anaconda und Google Colab bereits enthalten; andernfalls lässt es sich mit `pip install ipywidgets` installieren.

Parameterstudien mit `interact`

Als Beispiel betrachten wir einen gedämpften Einmassenschwinger, z.B. eine Maschine auf elastischen Füßen, die ausgelenkt und dann losgelassen wird. Die Auslenkung folgt

$$x(t) = x_0\, e^{-D\omega_0 t} \cos\left(\omega_0 \sqrt{1 - D^2}\, t\right)$$

mit der Eigenkreisfrequenz $\omega_0 = 2\pi f_0$ und dem Dämpfungsgrad $D < 1$. Zunächst schreiben wir eine ganz normale Funktion, die den Verlauf für gegebene Parameter zeichnet:

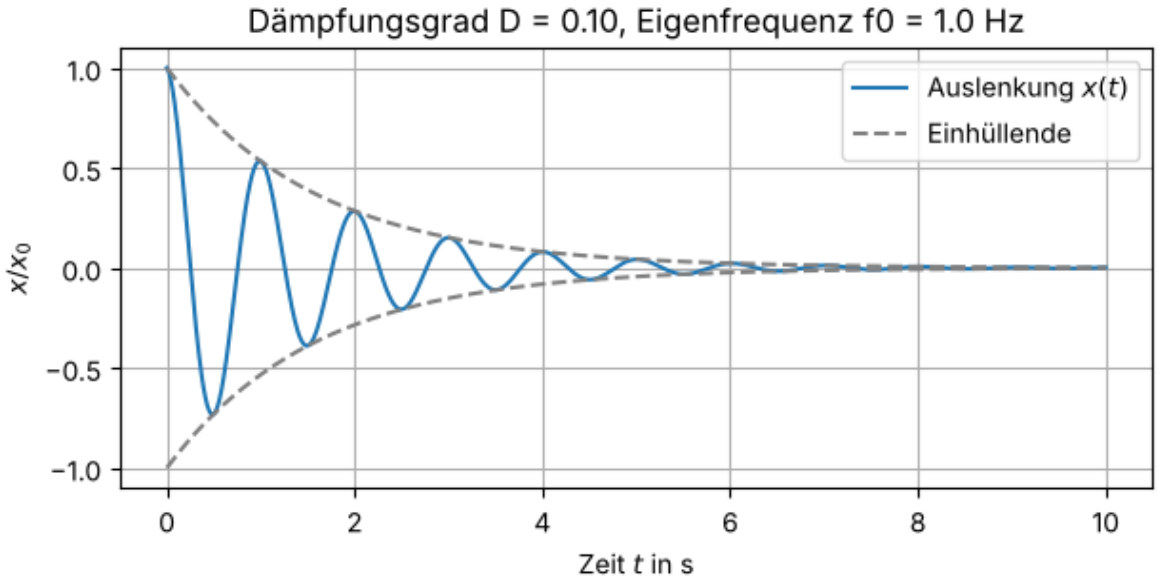
```python
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 10, 1000)  # Zeit in s

def plot_oscillation(D=0.1, f0=1.0):
    """Zeichnet die freie Schwingung eines gedämpften Einmassenschwingers."""
    omega0 = 2 * np.pi * f0
    x = np.exp(-D * omega0 * t) * np.cos(omega0 * np.sqrt(1 - D**2) * t)
    envelope = np.exp(-D * omega0 * t)

    plt.figure(figsize=(7, 3))
    plt.plot(t, x, label="Auslenkung $x(t)$")
    plt.plot(t, envelope, "--", color="gray", label="Einhüllende")
    plt.plot(t, -envelope, "--", color="gray")
    plt.ylim(-1.1, 1.1)
    plt.xlabel("Zeit $t$ in s")
    plt.ylabel("$x / x_0$")
    plt.title(f"Dämpfungsgrad D = {D:.2f}, Eigenfrequenz f0 = {f0:.1f} Hz")
    plt.legend(loc="upper right")
    plt.grid(True)
    plt.show()

plot_oscillation(D=0.1, f0=1.0)
```

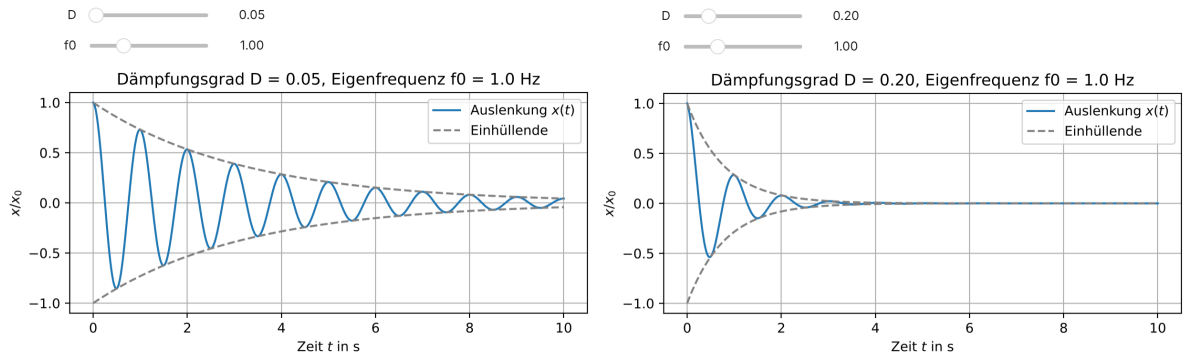


Um die Parameter interaktiv zu verändern, genügt eine einzige Zeile: `interact()` erhält die Funktion und für jeden Parameter einen Wertebereich `(min, max, schrittweite)`. Daraus erzeugt es automatisch Schieberegler. Bei jeder Änderung wird die Funktion mit den neuen Werten aufgerufen und das Diagramm neu gezeichnet – `plot_oscillation` ist hier also der Callback.

```
from ipywidgets import interact

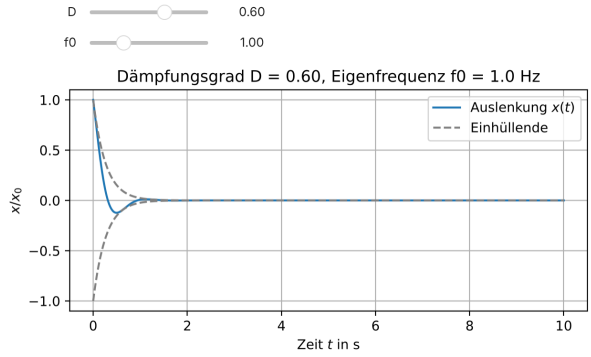
interact(plot_oscillation, D=(0.0, 0.95, 0.05), f0=(0.2, 3.0, 0.1));  # ;
 ↪  unterdrückt die Textausgabe
```

> **Widgets brauchen einen laufenden Python-Kernel**
>
> Die Schieberegler funktionieren nur im laufenden Notebook (Jupyter, VS Code, Colab), weil bei jeder Bewegung Python-Code ausgeführt werden muss. In der HTML- oder PDF-Version dieses Skripts sind die Regler deshalb nicht bedienbar. Die folgenden Screenshots zeigen beispielhaft, wie sich das Diagramm bei verschiedenen Reglerstellungen verändert. Probieren Sie die Zelle unbedingt selbst aus!

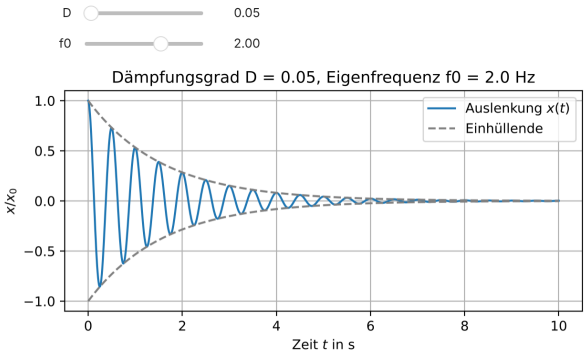


(a) D = 0,05, f0 = 1 Hz

(b) D = 0,20, f0 = 1 Hz



(c) D = 0,60, f0 = 1 Hz



(d) D = 0,05, f0 = 2 Hz

Abbildung C.2.: Der Einmassenschwinger bei verschiedenen Reglerstellungen: Je größer der Dämpfungsgrad D, desto schneller klingt die Schwingung ab (a–c). Bei gleicher Dämpfung und doppelter Eigenfrequenz (d) schwingt die Maschine doppelt so schnell, und die Schwingung klingt ebenfalls schneller ab.

Je nach Datentyp des Wertebereichs wählt `interact` ein passendes Bedienelement: Zahlentupel ergeben Schieberegler, eine Liste von Strings eine Auswahlliste, `True`/`False` eine Checkbox.

C.3. Desktop-Anwendungen mit tkinter

Für ein eigenständiges Werkzeug mit eigenem Fenster bietet sich `tkinter` an. Das Modul gehört zur Standardbibliothek von Python, muss also nicht installiert werden, und läuft unter Windows, macOS und Linux.

> **!** tkinter-Programme als Skript ausführen
>
> Ein tkinter-Fenster öffnet sich auf dem Rechner, auf dem Python läuft – in Google Colab also nicht. In einem lokalen Notebook bleibt die Zelle blockiert, bis Sie das Fenster schließen. Speichern Sie die Beispiele daher als `.py`-Datei und starten Sie sie z.B. in Spyder, VS Code oder mit `python dateiname.py`. Im Skript werden diese Zellen nicht ausgeführt; stattdessen sehen Sie einen Screenshot.

Beispiel: ein Biegespannungsrechner

Als Beispiel bauen wir einen Rechner für die Biegespannung in einem Balken mit Rechteckquerschnitt, $\sigma = 6M/(bh^2)$ (vgl. Anhang B). Die Berechnung steckt in einer eigenen Funktion. Der Callback liest die Eingabefelder (`Entry`) mit `get()` als **String** aus, wandelt die Eingaben um (deutsches Dezimalkomma inklusive) und zeigt das Ergebnis an. Ungültige Eingaben fangen wir mit `try`/`except` ab und melden sie in einem Dialogfenster (`messagebox`), statt das Programm abstürzen zu lassen:

```
import tkinter as tk
from tkinter import messagebox


def bending_stress_rect(moment_Nmm, width_mm, height_mm):
    """Biegespannung in N/mm² (MPa) für einen Rechteckquerschnitt."""
    if width_mm <= 0 or height_mm <= 0:
        raise ValueError("Breite und Höhe müssen positiv sein.")
    return 6 * moment_Nmm / (width_mm * height_mm**2)


def on_calculate():
    """Callback: wird bei jedem Klick auf den Button aufgerufen."""
    try:
        moment = float(entry_moment.get().replace(",", ".")) * 1000  # Nm ->
 ↪  Nmm
        width = float(entry_width.get().replace(",", "."))
        height = float(entry_height.get().replace(",", "."))
        sigma = bending_stress_rect(moment, width, height)
    except ValueError as e:
        messagebox.showerror("Ungültige Eingabe", str(e))
        return
    label_result.config(text=f"σ = {sigma:.1f} MPa")
```

```
window = tk.Tk()
window.title("Biegespannungsrechner")

# Beschriftungen und Eingabefelder in einem Raster (grid) anordnen
tk.Label(window, text="Biegemoment M [Nm]:").grid(row=0, column=0,
 ↪  sticky="w", padx=10, pady=5)
tk.Label(window, text="Breite b [mm]:").grid(row=1, column=0, sticky="w",
 ↪  padx=10, pady=5)
tk.Label(window, text="Höhe h [mm]:").grid(row=2, column=0, sticky="w",
 ↪  padx=10, pady=5)

entry_moment = tk.Entry(window, width=12)
entry_width = tk.Entry(window, width=12)
entry_height = tk.Entry(window, width=12)
entry_moment.grid(row=0, column=1, padx=10)
entry_width.grid(row=1, column=1, padx=10)
entry_height.grid(row=2, column=1, padx=10)

button = tk.Button(window, text="Berechnen", command=on_calculate)
button.grid(row=3, column=0, columnspan=2, pady=10)

label_result = tk.Label(window, text="σ = ?", font=("Arial", 14, "bold"))
label_result.grid(row=4, column=0, columnspan=2, pady=(0, 10))

window.mainloop()
```

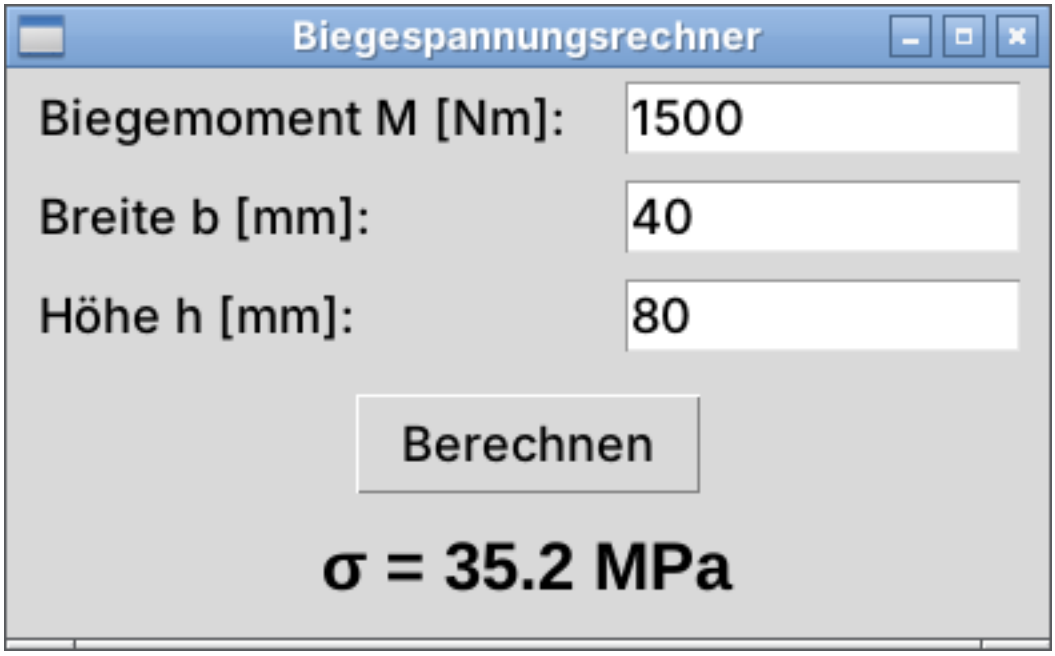


Abbildung C.3.: Der Biegespannungsrechner nach einem Klick auf „Berechnen“.

Jedes tkinter-Programm folgt demselben Grundmuster:

- **Hauptfenster und Widgets erzeugen:** `tk.Tk()` erzeugt das Fenster; jedes Widget erhält beim Erzeugen als erstes Argument das Fenster, in dem es liegen soll.
- **Widgets platzieren:** Ein erzeugtes Widget ist noch unsichtbar. Erst ein Geometriemanager platziert es: `grid()` ordnet in Zeilen und Spalten an, `pack()` stapelt einfach untereinander. `sticky="w"` richtet links (*west*) aus, `padx`/`pady` legen Abstände in Pixeln fest.
- **Callback registrieren:** `command=on_calculate` übergibt die **Funktion selbst, ohne Klammern**. Mit `command=on_calculate()` würde Python sie sofort beim Erzeugen des Buttons aufrufen, statt sie für den Klick zu hinterlegen – ein typischer Anfängerfehler!
- **Ereignisschleife starten:** `mainloop()` kehrt erst zurück, wenn das Fenster geschlossen wird; Code danach läuft also erst am Programmende.

Mit `label_result.config(text=...)` ändert der Callback die Beschriftung eines bestehenden Widgets nachträglich.

C.4. Weitere Werkzeuge

tkinter ist schlicht, aber für kleine Werkzeuge völlig ausreichend. Für umfangreiche Desktop-Anwendungen (z.B. Mess- und Steuersoftware) sind **PyQt** bzw. **PySide** verbreitet; Auswer-

tewerkzeuge und Dashboards im Browser lassen sich mit **Streamlit**, **Dash** oder **Panel** mit wenig Code erstellen.

Übungen:

1. Erweitern Sie `plot_oscillation` um einen Parameter `x0` für die Anfangsauslenkung und ergänzen Sie einen passenden Schieberegler. Passen Sie auch die y-Achse an.
2. Schreiben Sie mit tkinter einen Rechner für das ohmsche Gesetz: Spannung U und Strom I werden eingegeben, nach einem Klick auf einen Button wird der Widerstand $R = U/I$ angezeigt. Ungültige Eingaben und $I = 0$ sollen mit einer verständlichen Meldung abgefangen werden.

```
# Übung 1 (im Notebook ausführen)
def plot_oscillation(D=0.1, f0=1.0, x0=1.0):
    omega0 = 2 * np.pi * f0
    envelope = x0 * np.exp(-D * omega0 * t)
    x = envelope * np.cos(omega0 * np.sqrt(1 - D**2) * t)
    plt.figure(figsize=(7, 3))
    plt.plot(t, x, label="Auslenkung $x(t)$")
    plt.plot(t, envelope, "--", color="gray", label="Einhüllende")
    plt.plot(t, -envelope, "--", color="gray")
    plt.ylim(-5.5, 5.5)  # fest, damit man die Änderung von x0 sieht
    plt.xlabel("Zeit $t$ in s")
    plt.ylabel("$x$ in mm")
    plt.legend(loc="upper right")
    plt.grid(True)
    plt.show()

interact(plot_oscillation, D=(0.0, 0.95, 0.05), f0=(0.2, 3.0, 0.1), x0=(0.5,
 ↪  5.0, 0.5));

# Übung 2 (als .py-Datei speichern und ausführen)
import tkinter as tk
from tkinter import messagebox


def on_calculate():
    try:
        voltage = float(entry_u.get().replace(",", "."))
        current = float(entry_i.get().replace(",", "."))
        resistance = voltage / current
    except ValueError:
        messagebox.showerror("Ungültige Eingabe", "Bitte Zahlen für U und I
 ↪  eingeben.")
```

```
        return
    except ZeroDivisionError:
        messagebox.showerror("Ungültige Eingabe", "Der Strom darf nicht 0
 ↪  sein.")
        return
    label_result.config(text=f"R = {resistance:.2f} Ω")


window = tk.Tk()
window.title("Ohmsches Gesetz")

tk.Label(window, text="Spannung U [V]:").grid(row=0, column=0, sticky="w",
 ↪  padx=10, pady=5)
tk.Label(window, text="Strom I [A]:").grid(row=1, column=0, sticky="w",
 ↪  padx=10, pady=5)
entry_u = tk.Entry(window, width=12)
entry_i = tk.Entry(window, width=12)
entry_u.grid(row=0, column=1, padx=10)
entry_i.grid(row=1, column=1, padx=10)

tk.Button(window, text="R berechnen", command=on_calculate).grid(row=2,
 ↪  column=0, columnspan=2, pady=10)
label_result = tk.Label(window, text="R = ?", font=("Arial", 14, "bold"))
label_result.grid(row=3, column=0, columnspan=2, pady=(0, 10))

window.mainloop()
```

D. MicroPython: Python auf Mikrocontrollern

Vielleicht sind Ihnen schon kleine Platinen wie der *Raspberry Pi Pico* oder ein *ESP32* begegnet, etwa in einem Laborpraktikum oder einem Bastelprojekt. Auf solchen **Mikrocontrollern** läuft kein Betriebssystem wie Windows oder Linux, und der Arbeitsspeicher ist mit einigen Hundert Kilobyte winzig. Trotzdem lassen sie sich in Python programmieren – mit **MicroPython**, einer schlanken Umsetzung von Python 3 für Mikrocontroller.

Dieser Anhang ist kein Kurs, sondern soll Ihnen einen ersten Eindruck vermitteln: Was ist gleich, was ist anders, und wie sieht ein erstes Programm aus? Als Beispiel dient der Raspberry Pi Pico. Hardware brauchen Sie dafür nicht: Die Beispiele lassen sich im Simulator Wokwi direkt im Browser ausprobieren.

D.1. Was ist gleich, was ist anders?

Fast alles, was Sie in diesem Skript gelernt haben, gilt auch in MicroPython: Variablen, Datentypen, Verzweigungen, Schleifen, Funktionen, Listen und Dictionaries, sogar Klassen. Die Unterschiede liegen vor allem in der Umgebung:

	Python auf dem PC	MicroPython auf dem Mikrocontroller
Speicher	mehrere Gigabyte	einige Hundert Kilobyte
Bibliotheken	riesige Auswahl, z.B. NumPy, pandas, Matplotlib	kleine Standardbibliothek; NumPy, pandas und Matplotlib gibt es nicht
Hardwarezugriff	nur über Zusatzpakete	eingebaut: Modul `machine` für Pins, Analog-Digital-Umsetzer usw.
Programmstart	Skript oder Notebook-Zelle ausführen	Datei `main.py` startet automatisch nach dem Einschalten
Programmende	nach der letzten Anweisung	oft nie: Endlosschleife `while True:`

Der letzte Punkt ist typisch für eingebettete Systeme: Ein Mikrocontroller, der eine Temperatur überwacht, soll das ohne Unterbrechung tun, solange er mit Strom versorgt wird. Die Endlosschleife, die im Kapitel *Kontrollstrukturen* als gefürchteter Fehler auftauchte, ist hier also ausdrücklich gewollt.

D.2. Das „Hello, World“ der Mikrocontroller: eine blinkende LED

Ein Mikrocontroller hat oft kein Display, auf dem man „Hello, World“ ausgeben könnte. Das klassische erste Programm lässt deshalb eine LED blinken. Der Raspberry Pi Pico hat dafür eine LED direkt auf der Platine:

```
from machine import Pin
import time

led = Pin("LED", Pin.OUT)  # eingebaute LED als Ausgang

while True:
    led.toggle()           # LED umschalten: an -> aus -> an ...
    time.sleep(0.5)        # 0,5 Sekunden warten
```

Einige Details:

- Ein **Pin** ist ein Anschluss des Mikrocontrollers. `Pin.OUT` macht ihn zu einem Ausgang, der entweder eine Spannung ausgibt (an) oder nicht (aus). `"LED"` ist ein vordefinierter Name für den Pin, an dem die eingebaute LED hängt. Externe Bauteile sprechen Sie über die Nummer des Pins an, z.B. `Pin(15, Pin.OUT)`.
- Statt `toggle()` können Sie die LED auch gezielt mit `led.on()` und `led.off()` schalten.
- Das Modul `time` kennen Sie vom PC. Gestoppt wird das Programm in der Entwicklungsumgebung mit dem Stopp-Button oder mit Strg+C.

D.3. Eine Spannung messen

Viele Sensoren, z.B. für Temperatur, Druck oder Helligkeit, liefern eine **analoge Spannung**. Damit kann ein Mikrocontroller nicht direkt rechnen: Ein **Analog-Digital-Umsetzer** (*Analog-to-Digital Converter*, ADC) wandelt die Spannung zunächst in eine ganze Zahl um. In MicroPython liefert die Methode `read_u16()` einen Wert zwischen 0 und 65535 (also $2^{16} - 1$, eine 16-Bit-Zahl; der ADC des Pico misst intern mit 12 Bit, MicroPython rechnet den Wert auf diesen einheitlichen Bereich um). Beim Pico entspricht das dem Spannungsbereich von 0 bis 3,3 V.

Das folgende Programm misst einmal pro Sekunde die Spannung an Pin GP26, an dem z.B. ein Potentiometer oder ein Sensor angeschlossen ist, rechnet sie in Volt um und schaltet die LED ein, solange ein Grenzwert überschritten ist:

```
from machine import Pin, ADC
import time

adc = ADC(Pin(26))          # Analog-Digital-Umsetzer an Pin GP26
led = Pin("LED", Pin.OUT)
limit_V = 2.5               # Grenzwert in Volt

while True:
    raw = adc.read_u16()            # Rohwert zwischen 0 und 65535
    voltage = raw * 3.3 / 65535     # Umrechnung in Volt
    print(f"Rohwert: {raw:5d}   Spannung: {voltage:.2f} V")
    if voltage > limit_V:
        led.on()
    else:
        led.off()
    time.sleep(1)
```

Bis auf den Zugriff auf die Hardware über das Modul `machine` enthält das Programm nur Bekanntes: eine Schleife, eine Verzweigung, eine Umrechnung mit Gleitkommazahlen und eine formatierte Ausgabe. Die `print()`-Ausgaben erscheinen übrigens nicht auf dem Mikrocontroller selbst, sondern werden über das USB-Kabel an die Konsole der Entwicklungsumgebung übertragen.

> **Andere Boards**
>
> Auf anderen Mikrocontrollern, z.B. einem ESP32, funktioniert der Code fast genauso. Unterschiede gibt es vor allem bei den Pin-Nummern (nicht jedes Board hat eine eingebaute LED mit dem Namen `"LED"`) und beim Messbereich des ADC. Die Kurzreferenz (*Quick reference*) für das jeweilige Board in der MicroPython-Dokumentation nennt diese Details.

D.4. REPL und `main.py`

Eine einsteigerfreundliche Entwicklungsumgebung für MicroPython ist Thonny. Zunächst wird einmalig die MicroPython-Firmware auf den Mikrocontroller kopiert; Thonny bietet dafür eine eigene Installationsfunktion. Danach verbindet sich Thonny über USB mit dem Board, und es gibt zwei Arbeitsweisen:

- **REPL** (*Read-Eval-Print Loop*): eine interaktive Konsole, die direkt auf dem Mikrocontroller läuft. Jede eingegebene Zeile wird sofort ausgeführt – ideal, um z.B. nach `led = Pin("LED", Pin.OUT)` die LED testweise mit `led.on()` einzuschalten. Das entspricht dem schrittweisen Ausprobieren in einer Notebook-Zelle.
- **Datei `main.py`:** Ein Programm, das unter diesem Namen auf dem Mikrocontroller gespeichert ist, startet nach dem Einschalten automatisch – auch ohne angeschlossenen PC, z.B. mit einer Batterie als Stromversorgung.

> **Ausprobieren ohne Hardware**
>
> Wählen Sie auf wokwi.com den Raspberry Pi Pico mit MicroPython als Vorlage, fügen Sie das Blink-Programm in die Datei `main.py` ein und starten Sie die Simulation. Für das Messprogramm ergänzen Sie ein Potentiometer und verbinden dessen mittleren Anschluss mit GP26 und die beiden äußeren mit 3V3 und GND. Drehen Sie während der Simulation am Potentiometer und beobachten Sie die Ausgabe und die LED.

D.5. Zum Weiterlesen

- MicroPython-Dokumentation mit Kurzreferenzen für die einzelnen Boards, z.B. für den Raspberry Pi Pico
- Wokwi: Simulator für Mikrocontroller im Browser, mit LEDs, Potentiometern und vielen Sensoren
- Thonny: einfache Python-Entwicklungsumgebung mit MicroPython-Unterstützung